

Chinith: Algorithm Specifications of Signature Schemes SM4th, uBlockith, and Vistrutith

Weiham Li¹, Yuchen Wang¹, Zhelei Zhou¹, Cheng Hong¹, and Tao Wei¹

Ant Group, Hangzhou, China

{liweihan.lwh, tianwu.wyc, zhouzhelei.zzl, vince.hc, lenx.wei}@antgroup.com

June 30, 2026

Contents

Chinith: Algorithm Specifications of Signature Schemes SM4th, uBlockith, and Vistrutith ..	1
<i>Weihan Li, Yuchen Wang, Zhelei Zhou, Cheng Hong, and Tao Wei</i>	
1 Introduction	4
2 Algorithm Description: An Overview	5
2.1 Notations	5
2.2 Generating VOLE Correlations from VOLE Commitments	6
2.3 Zero-Knowledge Proof Phase	7
2.4 Putting Things Together	8
3 Design Rationale	9
3.1 Mathematical Hard Problems and Hardness Assumptions	9
3.2 Parameters and Criterion for Parameter Selection	9
3.3 Analysis of Failure Probability	14
3.4 Statements of Differences	15
3.5 Statements of Additional Functionalities	16
4 Algorithm Description: Cryptographic Primitives	16
4.1 The SM4 Block Cipher	16
4.1.1 The Key Expansion Routine of SM4	18
4.1.2 The Encryption Routine of SM4	19
4.2 The uBlock Block Cipher	19
4.2.1 The Key Expansion Routine of uBlock	21
4.2.2 The Encryption Routine of uBlock	22
4.3 The Ballet Block Cipher	22
4.4 The Vistrutah Block Cipher	23
4.4.1 The Encryption Routine of Vistrutah	25
4.5 Pseudo-Random Generators	26
4.6 Linear Universal Hash Functions	29
5 Algorithm Description: General Building Blocks	31
5.1 VOLE-in-the-Head Functions	32
5.1.1 Batch All-but-One Vector Commitments	32
5.1.2 Seed Expansions and Conversion to VOLE	35
5.1.3 Construction of VOLE Commitments	38
5.2 Zero-Knowledge Operations over VOLE Commitments	40
6 Algorithm Description: the SM4th Signature Scheme	40
6.1 Zero-Knowledge Building Blocks for SM4th	42
6.1.1 Building Blocks for Non-EM Variants	45
6.2 Witness Extension	50
6.3 Deriving Constraints for the Key Expansion	51
6.4 Deriving Constraints for the Encryption	53
6.5 Constraints for the OWF	55
6.6 Construction of SM4th	59
6.6.1 Key Generation	59
6.6.2 Signing	59
6.6.3 Verifying	62
7 Algorithm Description: the uBlockith Signature Scheme	63
7.1 Zero-Knowledge Building Blocks for uBlockith	63
7.2 Witness Extension	63
7.3 Deriving Constraints for the Key Expansion	65
7.4 Deriving Constraints for the Encryption	65
7.5 Constraints for the OWF	67
7.6 Construction of uBlockith	70

7.6.1	Key Generation	70
7.6.2	Signing	70
7.6.3	Verifying	72
8	Algorithm Description: the Vistrutith Signature Scheme	72
8.1	Zero-Knowledge Building Blocks for Vistrutith	73
8.2	Witness Extension	82
8.3	Deriving Constraints for the Encryption	83
8.4	Complete OWF Constraints	89
8.5	Construction of Vistrutith	91
8.5.1	Key Generation	91
8.5.2	Signing	92
8.5.3	Verifying	93
9	Security Statements and Analysis	93
9.1	Theoretical Security	93
9.1.1	Security Targets and Model	93
9.1.2	Building Blocks	95
9.1.3	Security Analysis of Block Ciphers as OWFs	98
9.1.4	Security Analysis of VOLE Commitments	99
9.1.5	Existentially Unforgeable against Chosen Message Attacks in the ROM	101
9.1.6	Existentially Unforgeable against Chosen Message Attacks in the QROM	102
9.1.7	Query Complexity Analysis in Bits	103
9.2	Practical Security against Known Attacks	110
9.2.1	Brute Force the Public Key	110
9.2.2	Brute Force the PRG and Multi-Key Attacks	110
9.2.3	Attack Soundness of the ZK Protocol	111
10	Performance Evaluation	112
10.1	Performance Analysis	112
10.2	Detailed Configuration of Implementation Platforms	113
10.3	Acceleration Techniques	114
10.3.1	Acceleration Techniques of PRGs	114
10.3.2	Acceleration Techniques of Field Operations	116
10.4	Performance Test Results	117
11	Feature Statements	121
11.1	Features Required by ICCS	121
11.2	Additional Advantages	122
11.3	Limitations	122

1 Introduction

This document describes and specifies Chinith¹, including SM4th, uBlockith and Vistrutith, a family of signature schemes based on block ciphers and zero-knowledge proofs (ZKPs) [1] following the “in-the-head” paradigm, targeting 128-bit, 256-bit, and 512-bit classical security, respectively. These schemes are designed to achieve post-quantum security using only information-theoretic and symmetric-key cryptographic primitives. Their security is bound to the security of the Chinese national standard SM3 hash function [2] and the underlying block ciphers. In particular, the block ciphers employed in Chinith include the 128-bit-block Chinese national standard SM4 [3], as well as the 256-bit-block uBlock [4] and Ballet [5], which are solely designed by Chinese researchers and were the only two first-prize winners in the block-cipher category of the Chinese Cryptographic Algorithm Design Competition². For 512-bit-block algorithms, since no suitable Chinese-designed block ciphers were available at the moment, we therefore selected Vistrutah [6], and will switch to a Chinese-designed 512-bit block cipher if one becomes available in the future.

Let $E_k(x)$ be a one-way function (OWF) with input binary vector x and secret input k . In our case, it can be the encryption algorithm of a block cipher with key k and plaintext x . A signature is generated by a ZKP, in which the signer acts as the prover and proves to the receiver (*i.e.*, verifier) that it knows k such that $E_k(x) = y$ for public x, y and $E(\cdot)$. Such a ZKP is typically originally a public-coin interactive argument and can be converted into non-interactive via the Fiat-Shamir transformation [7], where the public-coin challenges are generated by hashing each-round transcripts with the hashes modeled as random oracles. Here, the public key is (x, y) , the secret key is (x, k) , and the signature is the ZKP proof with the message embedded in the first-round transcript. Technically, the ZKPs are built based on the *Vector Oblivious Linear Evaluation in the Head* (VOLEitH) [8, 9] framework, and specifically, a variant of the QuickSilver protocol [10].

For security, our signature schemes achieve the standard property of existential unforgeability against chosen-message attacks (EUF-CMA). Further, two security proofs are presented, with one in the random oracle model (ROM) and the other in the quantum-accessible random oracle model (QROM). Importantly, the bound of the proof in QROM remains close to that in ROM, apart from the expected losses from quantum search and collision-finding.

Related works and comparisons. Our signature schemes are similar to other post-quantum signature schemes via the “in-the-Head” techniques. The original work is PICNIC [11], which is constructed from MPC-in-the-Head (MPCith) [12] and using LowMC as OWF [13]. The MPCith framework is later improved by the threshold-computation-in-the-head (TCitH) [14] framework. Subsequent works combine the MPCith/TCitH framework with various OWFs to build signatures with different trade-offs, such as Banquet [15] from AES, Mirath [16] from the Min-Rank problem, MQOM [17] from the multivariate quadratic problem, PERK [18] from the permuted kernel problem, and RYDE [19] or SDitH [20] from syndrome decoding. Recently, VOLEitH [9] is proposed, which can lead to smaller signatures sizes due to the more efficient repetition method to achieve target soundness, such as FAEST [21] from AES and ReSolveD [22] from syndrome decoding.

There also exist other post-quantum signatures based on different assumptions and techniques, such as lattice-based Dilithium [23], FALCON [24], and HAWK [25], code-based CROSS [26] and LESS [27], multivariate-based MAYO [28], isogeny-based SQIsign [29], and solely hash-based SPHINCS⁺ [30]. Our signatures achieve different trade-offs in security and performance.

Compared to them, our most promising feature is that the security of our signature schemes relies solely on symmetric ciphers and does not depend on any number-theoretic or structured hardness assumptions. Among the related works, FAEST and SPHINCS⁺ also achieve this property. However, this advantage comes at the cost that our scheme (as well as FAEST and SPHINCS⁺) is expected to have worse performance than those schemes based on structured hardness assumptions. Nevertheless, our schemes feature smaller signature sizes, faster signing time and comparable verification time compared with the standardized SLH-DSA version of SPHINCS⁺.

¹ CHINEse block ciphers In The Head

² <https://www.cacrnet.org.cn/site/content/854.html>

Outline of this document. In Section 2, we present an overview of the algorithms, which illustrates the main ideas of the design. Section 3 gives the rationales for the design, including assumptions and hard problems, parameters, and their criteria.

Section 4 introduces the cryptographic primitives used our schemes, including block ciphers, PRGs, and universal hashes. Section 5 specifies several general building blocks of all signature schemes, including VOLE commitments and zero-knowledge operations over VOLE commitments.

Section 6, Section 7, and Section 8 propose the concrete construction of SM4th, uBlockith, and Vistruthith, respectively.

Section 9 analyzes the security of our schemes, including theoretical security, concrete security, and practical security against known attacks. Section 10 provides the performance evaluation, including signature-size analysis, implementation platforms, intersection-set acceleration techniques, and performance results. Finally, in Section 11, we summarize the features of our schemes.

2 Algorithm Description: An Overview

This section provides a high-level overview of the proposed signature schemes. Let E be a public block-cipher-based OWF. In the standard OWF, the public key is defined as $\text{pk} = (\mathbf{x}, \mathbf{y})$ with $\mathbf{y} = E_{\mathbf{k}}(\mathbf{x})$, where $\mathbf{x}$ is public and $\mathbf{k}$ is secret. In the Even–Mansour OWF, the public key is again $\text{pk} = (\mathbf{x}, \mathbf{y})$ with $\mathbf{y} = E_{\mathbf{x}}(\mathbf{k}) \oplus \mathbf{k}$, where $\mathbf{x}$ is the public block-cipher key and $\mathbf{k}$ is the secret input block. In the concrete algorithms below, the secret key is encoded as $\text{sk} = (\mathbf{x}, \mathbf{k})$, so the second component is always the secret witness/key component. A signature is generated via ZKP, where the signer acts as the prover and proves to the receiver (or verifier) that it knows $\mathbf{k}$ satisfying the corresponding OWF relation for public $\mathbf{x}$, $\mathbf{y}$, and $E(\cdot)$.

The proposed signatures use a variant of QuickSilver [10] as the underlying ZKP. Such ZKPs assume a preprocessing phase to generate a sub-field VOLE correlation, which, saying of length ℓ , is defined by a random bit vector $\mathbf{u} \in \mathbb{F}_2^\ell$, a random field-element vector $\mathbf{v} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^\ell$, a random global key $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, and local keys $\mathbf{q} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^\ell$ such that for $i \in [\ell]$,

$$q_i = u_i \cdot \Delta + v_i. \quad (1)$$

The values u_i and v_i should be known only to the prover, while Δ should be picked by the verifier, and q_i should be given to the verifier. In our signature schemes, such VOLE correlations are produced by the VOLEitH technique [9], where the prover generates u_i, v_i and commits to them using a VOLE commitment. After receiving these commitments, the verifier picks and returns a random Δ . Then, the prover opens the commitment such that the verifier learns the q_i values satisfying Equation (1) without anything extra.

VOLE correlations in Equation (1) can be alternatively seen as the prover committing a bit u_i with commitment q_i and openings u_i and v_i . This commitment scheme is (perfectly) hiding due to the mask v_i , and is (computationally) binding because opening to a different u'_i means the prover knows $\Delta = (v_i - q_i + u'_i \Delta) / (u'_i - u_i)$, which happens with a probability of $2^{-\lambda}$. The commitment scheme has linear homomorphism such that if q_i and q_j are commitments to u_i and u_j , respectively, then $q_i + q_j$ is exactly the commitment to $u_i + u_j$. Further, $q_i \cdot q_j = u_i u_j \cdot \Delta^2 + (u_i v_j + u_j v_i) \Delta + v_i v_j$ can be seen as a degree-2 commitment to $u_i \cdot u_j$ such that the degree-2 term stores the secret and the degree-1 or degree-0 term is used for masking.

This section is outlined as follows. Section 2.2 describes how to generate and open VOLE commitments. Section 2.3 introduces the interactive zero-knowledge proof phase given VOLE correlations. In Section 2.4, we show how VOLE commitments and ZKP are combined into a non-interactive zero-knowledge argument.

2.1 Notations

Let $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ denote a finite field with $2^{\lambda_{\mathbb{F}}}$ elements, where $\lambda_{\mathbb{F}}$ is an integer. Let $f(x) \in \mathbb{F}[x]$ denote a polynomial over $\mathbb{F}$ in the variable x . We construct $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ explicitly as $\mathbb{F}_2[z]/f(z)$, where $f(z) \in \mathbb{F}_2[z]$ is an irreducible polynomial of degree $\lambda_{\mathbb{F}}$. We sometimes also use $\text{GF}(2^{\lambda_{\mathbb{F}}})$, *i.e.*, *Galois Field*, to represent such finite fields alternatively with an implicit irreducible polynomial.

Normal lowercase letters such as a mean integers or field elements, according to the context. Bold lowercase letters such as $\mathbf{a} = (a_0, \dots, a_{n-1}) \in \mathbb{F}^n$ denote vectors of length n . Bold uppercase letters like $\mathbf{A}$ denote matrices. Normal lowercase letters may represent integers or vectors, especially constant ones. For vector $\mathbf{a}$, $a[i]$ or a_i means its i -th entry, and $\mathbf{a}[i..j]$ means the sub-vector constructed from $a[i]$ to $a[j]$. We use $\mathbf{a} \in (\mathbb{F}^m)^n$ to represent that $\mathbf{a}$ is split into m sub-vectors each of length n , i.e., $\mathbf{a} = (\mathbf{a}_0, \dots, \mathbf{a}_m)$. Let $\mathbf{a}[i]$ to denote the i -th sub-vector $\mathbf{a}_i$.

We use $\cdot$ to represent multiplications between field elements or scalar multiplications between field elements and vectors. We use $\log a$ to denote the base-2 logarithm of an integer a . The set $\{1, \dots, n\}$ is denoted by $[n]$ for integer n . We use $[m, n]$ and $[m, n)$ to represent the sets $\{m, m+1, \dots, n\}$ and $\{m, m+1, \dots, n-1\}$ for non-negative integers $m < n$, respectively. The symbol $\oplus$ represents the bitwise exclusive-OR (XOR) operation over vectors. Additionally, $\mathbf{v} \lll i$ denotes the circular left shift of a binary vector $\mathbf{v}$ by i positions; $\mathbf{v}(\lll i)_{32}$ means that $\mathbf{c}$ is partitioned into 32-bit blocks, and each 32-bit block is circularly left-shifted by 8 positions independently.

2.2 Generating VOLE Correlations from VOLE Commitments

Generating VOLE correlations from all-but-one vector commitments. We describe how to produce a VOLE correlation of length ℓ over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ via a all-but-one vector commitment, where the prover commits to N seeds via a single hash commitment, and can later open $N-1$ seeds with a communication complexity of $O(\log N)$ and a concrete communication of $(\log N + 2) \cdot \lambda_{\mathbb{F}}$ bits. This all-but-one vector commitment is built from the Goldreich-Goldwasser-Micali (GGM) [31] tree technique, which builds a complete binary tree with $N = 2^{\lambda_{\mathbb{F}}}$ leaves, where, starting from the root node with a random N_{block} -bit seed, the two children of any node are defined by evaluating the parent seed with a length-doubling PRG that outputs two new seeds. Each random bits of these N leaves are used as the randomness in a [BAVC.LeafCommit](#) algorithm, to derive a new seed $\mathbf{sd}$ and a leaf commitment $\mathbf{com}$. The seed $\mathbf{sd}$ is then expanded into ℓ bits via another PRG, contributing to both $\mathbf{u}$ and $\mathbf{v}$ in Equation (1). More concretely, suppose the N committed seeds $\mathbf{sd}_0, \dots, \mathbf{sd}_{N-1}$ are expanded to N bit strings $\mathbf{r}_0, \dots, \mathbf{r}_{N-1} \in \{0, 1\}^{\ell}$. Then, $\mathbf{u}$ and $\mathbf{v}$ are defined as

$$\mathbf{u} = \sum_{i \in [0, N)} \mathbf{r}_i, \quad \mathbf{v} = \sum_{i \in [0, N)} i \cdot \mathbf{r}_i, \quad (2)$$

where the multiplication is over the field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$.

Consider a verifier who learns seeds $\mathbf{sd}_i$ for all $i \neq j^*$ for some index $j^* \in [0, N)$. Note that as $N = 2^{\lambda_{\mathbb{F}}}$, j^* can be uniquely mapped into a field element $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Then, the verifier can compute

$$\mathbf{q} = \sum_{i \in [0, N)} (j^* - i) \cdot \mathbf{r}_i = j^* \cdot \sum_{i \in [0, N)} \mathbf{r}_i - \sum_{i \in [0, N)} i \cdot \mathbf{r}_i = j^* \cdot \mathbf{u} - \mathbf{v}. \quad (3)$$

Note that the field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ has 2-linearity, so Equation (3) is consistent with Equation (1).

Generating τ small VOLE correlations from batch all-but-one vector commitments. There exists one efficiency problem of the “one-tree” approach above: directly generating VOLE correlations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ requires $O(2^{\lambda_{\mathbb{F}}})$ calls of PRG, which is exponential and unacceptable. To solve this problem, VOLEith first generates multiple VOLE instances over smaller finite fields and then combines them into ones over the target large field. Specifically, VOLEith creates τ_0 VOLE instances over field $\mathbb{F}_{2^{k_0}}$ and τ_1 VOLE instances over field $\mathbb{F}_{2^{k_1}}$. If the prover commits the *same* $\mathbf{u}$ in each instance, and $\tau_0 k_0 + \tau_1 k_1 = \lambda_{\mathbb{F}} - w_{\text{grind}}$, then these smaller VOLE instances can be concatenated to a large VOLE correlation in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. More details can be found in this section later.

The vector commitment above can be directly constructed by independent $\tau = \tau_0 + \tau_1$ GGM trees, but the concrete communication would be increased to $\tau \cdot (\log N + n_{\text{leafcom}}) \cdot N_{\text{block}}$ bits, where $n_{\text{leafcom}} N_{\text{block}}$ is the length of a leaf commitment. We use the batch all-but-one vector commitment (BAVC) technique [32] to optimize the communication. Specifically, the prover interleaves multiple vector commitments such that the i -th leaves of multiple trees become neighbors instead of the original neighbored i -th and $(i+1)$ -th leaves of a single tree. When opening all-but- τ leaves, the concrete communication depends on how much the τ active paths in the interleaved GGM

tree overlap. In the best case, when the τ unopened leaves form an aligned subtree of size τ , the number of revealed internal seeds is $\log(L/\tau)$. For equal-length vectors, where $L = \tau N$, this becomes $\log N$, and hence the total communication is $(\log N + \tau n_{\text{leafcom}}) \cdot N_{\text{block}}$ bits. If the τ unopened leaves do not align perfectly into such a subtree, the opening size would be larger. In the worst case, the τ active paths in the interleaved GGM tree have little overlap, so the one-tree BAVC opening provides essentially no asymptotic improvement over using τ independent trees, with a total communication $(\tau \log N + \tau n_{\text{leafcom}}) \cdot N_{\text{block}}$ bits. In our setting, the τ unopened leaves are determined by the verifier's challenge, and therefore the actual BAVC opening size is a random variable depending on the overlap pattern of the corresponding active paths in the interleaved tree.

To control the concrete communication, VOLEith introduces a threshold parameter T_{open} , which upper-bounds the number of internal seeds that may be revealed in a valid opening. Concretely, after deriving a challenge, the prover computes the associated set of unopened leaves and the resulting opening size. If this size is at most T_{open} , the prover proceeds normally; otherwise, the challenge is discarded and a fresh one is derived. Based on the Fiat-Shamir transformation, this retry step can be implemented non-interactively by appending a counter to the transcript and re-hashing until a challenge satisfying the threshold is obtained. This rejection sampling technique reduces the signature size and does not affect the security [32].

From τ small VOLEs to a large VOLE. Without loss of generality, suppose the parties hold τ independent small-field VOLE correlations $\{(\mathbf{q}_i, \Delta_i, \mathbf{u}_i, \mathbf{v}_i)\}_{i \in [\tau]}$, where $\mathbf{u}_i \in \mathbb{F}_2^\ell$, $\mathbf{v}_i \in \mathbb{F}_{2^k}^\ell$, and $\mathbf{q}_i = \mathbf{u}_i \cdot \Delta_i + \mathbf{v}_i$. For convenience, we represent each $\mathbf{v}_i$ as a binary matrix $\mathbf{V}_i \in \{0, 1\}^{\ell \times k}$ instead of a vector over $\mathbb{F}_{2^k}$, where the j -th row is the bit decomposition of the j -th element of $\mathbf{v}_i$.

Let the bit decomposition of $\Delta_i \in \mathbb{F}_{2^k}$ be $(\delta_0^{(i)}, \dots, \delta_{k-1}^{(i)}) \in \{0, 1\}^k$. Then the verifier's output can be written as an $\ell \times k$ binary matrix $\mathbf{Q}_i = (\mathbf{u}_i \cdot \delta_0^{(i)} \parallel \dots \parallel \mathbf{u}_i \cdot \delta_{k-1}^{(i)}) + \mathbf{V}_i$, where $\parallel$ denotes the horizontal concatenation of column vectors.

To concatenate these τ small VOLEs into a single VOLE over $\mathbb{F}_{2^{\lambda_F}}$ with $\lambda = \tau k$, it is necessary that all correlations use the same binary vector $\mathbf{u}$. Let us choose $\mathbf{u} = \mathbf{u}_0$. For each $i \in [\tau - 1]$, the prover sends the correction value $\mathbf{c}_i = \mathbf{u}_i - \mathbf{u}_0$. Using $\mathbf{c}_i$, the verifier can adjust its local view by computing $\mathbf{u}_0 = \mathbf{u}_i - \mathbf{c}_i$ so that the i -th VOLE is now interpreted with respect to the common vector $\mathbf{u} = \mathbf{u}_0$. Denote the corrected instances by $\{(\mathbf{u}, \mathbf{V}_i, \mathbf{Q}_i, \Delta_i)\}_{i \in [\tau]}$.

The prover and verifier then form the concatenated matrices $\mathbf{V} = (\mathbf{V}_0 \parallel \dots \parallel \mathbf{V}_{\tau-1}) \in \{0, 1\}^{(\ell \times k) \parallel (\ell \times k) \dots \parallel (\ell \times k)}$, and $\mathbf{Q} = (\mathbf{Q}_0 \parallel \dots \parallel \mathbf{Q}_{\tau-1}) \in \{0, 1\}^{(\ell \times k) \parallel (\ell \times k) \dots \parallel (\ell \times k)}$, or equivalently, $\mathbf{V}, \mathbf{Q} \in \{0, 1\}^{\ell \times \tau k}$ and hence $\mathbf{V}, \mathbf{Q} \in \{0, 1\}^{\ell \times \lambda}$. They also concatenate the bit decompositions of the Δ_i 's to define $\Delta = (\delta_0^{(0)}, \dots, \delta_{k-1}^{(0)}, \delta_0^{(1)}, \dots, \delta_{k-1}^{(1)}, \dots, \delta_0^{(\tau-1)}, \dots, \delta_{k-1}^{(\tau-1)}) \in \mathbb{F}_{2^{\lambda_F}}$. Viewing the j -th row of $\mathbf{V}$ and $\mathbf{Q}$ as an element of $\mathbb{F}_{2^{\lambda_F}}$, we obtain a single large-field VOLE correlation $\mathbf{Q}[j] = \mathbf{V}[j] + \mathbf{u}[j] \cdot \Delta$ for $j \in [\ell]$. Thus, τ VOLE instances over $\mathbb{F}_{2^k}$ are combined into one VOLE instance over $\mathbb{F}_{2^{\lambda_F}}$.

VOLE consistency check. A malicious prover may send inconsistent correlation value $\mathbf{c}_i$. To guarantee the consistency, the prover must additionally prove that the same $\mathbf{u}$ is used across all instances. This is done using the VOLE consistency check from [21, 33], where the verifier challenges the prover to open a random linear universal hash function applied to $\mathbf{u}$ and $\mathbf{V}$. To this end, let $\mathbf{H}$ be the linear hash matrix related to the verifier's challenge. Then, the prover sends $\tilde{\mathbf{u}} = \mathbf{H}\mathbf{u}$ to the verifier. As the verifier knows $\mathbf{Q}$ and Δ , after receiving $\tilde{\mathbf{u}}$, it can compute $\tilde{\mathbf{Q}} = \mathbf{H}\mathbf{Q}$ and $\tilde{\mathbf{V}} = \tilde{\mathbf{Q}} - \Delta \cdot \tilde{\mathbf{u}}$, and finally checks if $\tilde{\mathbf{Q}}[j] = \tilde{\mathbf{u}}[j] \cdot \Delta + \tilde{\mathbf{V}}[j]$ for $j \in [\ell]$.

Padding the extended witness due to VOLE consistency check. The above description assumes $\mathbf{u}$ is of length ℓ bits, which is the same as the extended witness for the OWF. To achieve zero knowledge, the extended witness would be increased by a masked sub-witness of length $(\lambda_F + B)$ bits, as the VOLE consistency check reveals a $(\lambda_F + B)$ -bit linear combination of $\mathbf{u}$ to the verifier.

2.3 Zero-Knowledge Proof Phase

After generating the VOLE correlations, the prover (or signer) and verifier (or receiver) would invoke the interactive ZKP phase based on QuickSilver [10] to prove the knowledge of secret key consistent with the public key. Below, we assume that the prover and verifier hold random VOLE

correlations $\{(q_i, \Delta, u_i, v_i)\}_{i \in [\ell]}$ such that $q_i, v_i, \Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, $u_i \in \mathbb{F}_2$, and $q_i = u_i \cdot \Delta + v_i$. The proof phase proceeds in three main stages.

Witness extension. The prover takes the secret key $\mathbf{sk} = (\mathbf{x}, \mathbf{k})$ as inputs, and executes the block cipher key expansion and encryption routines to compute $\mathbf{y} = E_{\mathbf{k}}(\mathbf{x})$. Specified bits related to the S-box inputs and outputs are extracted during the execution, and are then recorded as the extended witness $\mathbf{w} \in \{0, 1\}^{\ell}$. After extracting $\mathbf{w}$, the prover commits to it by sending $\mathbf{d} = \mathbf{u} + \mathbf{w}$ to the verifier. This allows the verifier to update the VOLE commitments to $\mathbf{w}$, *i.e.*, $q'_i = q_i + d_i \cdot \Delta = w_i \cdot \Delta + v_i$ for all $i \in [\ell]$.

Evaluating constraints. Next, the parties evaluate all constraints over the committed extended witness. Suppose there are C constraints determined by functions $f_1, \dots, f_C$, where each function can be represented as a degree- d circuit. Then, the goal of the ZKP is showing that for all $i \in [\ell]$, $f_i(\mathbf{w}) = 0$. The circuit evaluation follows a gate-by-gate manner. As the circuit is public and the VOLE commitments have linear homomorphism, the prover and verifier can evaluate constraints for addition gates freely. For multiplication gates, the prover sees its (w_i, v_i) as the coefficients of a secret degree-1 polynomial $\rho_i(X) = w_i X + v_i$. Then, multiplying two VOLE commitments for w_i, w_j is done by multiplying the polynomials $\rho_i(X)\rho_j(X)$, obtaining a degree-2 secret polynomial. Meanwhile, the verifier can multiply its VOLE values q_i and q_j by computing $q_i \cdot q_j = \rho_i(\Delta) \cdot \rho_j(\Delta)$. As the polynomial degree increases, the prover needs to additionally send terms $w_i v_j + w_j v_i$ and $v_i v_j$ to the verifier for the consistency check. The above procedure can be generalized for any low-degree polynomials, as long as the secrets are aligned (by increasing degrees if necessary) in the highest coefficient. The highest degree among $f_i(\mathbf{X})$ is a parameter in our schemes.

Proving constraints. After evaluating all constraints gate-by-gate over the VOLE-committed extended witness, the prover must convince the verifier that every constraint output equals zero, without leaking any information about the witness itself. Recall that each constraint (supposing degree-2), when evaluated over VOLE commitments, yields a secret polynomial of the form $\rho(X) = a_0 + a_1 X + z X^2$, where the leading coefficient z vanishes if and only if the constraint is satisfied. The prover holds the plaintext coefficient vectors (a_0, a_1) , while the verifier holds $\rho(\Delta)$.

To batch all C constraints $\{(a_0^{(i)}, a_1^{(i)}), \rho_i(\Delta)\}$ into a single one, the verifier picks a random challenge chall . The prover then invokes another linear universal hash (called as ZKHash) with chall as a public seed to compress each coefficient vector into a single field element: intuitively, ZKHash evaluates a degree- ℓ polynomial over the entries of $\{(a_0^{(i)}, a_1^{(i)})\}_{i \in [\ell]}$ at a random point derived from chall , producing 2 compact hash outputs $\tilde{a}_0, \tilde{a}_1 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. The verifier applies ZKHash to obtain a corresponding value $\tilde{q} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. By the linearity of both ZKHash and the underlying VOLE structure, it holds that $\tilde{q} = \tilde{a}_0 + \tilde{a}_1 \cdot \Delta$ if and only if every constraint output vanishes (*i.e.*, $z = 0$). The verifier accepts if $\tilde{q} - \tilde{a}_1 \cdot \Delta$ matches the prover-supplied $\tilde{a}_0$. This proving phase can be naturally generalized into the high-degree case such that for degree- d constraints, the prover computes $\tilde{a}_0, \dots, \tilde{a}_{d-1}$ from $\{(a_0^{(i)}, a_1^{(i)}, \dots, a_{d-1}^{(i)})\}_{i \in [\ell]}$ and sends $\tilde{a}_1, \dots, \tilde{a}_{d-1}$ to the verifier, and the verifier checks if $\tilde{q} - \tilde{a}_{d-1} \cdot \Delta^{d-1} \dots - \tilde{a}_1 \cdot \Delta$ matches the prover-supplied $\tilde{a}_0$.

Padding the extended witness due to ZKHash. Like the VOLE consistency check, to achieve zero knowledge, the extended witness would be increased by a masked sub-witness of length $(d - 1) \cdot \lambda_{\mathbb{F}}$ bits, as the ZKHash reveals a $(d - 1) \cdot \lambda_{\mathbb{F}}$ -bit linear combination of $\{(a_0^{(i)}, a_1^{(i)}, \dots, a_{d-1}^{(i)})\}_{i \in [\ell]}$.

2.4 Putting Things Together

In our construction, VOLE commitments and the QuickSilver-style ZKP are combined into a 3-challenge interactive argument. At a high level, the protocol consists of the following stages.

First, the prover and verifier generate VOLE-based commitments and correction values. The verifier then sends the first challenge chall_1 , which is used to derive the VOLE consistency check. After receiving chall_1 , the prover answers this check and, at the same time, commits to the extended witness derived from the secret key and the OWF evaluation. Next, the verifier sends a second challenge chall_2 . Using this challenge, the prover evaluates the OWF constraints over the VOLE-committed witness and generates the proof for the OWF ZKP phase.

Finally, the verifier sends the third challenge chall_3 , *i.e.*, Δ , which determines which parts of the VOLE commitment must be opened. The prover returns the requested VOLE opening information, and the verifier checks both the VOLE consistency and the correctness of the compressed constraint proof. If all checks pass, the verifier accepts.

To obtain a non-interactive signature scheme, we apply the Fiat–Shamir transformation in the random oracle model. Instead of receiving challenges from the verifier, the signer derives them by hashing the transcript incrementally. More precisely, letting μ be the hash of the signature message and the public key, the challenges are defined as $\text{chall}_i = H(\text{chall}_{i-1} \parallel \text{msg}_i)$, where msg_i denotes the prover’s i -th message and $\text{chall}_0 \leftarrow \mu$.

A straightforward Fiat–Shamir transcript would include all prover messages. However, some parts of the early messages can be reconstructed from the final challenge chall_3 together with the VOLE opening. Therefore, the signature can be represented more compactly by keeping only the information that is not reconstructible, while still allowing the verifier to recompute the full transcript, reconstruct all challenges, and verify that the resulting transcript is consistent.

3 Design Rationale

This section describes the parameters for our signature schemes, including the main parameter param , the OWF parameter $\text{param}_{\text{OWF}}$, and the VOLE parameter $\text{param}_{\text{VOLE}}$.

3.1 Mathematical Hard Problems and Hardness Assumptions

Mathematical hard problems. Our signature schemes do not rely on any number-theoretic or structured hardness assumptions. We only assume the security of block ciphers as PRGs and OWFs, as well as hash functions as PRFs and random oracles.

Hardness assumptions. We make the following hardness assumptions in the proposed signatures.

First, we rely on the PRP and key-recovery security of the underlying block ciphers in both the classical and quantum models, with the latter one estimated according to [34, 35]. We also assume that block ciphers and the Even-Mansour constructions [36] of block ciphers have one-wayness properties, which have been implied in previous works [37, 38].

Second, we rely on the security of PRG-based leaf commitments in the VOLE commitments, which assume the collision-resistance of 2λ -bit outputs of block ciphers for the security parameter λ . This leaf commitment is adapted from FAEST-EM variants for better performance, which can be changed as standard hash functions or the universal-hash-based leaf commitment in non-EM variants of FAEST [21].

Finally, to make the ZKP non-interactive, we rely on the ROM and QROM. Although idealized models, they are widely used in cryptography, including the NIST selected standardized post-quantum signature schemes ML-DSA, FALCON [24], and SLH-DSA [30].

3.2 Parameters and Criterion for Parameter Selection

This section introduces the parameters used in our signature schemes and the criterion for parameter selection. There are four sets of parameters, the main parameter param , the OWF parameter $\text{param}_{\text{OWF}}$, the VOLE parameter $\text{param}_{\text{VOLE}}$, and other parameters $\text{param}_{\text{other}}$ including hashes and IVs. The first three sets of parameters are determined by the main parameter param .

Main parameter. The main parameter param , which determines $\text{param}_{\text{OWF}}$ and $\text{param}_{\text{VOLE}}$, is

$$\text{param} = (\lambda, \lambda_Q, \lambda_{\mathbb{F}}, \tau, w_{\text{grind}}, T_{\text{open}}, d, B, \text{OWF}, \ell, \lambda_{\text{iv}}, N_{\text{block}}).$$

The meanings of these parameters are given below with a further description below.

- λ , security parameter targeting for a specified classical security, as required by ICCS.
- λ_Q , security parameter targeting for a specified quantum security, as required by ICCS.

- $\lambda_{\mathbb{F}}$, the logarithm of the field size. This is determined by λ and $\lambda_{\mathbb{Q}}$ and the round-by-round soundness of signature schemes.
- τ , the total number of sub-GGM trees in the BAVC technique, as introduced in Section 2.2.
- w_{grind} . Grinding (a lightweight proof-of-work), a technique practically applied in ZKPs and blockchain, is used to make the Fiat–Shamir challenge grinding expensive for a malicious prover. After committing, the prover must find a nonce such that hashing it with the transcript state yields an output with a number of w_{grind} leading zeros before deriving Fiat–Shamir randomness. Any change to commitments forces repeating this work, while the verifier checks the nonce with a single hash. A grinding parameter w_{grind} is estimated to provide w_{grind} -bit security.
- T_{open} , the upper bound of the number of revealed seeds in the opening of BAVC, as introduced in Section 2.2.
- d , the highest degree of polynomial constraints in the ZKP for OWFs. This is determined by the proving method for OWFs, and we choose the method with the optimized signature size.
- B , used for extra length padding for the VOLE consistency check. This is fixed as 16 to meet the target security,
- OWF. The OWF is either $y = E_k(x)$ (the standard variant) or $y = E_x(k) \oplus k$ (the EM variant), where E is the encryption function of a specified block cipher. (x, y) are the public key, and the concrete secret-key encoding is (x, k) so that the second component is the secret witness/key component. The signer proves that it knows the witness k such that $y = E_k(x)$ or $y = E_x(k) \oplus k$ holds for public inputs (x, y, E) via a ZKP.
- ℓ , the extended witness length for the underlying ZKP, which is determined by the OWF and the proving method.
- λ_{iv} , the length of initialization vectors (IVs), which is no more than the block size of PRGs.
- N_{block} , the block size used in PRGs.

Table 1 lists the concrete values of the main parameters `param`, and the computed sizes of public key, secret key and signature for all proposed variants of `SM4th`, `uBlockith`, and `Vistrutith`. Specifically, the key words of variants and their meanings are as follows.

- `SM4th`, `uBlockith`, or `Vistrutith`: the block cipher used as the OWF in the scheme.
- “EM” or no “EM”: whether the scheme adopts the EM or non-EM construction as the OWF.
- “s” or “f”: whether the variant focuses on smaller signatures or faster running times.
- “128”, “256”, or “512”: the targeted classical security level, in bits.
- “lo” or “ti”: whether the variant targets the loose or tight security requirements.

Below we give necessary explanations.

Choice of loose and tight variants with different fields, λ_{iv} , and PRGs. For `SM4th` with 128-bit classical security, we give two sets of schemes, loose (-lo) variants and tight (-ti) variants, with different λ_{iv} , PRGs, and finite fields. The tight variants achieve the bit-security requirements of ICCS strictly from the theoretical perspective. Meanwhile, the loose variants aim for the practical security, achieving bit-security requirements of ICCS against the best-known concrete attacks.

`SM4th-lo` uses SM4 as PRGs with $\lambda_{\text{iv}} = 128$, and `SM4th-ti` uses Ballet as PRGs with $\lambda_{\text{iv}} = 256$, where the size- λ_{iv} IV contributes to the plaintext of block-cipher-based PRGs. The choices of block ciphers are flexible, *e.g.*, `SM4th-ti` can use PRGs with block sizes no less than 160 bits. Meanwhile, `SM4th-lo` uses a 128-bit finite field in contrast to `SM4th-ti` using a larger 160-bit finite field.

The reason for introducing these two variants is to satisfy the signing-query bound and the quantum concrete security of ICCS, which requires that:

1. The upper bound of signing queries is $Q_{\text{sig}} = 2^{80}$.
2. Each pair of keys can be used at most 2^{64} times.
3. The concrete security (in bits) in the quantum model corresponding to the classical 128-bit security should be 80 bits.

For SM4-based PRGs with $\lambda_{\text{iv}} = 128$, IV collisions are bounded by the birthday paradox. Consequently, a single key supports at most roughly 2^{64} signatures before the collision probability becomes non-negligible. To meet the ICCS target of $Q_{\text{sig}} = 2^{80}$, the *loose variants* retain the native

SM4 PRG and interpret the 2^{80} signing queries in a multi-key EUF-CMA setting, for example, with 2^{16} keys each used for at most 2^{64} signatures. The total query budget is therefore distributed across many independently generated keys, satisfying ICCS requirements.

In contrast, the *tight variants* achieve single-key security by setting $\lambda_{iv} = 256$ and using Ballet as the PRG. This enables the full $Q_{sig} = 2^{80}$ bound to be supported under a single key with negligible IV-collision probability.

Thus, the loose variants preserve the SM4-only PRG design, while the tight variants provide a single-key interpretation of the ICCS signing-query requirement.

For the field size, our theoretical security proof in the QROM contains a term of the form $Q^2/2^{\lambda_F}$ in the round-by-round soundness error of the ZKP, where Q denotes the number of random-oracle queries and λ_F is the bit size of the finite field. This term comes from the probability that the adversary guesses a bad value of Δ in the VOLE correlation of Equation (1). Therefore, **in the tight variants, we set $\lambda_F = 160$** , so that this proof term remains near the 80-bit quantum-security target for the considered query regime. **For the loose variants, we instead use $\lambda_F = 128$** , and consider the concrete security against best known attacks. Currently, we are not aware of any quantum attack of guessing Δ that performs better than the Grover algorithm. A concrete Grover attack against the 128-bit field would have to search over a 128-bit space of possible Δ values, which requires about 2^{64} Grover iterations. Moreover, each Grover iteration has to evaluate the corresponding predicate, which in our setting involves an invocation of SM3. Using the same depth-limited cost model as in [34, 39, 40], we estimate the depth-limited cost of this Grover search is approximately $2^{160}/\text{MAXDEPTH}$ (Details can be found in Section 9.2.3). For the concrete value $\text{MAXDEPTH} = 2^{56}$ used in our evaluation, the resulting attack cost is approximately 2^{104} , which satisfies the requirements of 80-bit quantum security. We therefore use $\lambda_F = 128$ for the loose variants to obtain a more efficient scheme, while retaining the 160-bit field in the tight variants if a tighter theoretic QROM bound is desired.

Choice of τ , w_{grind} , and T_{open} . These parameters are chosen jointly for the BAVC opening and Fiat-Shamir challenge space. Let $m = \lambda_F - w_{grind}$. For a fixed τ , the implementation splits the m challenge-index bits almost evenly among the τ sub-GGM trees by setting $k = \lfloor m/\tau \rfloor + 1$, $\tau_1 = m \bmod \tau$, and $\tau_0 = \tau - \tau_1$. Thus τ_1 trees have depth k and τ_0 trees have depth $k - 1$, so that $\tau_1 k + \tau_0 (k - 1) = m$ and the total challenge-index space is $\prod_i N_i = 2^{\lambda_F - w_{grind}}$. The parameter w_{grind} gives the grinding cost: the signer searches for a nonce such that the last w_{grind} challenge bits are zero, giving an expected cost of about $2^{w_{grind}}$ hash trials for any changed transcript, while verification checks the recorded nonce with one hash. Finally, T_{open} is selected as the fixed upper bound on the number of sibling seeds included in the BAVC opening. If n_h denotes the number of nodes in the union of the opened root-to-leaf paths, the implementation accepts only openings satisfying $n_h - 2\tau + 1 \leq T_{open}$. In summary, smaller τ reduces the signature size, while larger τ leads to faster running times. Smaller T_{open} reduces the signature size, whereas larger T_{open} increases the acceptance probability of the grinding loop. All these concrete values are chosen to balance signature size and signing/verifying cost for each parameter set.

Our concrete choices follow the same tuning principle as FAEST: we keep the small-GGM tree depths close to the corresponding FAEST instances, and then choose T_{open} so that the expected number of signing attempts remains in the same range. For the 128-bit loose variants, we adapt the parameters from FAEST. For the SM4-based 128-bit tight schemes, after enlarging the proof field to $\lambda_F = 160$, keeping the original FAEST values $\tau = 11$ and $\tau = 16$ would make the BAVC leaf budget much larger. We therefore increase τ so that the maximum tree depth is preserved: the small variants use $\tau = 14$, for which $\lambda_F - w_{grind} = 153$ is split into trees of depth at most 11, matching the FAEST-128s depth, while the fast variants use $\tau = 21$, for which the maximum depth is 8, matching the FAEST-128f depth. We keep $w_{grind} = 7$ for SM4th-s and $w_{grind} = 8$ for SM4th-f variants. With these choices, we set $T_{open} = 131$ for SM4th-d3-128s and $T_{open} = 132$ for SM4th-EM-d2-128s, giving an expected signing-retry cost comparable to FAEST-128s. For the fast variants, we set $T_{open} = 138$ for SM4th-d3-128f and $T_{open} = 140$ for SM4th-EM-d2-128f, giving an expected retry cost comparable to FAEST-128f.

For the uBlock-based 256-bit schemes, we keep the FAEST-256-style BAVC split directly. The small variants use $\tau = 22$ and $w_{grind} = 6$, so we have $\tau_1 = 8$, $\tau_0 = 14$, and $k = 12$, giving trees

Table 1: Concrete values of the main parameter **param**, where “s” means a variant with smaller signature sizes but slower running times, and “f” means a variant with faster running times and larger signature sizes

Scheme	OWF $E_k(x)$ or $E_x(k) \oplus k$	(λ, λ_Q)	λ_F	ℓ	τ	w_{grind}	T_{open}	B	d	λ_{iv}	N_{block}
SM4th-d3-128s-lo	$SM4_k(x)$	(128, 80)	128	1808	11	7	102	16	3	128	128
SM4th-d3-128f-lo		(128, 80)	128	1808	16	8	110	16	3	128	128
SM4th-EM-d2-128s-lo	$SM4_x(k) \oplus k$	(128, 80)	128	1024	11	7	103	16	2	128	128
SM4th-EM-d2-128f-lo		(128, 80)	128	1024	16	8	112	16	2	128	128
SM4th-d3-128s-ti	$SM4_k(x)$	(128, 80)	160	1808	14	7	131	16	3	256	256
SM4th-d3-128f-ti		(128, 80)	160	1808	21	8	138	16	3	256	256
SM4th-EM-d2-128s-ti	$SM4_x(k) \oplus k$	(128, 80)	160	1024	14	7	132	16	2	256	256
SM4th-EM-d2-128f-ti		(128, 80)	160	1024	21	8	140	16	2	256	256
uBlockith-d3-256s	$uBlock_k(x)$	(256, 128)	256	4608	22	6	245	16	3	256	256
uBlockith-d3-256f		(256, 128)	256	4608	32	8	246	16	3	256	256
uBlockith-EM-d3-256s	$uBlock_x(k) \oplus k$	(256, 128)	256	3072	22	6	218	16	3	256	256
uBlockith-EM-d3-256f		(256, 128)	256	3072	32	8	234	16	3	256	256
Vistrutith-d3-512s	$Vistrutah_k(x)$	(512, 256)	512	6912	44	5	480	16	3	256	512
Vistrutith-d3-512f		(512, 256)	512	6912	64	8	481	16	3	256	512

of depth at most 12 and leaf budget $8 \cdot 2^{12} + 14 \cdot 2^{11}$, matching the FAEST-256s depth. The fast variants use $\tau = 32$ and $w_{\text{grind}} = 8$, so we have $\tau_1 = 24$, $\tau_0 = 8$, and $k = 8$, giving leaf budget $24 \cdot 2^8 + 8 \cdot 2^7$, matching the FAEST-256f depth. With these choices, we set $T_{\text{open}} = 245$ for uBlockith-d3-256s and $T_{\text{open}} = 246$ for uBlockith-d3-256f, near the high-acceptance region of the BAVC opening distribution. For the EM variants, the witness length and hence $\hat{\ell}$ are smaller, so we use the slightly tighter bounds $T_{\text{open}} = 218$ for uBlockith-EM-d3-256s and $T_{\text{open}} = 234$ for uBlockith-EM-d3-256f to further reduce signature size while keeping the expected retry cost practical and comparable to the corresponding FAEST-256-style variants.

For the 512-bit instantiation, the choices of τ , w_{grind} , and T_{open} for Vistrutith then follow the same FAEST-style depth-preserving rule, using the FAEST-256 instances as the reference point. The small variant uses $\tau = 44$, approximately twice the FAEST-256s value $\tau = 22$, so that the maximum small-tree depth remains 12. We set $w_{\text{grind}} = 5$ for this variant because increasing it to 6 would only slightly reduce the leaf budget but would double the grinding cost, which is less attractive for the more expensive 512-bit field operations. The fast variant uses $\tau = 64$, matching twice the FAEST-256f value $\tau = 32$, and keeps $w_{\text{grind}} = 8$, giving trees of depth at most 8 and a much smaller BAVC leaf budget. Finally, $T_{\text{open}} = 480$ and $T_{\text{open}} = 481$ are chosen near the high-acceptance region of the BAVC opening distribution, so that the grinding loop is dominated mainly by the explicit $2^{w_{\text{grind}}}$ factor rather than by frequent BAVC aborts.

One-way functions and their parameters. We introduce OWFs used in SM4th, uBlockith, and Vistrutith and extra parameters $\text{param}_{\text{OWF}}$ related to these OWFs.

Our signatures use $y = E_k(x)$ as OWF where E is the encryption routine of the underlying block cipher, x is the plaintext, y is the ciphertext, and k is the key of corresponding block ciphers. The public key is (x, y) , and the concrete secret-key encoding is (x, k) . In SM4th-EM and uBlockith-EM, the one-way function is $y = E_x(k) \oplus k^3$, where x is the key, and k is the plaintext of corresponding block ciphers. The public key is (x, y) , and the concrete secret-key encoding is

³ As long as $E_x(k)$ can be modeled as a random permutation, $y = E_x(k) \oplus k$ is indeed a OWF. To the best of our knowledge, no attacks are known against full round SM4 [41] or uBlock [42] that would undermine this construction.

Table 2: Concrete values of the OWF parameter $\text{param}_{\text{OWF}}$ for all parameter sets.

Scheme	OWF	R	S_{ke}	S_{enc}	ℓ_{ke}	ℓ_{enc}	ℓ	C
SM4th-d3-128s-lo	$\text{SM4}_k(x)$	32	128	128	1024	784	1808	453
SM4th-d3-128f-lo		32	128	128	1024	784	1808	453
SM4th-EM-d2-128s-lo	$\text{SM4}_x(k) \oplus k$	32	0	128	0	1024	1024	257
SM4th-EM-d2-128f-lo		32	0	128	0	1024	1024	257
SM4th-d3-128s-ti	$\text{SM4}_k(x)$	32	128	128	1024	784	1808	453
SM4th-d3-128f-ti		32	128	128	1024	784	1808	453
SM4th-EM-d2-128s-ti	$\text{SM4}_x(k) \oplus k$	32	0	128	0	1024	1024	257
SM4th-EM-d2-128f-ti		32	0	128	0	1024	1024	257
uBlockith-d3-256s	$\text{uBlock}_k(x)$	24	384	1536	1792	2816	4608	4609
uBlockith-d3-256f		24	384	1536	1792	2816	4608	4609
uBlockith-EM-d3-256s	$\text{uBlock}_x(k) \oplus k$	24	0	1536	0	3072	3072	3073
uBlockith-EM-d3-256f		24	0	1536	0	3072	3072	3073
Vistruth-d3-512s	$\text{Vistrutah}_k(x)$	18	0	1152	0	6912	6912	2881
Vistruth-d3-512f		18	0	1152	0	6912	6912	2881

again (x, k) . As Vistrutah does not involve any non-linear operations in its key-schedule routine, we do not construct EM variants of Vistruth.

Table 2 describes the parameters of these one-way functions used in ZKP, which is denoted as

$$\text{param}_{\text{OWF}} = (R, S_{\text{ke}}, S_{\text{enc}}, \ell_{\text{ke}}, \ell_{\text{enc}}, \ell, C).$$

The meanings of $\text{param}_{\text{OWF}}$ are as follows.

- R : the number of rounds for either the key expansion routine or the encryption routine;
- S_{ke} : the number of S-boxes in the key expansion routine;
- S_{enc} : the number of S-boxes in the encryption routine;
- ℓ_{ke} : the length of extended witness in bits for the key expansion routine;
- ℓ_{enc} : the length of extended witness in bits for the encryption routine;
- ℓ : the total length of extended witness in bits;
- C : the number of constraints in OWFs.

VOLE-in-the-head parameters. Table 3 describes the VOLE-in-the-head parameter $\text{param}_{\text{VOLE}}$, which is defined as

$$\text{param}_{\text{VOLE}} = (\tau_1, \tau_0, k, L, \{N_i\}_{i=0}^{\tau-1}, B, \hat{\ell}, N_{\text{block}}).$$

The meanings of $\text{param}_{\text{VOLE}}$ are as follows.

- τ_1 : the number of *larger* small-VOLE instances, computed as $\tau_1 = (\lambda_{\mathbb{F}} - w_{\text{grind}}) \bmod \tau$;
- τ_0 : the number of *smaller* small-VOLE instances, computed as $\tau_0 = \tau - \tau_1$;
- k : the bit-length of the larger small-VOLE instances, computed as $k = \lfloor (\lambda_{\mathbb{F}} - w_{\text{grind}}) / \tau \rfloor + 1$, so that the bit-length of the smaller instances is $k - 1$;
- N_i : the field size of the i -th small-VOLE instance, equivalently the number of leaves in the i -th GGM tree, defined as

$$N_i = \begin{cases} 2^k & \text{if } i < \tau_1, \\ 2^{k-1} & \text{otherwise;} \end{cases}$$

- L : the total number of leaves across all GGM trees, given by

$$L = \sum_{i=0}^{\tau-1} N_i = \tau_1 \cdot 2^k + \tau_0 \cdot 2^{k-1}.$$

Then, the VOLE challenge space is $\prod_{i=0}^{\tau-1} N_i = 2^{\lambda_{\mathbb{F}} - w_{\text{grind}}}$.

Table 3: Concrete values of the VOLE-in-the-head parameter $\text{param}_{\text{VOLE}}$ for all parameter sets

Scheme	τ_1	τ_0	k	L	$\{N_i\}_{i=0}^{\tau-1}$	B	$\hat{\ell}$	N_{block}
SM4th-d3-128s-lo	0	11	12	22528	11×2^{11}	16	2208	128
SM4th-d3-128f-lo	8	8	8	3072	$8 \times 2^8, 8 \times 2^7$	16	2208	128
SM4th-EM-d2-128s-lo	0	11	12	22528	11×2^{11}	16	1296	128
SM4th-EM-d2-128f-lo	8	8	8	3072	$8 \times 2^8, 8 \times 2^7$	16	1296	128
SM4th-d3-128s-ti	13	1	11	27648	$13 \times 2^{11}, 1 \times 2^{10}$	16	2304	256
SM4th-d3-128f-ti	5	16	8	3328	$5 \times 2^8, 16 \times 2^7$	16	2304	256
SM4th-EM-d2-128s-ti	13	1	11	27648	$13 \times 2^{11}, 1 \times 2^{10}$	16	1360	256
SM4th-EM-d2-128f-ti	5	16	8	3328	$5 \times 2^8, 16 \times 2^7$	16	1360	256
uBlockith-d3-256s	8	14	12	61440	$8 \times 2^{12}, 14 \times 2^{11}$	16	5392	256
uBlockith-d3-256f	24	8	8	7168	$24 \times 2^8, 8 \times 2^7$	16	5392	256
uBlockith-EM-d3-256s	8	14	12	61440	$8 \times 2^{12}, 14 \times 2^{11}$	16	3856	256
uBlockith-EM-d3-256f	24	8	8	7168	$24 \times 2^8, 8 \times 2^7$	16	3856	256
Vistruth-d3-512s	23	21	12	137216	$23 \times 2^{12}, 21 \times 2^{11}$	16	8464	512
Vistruth-d3-512f	56	8	8	15360	$56 \times 2^8, 8 \times 2^7$	16	8464	512

- B : the extra bit-length padding for the VOLE consistency check, fixed at $B = 16$;
- $\hat{\ell}$: the extended witness length including extra masks for the VOLE consistency check and ZKHash, computed as $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$.
- N_{block} : the block size of the used block ciphers as PRGs.

Hash functions. Let λ denote the seed/security parameter, $\lambda_{\mathbb{F}}$ denote the VOLE/QuickSilver field size, and $\lambda_{\text{iv}} = 128$ for SM4th-lo variants or 256 for all other variants. Our schemes use the following hash functions.

- $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^{2N_{\text{block}}}$, the hash function for vector commitments.
- $H_2^0 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, the hash function for deriving the message-dependent digest μ .
- H_2^i : hash functions for the Fiat-Shamir transformation, where $i \in \{0, 1, 2, 3\}$. Specifically,
 - $H_2^0 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$
 - $H_2^1 : \{0, 1\}^* \rightarrow \{0, 1\}^{5\lambda_{\mathbb{F}}+64}$,
 - $H_2^2 : \{0, 1\}^* \rightarrow \{0, 1\}^{3\lambda_{\mathbb{F}}+64}$,
 - $H_2^3 : \{0, 1\}^* \rightarrow \{0, 1\}^{\lambda_{\mathbb{F}}}$.
- $H_3 : \{0, 1\}^{4\lambda+\xi} \rightarrow \{0, 1\}^{N_{\text{block}}+\lambda_{\text{iv}}}$, the hash function for root seed and pre-processing initialization vector (IV). H_3 is modeled as a PRF, with the last ξ bits as the key. We set $\xi = \lambda + 128$.
- $H_4 : \{0, 1\}^{\lambda_{\text{iv}}} \rightarrow \{0, 1\}^{\lambda_{\text{iv}}}$, the hash function from pre-processing IV to IV.

These hash functions are instantiated with SM3 [2] of 256-bit output length or XOFs implemented by SM3 provided by ICCS⁴. We use $H(x; \ell)$ to specify the input x and the output length ℓ in bits.

3.3 Analysis of Failure Probability

The only algorithmic failure event comes from the grinding step in the generation of chall_3 . For each counter value, the hash output is modeled as uniform, and the grinding condition requires the last w_{grind} bits to be zero. Hence one trial succeeds with probability $2^{-w_{\text{grind}}}$, and after at most T trials the failure probability is

$$\Pr[\text{Fail}] = (1 - 2^{-w_{\text{grind}}})^T \leq \exp(-T/2^{w_{\text{grind}}}).$$

⁴ https://niccs.org.cn/niccs/Download/API/API_PKC.zip

Taking SM4th-d3-128s variants as an example, $w_{\text{grind}} = 7$, so the expected number of trials is $2^7 = 128$. Our implementation uses a 32-bit counter, then $T = 2^{32}$ and $\Pr[\text{Fail}] \leq \exp(-2^{25})$, which is very small. Note that the counter length is adjustable if a smaller probability is preferred, which only slightly affects the concrete signature sizes.

This failure event has no practical impact on correctness or security. It only introduces a variable signing time due to the distribution of the number of grinding trials.

3.4 Statements of Differences

Our signature schemes follow the high-level VOLE-in-the-head framework of FAEST, but differ from FAEST in several important aspects. FAEST instantiates its schemes with AES and aligns the main security parameter across the construction: for a λ -bit security level, it uses an AES variant with a λ -bit key as both the OWF and the PRG, performs the ZKP over $\mathbb{F}_{2^\lambda}$, and chooses hash output lengths linear in λ . In contrast, Chinith decouples these roles when necessary and replaces the AES-based components with primitives that are better aligned with Chinese cryptographic standards and with the algebraic structure required by our proofs.

First, we distinguish the proof field used by the loose and tight SM4-based instantiations. For all SM4-based instantiations, the seed and OWF security parameter remains $\lambda = 128$, while the target quantum security is $\lambda_Q = 80$. For the tight variants, we follow the strict theoretical proof, where the round-by-round soundness term is roughly of the form $Q^2 \cdot 2^{-\lambda_F}$, so a quantum adversary making $Q \approx 2^{80}$ queries requires a challenge space of size about 2^{160} . We therefore set $\lambda_F = 160$ for the tight variants of SM4th and use $\mathbb{F}_{2^{\lambda_F}}$ in VOLE, QuickSilver, ZKHash, and the Fiat-Shamir challenges that are interpreted as field elements. For the loose variants, we follow a concrete attack analysis to the soundness considering the concrete cost of Grover’s algorithm, and results show that it suffices to set $\lambda_F = 128$, keeping the proof field aligned with the OWF security parameter.

Second, we distinguish the lengths of IVs between the loose and tight variants. In FAEST, the AES-CTR-based PRG uses a 128-bit IV, which is natural for a 128-bit block cipher but only supports the existing divergence-based analysis up to about 2^{64} signatures under one key. For the SM4-based loose variants, denoted by the suffix “lo”, we keep the SM4-based PRG and its 128-bit IV for better performance and compatibility with SM4 hardware acceleration. These variants therefore interpret the required $Q_{\text{sig}} = 2^{80}$ signing-query bound in a multi-key sense, for example as 2^{16} independent keys each used for at most 2^{64} signatures. For the tight variants, denoted by the suffix “ti”, we use 256-bit block cipher and IV, so that IV collisions remain negligible even for $Q_{\text{sig}} = 2^{80}$ under a single key. Thus, the loose variants prioritize efficiency under a multi-key interpretation, whereas the tight variants provide the desired single-key signing-query bound.

Third, we replace and redesign the OWF and PRG instantiations. For the 128-bit instantiations, SM4 provides the OWF, while SM4 is used as the PRG in the loose variants and Ballet is used as the PRG in the tight variants. For the 256-bit instantiations, uBlock is used as the OWF, while Ballet is used as a software-friendly PRG. For the 512-bit instantiations, we use Vistrutah, which provides a large-block primitive suitable for the target security level.

Fourth, we redesign the zero-knowledge constraint systems for the underlying block ciphers. FAEST is tailored to AES and its algebraic representation, whereas Chinith must handle the different nonlinear layers of SM4, uBlock, and Vistrutah. For SM4, we exploit the patched inversion representation of the SM4 S-box and give two proof strategies for optimized transmission costs: a non-EM variant based on a norm-style technique, and an EM variant that directly commits to S-box outputs and obtains degree-2 constraints. For uBlock, we use the low-degree Boolean structure of its S-boxes and a meet-in-the-middle style representation. For Vistrutah, we adapt AES-like patched-inversion techniques to a 512-bit-block setting.

Fifth, our hash functions are instantiated with SM3 or SM3-based extendable-output functions rather than SHAKE. Their output lengths are chosen according to their roles in the protocol: commitment hashes remain tied to the seed/security parameter λ , while Fiat-Shamir challenges and VOLE/ZK hash seeds are tied to the proof field size λ_F . This distinction is especially important for SM4th, where $\lambda = 128$ but $\lambda_F = 160$.

Finally, we adopt the PRG-based leaf commitments introduced for FAEST-EM and use them in both our EM and non-EM variants. This reduces the cost of leaf commitments and improves

performance, but it also requires an assumption that the output of the PRG is sufficiently binding as a commitment to its input seed. In the ideal-cipher heuristic, this assumption is expected to hold with the claimed concrete security because the commitment output has length 2λ , but it should still be stated separately from the standard PRG or PRP assumptions.

Overall, Chinith keeps the modular VOLE-in-the-head architecture of FAEST, but replaces and retunes the concrete primitives, field sizes, hash lengths, commitment mechanisms, and ZK constraint systems to match the selected block ciphers and the desired concrete security levels.

3.5 Statements of Additional Functionalities

This section specifies the additional functionalities required by NGGC of ICCS.

First, our signature schemes support messages with a length of up to 2^{63} bits as long as the hash functions used as random oracles support inputs with a length no smaller than 2^{64} bits. Many hash functions, including SM3 [2], satisfy this requirement. This is because the message, after being concatenated with the public key (with length less than 512 bytes in our case), would act as the input of a hash function. However, we note that the submitted C implementations expose a byte-array API and process these hash inputs as in-memory buffers before invoking the SM3/SM3-XOF based hash routines. Therefore, the practical maximum message length of the current implementations is bounded by the machine word size, address space, and available memory. A streaming implementation of the random-oracle wrappers would realize the full 2^{63} -bit message bound without changing the mathematical scheme, public interface, key format, or signature format.

Second, our signature schemes support to sign for 2^{64} times using one pair of keys. This is because: 1) there seems no way to perform a multi-target attack across multiple OWF instances; 2) the IVs are randomly sampled for each signature, which have sizes of no less than 128 bits.

4 Algorithm Description: Cryptographic Primitives

This section introduces the symmetric cryptographic primitives used in our signature schemes.

4.1 The SM4 Block Cipher

The SM4 algorithm [3] is a Chinese standard block cipher with a 128-bit block size and a 128-bit key size, running in $R = 32$ rounds for both the key expansion and encryption/decryption routines. In SM4th, we ignore the decryption routine and use the encryption routine (along with the key expansion) as the OWF and PRG. SM4 uses the following building blocks.

The SM4 S-box. S-box is a non-linear function mapping 8 bits to 8 bits. It is originally given as a substitution table in Table 4. Its algebraic representation is analyzed in [43] with details below.

Given a $\mathbf{x}$ of 8 bits, the S-box function $S(\cdot) : \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$ is defined as

$$S(\mathbf{x}) = \mathbf{A} \cdot I(\mathbf{A} \cdot \mathbf{x} + \mathbf{c}) + \mathbf{c}. \quad (4)$$

Here, $I(\mathbf{x})$ is the patched inversion over $\mathbb{F}_{2^8}$. It first transforms $\mathbf{x}$ into a field element $x \in \mathbb{F}_{2^8}$, and then outputs the inverse of x if $x \neq 0$, otherwise outputs 0. The construction of $\mathbf{A} \in \mathbb{F}_2^{8 \times 8}$ and $\mathbf{c} \in \mathbb{F}_2^8$ are as follows:

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}, \quad \mathbf{c} = (1, 1, 0, 0, 1, 0, 1, 1). \quad (5)$$

The irreducible polynomial is

$$m(t) = t^8 + t^7 + t^6 + t^5 + t^4 + t^2 + 1 \in \mathbb{F}_2[t]. \quad (6)$$

In SM4th, we also need the inverse matrix of $\mathbf{A}$, which is

$$\mathbf{A}^{-1} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \end{pmatrix}. \quad (7)$$

Table 4: The SM4 S-box Substitution Table

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
A	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
B	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
C	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
D	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
E	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
F	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

The $T(\cdot)$ function. $T(\cdot) : \mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$ is a composite substitution function composed of S-boxes and a linear transformation $L(\cdot) : \mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$. Two linear transformations with slight differences, denoted as $L'(\cdot)$ and $L(\cdot)$, are used in the key expansion and encryption routines, respectively. The function $T(\cdot)$ includes two phases as below.

- S-box phases. Assume the input is $\mathbf{a} = (\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3) \in (\mathbb{F}_2^8)^4$, and the output is $\mathbf{b} = (\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3) \in (\mathbb{F}_2^8)^4$. Then we have

$$(\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3) = (S(\mathbf{a}_0), S(\mathbf{a}_1), S(\mathbf{a}_2), S(\mathbf{a}_3)).$$

Define a function $\text{SubWord}(\cdot) : \mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$ such that $\text{SubWord}(\mathbf{a}) = \mathbf{b}$.

- Linear transformations $L(\cdot)$ and $L'(\cdot)$. Assume the input is $\mathbf{b} = (\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3) \in (\mathbb{F}_2^8)^4$, and the output is $\mathbf{c} = (\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3) \in (\mathbb{F}_2^8)^4$. In the encryption routine, we have

$$\mathbf{c} = L(\mathbf{b}) = \mathbf{b} \oplus (\mathbf{b} \lll 2) \oplus (\mathbf{b} \lll 10) \oplus (\mathbf{b} \lll 18) \oplus (\mathbf{b} \lll 24). \quad (8)$$

In the key expansion routine, we have

$$\mathbf{c} = L'(\mathbf{b}) = \mathbf{b} \oplus (\mathbf{b} \lll 13) \oplus (\mathbf{b} \lll 23). \quad (9)$$

In SM4th, we also need the inversions of $L(\cdot)$ and $L'(\cdot)$:

$$\begin{aligned} \mathbf{b} = L^{-1}(\mathbf{c}) = & \mathbf{c} \oplus (\mathbf{c} \lll 2) \oplus (\mathbf{c} \lll 4) \oplus (\mathbf{c} \lll 8) \oplus (\mathbf{c} \lll 12) \oplus (\mathbf{c} \lll 14) \oplus (\mathbf{c} \lll 16) \\ & \oplus (\mathbf{c} \lll 18) \oplus (\mathbf{c} \lll 22) \oplus (\mathbf{c} \lll 24) \oplus (\mathbf{c} \lll 30) \end{aligned} \quad (10)$$

and

$$\begin{aligned} \mathbf{b} = L'^{-1}(\mathbf{c}) = & \mathbf{c} \oplus (\mathbf{c} \lll 2) \oplus (\mathbf{c} \lll 4) \oplus (\mathbf{c} \lll 8) \\ & \oplus (\mathbf{c} \lll 11) \oplus (\mathbf{c} \lll 12) \oplus (\mathbf{c} \lll 14) \oplus (\mathbf{c} \lll 17) \\ & \oplus (\mathbf{c} \lll 22) \oplus (\mathbf{c} \lll 23) \oplus (\mathbf{c} \lll 24) \oplus (\mathbf{c} \lll 30) \oplus (\mathbf{c} \lll 31). \end{aligned} \quad (11)$$

4.1.1 The Key Expansion Routine of SM4

The key expansion algorithm [SM4.KeyExpansion](#) takes the SM4 initial key $\mathbf{k} \in \{0,1\}^{128}$ as inputs, and outputs an expanded key $\bar{\mathbf{k}} \in (\{0,1\}^{32})^{R+4}$, which is used in the encryption routine. [SM4.KeyExpansion](#) also uses the system parameters FK and fixed parameters CK as in Table 5.

Table 5: SM4 system parameters $\{FK_i\}_{i \in [0,4)}$ and fixed parameters $\{CK_i\}_{i \in [0,32)}$

i	0	1	2	3
FK_i	0xA3B1BAC6	0x56AA3350	0x677D9197	0xB27022DC

i	0	1	2	3
CK_i	0x00070E15	0x1C232A31	0x383F464D	0x545B6269
CK_{i+4}	0x70777E85	0x8C939AA1	0xA8AFB6BD	0xC4CBD2D9
CK_{i+8}	0xE0E7EEF5	0xFC030A11	0x181F262D	0x343B4249
CK_{i+12}	0x50575E65	0x6C737A81	0x888F969D	0xA4ABB2B9
CK_{i+16}	0xC0C7CED5	0xDCE3EAF1	0xF8FF060D	0x141B2229
CK_{i+20}	0x30373E45	0x4C535A61	0x686F767D	0x848B9299
CK_{i+24}	0xA0A7AEB5	0xBCC3CAD1	0xD8DFE6ED	0xF4FB0209
CK_{i+28}	0x10171E25	0x2C333A41	0x484F565D	0x646B7279

[SM4.KeyExpansion](#)($\mathbf{k}, \{FK_i\}_{i \in [0,4)}, \{CK_i\}_{i \in [0,32)}, \text{param}$)

INPUT: $\mathbf{k} = (\mathbf{k}_0, \mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3) \in (\{0,1\}^{32})^4$

OUTPUT: $\bar{\mathbf{k}} \in (\{0,1\}^{32})^{R+4}$

```

1: new  $\bar{\mathbf{k}} \in (\{0,1\}^{32})^{R+4}$ 
2: new tmp  $\in \{0,1\}^{32}$  // temporary values
3: for  $i \in [0..4)$  do
4:    $\bar{\mathbf{k}}[i] \leftarrow \mathbf{k}[i] \oplus FK_i$ .
5: end for
6: for  $i \in [0..R)$  do
7:   tmp  $\leftarrow \bar{\mathbf{k}}[i+1] \oplus \bar{\mathbf{k}}[i+2] \oplus \bar{\mathbf{k}}[i+3] \oplus CK_i$ 
8:   tmp  $\leftarrow \text{SubWord}(\text{tmp})$ 
9:   tmp  $\leftarrow L'(\text{tmp})$  //  $L'(\cdot)$  is defined in Equation (9)
10:   $\bar{\mathbf{k}}[i+4] \leftarrow \bar{\mathbf{k}}[i] \oplus \text{tmp}$ 
11: end for
12: return  $\bar{\mathbf{k}}$ 
```

[SM4.KeyExpansion](#) takes the 128-bit initial key $\mathbf{k} = (\mathbf{k}_0, \mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3)$ as input and derives a sequence of round keys. First, the initial words are mixed with the system parameters FK_i . Then,

for each round i , a temporary value is computed from the previous words and the constant CK_i , followed by the nonlinear transformation [SubWord](#) and the linear transformation $L'(\cdot)$. Finally, the next expanded key word is generated by XORing this result with $\bar{k}[i]$.

4.1.2 The Encryption Routine of SM4

The encryption algorithm [SM4.Encryption](#) takes the plaintext $\mathbf{in} \in \{0, 1\}^{128}$ as inputs, and outputs the ciphertext $\mathbf{out} \in \{0, 1\}^{128}$. Specifically, it takes a 128-bit plaintext $\mathbf{in}$ and the expanded key $\bar{k}$ as input, and outputs a 128-bit ciphertext $\mathbf{out}$. It first initializes the internal state $\mathbf{z}$ with the 4 32-bit words of $\mathbf{in}$. Then, in each round i , a temporary value $\mathbf{tmp}$ is computed from $\mathbf{z}[i+1]$, $\mathbf{z}[i+2]$, $\mathbf{z}[i+3]$, and the round key $\bar{k}[i+4]$, followed by the nonlinear transformation [SubWord](#) and the linear transformation $L(\cdot)$. Finally, the next state word is obtained as $\mathbf{z}[i+4] = \mathbf{z}[i] \oplus \mathbf{tmp}$, and the ciphertext is formed by reversing the order of the last four state words.

[SM4.Encryption](#)($\mathbf{in}, \bar{k}, \text{param}$)

INPUT: $\mathbf{in} \in (\{0, 1\}^{32})^4, \bar{k} \in (\{0, 1\}^{32})^{4+R}$
 OUTPUT: $\mathbf{out} \in (\{0, 1\}^{32})^4$

```

1: new  $\mathbf{out} \in (\{0, 1\}^{32})^4$ 
2: new  $\mathbf{z} \in (\{0, 1\}^{32})^{R+4}$ 
3: new  $\mathbf{tmp} \in \{0, 1\}^{32}$ 
4: for  $i \in [0..4)$  do
5:    $\mathbf{z}[i] \leftarrow \mathbf{in}[i]$ 
6: end for
7: for  $i \in [0..R)$  do
8:    $\mathbf{tmp} \leftarrow \mathbf{z}[i+1] \oplus \mathbf{z}[i+2] \oplus \mathbf{z}[i+3] \oplus \bar{k}[i+4]$ 
9:    $\mathbf{tmp} \leftarrow \text{SubWord}(\mathbf{tmp})$ 
10:   $\mathbf{tmp} \leftarrow \mathbf{tmp} \oplus L(\mathbf{tmp})$  //  $L(\cdot)$  is defined in Equation (8)
11:   $\mathbf{z}[i+4] \leftarrow \mathbf{z}[i] \oplus \mathbf{tmp}$ 
12: end for
13: return ( $\mathbf{z}[35], \mathbf{z}[34], \mathbf{z}[33], \mathbf{z}[32]$ )
```

4.2 The uBlock Block Cipher

The uBlock algorithm [4] is a family of block ciphers with block sizes and key sizes of 128 or 256 bits. It is designed by Wu, Zhang, Zheng, and Li, all Chinese researchers. This document only involves the uBlock variant with a 256-bit block size and a 256-bit key size, and we use the term uBlock to represent such a variant for ease of notation. uBlock runs in $R = 24$ rounds for both the key expansion and encryption/decryption routines. In uBlockith, we use the key expansion and encryption routines of uBlock to construct the OWF, while we use the software-friendly Ballet [5] to construct the PRGs. Recall that uBlock involves the following building blocks.

The uBlock S-box. S-box is a function mapping 4 bits to 4 bits. Its substitution table version and inverse are presented in Table 6. In uBlockith, we also need their algebraic representations.

Table 6: The uBlock substitution table and its inverse

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	7	4	9	C	B	A	D	8	F	E	1	6	0	3	2	5
$S^{-1}(x)$	C	A	E	D	1	F	B	0	7	2	5	4	3	6	9	8

Given the input $(x_0, x_1, x_2, x_3) \in \mathbb{F}_2^4$, the output (y_0, y_1, y_2, y_3) of S-boxes is computed as

$$\begin{aligned} y_0 &= 1 \oplus x_0 \oplus x_2 x_3, \\ y_1 &= 1 \oplus x_0 \oplus x_1 \oplus x_0 x_1 \oplus x_0 x_2 \oplus x_0 x_1 x_2 \oplus x_0 x_3 \oplus x_2 x_3, \\ y_2 &= 1 \oplus x_1 \oplus x_0 x_1 \oplus x_2 \oplus x_1 x_2 x_3, \\ y_3 &= x_1 \oplus x_2 \oplus x_3 \oplus x_1 x_2. \end{aligned} \tag{12}$$

For the inverse S-box, given the input (y_0, y_1, y_2, y_3) , the output (z_0, z_1, z_2, z_3) is computed as

$$\begin{aligned} z_0 &= y_0 y_1 \oplus y_2 \oplus y_3 \oplus y_0 y_3 \oplus y_2 y_3 \oplus y_0 y_1 y_3, \\ z_1 &= y_0 \oplus y_1 \oplus y_3 \oplus y_0 y_3, \\ z_2 &= 1 \oplus y_0 \oplus y_2 \oplus y_0 y_1, \\ z_3 &= 1 \oplus y_2 \oplus y_3 \oplus y_0 y_2 \oplus y_1 y_2 \oplus y_2 y_3 \oplus y_0 y_2 y_3. \end{aligned} \tag{13}$$

Clearly, both the S-box and inverse S-box can be represented as degree-3 binary circuits.

Like SM4, we define two functions [SubDWord\(\$\cdot\$ \)](#) and [SubQWord\(\$\cdot\$ \)](#). For the former, assume the input is $\mathbf{a} = (a_0, a_1, \dots, a_{15}) \in (\mathbb{F}_2^4)^{16}$. Then we have

$$\text{SubDWord}(\mathbf{a}) = (S(a_0), S(a_1), \dots, S(a_{15})). \tag{14}$$

For the latter, assume the input is $\mathbf{b} = (b_0, b_1, \dots, b_{31}) \in (\mathbb{F}_2^4)^{32}$. Then we have

$$\text{SubQWord}(\mathbf{b}) = (S(b_0), S(b_1), \dots, S(b_{31})). \tag{15}$$

Vector permutations $PL(\cdot)$ and $PR(\cdot)$. Both $PL(\cdot)$ and $PR(\cdot)$ are byte-wise permutations on 16 bytes, as listed in Table 7. For example, $PL(\cdot)$ is defined as

$$PL : (\{0, 1\}^8)^{16} \rightarrow (\{0, 1\}^8)^{16},$$

which maps $\mathbf{y} = (y_0, y_1, \dots, y_{15})$ to $\mathbf{z} = (z_0, z_1, \dots, z_{15})$, where

$$\begin{aligned} z_0 &= y_2, & z_1 &= y_7, & z_2 &= y_8, & z_3 &= y_{13}, & z_4 &= y_3, & z_5 &= y_6, & z_6 &= y_9, & z_7 &= y_{12}, \\ z_8 &= y_1, & z_9 &= y_4, & z_{10} &= y_{15}, & z_{11} &= y_{10}, & z_{12} &= y_{14}, & z_{13} &= y_{11}, & z_{14} &= y_5, & z_{15} &= y_0. \end{aligned}$$

Table 7: The uBlock Permutation Table

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$PL(i)$	2	7	8	13	3	6	9	12	1	4	15	10	14	11	5	0
$PR(i)$	6	11	1	12	9	4	2	15	7	0	13	10	14	3	8	5

Vector permutation $PK(\cdot)$. $PK(\cdot)$ is a nibble-wise permutation on 32 nibbles, as listed in Table 8. It is defined as

$$PK : (\{0, 1\}^4)^{32} \rightarrow (\{0, 1\}^4)^{32}, \tag{16}$$

which maps $\mathbf{y} = (y_0, y_1, \dots, y_{31})$ to $\mathbf{z} = (z_0, z_1, \dots, z_{31})$, where

$$\begin{aligned} z_0 &= y_{10}, & z_1 &= y_5, & z_2 &= y_{15}, & z_3 &= y_0, & z_4 &= y_2, & z_5 &= y_7, & z_6 &= y_8, & z_7 &= y_{13}, \\ z_8 &= y_{14}, & z_9 &= y_6, & z_{10} &= y_4, & z_{11} &= y_{12}, & z_{12} &= y_1, & z_{13} &= y_3, & z_{14} &= y_{11}, & z_{15} &= y_9, \\ z_{16} &= y_{24}, & z_{17} &= y_{25}, & z_{18} &= y_{26}, & z_{19} &= y_{27}, & z_{20} &= y_{28}, & z_{21} &= y_{29}, & z_{22} &= y_{30}, & z_{23} &= y_{31}, \\ z_{24} &= y_{16}, & z_{25} &= y_{17}, & z_{26} &= y_{18}, & z_{27} &= y_{19}, & z_{28} &= y_{20}, & z_{29} &= y_{21}, & z_{30} &= y_{22}, & z_{31} &= y_{23}. \end{aligned}$$

Table 8: The uBlock key schedule permutation $PK(\cdot)$

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$PK(i)$	10	5	15	0	2	7	8	13	14	6	4	12	1	3	11	9
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$PK(i)$	24	25	26	27	28	29	30	31	16	17	18	19	20	21	22	23

The $T(\cdot)$ function. uBlock uses $T(\cdot) : \mathbb{F}_2^{64} \rightarrow \mathbb{F}_2^{64}$ as a linear transformation that operates on each nibble of its input independently. It multiplies every nibble by 2 in the finite field $\mathbb{F}_{2^4}$, where the field is defined by the irreducible polynomial

$$m(t) = t^4 + t + 1 \in \mathbb{F}_2[t].$$

For an input $\mathbf{x} = (x_0, x_1, \dots, x_{15})$ with $x_i \in \{0, 1\}^4$, we first transform x_i into a field element $x_i \in \mathbb{F}_{2^4}$, and then define

$$T(\mathbf{x}) = (2 \cdot x_0, 2 \cdot x_1, \dots, 2 \cdot x_{15}). \quad (17)$$

The field multiplication for every 4-bit inputs $(y_0, \dots, y_3)$ (corresponding to the field element $\sum_{i \in [0,4)} y_i \cdot t^i$ for the primitive element t) and 4-bit outputs $(z_0, \dots, z_3)$ can be alternatively represented as

$$z_0 = y_1, \quad z_1 = y_2, \quad z_2 = y_3 \oplus y_0, \quad z_3 = y_0. \quad (18)$$

4.2.1 The Key Expansion Routine of uBlock

The key expansion algorithm `uBlock.KeyExpansion` takes the uBlock initial key $\mathbf{k} \in \{0, 1\}^{256}$ as inputs, and outputs an expanded key $\bar{\mathbf{k}} \in (\{0, 1\}^{256})^{R+1}$, which is used in the encryption routine. `uBlock.KeyExpansion` uses the round constants RC as described in Table 9.

Table 9: uBlock round constants $\{RC_i\}_{i \in [24]}$

i	1	2	3	4
RC_i	0x988CC9DD	0xF0E4A1B5	0x21357064	0x8397D2C6
RC_{i+4}	0xC7D39682	0x4F5B1E0A	0x5E4A0F1B	0x7C682D39
RC_{i+8}	0x392D687C	0xB3A7E2F6	0xA7B3F6E2	0x8E9ADFCB
RC_{i+12}	0xDCC88D99	0x786C293D	0x30246175	0xA1B5F0E4
RC_{i+16}	0x8296D3C7	0xC5D19480	0x4A5E1B0F	0x55410410
RC_{i+20}	0x6B7F3A2E	0x17034652	0xEFFBBEAA	0x1F0B4E5A

Specifically, the key $\mathbf{k}$ is first split into four 64-bit double words $(\mathbf{k}_0, \mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3)$, and the initial round key is set as $\bar{\mathbf{k}}[0] \leftarrow \mathbf{k}$. At each round $i \in [1 .. R]$, the state is updated as follows:

$$\begin{aligned}
(\mathbf{k}_0, \mathbf{k}_1) &\leftarrow PK_t(\mathbf{k}_0, \mathbf{k}_1), \\
\mathbf{k}_2 &\leftarrow \mathbf{k}_2 \oplus \text{SubDWord}(\mathbf{k}_0 \oplus RC_i), \\
\mathbf{k}_3 &\leftarrow \mathbf{k}_3 \oplus T(\mathbf{k}_1), \\
\mathbf{k} &\leftarrow (\mathbf{k}_2, \mathbf{k}_3, \mathbf{k}_1, \mathbf{k}_0),
\end{aligned}$$

after which the round key is extracted as $\bar{\mathbf{k}}[i] \leftarrow \mathbf{k}$.

uBlock.KeyExpansion($\mathbf{k}, \{RC_i\}_{i \in [R]}, PK(\cdot), \text{param}$)

INPUT: $\mathbf{k} = (\mathbf{k}_0^{(0)}, \mathbf{k}_1^{(0)}, \mathbf{k}_2^{(0)}, \mathbf{k}_3^{(0)}) \in (\{0, 1\}^{64})^4$, round constants $\{RC_i\}_{i \in [R]}$, permutation PK

OUTPUT: $\bar{\mathbf{k}} \in (\{0, 1\}^{256})^{R+1}$

```

1: new  $\bar{\mathbf{k}} \in (\{0, 1\}^{256})^{R+1}$ 
2:  $\bar{\mathbf{k}}[0] \leftarrow (\mathbf{k}_0^{(0)}, \mathbf{k}_1^{(0)}, \mathbf{k}_2^{(0)}, \mathbf{k}_3^{(0)})$ 
3: for  $i \in [1..R]$  do
4:   new  $(\mathbf{k}_0^{(i)}, \mathbf{k}_1^{(i)}, \mathbf{k}_2^{(i)}, \mathbf{k}_3^{(i)})$ 
5:    $(\mathbf{k}_0^{(i)}, \mathbf{k}_1^{(i)}) \leftarrow PK(\mathbf{k}_0^{(i-1)}, \mathbf{k}_1^{(i-1)})$  //  $PK(\cdot)$  is defined in Equation (16)
6:    $\mathbf{k}_2^{(i)} \leftarrow \mathbf{k}_2^{(i-1)} \oplus \text{SubDWord}(\mathbf{k}_0^{(i)} \oplus RC_i)$  //  $\text{SubDWord}(\cdot)$  is defined in Equation (14)
7:    $\mathbf{k}_3^{(i)} \leftarrow \mathbf{k}_3^{(i-1)} \oplus T(\mathbf{k}_1^{(i)})$  //  $T(\cdot)$  is defined in Equation (17)
8:    $\bar{\mathbf{k}}[i] \leftarrow (\mathbf{k}_2^{(i)}, \mathbf{k}_3^{(i)}, \mathbf{k}_1^{(i)}, \mathbf{k}_0^{(i)})$ 
9:    $(\mathbf{k}_0^{(i)}, \mathbf{k}_1^{(i)}, \mathbf{k}_2^{(i)}, \mathbf{k}_3^{(i)}) \leftarrow (\mathbf{k}_2^{(i)}, \mathbf{k}_3^{(i)}, \mathbf{k}_1^{(i)}, \mathbf{k}_0^{(i)})$ 
10: end for
11: return  $\bar{\mathbf{k}}$ 

```

4.2.2 The Encryption Routine of uBlock

The encryption algorithm **uBlock.Encryption** takes a 256-bit plaintext $\mathbf{in}$ and the expanded key $\bar{\mathbf{k}} \in (\{0, 1\}^{256})^{R+1}$ as input, and outputs a 256-bit ciphertext $\mathbf{out}$. It first splits $\mathbf{in}$ into two 128-bit branches $(\mathbf{x}_0, \mathbf{x}_1)$. Then, in each round $i \in [0..R]$, the round key $\bar{\mathbf{k}}[i]$ is divided into two 128-bit subkeys $(\mathbf{k}_0, \mathbf{k}_1)$, which are XORed with $\mathbf{x}_0$ and $\mathbf{x}_1$, respectively, before applying the nonlinear function **SubQWord**. After that, the two state branches are updated through a sequence of XOR operations and word-wise cyclic left shifts by 4, 8, 8, and 20 bits, followed by the vector permutations $PL(\cdot)$ and $PR(\cdot)$ applied to $\mathbf{x}_0$ and $\mathbf{x}_1$, respectively. Finally, after R rounds, the ciphertext is obtained as $\mathbf{out} = \bar{\mathbf{k}}[R] \oplus (\mathbf{x}_0 \parallel \mathbf{x}_1)$.

uBlock.Encryption($\mathbf{in}, \bar{\mathbf{k}}, \text{param}$)

INPUT: $\mathbf{in} \in \{0, 1\}^{256}$, $\bar{\mathbf{k}} \in (\{0, 1\}^{256})^{R+1}$

OUTPUT: $\mathbf{out} \in \{0, 1\}^{256}$

```

1: new  $\mathbf{x}_0, \mathbf{x}_1 \in \{0, 1\}^{128}$ 
2: new  $\mathbf{k}_0, \mathbf{k}_1 \in \{0, 1\}^{128}$ 
3:  $(\mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathbf{in}$ 
4: for  $i \in [0..R]$  do
5:    $(\mathbf{k}_0, \mathbf{k}_1) \leftarrow \bar{\mathbf{k}}[i]$ 
6:    $\mathbf{x}_0 \leftarrow \text{SubQWord}(\mathbf{x}_0 \oplus \mathbf{k}_0)$  // defined in Equation (15)
7:    $\mathbf{x}_1 \leftarrow \text{SubQWord}(\mathbf{x}_1 \oplus \mathbf{k}_1)$ 
8:    $\mathbf{x}_1 \leftarrow \mathbf{x}_1 \oplus \mathbf{x}_0$ 
9:    $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \oplus (\mathbf{x}_1 \lll 4)_{32}$  // word-wise cyclic left shifts
10:   $\mathbf{x}_1 \leftarrow \mathbf{x}_1 \oplus (\mathbf{x}_0 \lll 8)_{32}$ 
11:   $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \oplus (\mathbf{x}_1 \lll 8)_{32}$ 
12:   $\mathbf{x}_1 \leftarrow \mathbf{x}_1 \oplus (\mathbf{x}_0 \lll 20)_{32}$ 
13:   $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \oplus \mathbf{x}_1$ 
14:   $\mathbf{x}_0 \leftarrow PL(\mathbf{x}_0)$  // defined in Table 7
15:   $\mathbf{x}_1 \leftarrow PR(\mathbf{x}_1)$  // defined in Table 7
16: end for
17:  $\mathbf{out} \leftarrow \bar{\mathbf{k}}[R] \oplus (\mathbf{x}_0 \parallel \mathbf{x}_1)$ 
18: return  $\mathbf{out}$ 

```

4.3 The Ballet Block Cipher

The Ballet algorithm [5] is a family of block ciphers with block sizes and key sizes of 128 or 256 bits. It is designed by Cui, Wang, Fan, Hu, Fu and Huang, all Chinese researchers. This document

only involves the Ballet variant with a 256-bit block size and a 256-bit key size, and we use the term Ballet to represent such a variant for ease of notation. Ballet is software-friendly and we use Ballet as PRGs in uBlockith and SM4th-ti. We omit to introduce the concrete algorithms of Ballet in this document, and kindly refer to the design [5] for details.

4.4 The Vistrutah Block Cipher

The Vistrutah algorithm [6] is a family of larger block ciphers with block sizes and key sizes of 256 or 512 bits. It is designed by Avanzi, Chakraborti, Chakraborty and List. We use the variant with a block size and key size of 512 bits, and call it Vistrutah in this document for ease of notation. Vistrutah offers a long version with $R = 9$ rounds and a short version with 6 rounds, and we use the former one aimed for standard usages. On a high level, Vistrutah iterates a step function that applies two AES rounds to each 128-bit block of the state, followed by a state-wide cell permutation. We next introduce the building blocks of AES and Vistrutah.

The AES State. The state of AES is represented as a 4×4 array of bytes, denoted by $\mathbf{s}$. Each byte of the state is written as $s_{r,c}$, where $0 \leq r < 4$ and $0 \leq c < 4$ denote the row and column indices, respectively. Hence, the state is of the form

$$\mathbf{s} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix}.$$

The state is interpreted in column-major order. That is, when viewing $\mathbf{s}$ as a 16-byte string, one reads the entries column by column:

$$\mathbf{s} = s_{0,0}, s_{1,0}, s_{2,0}, s_{3,0}, s_{0,1}, s_{1,1}, s_{2,1}, s_{3,1}, s_{0,2}, s_{1,2}, s_{2,2}, s_{3,2}, s_{0,3}, s_{1,3}, s_{2,3}, s_{3,3}.$$

The AES S-box. The AES S-box is a non-linear function mapping 8 bits to 8 bits. Given an input vector $\mathbf{x} = (x_0, x_1, \dots, x_7) \in \mathbb{F}_2^8$, the S-box function $\text{SubByte}(\cdot): \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$ is defined as

$$S(\mathbf{x}) = \mathbf{A} \cdot I(\mathbf{x}) + \mathbf{c}, \quad (19)$$

where $I(\mathbf{x})$ is the patched inversion over $\text{GF}(2^8)$. It first transforms $\mathbf{x}$ into a field element $x \in \text{GF}(2^8)$, and then outputs the inverse of x if $x \neq 0$, otherwise outputs 0. The form of $\mathbf{A}$ and $\mathbf{c}$ is

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}.$$

The irreducible polynomial is

$$m(t) = t^8 + t^4 + t^3 + t + 1 \in \mathbb{F}_2[t]. \quad (20)$$

For an AES state $\mathbf{s}$, define $\text{SubBytes}(\mathbf{s})$ such that each byte is applied via SubByte independently.

Accordingly, the inverse affine component of the AES S-box is given by

$$\mathbf{u} = \mathbf{A}^{-1} \cdot \mathbf{y} + \mathbf{c}',$$

where

$$\mathbf{A}^{-1} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{c}' = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Equivalently, for each bit position $0 \leq j < 8$, one has

$$u_j = y_{(j-1) \bmod 8} \oplus y_{(j-3) \bmod 8} \oplus y_{(j-6) \bmod 8} \oplus c'_j, \quad (21)$$

where $c'_j = 1$ if $j \in \{0, 2\}$, and $c'_j = 0$ otherwise.

ShiftRows. The [ShiftRows](#) transformation cyclically shifts the bytes in each row of the AES state to the left by different offsets. For AES with a 4×4 state, the first row is not shifted, while the second, third, and fourth rows are shifted to the left by 1, 2, and 3 byte positions, respectively.

More precisely, for a state

$$\mathbf{s} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix},$$

the output of [ShiftRows](#) is

$$\text{ShiftRows}(\mathbf{s}) = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{pmatrix}.$$

MixColumns. The [MixColumns](#) transformation applies an $\mathbb{F}_2^8$ -linear transformation to each column of the AES state independently. More precisely, each column of the state is viewed as a vector of four bytes over $\text{GF}(2^8)$, and is multiplied by a fixed 4×4 matrix over $\text{GF}(2^8)$.

Let $\mathbf{s}' = \text{MixColumns}(\mathbf{s})$. Then, for each column index $0 \leq c < 4$, it holds that

$$\begin{pmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{pmatrix}.$$

Here, the bytes of the state are interpreted as elements of $\text{GF}(2^8)$. The constants $\{01\}$, $\{02\}$, and 03 denote field elements in $\text{GF}(2^8)$, where $\{01\} = 1$, $\{02\} = \alpha$, and $03 = \alpha + 1$, with α being the generator of $\text{GF}(2^8)$ such that $m(\alpha) = 0$ in Equation (20).

We also need the inverse operation [InvMixColumns](#). Let $\mathbf{s} = \text{InvMixColumns}(\mathbf{s}')$. Then, for each column index $0 \leq c < 4$, it holds that

$$\begin{pmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{pmatrix} = \begin{pmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{pmatrix} \begin{pmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{pmatrix}.$$

Here, the bytes of the state are interpreted as elements of $\text{GF}(2^8)$. The constants 09 , $0b$, $0d$, and $0e$ denote field elements in $\text{GF}(2^8)$, where

$$09 = \alpha^3 + 1, \quad 0b = \alpha^3 + \alpha + 1, \quad \{0d\} = \alpha^3 + \alpha^2 + 1, \quad 0e = \alpha^3 + \alpha^2 + \alpha.$$

AESRound and AESFinalRound. Given $\mathbf{x}, \mathbf{k} \in (\mathbb{F}_2^8)^{16}$, define **AESRound**($\mathbf{x}, \mathbf{k}$) as

$$\text{AESRound}(\mathbf{x}, \mathbf{k}) = \text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(\mathbf{x}))) \oplus \mathbf{k}.$$

Similarly, define the final AES round **AESFinalRound** omitting the **MixColumns** layer, *i.e.*,

$$\text{AESFinalRound}(\mathbf{x}, \mathbf{k}) = \text{ShiftRows}(\text{SubBytes}(\mathbf{x})) \oplus \mathbf{k}.$$

We next introduce the building blocks of Vistrutah.

RotByte. **Vistrutah.RotByte** _{s} ($\cdot$) denotes the cyclic rotation of a 128-bit branch by s bytes.

Vistrutah.RotByte _{s} ($\mathbf{x}$)

INPUT: A 128-bit string $\mathbf{x} = (\mathbf{x}[0], \dots, \mathbf{x}[15])$ represented as 16 bytes, and a shift value $s \in \{0, \dots, 15\}$

OUTPUT: A 128-bit string $\mathbf{y} = (\mathbf{y}[0], \dots, \mathbf{y}[15])$

```

1: for  $i = 0$  to 15 do
2:    $\mathbf{y}[i] \leftarrow \mathbf{x}[(i + s) \bmod 16]$ 
3: end for
4: return  $\mathbf{y}$ 

```

Mix. **Vistrutah.Mix** denotes the byte-wise inter-branch permutation over the 512-bit state.

Vistrutah.Mix($\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3$)

INPUT: Four 128-bit branches $\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3$, each viewed as 16 bytes

OUTPUT: Four 128-bit branches ($\mathbf{s}'_0, \mathbf{s}'_1, \mathbf{s}'_2, \mathbf{s}'_3$)

```

1: Parse  $\mathbf{s}_b = (\mathbf{s}_b[0], \dots, \mathbf{s}_b[15]) \in (\mathbb{F}_2^8)^{16}$  for each  $b \in [0, 3]$ 
2: Let  $\pi_{\text{mix}} = (0, 1, 2, 3, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13, 14, 15)$ .
3: new  $\mathbf{t}[0..63] \in (\mathbb{F}_2^8)^{64}$ 
4: for  $j = 0$  to 15 do
5:    $u \leftarrow \pi_{\text{mix}}[j]$ 
6:    $\mathbf{t}[4j + 0] \leftarrow \mathbf{s}_0[u]$ 
7:    $\mathbf{t}[4j + 1] \leftarrow \mathbf{s}_1[u]$ 
8:    $\mathbf{t}[4j + 2] \leftarrow \mathbf{s}_2[u]$ 
9:    $\mathbf{t}[4j + 3] \leftarrow \mathbf{s}_3[u]$ 
10: end for
11: Parse  $\mathbf{s}'_0 \parallel \mathbf{s}'_1 \parallel \mathbf{s}'_2 \parallel \mathbf{s}'_3 \leftarrow \mathbf{t}[0] \parallel \mathbf{t}[1] \parallel \dots \parallel \mathbf{t}[63]$ .
12: return ( $\mathbf{s}'_0, \mathbf{s}'_1, \mathbf{s}'_2, \mathbf{s}'_3$ )

```

Shuffle. The **Vistrutah.Shuffle** transformation is a byte permutation on a 256-bit input. It takes a 256-bit string represented as 32 bytes and rearranges these bytes according to a fixed permutation.

4.4.1 The Encryption Routine of Vistrutah

The key schedule routine of Vistrutah does not involve any S-boxes, so we describe the key schedule and encryption routines in one function named as **Vistrutah.Encryption**.

Vistrutah.Encryption operates on a 512-bit input $\mathbf{in}$, which is parsed into 4 128-bit branches $\mathbf{in}_0, \mathbf{in}_1, \mathbf{in}_2, \mathbf{in}_3$. The 512-bit initial key $\mathbf{k}$ is parsed into 4 128-bit branches $\mathbf{k}_0, \mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3$.

First, the fixed branch keys are defined by $FK_0 = \mathbf{k}_0$ and $FK_1 = \mathbf{k}_1$, while $FK_2 \parallel FK_3 = \text{Vistrutah.Shuffle}(\mathbf{k}_2 \parallel \mathbf{k}_3)$. The initial round-key state is then set as $RK_0 = FK_1$, $RK_1 = FK_0$, $RK_2 = FK_3$, and $RK_3 = FK_2$. Encryption starts with a whitening step, where each input branch is xored with the corresponding round-key branch. The resulting state branches are then independently processed by one application of **AESRound** using the fixed keys FK_0, FK_1, FK_2, FK_3 .

Vistrutah.Shuffle(x)

INPUT: A 256-bit string $x = (x[0], \dots, x[31])$, each is a byteOUTPUT: A 256-bit string $y = (y[0], \dots, y[31])$, each is a byte

1: Let

$$\pi_{\text{Shuffle}} = (30, 29, 8, 23, 10, 9, 20, 3, 22, 21, 0, 31, 2, 1, 28, 11, 14, 13, 24, 7, 26, 25, 4, 19, 6, 5, 16, 15, 18, 17, 12, 27).$$

2: **for** $i = 0$ to 31 **do**3: $y[i] \leftarrow x[\pi_{\text{kexp}}[i]]$ 4: **end for**5: **return** y

In each intermediate step, the four state branches first undergo one zero-key AES-like round, then the inter-branch mixing layer [Vistrutah.Mix](#), and then a round-key update. More precisely, RK_0 and RK_2 are rotated by 5 bytes, whereas RK_1 and RK_3 are rotated by 10 bytes. The updated round keys are xored into the four state branches, and in addition the round constant RC_{i-1} is injected into the first branch only. The round constants $RC_0, RC_1, \dots, RC_7 \in \{0, 1\}^{128}$ are defined as

$$\begin{aligned} RC_0 &= 243F6A8885A308D313198A2E03707344, \\ RC_1 &= A4093822299F31D0082EFA98EC4E6C89, \\ RC_2 &= 452821E638D01377BE5466CF34E90C6C, \\ RC_3 &= C0AC29B7C97C50DD3F84D5B5B5470917, \\ RC_4 &= 9216D5D98979FB1BD1310BA698DFB5AC, \\ RC_5 &= 2FFD72DBD01ADFB7B8E1AFED6A267E96, \\ RC_6 &= BA7C9045F12C7F9924A19947B3916CF7, \\ RC_7 &= 0801F2E2858EFC16636920D871574E69. \end{aligned}$$

After that, the four branches are again processed independently by [AESRound](#) with the fixed keys FK_0, FK_1, FK_2, FK_3 .

After the last intermediate step, the round keys are rotated once more in the same manner, and the cipher terminates with a final AES-like layer [AESFinalRound](#) applied independently to the four 128-bit branches using RK_0, RK_1, RK_2, RK_3 , respectively. The ciphertext is finally obtained as $\text{out} = s_0 \parallel s_1 \parallel s_2 \parallel s_3$.

4.5 Pseudo-Random Generators

Our signature schemes instantiate the corresponding CTR-based block ciphers as pseudo-random generators with a 128-bit initialization vector iv and a 32-bit tweak twk . These parameters provide domain separation across different uses of the PRG and thereby improve concrete security against multi-target attacks. The value iv is sampled freshly for each signature, whereas twk is varied for each distinct use within the signing and verification procedures. The PRG uses four auxiliary functions: [BitDec](#), [NumRec](#), [AddToUpperWord](#), and [AddToLowerWord](#).

- [BitDec](#) decomposes an integer i into a d -bit string in LSB-first order.
- [NumRec](#) is the inverse reconstruction procedure, which maps a bit string of length d uniquely to an integer in the interval $[0, 2^d)$. Clearly, $\text{NumRec}(d, \text{BitDec}(i, d)) = i$ for all $i \in [0, 2^d)$.
- [AddToUpperWord](#) takes a 128-bit input $iv = (iv_0, iv_1, iv_2, iv_3)$ and a 32-bit word α , and outputs an updated 128-bit value iv' . It interprets iv_3 as the upper 32-bit word of iv in little-endian order, converts it to an integer via [NumRec](#), adds α modulo 2^{32} , converts the result back to a 32-bit string via [BitDec](#), and replaces the last word accordingly. The other three words remain unchanged, leaving the lower 96 bits unchanged.

Vistrutah.Encryption(*in*, *k*)

INPUT: *in* $\in \{0, 1\}^{512}$, *K* $\in \{0, 1\}^{512}$ OUTPUT: *out* $\in \{0, 1\}^{512}$

```
1: new in0, in1, in2, in3  $\in \{0, 1\}^{128} \leftarrow in$ 
2: new k0, k1, k2, k3  $\in \{0, 1\}^{128}$ 
3: new FK0, FK1, FK2, FK3  $\in \{0, 1\}^{128}$ 
4: new RK0, RK1, RK2, RK3  $\in \{0, 1\}^{128}$ 
5: new s0, s1, s2, s3  $\in \{0, 1\}^{128}$ 
6: (k0, k1, k2, k3)  $\leftarrow K$ 
7: FK0  $\leftarrow k_0$ , FK1  $\leftarrow k_1$ 
8: (FK2  $\parallel$  FK3)  $\leftarrow \text{Vistrutah.Shuffle}(k_2 \parallel k_3)$ 
9: RK0  $\leftarrow FK_1$ , RK1  $\leftarrow FK_0$ , RK2  $\leftarrow FK_3$ , RK3  $\leftarrow FK_2$ 
10: s0  $\leftarrow in_0 \oplus RK_0$ 
11: s1  $\leftarrow in_1 \oplus RK_1$ 
12: s2  $\leftarrow in_2 \oplus RK_2$ 
13: s3  $\leftarrow in_3 \oplus RK_3$ 
14: s0  $\leftarrow \text{AESRound}(s_0, FK_0)$ 
15: s1  $\leftarrow \text{AESRound}(s_1, FK_1)$ 
16: s2  $\leftarrow \text{AESRound}(s_2, FK_2)$ 
17: s3  $\leftarrow \text{AESRound}(s_3, FK_3)$ 
18: for i  $\in [1..R/2]$  do
19:   s0  $\leftarrow \text{AESRound}(s_0, \mathbf{0}^{128})$ 
20:   s1  $\leftarrow \text{AESRound}(s_1, \mathbf{0}^{128})$ 
21:   s2  $\leftarrow \text{AESRound}(s_2, \mathbf{0}^{128})$ 
22:   s3  $\leftarrow \text{AESRound}(s_3, \mathbf{0}^{128})$ 
23:   (s0, s1, s2, s3)  $\leftarrow \text{Vistrutah.Mix}(s_0, s_1, s_2, s_3)$ 
24:   RK0  $\leftarrow \text{Vistrutah.RotByte}_5(RK_0)$ 
25:   RK1  $\leftarrow \text{Vistrutah.RotByte}_{10}(RK_1)$ 
26:   RK2  $\leftarrow \text{Vistrutah.RotByte}_5(RK_2)$ 
27:   RK3  $\leftarrow \text{Vistrutah.RotByte}_{10}(RK_3)$ 
28:   s0  $\leftarrow s_0 \oplus RK_0 \oplus RC_{i-1}$ 
29:   s1  $\leftarrow s_1 \oplus RK_1$ 
30:   s2  $\leftarrow s_2 \oplus RK_2$ 
31:   s3  $\leftarrow s_3 \oplus RK_3$ 
32:   s0  $\leftarrow \text{AESRound}(s_0, FK_0)$ 
33:   s1  $\leftarrow \text{AESRound}(s_1, FK_1)$ 
34:   s2  $\leftarrow \text{AESRound}(s_2, FK_2)$ 
35:   s3  $\leftarrow \text{AESRound}(s_3, FK_3)$ 
36: end for
37: RK0  $\leftarrow \text{Vistrutah.RotByte}_5(RK_0)$ 
38: RK1  $\leftarrow \text{Vistrutah.RotByte}_{10}(RK_1)$ 
39: RK2  $\leftarrow \text{Vistrutah.RotByte}_5(RK_2)$ 
40: RK3  $\leftarrow \text{Vistrutah.RotByte}_{10}(RK_3)$ 
41: s0  $\leftarrow \text{AESFinalRound}(s_0, RK_0)$ 
42: s1  $\leftarrow \text{AESFinalRound}(s_1, RK_1)$ 
43: s2  $\leftarrow \text{AESFinalRound}(s_2, RK_2)$ 
44: s3  $\leftarrow \text{AESFinalRound}(s_3, RK_3)$ 
45: out  $\leftarrow s_0 \parallel s_1 \parallel s_2 \parallel s_3$ 
46: return out
```

- **AddToLowerWord** takes a 128-bit input $iv = (iv_0, iv_1, iv_2, iv_3)$ and a 32-bit word α , and outputs an updated 128-bit value iv' . It interprets iv_0 as the lower 32-bit word of iv in little-endian order, converts it to an integer via **NumRec**, adds α modulo 2^{32} , converts the result back to a 32-bit string via **BitDec**, and replaces the first word accordingly. The other three words remain unchanged, leaving the upper 96 bits unchanged.

PRG realizes a tweakable pseudorandom generator from block ciphers in CTR mode. Given a seed sd , an initial value iv , a tweak twk , and a desired output length m , the algorithm first derives the round-key vector $\bar{k}$ using the key schedule routine. It then computes

$$iv' \leftarrow \text{AddToUpperWord}(iv, twk),$$

thereby injecting the tweak into the upper 32-bit word of the iv . Next, for successive counter values $i = 0, 1, 2, \dots$, it encrypts

$$\text{AddToLowerWord}(iv', i),$$

so that the lower 32-bit word acts as the CTR counter and is incremented modulo 2^{32} , while the upper 32-bit word remains reserved for domain separation through twk . The resulting ciphertext blocks are concatenated, and the output is truncated to its first m bits. Thus, the upper word of iv is used for tweak incorporation, whereas the lower word carries the block counter, keeping the two roles cleanly separated.

BitDec(i, d)

INPUT: $i \in \mathbb{N}, d \in \mathbb{N}$

OUTPUT: $(b_0, \dots, b_{d-1}) \in \{0, 1\}^d$

- 1: **for** $j \in [0..d-1]$ **do**
 - 2: $b_j \leftarrow i \bmod 2$
 - 3: $i \leftarrow (i - b_j)/2$
 - 4: **end for**
 - 5: **return** $(b_0, \dots, b_{d-1})$
-

NumRec($d, (b_0, \dots, b_{d-1})$)

INPUT: $d \in \mathbb{N}, (b_0, \dots, b_{d-1}) \in \{0, 1\}^d$

OUTPUT: $x \in \mathbb{N}$

- 1: **return** $\sum_{j=0}^{d-1} b_j \cdot 2^j$
-

AddToUpperWord($iv = (iv_0, \dots, iv_3), \alpha$)

INPUT: $iv_i \in \{0, 1\}^{32}, \alpha \in [0..2^{32})$

OUTPUT: $iv' \in \{0, 1\}^{128}$

- 1: // iv_3 is “upper” 32 bits in little-endian order
 - 2: $iv'_3 \leftarrow \text{NumRec}(32, iv_3) + \alpha \bmod 2^{32}$
 - 3: **return** $(iv_0, iv_1, iv_2, \text{BitDec}(iv'_3))$
-

AddToLowerWord(iv, α)

INPUT: $\text{iv}_i \in \{0, 1\}^{32}$, $\alpha \in [0..2^{32})$

OUTPUT: $\text{iv}' \in \{0, 1\}^{128}$

- 1: // iv_0 is “lower” 32 bits in little-endian order
 - 2: $\text{iv}'_0 \leftarrow \text{NumRec}(32, \text{iv}_0) + \alpha \bmod 2^{32}$
 - 3: **return** (BitDec(iv'_0), iv_1 , iv_2 , iv_3)
-

PRG(sd, iv, twk; m)

INPUT: $\text{sd} \in \{0, 1\}^\lambda$, $\text{iv} \in \{0, 1\}^{128}$, $\text{twk} \in \{0, 1\}^{32}$, $m \in \mathbb{N}$

OUTPUT: $\mathbf{s} \in \{0, 1\}^m$

- 1: $h \leftarrow \lfloor m/128 \rfloor$
 - 2: $\text{rem} \leftarrow m - 128 \cdot h$
 - 3: $\bar{\mathbf{k}} \leftarrow \text{SM4.KeyExpansion}(\text{sd})$ or $\text{uBlock.KeyExpansion}(\text{sd})$
 - 4: $\text{iv}' \leftarrow \text{AddToUpperWord}(\text{iv}, \text{twk})$
 - 5: **for** $i \in [0..h)$ **do**
 - 6: $\mathbf{s}_i \leftarrow \text{SM4.SM4.Encryption}(\bar{\mathbf{k}}, \text{AddToLowerWord}(\text{iv}', i))$
 - 7: **end for**
 - 8: **return** $\mathbf{s}_0 \parallel \dots \parallel \mathbf{s}_{h-2} \parallel \mathbf{s}_{h-1}[0..\text{rem} - 1]$
-

4.6 Linear Universal Hash Functions

The VOLE consistency check and the constraint validity check are achieved via linear universal hash functions. We adapt the two functions from FAEST [21] with ϵ -almost universal and ϵ -uniform properties as defined below.

Definition 1 (ϵ -almost universal and uniform hash [21]). *A family of linear hash functions is a family of matrices $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$. The family is ϵ -almost universal if for any non-zero $\mathbf{x} \in \mathbb{F}_q^n$,*

$$\Pr_{\mathbf{H} \leftarrow \mathcal{H}}[\mathbf{H}\mathbf{x} = \mathbf{0}] \leq \epsilon.$$

The family is ϵ -almost uniform, if for any non-zero $\mathbf{x} \in \mathbb{F}_q^n$ and for any $\mathbf{v} \in \mathbb{F}_q^r$,

$$\Pr_{\mathbf{H} \leftarrow \mathcal{H}}[\mathbf{H}\mathbf{x} = \mathbf{v}] \leq \epsilon.$$

Our hashes must also satisfy the following hiding property, and Lemma 1 gives a method to transform a uniform hash family into a hiding one.

Definition 2 (Hiding property of linear hash). *A matrix $\mathbf{H} \in \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$ -hiding if the distribution of $\mathbf{H}\mathbf{v}$ is independent from $\mathbf{v}_{[0..n]}$ when $\mathbf{v}_{[n..n+h]} \xleftarrow{\$} \mathbb{F}_q^h$. A hash family $\mathcal{H} \subseteq \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$ -hiding if every $\mathbf{H} \in \mathcal{H}$ is $\mathbb{F}_q^n$ -hiding.*

Lemma 1 (Transforming a uniform linear hash to universal and hiding [21]). *Suppose a ϵ -almost uniform hash family $\mathcal{H} \subseteq \mathbb{F}_q^{r \times (n+h)}$. Let $\mathcal{H}' \subseteq \mathbb{F}_q^{r \times (n+r)}$ be another family $\{[\mathbf{H}, \mathbf{I}_r], \mathbf{H} \in \mathcal{H}\}$, where $\mathbf{I}_r \in \mathbb{F}_q^{r \times r}$ is an identity matrix. Then, it holds that $\mathcal{H}'$ is a ϵ -almost universal and $\mathbb{F}_q^n$ -hiding linear hash family.*

We next give the concrete constructions of **VOLEHash** and **ZKHash**, used in the VOLE consistency check and the constraint validity check, respectively. Before presenting these constructions, we first introduce two auxiliary functions: **ToField** and **ToBits**. The **ToField** algorithm maps a bit string $\mathbf{x} \in \{0, 1\}^{nk}$, for an integer $n \geq 1$, into a vector of field elements $\mathbf{y} \in \mathbb{F}_{2^k}^n$ using little-endian

ToField(x, k)

INPUT: $x \in \{0, 1\}^{nk}$

OUTPUT: $y \in \mathbb{F}_{2^k}^n$

```

1: let  $\alpha_k \in \mathbb{F}_{2^k}$  // The generator of  $\mathbb{F}_{2^k}$ 
2: if  $x \in \{0, 1\}^k$  then
3:   return  $y \leftarrow \sum_{i=0}^{k-1} x[i] \cdot \alpha_k^i$ 
4: else if  $x \in \{0, 1\}^{nk}$  then
5:   new  $y \in \mathbb{F}_{2^k}^n$ 
6:   for  $i \in [0..n)$  do
7:      $y[i] \leftarrow \sum_{j=0}^{k-1} x[ik + j] \cdot \alpha_k^j$ 
8:   end for
9:   return  $y$ 
10: else
11:   return  $\perp$ 
12: end if

```

ToBits($x; k, n$)

```

1: new  $x \in \{0, 1\}^{nk}$ 
2: for  $i \in [0..n)$  do
3:   Parse  $x_0 + x_1\alpha_k + \dots + x_{k-1}\alpha_k^{k-1} = x[i]$  for  $x_j \in \mathbb{F}_2$ 
4:    $x_i \leftarrow x_0 \parallel \dots \parallel x_{k-1} \in \{0, 1\}^k$ 
5:    $x \leftarrow x \parallel x_i$ 
6: end for
7: return  $x \in \{0, 1\}^{nk}$ 

```

ordering. Conversely, the **ToBits** algorithm maps a vector of field elements $x \in \mathbb{F}_{2^k}^n$, for an integer $n \geq 1$, into a bit string $y \in \{0, 1\}^{nk}$.

VOLEHash. The **VOLEHash** algorithm verifies the consistency of VOLE correlations by operating on inputs in $\mathbb{F}_2^{\hat{\ell}}$, where $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$. Here, ℓ denotes the witness length, and $B = 16$ is a security parameter adapted from [9].

To construct the hash, we first map the seed sd into $(r_0, r_1, r_2, r_3, s, t) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^5 \times \mathbb{F}_{2^{64}}$ via **ToField**. The input $x \in \mathbb{F}_2^{\hat{\ell}}$ is split into (x_0, x_1) , where $x_0 \in \{0, 1\}^{\hat{\ell} - \lambda_{\mathbb{F}} - B}$ is padded to a length that is a multiple of $\lambda_{\mathbb{F}}$, and $x_1 \in \{0, 1\}^{\lambda_{\mathbb{F}} + B}$. We then parse x_0 via **ToField** in two ways: first as a vector $\hat{y}$ of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ elements, and second as a vector $\bar{y}$ of $\mathbb{F}_{2^{64}}$ elements. Next, we compute

$$\begin{aligned}
h_0 &= \hat{y}_0 s^{\ell'/\lambda_{\mathbb{F}}-1} + \hat{y}_1 s^{\ell'/\lambda_{\mathbb{F}}-2} + \dots + \hat{y}_{\ell'/\lambda_{\mathbb{F}}-2} s + \hat{y}_{\ell'/\lambda_{\mathbb{F}}-1} \quad \text{in } \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}, \\
h_1 &= \bar{y}_0 t^{\ell'/64-1} + \bar{y}_1 t^{\ell'/64-2} + \dots + \bar{y}_{\ell'/64-2} t + \bar{y}_{\ell'/64-1} \quad \text{in } \mathbb{F}_{2^{64}},
\end{aligned}$$

where

$$\ell' = \lambda_{\mathbb{F}} \cdot \left\lceil \frac{\hat{\ell} - \lambda_{\mathbb{F}} - B}{\lambda_{\mathbb{F}}} \right\rceil$$

is the padded length of x_0 . To combine these two hash values, we view h_1 as an element of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ by zero-padding:

$$h'_1 = \text{ToField}(\text{ToBits}(h_1) \parallel 0^{\lambda_{\mathbb{F}}-64}, \lambda_{\mathbb{F}}).$$

The hash values are then linearly combined in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ as

$$\begin{bmatrix} h_2 \\ h_3 \end{bmatrix} = \begin{bmatrix} r_0 & r_1 \\ r_2 & r_3 \end{bmatrix} \begin{bmatrix} h_0 \\ h'_1 \end{bmatrix}.$$

Finally, we extract the first $\lambda_{\mathbb{F}} + B$ bits from the concatenation of h_2 and h_3 , and XOR this with x_1 to obtain the output

$$h = (\text{ToBits}(h_2) \parallel \text{ToBits}(h_3)[0..B]) \oplus x_1.$$

We argue the security of **VOLEHash** below, which follows from [21] with the proof field size $\lambda_{\mathbb{F}}$.

VOLEHash($\text{sd}, (\mathbf{x}_0, \mathbf{x}_1) \in \{0, 1\}^{\hat{\ell}-\lambda_{\mathbb{F}}-B} \times \{0, 1\}^{\lambda_{\mathbb{F}}+B}$)

INPUT: $\mathbf{x}_0 \in \{0, 1\}^{\hat{\ell}-\lambda_{\mathbb{F}}-B}$, $\mathbf{x}_1 \in \{0, 1\}^{\lambda_{\mathbb{F}}+B}$, $\text{sd} \in \{0, 1\}^{5\lambda_{\mathbb{F}}+64}$

OUTPUT: $\mathbf{h} \in \{0, 1\}^{\lambda_{\mathbb{F}}+B}$

```

1: Parse  $\text{sd} = (\mathbf{r}_0 \parallel \mathbf{r}_1 \parallel \mathbf{r}_2 \parallel \mathbf{r}_3 \parallel \mathbf{s} \parallel \mathbf{t}) \in \{0, 1\}^{5\lambda_{\mathbb{F}}+64}$  //  $\lambda_{\mathbb{F}}$ -bit  $r_i, s$ ; 64-bit  $t$ 
2:  $r_i \leftarrow \text{ToField}(\mathbf{r}_i, \lambda_{\mathbb{F}})$ , for  $i \in [0..3]$ 
3:  $s \leftarrow \text{ToField}(\mathbf{s}, \lambda_{\mathbb{F}})$ 
4:  $t \leftarrow \text{ToField}(\mathbf{t}, 64)$ 
5:  $\ell' \leftarrow \lambda_{\mathbb{F}} \cdot \left\lceil (\hat{\ell} - \lambda_{\mathbb{F}} - B) / \lambda_{\mathbb{F}} \right\rceil$ 
6:  $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \parallel 0^{\ell' - (\hat{\ell} - \lambda_{\mathbb{F}} - B)}$  // pad to multiple of  $\lambda_{\mathbb{F}}$ 
7:  $\hat{\mathbf{y}} \leftarrow \text{ToField}(\mathbf{x}_0, \lambda_{\mathbb{F}})$ 
8:  $\bar{\mathbf{y}} \leftarrow \text{ToField}(\mathbf{x}_0, 64)$ 
9:  $h_0 \leftarrow \sum_{i=0}^{\ell'/\lambda_{\mathbb{F}}-1} s^{\ell'/\lambda_{\mathbb{F}}-1-i} \cdot \hat{\mathbf{y}}[i]$  (in  $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ )
10:  $h_1 \leftarrow \sum_{i=0}^{\ell'/64-1} t^{\ell'/64-1-i} \cdot \bar{\mathbf{y}}[i]$  (in  $\mathbb{F}_{2^{64}}$ )
11:  $h'_1 \leftarrow \text{ToField}(\text{ToBits}(h_1) \parallel 0^{\lambda_{\mathbb{F}}-64}, \lambda_{\mathbb{F}})$ 
12:  $(h_2, h_3) \leftarrow (r_0 h_0 + r_1 h'_1, r_2 h_0 + r_3 h'_1)$ 
13:  $\mathbf{h} \leftarrow (\text{ToBits}(h_2) \parallel \text{ToBits}(h_3)[0..B]) \oplus \mathbf{x}_1$ 
14: return  $\mathbf{h}$ 

```

Lemma 2 ([21]). **VOLEHash** is an ϵ_V -almost universal hash family in $\mathbb{F}_2^{(\lambda_{\mathbb{F}}+B) \times \hat{\ell}}$, for

$$\epsilon_V = 2^{-\lambda_{\mathbb{F}}-B} (1 + 2^{B-50}),$$

provided that

$$\ell' = \lambda_{\mathbb{F}} \cdot \left\lceil \frac{\hat{\ell} - \lambda_{\mathbb{F}} - B}{\lambda_{\mathbb{F}}} \right\rceil \leq 2^{13} \quad \text{and} \quad 2 \log_2(\tau) \leq \min\{B, 50\}.$$

Furthermore, **VOLEHash** is $\mathbb{F}_2^{\hat{\ell}-\lambda_{\mathbb{F}}-B} = \mathbb{F}_2^{\ell+(d-1)\lambda_{\mathbb{F}}}$ -hiding.

ZKHash. The **ZKHash** algorithm compresses inputs in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^n \times \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ into a $\lambda_{\mathbb{F}}$ -bit output. Given a seed $\text{sd} \in \{0, 1\}^{3\lambda_{\mathbb{F}}+64}$, we first parse it as $(r_0, r_1, s, t) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^3 \times \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ via **ToField**, where r_0, r_1, s are obtained by interpreting $\lambda_{\mathbb{F}}$ -bit substrings as elements of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, and t is derived from a 64-bit substring by zero-padding it to $\lambda_{\mathbb{F}}$ bits and then applying **ToField**.

The input is given as $(\mathbf{x}_0, x_1)$, where $\mathbf{x}_0 = (x_{0,0}, \dots, x_{0,n-1}) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^n$ and $x_1 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. The vector $\mathbf{x}_0$ is hashed twice using two independent algebraic accumulations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$:

$$\begin{aligned} h_0 &= x_{0,0}s^{n-1} + x_{0,1}s^{n-2} + \dots + x_{0,n-2}s + x_{0,n-1}, \\ h_1 &= x_{0,0}t^{n-1} + x_{0,1}t^{n-2} + \dots + x_{0,n-2}t + x_{0,n-1}. \end{aligned}$$

That is, h_0 and h_1 are two polynomial evaluations of the same input vector under the field points s and t , respectively. Next, these two hash values are linearly combined using the coefficients r_0, r_1 , and the extra field element x_1 is added as an offset to obtain $h = r_0 h_0 + r_1 h_1 + x_1$. Finally, the output of **ZKHash** is the $\lambda_{\mathbb{F}}$ -bit representation of h , i.e., $\mathbf{h} = \text{ToBits}(h)$.

Lemma 3 ([21]). **ZKHash** is an ϵ_{zk} -almost universal hash family in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{1 \times (n+1)}$ for

$$\epsilon_{\text{zk}} = 2^{-\lambda_{\mathbb{F}}} (1 + 2^{-38})$$

if $n+1 \leq 2^{13}$. Furthermore, **ZKHash** is $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^n$ -hiding.

5 Algorithm Description: General Building Blocks

In this section, we describe the algorithms required for all signature schemes, including VOLEith functions in Section 5.1 and operations for ZKPs in Section 5.2.

ZKHash($\text{sd}, (x_0, x_1) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^n \times \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$)

```

1: //  $\lambda_{\mathbb{F}}$ -bit  $r_i, s$ ; 64-bit  $t$ 
2: Parse  $\text{sd} = (r_0 \| r_1 \| s \| t) \in \{0, 1\}^{3\lambda_{\mathbb{F}}+64}$ 
3:  $r_i \leftarrow \text{ToField}(r_i, \lambda_{\mathbb{F}})$ , for  $i \in \{0, 1\}$ 
4:  $s \leftarrow \text{ToField}(s, \lambda_{\mathbb{F}})$ 
5:  $t \leftarrow \text{ToField}(t \| 0^{\lambda_{\mathbb{F}}-64}, \lambda_{\mathbb{F}})$ 
6:  $h_0 \leftarrow \sum_{i=0}^{n-1} s^{n-1-i} \cdot x_0[i]$ 
7:  $h_1 \leftarrow \sum_{i=0}^{n-1} t^{n-1-i} \cdot x_0[i]$ 
8:  $h \leftarrow \text{ToBits}(r_0 h_0 + r_1 \cdot h_1 + x_1)$ 
9: return  $h$ 

```

5.1 VOLE-in-the-Head Functions

5.1.1 Batch All-but-One Vector Commitments

This section describes the construction of BAVC, which is adapted from FAEST [21]. It includes three algorithms: **BAVC.Commit**, **BAVC.Open** and **BAVC.Reconstruct**.

We first give some auxiliary algorithms.

- **BAVC.PosInTree**(i, j) computes the unique position $\alpha \in [L - 1 .. 2L - 1]$ of the j -th position in the i -th GGM tree. Recall that L , as defined in Table 3, is the number of all leaves in GGM trees, and N_i is the number of leaves in the i -th GGM tree.
- **BAVC.LeafCommit**(r, iv, twk) is the leaf commitment scheme for GGM trees. It returns the original seed r as leaf seed and uses the 2λ -bit PRG output as the commitment.

BAVC.LeafCommit(r, iv, twk)

```

INPUT:  $r \in \{0, 1\}^{N_{\text{block}}}, \text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}, \text{twk} \in \{0, 1\}^{32}$ 
OUTPUT:  $\text{sd} \in \{0, 1\}^{N_{\text{block}}}, \text{com} \in \{0, 1\}^{2N_{\text{block}}}$ 
1:  $\text{com} \leftarrow \text{PRG}(r, \text{iv}, \text{twk}; 2N_{\text{block}})$ 
2:  $\text{sd} \leftarrow r$ 
3: return  $(\text{sd}, \text{com})$ 

```

BAVC.PosInTree(i, j)

```

INPUT:  $i \in [0.. \tau), j \in [0.. N_i)$ 
OUTPUT:  $\alpha \in [L - 1 .. 2L - 1]$ 
1: if  $j < 2^{k-1}$  then
2:   return  $L - 1 + \tau j + i$ 
3: else
4:   return  $L - 1 + \tau 2^{k-1} + \tau_1(j \bmod 2^{k-1}) + i$ 
5: end if

```

We next describe the BAVC algorithms.

BAVC.Commit. The **BAVC.Commit** algorithm takes as input a root seed $\text{sd}_{\text{tree}} \in \{0, 1\}^{N_{\text{block}}}$ and an initialization vector $\text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$, and outputs a commitment com , a decommitment string decom , and the collection of leaf seeds $\{\text{sd}_{i,j}\}_{i \in [0.. \tau), j \in [0.. N_i)}$. Here $L = \sum_{i=0}^{\tau-1} N_i$ denotes the total number of leaves across all GGM trees.

It first sets the root node key as $k_0 \leftarrow \text{sd}_{\text{tree}}$. Then, for every internal node $\alpha \in [0.. L - 2]$, it expands k_α using a $2N_{\text{block}}$ -bit output of **PRG** to derive the two child keys $(k_{2\alpha+1}, k_{2\alpha+2}) \in$

$\{0, 1\}^{N_{\text{block}}} \times \{0, 1\}^{N_{\text{block}}}$. In this way, the full binary tree with $2L - 1$ nodes is deterministically generated from the root seed.

Next, for each layer $i \in [0.. \tau)$ and each leaf index $j \in [0.. N_i)$, the algorithm computes the corresponding tree position $\alpha \leftarrow \text{BAVC.PosInTree}(i, j)$ and invokes $\text{BAVC.LeafCommit}(k_\alpha, \text{iv}, i + L - 1)$ to obtain a leaf seed $\text{sd}_{i,j}$ together with a leaf commitment $\text{com}_{i,j}$. For every fixed i , all leaf commitments in that layer are concatenated and hashed to obtain

$$h_i \leftarrow H_1(\text{com}_{i,0} \parallel \dots \parallel \text{com}_{i,N_i-1}).$$

Finally, the global commitment is computed by hashing all layer hashes:

$$\text{com} \leftarrow H_1(h_0 \parallel \dots \parallel h_{\tau-1}).$$

The decommitment information consists of all tree node keys together with all leaf commitments, namely

$$\text{decom} \leftarrow ((k_\alpha)_{\alpha \in [0.. 2L-1]}, (\text{com}_{i,j})_{i \in [0.. \tau), j \in [0.. N_i)}).$$

The algorithm outputs $(\text{com}, \text{decom}, (\text{sd}_{i,j})_{i,j})$.

BAVC.Commit($\text{sd}_{\text{tree}}, \text{iv}, \text{param}$)

INPUT: $\text{sd}_{\text{tree}} \in \{0, 1\}^{N_{\text{block}}}$, $\text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$
 OUTPUT: $\text{com} \in \{0, 1\}^{2N_{\text{block}}}$, $\text{decom}, \{\text{sd}_{i,j} \in \{0, 1\}^{N_{\text{block}}}\}_{i \in [0.. \tau), j \in [0.. N_i)}$

- 1: $k_0 \leftarrow \text{sd}_{\text{tree}}$
- 2: **for** $\alpha \in [0.. L - 2]$ **do**
- 3: $(k_{2\alpha+1}, k_{2\alpha+2}) \leftarrow \text{PRG}(k_\alpha, \text{iv}, \alpha; 2N_{\text{block}})$
- 4: **end for**
- 5: **for** $i \in [0.. \tau)$ **do**
- 6: **for** $j \in [0.. N_i)$ **do**
- 7: $\alpha \leftarrow \text{BAVC.PosInTree}(i, j)$
- 8: $(\text{sd}_{i,j}, \text{com}_{i,j}) \leftarrow \text{BAVC.LeafCommit}(k_\alpha, \text{iv}, i + L - 1)$
- 9: **end for**
- 10: $h_i \leftarrow H_1(\text{com}_{i,0} \parallel \dots \parallel \text{com}_{i,N_i-1})$
- 11: **end for**
- 12: $\text{com} \leftarrow H_1(h_0 \parallel \dots \parallel h_{\tau-1})$
- 13: $\text{decom} \leftarrow ((k_\alpha)_{\alpha \in [0.. 2L-1]}, (\text{com}_{i,j})_{i \in [0.. \tau), j \in [0.. N_i)})$
- 14: **return** $(\text{com}, \text{decom}, (\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1})_{i \in [0.. \tau)})$

BAVC.Open. The algorithm $\text{BAVC.Open}(\text{decom}, I, \text{param})$ takes as input the decommitment string decom and an index tuple $I = (\Delta_0, \dots, \Delta_{\tau-1})$, where each $\Delta_i \in [0.. N_i)$ specifies the opened position in the i -th GGM tree. It outputs an opening string decom_I or $\perp$ if the requested opening exceeds the allowed size bound.

It first parses decom as $\text{decom} = ((k_\alpha)_{\alpha \in [0.. 2L-1]}, (\text{com}_{i,j})_{i \in [0.. \tau), j \in [0.. N_i)})$. The opening string is initialized with the τ selected leaf commitments, namely $\text{decom}_I \leftarrow (\text{com}_{0,\Delta_0}, \dots, \text{com}_{\tau-1,\Delta_{\tau-1}})$.

Next, the algorithm determines which tree nodes must remain hidden. To this end, it initializes a bit vector $S \in \{0, 1\}^{2L-1}$ to zero and a counter $n_h \leftarrow 0$. Then, for each layer $i \in [0.. \tau)$, it computes the corresponding leaf position $\alpha \leftarrow \text{BAVC.PosInTree}(i, \Delta_i)$, marks this node in S , and increments n_h . After that, it repeatedly moves upward along the path to the root, marking every previously unmarked ancestor. In this way, S records exactly those nodes that lie on the union of the root-to-opened-leaf paths, and n_h counts the number of such marked nodes.

The algorithm then checks whether the resulting opening is admissible. If $n_h - 2\tau + 1 > T_{\text{open}}$, it returns $\perp$. Otherwise, it scans the internal nodes in reverse order, from $L - 2$ down to 0. For each internal node i , it tests whether exactly one of its two children is marked, i.e., whether $S[2i + 1] \oplus S[2i + 2] = 1$. If so, the sibling key that is not on the marked path is appended to decom_I . Concretely, it sets $\alpha \leftarrow 2i + 1 + S[2i + 1]$ and appends k_α to the opening.

Finally, the algorithm zero-pads decom_I to the fixed length $2N_{\text{block}}\tau + T_{\text{open}}N_{\text{block}}$ bits and returns the resulting string. Thus, the output contains the selected leaf commitments together with the minimal set of auxiliary tree keys needed to reconstruct the unopened parts of the GGM tree.

BAVC.Open($\text{decom}, I = (\Delta_0, \dots, \Delta_{\tau-1}), \text{param}$)

INPUT: $\text{decom} \in \{0, 1\}^{(2L-1)N_{\text{block}} + 2N_{\text{block}}L}$, $I \in [0..N_0) \times \dots \times [0..N_{\tau-1})$
OUTPUT: $\text{decom}_I \in \{0, 1\}^{2N_{\text{block}}\tau + T_{\text{open}}N_{\text{block}}} \cup \{\perp\}$

- 1: Parse $\text{decom} \leftarrow ((k_\alpha)_{\alpha \in [0..2L-1)}, (\text{com}_{i,j})_{i \in [0..\tau), j \in [0..N_i)})$ // Initialize opening with τ leaf commitments
- 2: $\text{decom}_I \leftarrow (\text{com}_{0,\Delta_0}, \dots, \text{com}_{\tau-1,\Delta_{\tau-1}})$ // Mark tree nodes that will not be revealed
- 3: $S \leftarrow 0^{2L-1} \in \{0, 1\}^{2L-1}$
- 4: $n_h \leftarrow 0$ // number of marked hidden nodes
- 5: // Mark each unopened leaf by n_h and its ancestors by $S[\cdot]$
- 6: **for** $i \in [0..\tau)$ **do**
- 7: $\alpha \leftarrow \text{BAVC.PosInTree}(i, \Delta_i)$
- 8: $S[\alpha] \leftarrow 1$
- 9: $n_h \leftarrow n_h + 1$
- 10: **while** $\alpha > 0$ and $S[(\alpha - 1)/2] = 0$ **do**
- 11: $\alpha \leftarrow \lfloor (\alpha - 1)/2 \rfloor$
- 12: $S[\alpha] \leftarrow 1$
- 13: $n_h \leftarrow n_h + 1$
- 14: **end while**
- 15: **end for**
- 16: **if** $n_h - 2\tau + 1 > T_{\text{open}}$ **then**
- 17: **return** $\perp$
- 18: **end if**
- 19: // Add tree nodes to opening information
- 20: **for** $i = L - 2$ **to** 0 **do**
- 21: // If exactly one child needs to be opened, open it
- 22: **if** $S[2i + 1] \oplus S[2i + 2] = 1$ **then**
- 23: $\alpha \leftarrow 2i + 1 + S[2i + 1]$
- 24: Append k_α to decom_I
- 25: **end if**
- 26: **end for**
- 27: Zero-append decom_I to length $2N_{\text{block}}\tau + T_{\text{open}}N_{\text{block}}$ bits
- 28: **return** decom_I

BAVC.Reconstruct. The **BAVC.Reconstruct** algorithm describes the reconstruction procedure for the BAVC commitment from an opening decom_I corresponding to the queried index set $I = (\Delta_0, \dots, \Delta_{\tau-1})$. The algorithm first parses the opening decom_I into the revealed leaf commitments $(\text{com}_{0,\Delta_0}, \dots, \text{com}_{\tau-1,\Delta_{\tau-1}})$ and the auxiliary node set **nodes**, which contains exactly those seeds k_α that are needed to reconstruct the unrevealed sibling sub-trees along the paths from the opened leaves to the root. It then initializes a binary marker array $S \in \{0, 1\}^{2L-1}$ to indicate which nodes of the tree are already determined by the opening. For each queried leaf, the corresponding tree position is marked, and the algorithm traverses the tree upwards to identify all internal nodes lying on paths to the revealed leaves. Whenever exactly one child of an internal node is marked, the missing sibling seed must be provided in **nodes**; otherwise, the reconstruction fails and outputs $\perp$.

After this consistency check, the algorithm expands all unrevealed parts of the tree using the $2N_{\text{block}}$ -bit output **PRG**. More precisely, for every internal node whose value is not fixed by the opening, its two children are derived deterministically from the parent seed. Then, for each level $i \in [0..\tau)$ and each position $j \in [0..N_i)$, the corresponding leaf commitment is obtained either directly from the opened value (when $j = \Delta_i$) or by recomputing it from the reconstructed seed via **BAVC.LeafCommit**. The resulting leaf hashes are aggregated level by level as $h_i \leftarrow H_1(h_{i,0} \parallel \dots \parallel h_{i,N_i-1})$, and finally combined into the global commitment $\text{com} \leftarrow H_1(h_0 \parallel \dots \parallel h_{\tau-1})$.

The algorithm outputs this reconstructed commitment together with all unopened leaf seeds. Hence, **BAVC.Reconstruct** deterministically recovers the full commitment view implied by the partial opening, while rejecting malformed openings that do not match a valid tree structure.

BAVC.Reconstruct($\text{decom}_I, I = (\Delta_0, \dots, \Delta_{\tau-1}), \text{iv}; \text{param}$)

INPUT: $\text{decom}_I \in \{0, 1\}^{2^{N_{\text{block}} \tau + T_{\text{open}} N_{\text{block}}}}$, $I \in [0..N_0) \times \dots \times [0..N_{\tau-1})$, $\text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$
 OUTPUT: $\text{com} \in \{0, 1\}^{2^{N_{\text{block}}}}$ and $L - \tau$ seeds in $\{0, 1\}^{N_{\text{block}}}$

```

1: Parse  $\text{decom}_I = (\text{com}_{0, \Delta_0}, \dots, \text{com}_{\tau-1, \Delta_{\tau-1}}, \text{nodes})$ 
2: // Mark tree nodes that are not revealed
3:  $S \leftarrow 0^{2^{L-1}} \in \{0, 1\}^{2^{L-1}}$ 
4: for  $i \in [0.. \tau)$  do
5:    $\alpha \leftarrow \text{BAVC.PosInTree}(i, \Delta_i)$ 
6:    $S[\alpha] \leftarrow 1$ 
7: end for
8: // Walk up the tree, copying nodes from  $\text{decom}_I$ 
9: for  $i = L - 2$  to  $0$  do
10:   $S[i] \leftarrow S[2i + 1] \vee S[2i + 2]$ 
11:  // If exactly one child has been opened, take seed from the opening
12:  if  $S[2i + 1] \oplus S[2i + 2] = 1$  then
13:    if  $\text{nodes}$  is empty then
14:      return  $\perp$ 
15:    end if
16:     $\alpha \leftarrow 2i + 1 + S[2i + 1]$ 
17:    Parse  $\text{nodes} = (k_\alpha, \text{nodes}')$ 
18:     $\text{nodes} \leftarrow \text{nodes}'$ 
19:  end if
20: if  $\text{nodes}$  is not empty, or not all zeroes then
21:   return  $\perp$ 
22: end if
23: // Expand the tree
24: for  $i \in [0.. L - 1)$  do
25:   if  $S[i] = 0$  then
26:     $(k_{2i+1}, k_{2i+2}) \leftarrow \text{PRG}(k_i, \text{iv}, i; 2^{N_{\text{block}}})$ 
27:   end if
28: end for
29: for  $i \in [0.. \tau)$  do
30:   for  $j \in [0.. N_i)$  do
31:     $\alpha \leftarrow \text{BAVC.PosInTree}(i, j)$ 
32:    if  $S[\alpha] = 1$  then
33:       $h_{i,j} \leftarrow \text{com}_{i,j}$ 
34:    else
35:       $(\text{sd}_{i,j}, h_{i,j}) \leftarrow \text{BAVC.LeafCommit}(k_\alpha, \text{iv}, i + L - 1)$ 
36:    end if
37:   end for
38:    $h_i \leftarrow H_1(h_{i,0} \parallel \dots \parallel h_{i,N_i-1})$ 
39: end for
40:  $\text{com} \leftarrow H_1(h_0 \parallel \dots \parallel h_{\tau-1})$ 
41: return  $(\text{com}, (\text{sd}_{i,j})_{j \in [0.. N_i) \setminus \{\Delta_i\}})_{i \in [0.. \tau)}$ 

```

5.1.2 Seed Expansions and Conversion to VOLE

After running the BAVC, the j -th leaf in the i -th GGM tree is associated with a seed $\text{sd}_{i,j} \in \{0, 1\}_{\text{block}}^N$ and a commitment $\text{com}_{i,j}$. All these seeds are then expanded into a VOLE correlation

via [ConvertToVOLE](#). Without loss of generality, we focus on a single tree with $N = 2^D$ leaves, where $D \in \{k, k-1\}$, and we neglect the description about the tree index.

Let $\text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}) \in \{0, 1\}^{\hat{\ell}}$ denote the $\hat{\ell}$ -bit PRG expansion of seed $\text{sd}_i \in \{0, 1\}_{\text{block}}^N$. A VOLE correlation over $\mathbb{F}_{2^D}$ (viewing $\mathbb{F}_2 \subseteq \mathbb{F}_{2^D}$ in the natural way) targets

$$\mathbf{u} = \bigoplus_{i=0}^{N-1} \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}) \in \mathbb{F}_2^{\hat{\ell}}, \quad \mathbf{v} = \sum_{i=0}^{N-1} i \cdot \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}) \in \mathbb{F}_{2^D}^{\hat{\ell}}, \quad (22)$$

where the multiplication and the sum defining $\mathbf{v}$ are in $\mathbb{F}_{2^D}$, and each $\hat{\ell}$ -bit block $\text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}) \in \mathbb{F}_2^{\hat{\ell}}$ is embedded coordinate-wise into $\mathbb{F}_{2^D}^{\hat{\ell}}$.

The algorithm [ConvertToVOLE](#) outputs $\mathbf{u} \in \{0, 1\}^{\hat{\ell}}$ together with D vectors $\mathbf{v}_0, \dots, \mathbf{v}_{D-1} \in \{0, 1\}^{\hat{\ell}}$ such that

$$\mathbf{v} = \sum_{j=0}^{D-1} 2^j \cdot \mathbf{v}_j,$$

where $2^j \in \mathbb{F}_{2^D}$. Intuitively, $\mathbf{v}_j$ is the XOR of exactly those PRG outputs whose leaf index has j -th binary bit equal to 1.

Prover and verifier views. The procedure [ConvertToVOLE](#) is executed by both prover and verifier. The prover knows all seeds, hence $\text{sd}_0 \neq \perp$. On the verifier side, given an index $\Delta \in [0 .. N)$ and all seeds sd_i for $i \neq \Delta$, the verifier aims to compute

$$\mathbf{q} = \sum_{\substack{i=0 \\ i \neq \Delta}}^{N-1} (\Delta - i) \cdot \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}) \in \mathbb{F}_{2^D}^{\hat{\ell}}. \quad (23)$$

To align the verifier input with the all-but-one setting of [ConvertToVOLE](#), the verifier first applies the permutation

$$\pi_{\Delta}(i) \leftarrow \text{BitCom}(\text{BitDec}(i, D) \oplus \text{BitDec}(\Delta, D)),$$

where BitCom is a bit composition function defined as $\text{BitCom}(b_0, \dots, b_{D-1}) = \sum_{j \in [0, D-1]} 2^j \cdot b_j$. Hence, the permutation π_{Δ} is exactly the XOR map $i \mapsto i \oplus \Delta$ on D -bit indices. This mapping sends Δ to 0, so the missing seed is moved to position 0 in the permuted order.

The following lemma formalizes the consistency between the prover and verifier executions of [ConvertToVOLE](#).

Lemma 4 (Correctness of [ConvertToVOLE](#) [21]). *Let $(\text{sd}_0, \dots, \text{sd}_{N-1})$ and $(\text{sd}'_0, \dots, \text{sd}'_{N-1})$ be seeds with $\text{sd}_i \in \{0, 1\}_{\text{block}}^N$ and $\text{sd}'_i \in \{0, 1\}_{\text{block}}^N \cup \{\perp\}$ such that, for some $\Delta \in [0 .. N)$,*

$$\text{sd}'_0 = \perp, \quad \text{sd}'_i = \text{sd}_{\pi_{\Delta}(i)} \text{ for all } i \in [1 .. N), \quad \text{where } \pi_{\Delta}(i) = \text{BitCom}(\text{BitDec}(i, D) \oplus \text{BitDec}(\Delta, D)).$$

Let $(\delta_0, \dots, \delta_{D-1}) \leftarrow \text{BitDec}(\Delta, D)$, and define

$$\begin{aligned} (\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{D-1}) &\leftarrow \text{ConvertToVOLE}(\text{sd}_0, \dots, \text{sd}_{N-1}; \hat{\ell}), \\ (\mathbf{u}', \mathbf{q}_0, \dots, \mathbf{q}_{D-1}) &\leftarrow \text{ConvertToVOLE}(\text{sd}'_0, \dots, \text{sd}'_{N-1}; \hat{\ell}). \end{aligned}$$

Then, for all $j \in [0 .. D)$,

$$\mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u}.$$

We follow the idea in [21] to give a proof for Lemma 4.

Proof. For readability, define $\hat{\ell}$ -bit blocks

$$\mathbf{x}_i \leftarrow \begin{cases} \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}) & \text{if } \text{sd}_i \neq \perp, \\ 0^{\hat{\ell}} & \text{if } \text{sd}_i = \perp, \end{cases} \quad \mathbf{x}'_i \leftarrow \begin{cases} \text{PRG}(\text{sd}'_i, \text{iv}, \text{twk}; \hat{\ell}) & \text{if } \text{sd}'_i \neq \perp, \\ 0^{\hat{\ell}} & \text{if } \text{sd}'_i = \perp. \end{cases}$$

[ConvertToVOLE](#) initializes $\mathbf{r}_{0,i} = \mathbf{x}_i$ (and analogously $\mathbf{r}'_{0,i} = \mathbf{x}'_i$).

(1) *Correctness of $\mathbf{u}$.* We first establish an invariant for the intermediate values $\mathbf{r}_{j,i}$. For every $j \in [0..D]$ and every $i \in [0..N/2^j)$,

$$\mathbf{r}_{j,i} = \bigoplus_{t=0}^{2^j-1} \mathbf{x}_{i \cdot 2^j + t}. \quad (24)$$

For $j = 0$, this holds by initialization. Assuming it holds for level j , Line 10 sets

$$\mathbf{r}_{j+1,i} = \mathbf{r}_{j,2i} \oplus \mathbf{r}_{j,2i+1} = \left(\bigoplus_{t=0}^{2^j-1} \mathbf{x}_{(2i) \cdot 2^j + t} \right) \oplus \left(\bigoplus_{t=0}^{2^j-1} \mathbf{x}_{(2i+1) \cdot 2^j + t} \right) = \bigoplus_{t=0}^{2^{j+1}-1} \mathbf{x}_{i \cdot 2^{j+1} + t},$$

which is exactly Equation (24) at level $j + 1$. Therefore, Equation (24) holds for all levels. In particular, at the end (Line 13),

$$\mathbf{u} = \mathbf{r}_{D,0} = \bigoplus_{i=0}^{N-1} \mathbf{x}_i = \bigoplus_{i=0}^{N-1} \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell}).$$

(2) *Correctness of $\mathbf{v}_j$.* Fix $j \in [0..D)$. By Equation (24), each term $\mathbf{r}_{j,2i+1}$ equals the XOR of $\mathbf{x}_k$ over the interval

$$k \in [(2i+1) \cdot 2^j .. (2i+2) \cdot 2^j - 1].$$

These intervals (over all $i \in [0..N/2^{j+1})$) form a disjoint partition of exactly the indices $k \in [0..N)$ whose j -th bit is 1. Hence, Line 9 computes

$$\mathbf{v}_j = \bigoplus_{i=0}^{N/2^{j+1}-1} \mathbf{r}_{j,2i+1} = \bigoplus_{\substack{k \in [0..N) \\ \text{BitDec}(k,D)_j=1}} \mathbf{x}_k. \quad (25)$$

(3) *Relating prover and verifier outputs.* By the lemma assumptions, $\mathbf{x}'_0 = 0^{\hat{\ell}}$ and $\mathbf{x}'_i = \mathbf{x}_{\pi_{\Delta}(i)}$ for all $i \in [1..N)$. Applying Equation (25) to the verifier run gives

$$\mathbf{q}_j = \bigoplus_{\substack{i \in [0..N) \\ \text{BitDec}(i,D)_j=1}} \mathbf{x}'_i = \bigoplus_{\substack{i \in [1..N) \\ \text{BitDec}(i,D)_j=1}} \mathbf{x}_{\pi_{\Delta}(i)}.$$

Since $\pi_{\Delta}(i) = i \oplus \Delta$ is a bijection on $[0..N)$, we rename $k = i \oplus \Delta$ and note that

$$\text{BitDec}(i,D)_j = 1 \iff \text{BitDec}(k \oplus \Delta, D)_j = 1 \iff \text{BitDec}(k,D)_j = 1 \oplus \delta_j, \quad \text{where } (\delta_0, \dots, \delta_{D-1}) = \text{BitDec}(\Delta, D).$$

Therefore,

$$\mathbf{q}_j = \bigoplus_{\substack{k \in [0..N) \\ \text{BitDec}(k,D)_j=1 \oplus \delta_j}} \mathbf{x}_k.$$

If $\delta_j = 0$, then $\mathbf{q}_j = \mathbf{v}_j$ by Equation (25). If $\delta_j = 1$, then $\mathbf{q}_j$ is the XOR of the complementary set $\text{BitDec}(k,D)_j = 0$, hence

$$\mathbf{q}_j = \left(\bigoplus_{k=0}^{N-1} \mathbf{x}_k \right) \oplus \left(\bigoplus_{\substack{k \in [0..N) \\ \text{BitDec}(k,D)_j=1}} \mathbf{x}_k \right) = \mathbf{u} \oplus \mathbf{v}_j.$$

In both cases we obtain $\mathbf{q}_j = \mathbf{v}_j \oplus \delta_j \cdot \mathbf{u}$, as claimed.

ConvertToVOLE($\text{sd}_0, \dots, \text{sd}_{N-1}, \text{iv}, \text{twk}, \hat{\ell}$)

INPUT: $\text{sd}_i \in \{0, 1\}_{\text{block}}^N \cup \{\perp\}$ for all $i \in [0, N)$, $\text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$, $\text{twk} \in \{0, 1\}^{32}$, $\hat{\ell} \in \mathbb{N}$

OUTPUT: $\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{\log N-1} \in \{0, 1\}^{\hat{\ell}}$

```

1:  $D \leftarrow \log N$ 
2:  $\mathbf{r}_{0,0} \leftarrow 0^{\hat{\ell}}$  if  $\text{sd}_0 = \perp$  else PRG( $\text{sd}_0, \text{iv}, \text{twk}; \hat{\ell}$ )
3: for  $i \in [1..N)$  do
4:    $\mathbf{r}_{0,i} \leftarrow \text{PRG}(\text{sd}_i, \text{iv}, \text{twk}; \hat{\ell})$ 
5: end for
6:  $\mathbf{v}_0 = \dots = \mathbf{v}_{D-1} \leftarrow 0^{\hat{\ell}}$ 
7: for  $j \in [0..D)$  do
8:   for  $i \in [0..N/2^{j+1})$  do
9:      $\mathbf{v}_j \leftarrow \mathbf{v}_j \oplus \mathbf{r}_{j,2i+1}$ 
10:     $\mathbf{r}_{j+1,i} \leftarrow \mathbf{r}_{j,2i} \oplus \mathbf{r}_{j,2i+1}$ 
11:   end for
12: end for
13:  $\mathbf{u} \leftarrow \mathbf{r}_{D,0}$ 
14: return  $\mathbf{u}, \mathbf{v}_0, \dots, \mathbf{v}_{D-1}$ 

```

5.1.3 Construction of VOLE Commitments

Given the algorithms related to the BAVC, this section describes the construction of VOLE commitments, including **VOLECommit** and **VOLEReconstruct**. Before this, we first introduce the **DecodeChall** algorithm for challenge decomposition, which is used in **VOLEReconstruct**.

DecodeChall. The **DecodeChall** algorithm specifies the challenge decomposition procedure. Given a challenge $\text{chall} \in \{0, 1\}^{\lambda_{\mathbb{F}}}$, the algorithm decomposes it into τ sub-challenges $(\Delta_0, \dots, \Delta_{\tau-1})$, which are later used in the reconstruction procedure **VOLEReconstruct**. The parameters τ , τ_0 , τ_1 , k , and w_{grind} determine how the challenge is partitioned. More precisely, the algorithm first checks that the last w_{grind} bits of chall are zero, and then parses the first $\lambda_{\mathbb{F}} - w_{\text{grind}}$ bits as the challenge-index string. For each index $i \in [0, \tau)$, it determines the substring boundaries (lo, hi) according to whether i belongs to the first τ_1 positions or to the remaining ones. If $i < \tau_1$, then the i -th sub-challenge is extracted from a block of length k , with

$$\text{lo} \leftarrow i \cdot k \quad \text{and} \quad \text{hi} \leftarrow (i + 1) \cdot k - 1.$$

Otherwise, letting $t \leftarrow i - \tau_1$, the algorithm extracts a block of length $k - 1$, with

$$\text{lo} \leftarrow \tau_1 k + t(k - 1) \quad \text{and} \quad \text{hi} \leftarrow \tau_1 k + (t + 1)(k - 1) - 1.$$

Hence, the first τ_1 sub-challenges are represented using k bits each, while the remaining τ_0 sub-challenges are represented using $k - 1$ bits each. For each $i \in [0, \tau)$, the algorithm sets

$$\Delta_i \leftarrow \text{chall}[\text{lo} \dots \text{hi}],$$

interpreted as an integer in $[0..N_i)$, and after all indices have been processed, it returns the full tuple $(\Delta_0, \dots, \Delta_{\tau-1})$.

DecodeChall(chall; param)

INPUT: $\text{chall} \in \{0, 1\}^{\lambda_{\mathbb{F}}}$
OUTPUT: $(\Delta_0, \dots, \Delta_{\tau-1}) \in [0..N_0] \times \dots \times [0..N_{\tau-1}] \cup \{\perp\}$

- 1: **if** $\text{chall}[\lambda_{\mathbb{F}} - w_{\text{grind}} \dots \lambda_{\mathbb{F}} - 1] \neq 0^{w_{\text{grind}}}$ **then**
- 2: **return** $\perp$
- 3: **end if**
- 4: **for** $i \in [0, \tau)$ **do**
- 5: **if** $i < \tau_1$ **then**
- 6: $\text{lo} \leftarrow i \cdot k$
- 7: $\text{hi} \leftarrow (i + 1) \cdot k - 1$
- 8: **else**
- 9: $t \leftarrow i - \tau_1$
- 10: $\text{lo} \leftarrow \tau_1 k + t(k - 1)$
- 11: $\text{hi} \leftarrow \tau_1 k + (t + 1)(k - 1) - 1$
- 12: **end if**
- 13: $\Delta_i \leftarrow \text{chall}[\text{lo} \dots \text{hi}]$
- 14: **end for**
- 15: **return** $(\Delta_0, \dots, \Delta_{\tau-1})$

VOLECommit. The **VOLECommit** algorithm describes the VOLE commitment procedure. Given a root seed sd_{tree} , an initialization vector iv , and the target output length $\hat{\ell}$, the algorithm first invokes **BAVC.Commit** to generate the BAVC commitment com , the corresponding decommitment information decom , and, for each index $i \in [0, \tau)$, a set of GGM seeds $(\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1})$. Then, for each VOLE instance i , the algorithm applies **ConvertToVOLE** to derive a vector $\mathbf{u}_i \in \{0, 1\}^{\hat{\ell}}$ together with column vectors $\mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,k_i-1}$, which are assembled into a matrix $\mathbf{V}_i \in \mathbb{F}_2^{\hat{\ell} \times k_i}$. After concatenating all $\mathbf{V}_i$ and appending zero-padding of width w_{grind} , the algorithm obtains the global matrix $\mathbf{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$. Finally, it sets $\mathbf{u} = \mathbf{u}_0$ and defines the correction vectors $\mathbf{c}_i = \mathbf{u} \oplus \mathbf{u}_i$ for all $i \in [1, \tau)$, so that all VOLE instances are linked to the same reference vector $\mathbf{u}$. The algorithm outputs the commitment com , the decommitment string decom , the correction vectors $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$, and the resulting VOLE commitment values $(\mathbf{u}, \mathbf{V})$.

VOLECommit($\text{sd}_{\text{tree}}, \text{iv}, \hat{\ell}, \text{param}$)

INPUT: $\text{sd}_{\text{tree}} \in \{0, 1\}_{\text{block}}^N, \text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$
OUTPUT: $\text{com} \in \{0, 1\}^{2N_{\text{block}}}, \text{decom} \in \{0, 1\}^{(2L-1)N_{\text{block}} + 2N_{\text{block}}L}, \mathbf{c}_i, \mathbf{u} \in \{0, 1\}^{\hat{\ell}}, \mathbf{V} \in \{0, 1\}^{\hat{\ell} \times \lambda_{\mathbb{F}}}$

- 1: $(\text{com}, \text{decom}, (\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1})_{i \in [0, \tau)}) \leftarrow \text{BAVC.Commit}(\text{sd}_{\text{tree}}, \text{iv}, \text{param})$
- 2: **for** $i \in [0.. \tau)$ **do**
- 3: $k_i \leftarrow k$ if $i < \tau_1$, and $k_i \leftarrow k - 1$ otherwise
- 4: $(\mathbf{u}_i, \mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,k_i-1}) \leftarrow \text{ConvertToVOLE}(\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1}, \text{iv}, i + 2^{31}, \hat{\ell})$ // Recall that k_i is the bit-length of the current VOLE instance, and N_i is the number of leaves of the current GGM tree
- 5: $\mathbf{V}_i \leftarrow [\mathbf{v}_{i,0}, \dots, \mathbf{v}_{i,k_i-1}] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$
- 6: **end for**
- 7: $\mathbf{V} \leftarrow [\mathbf{V}_0, \dots, \mathbf{V}_{\tau-1}, \mathbf{0}_{\hat{\ell}, w_{\text{grind}}}] \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$ // $\mathbf{0}_{\hat{\ell}, w_{\text{grind}}}$ is $(\hat{\ell} \times w_{\text{grind}})$ bits of zero-padding
- 8: $\mathbf{u} \leftarrow \mathbf{u}_0$
- 9: **for** $i \in [1.. \tau)$ **do**
- 10: $\mathbf{c}_i \leftarrow \mathbf{u} \oplus \mathbf{u}_i$
- 11: **end for**
- 12: **return** $(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V})$

VOLEReconstruct. The **VOLEReconstruct** algorithm describes the VOLE reconstruction procedure from the BAVC opening information. Given the challenge chall_3 , the partial decommitment decom_I , the correction vectors $\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}$, and the initialization vector iv , the algorithm first

decodes chall_3 into the tuple $I = (\Delta_0, \dots, \Delta_{\tau-1})$ using **DecodeChall**. If decoding fails, it returns $\perp$. Otherwise, it calls **BAVC.Reconstruct** to recover the commitment com and the corresponding seed sets for all GGM trees. For each instance $i \in [0, \tau)$, the recovered seeds are reordered according to the offset Δ_i , and the reordered seeds are then passed to **ConvertToVOLE** to derive $\mathbf{u}'_i$ and the vectors $\mathbf{q}_{i,0}, \dots, \mathbf{q}_{i,k_i-1}$. Next, the algorithm computes the bit decomposition $(\delta_{i,0}, \dots, \delta_{i,k_i-1})$ of Δ_i . If $i = 0$, it directly forms $\mathbf{Q}_i$ from the vectors $\mathbf{q}_{i,0}, \dots, \mathbf{q}_{i,k_i-1}$. Otherwise, it adds the correction terms determined by the bits $\delta_{i,j}$ and the vector $\mathbf{c}_i$, thereby obtaining the corrected matrix $\mathbf{Q}_i$. Finally, it concatenates all matrices $\mathbf{Q}_i$ and appends zero-padding to obtain $\mathbf{Q} \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$. The algorithm outputs $(\text{com}, \mathbf{Q})$.

VOLEReconstruct($\text{chall}_3, \text{decom}_I, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}, \hat{\ell}; \text{param}$)

INPUT: $\text{chall}_3 \in \{0, 1\}^{\lambda_{\mathbb{F}}}$, $\text{decom}_I \in \{0, 1\}^{2N_{\text{block}}\tau + T_{\text{open}}N_{\text{block}}}$, $\mathbf{c}_i \in \{0, 1\}^{\hat{\ell}}$, $\text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$
OUTPUT: $\text{com} \in \{0, 1\}^{2N_{\text{block}}}$, $\mathbf{Q} \in \{0, 1\}^{\hat{\ell} \times \lambda_{\mathbb{F}}}$

- 1: $I = (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{DecodeChall}(\text{chall}_3; \text{param})$
- 2: **if** $I = \perp$ **return** $\perp$
- 3: $\text{rec} \leftarrow (\text{com}, (\text{sd}_{i,0}, \dots, \text{sd}_{i,N_i-1})_{i \in [0.. \tau)}) \leftarrow \text{BAVC.Reconstruct}(\text{decom}_I, I, \text{iv}; \text{param})$
- 4: **if** $\text{rec} = \perp$ **return** $\perp$
- 5: **for** $i \in [0.. \tau)$ **do**
- 6: $k_i \leftarrow k$ **if** $i < \tau_1$, **and** $k_i \leftarrow k - 1$ **otherwise**
- 7: **for** $j \in [1.. N_i)$ **do**
- 8: $\text{sd}'_{i,j} \leftarrow \text{sd}_{i,j} \oplus \Delta_i$ // XOR j and Δ_i as bit-strings
- 9: **end for**
- 10: $(\mathbf{u}'_i, \mathbf{q}_{i,0}, \dots, \mathbf{q}_{i,k_i-1}) \leftarrow \text{ConvertToVOLE}(\perp, \text{sd}'_{i,1}, \dots, \text{sd}'_{i,N_i-1}, \text{iv}, i + 2^{31}; \hat{\ell})$
- 11: $(\delta_{i,0}, \dots, \delta_{i,k_i-1}) \leftarrow \text{BitDec}(\Delta_i, k_i)$
- 12: **if** $i = 0$ **then**
- 13: $\mathbf{Q}_i \leftarrow [\mathbf{q}_{i,0} \cdots \mathbf{q}_{i,k_i-1}] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$ // stored in column major representation
- 14: **else**
- 15: $\mathbf{Q}_i \leftarrow [\mathbf{q}_{i,0} \cdots \mathbf{q}_{i,k_i-1}] + [\delta_{i,0} \cdot \mathbf{c}_i \cdots \delta_{i,k_i-1} \cdot \mathbf{c}_i] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$
- 16: **end if**
- 17: **end for**
- 18: $\mathbf{Q} \leftarrow [\mathbf{Q}_0 \cdots \mathbf{Q}_{\tau-1} \mathbf{0}_{\hat{\ell}, w_{\text{grind}}}] \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$ // $\mathbf{0}$ is $(\hat{\ell} \times w_{\text{grind}})$ bits of zero-padding
- 19: **return** $(\text{com}, \mathbf{Q})$

5.2 Zero-Knowledge Operations over VOLE Commitments

We use the notation $\langle x \rangle_e^d$ to denote a degree- d VOLE commitment to a field element $x \in \mathbb{F}_{2^e}$, where $e = 1$ or $e = \lambda_{\mathbb{F}}$. Elements in $\mathbb{F}_2$ are naturally embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. This means that the prover knows the coefficients of a secret polynomial $\rho_x(X) = \rho_0 + \rho_1 X + \dots + \rho_d X^d$, where $\rho_d = x$ and $\rho_0, \dots, \rho_{d-1} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, and the verifier holds the evaluation $\gamma_x = \rho_x(\Delta)$ for a challenge $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. VOLE commitments support addition and multiplication operations from both the prover and verifier sides, which is shown in Fig. 1. For ease of notations, we omit to use e and d when they equal 1. We use $\langle \mathbf{x} \rangle_e^d$, where $\mathbf{x} = (x_0, \dots, x_{n-1}) \in \mathbb{F}_{2^e}^n$, to mean the vector of VOLE commitments $\langle x_0 \rangle_e^d, \dots, \langle x_{n-1} \rangle_e^d$.

ConstantToVOLE. The Prover.**ConstantToVOLE** produces a vector of degree-1 commitments for public $\mathbf{x} = (x_0, \dots, x_{n-1}) \in \mathbb{F}_2^n$, storing the 0-string in the first component and the bit-string $\mathbf{x}$ in the second component. Conversely, Verifier.**ConstantToVOLE** multiplies $\mathbf{x}$ by the global challenge $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ and obtains the commitment $\langle x_0 \rangle, \dots, \langle x_{n-1} \rangle$ from the verifier's side.

6 Algorithm Description: the SM4th Signature Scheme

This section presents the construction of SM4th, which uses two proving methods for OWFs in the non-EM and EM variants, respectively. The EM version commits all outputs of S-boxes with degree-

Prover: VOLE Com. operations	Verifier: VOLE Com. operations
<i>// Represent $\langle x \rangle_e^d$ as $(x_0, \dots, x_{d-1}, x)$</i> <i>// Holds $\rho_x(X) = \sum_{i=0}^{d-1} x_i \cdot X^i + xX^d$</i> ADD: $\langle x \rangle_{e_1}^{d_1} + \langle y \rangle_{e_2}^{d_2} \rightarrow \langle z \rangle_e^d$ (where $d = \max(d_1, d_2)$, $e = \max(e_1, e_2)$) $\rho_z(X) \leftarrow X^{d-d_1} \rho_x(X) + X^{d-d_2} \rho_y(X)$ return $(z_0, \dots, z_d)$ (coefficients of ρ_z)	<i>// Represent $\langle x \rangle_e^d$ as γ_x</i> <i>// Holds the challenge Δ</i> ADD: $\langle x \rangle^{d_1} + \langle y \rangle^{d_2} \rightarrow \langle z \rangle^d$ return $\Delta^{d-d_1} \cdot \gamma_x + \Delta^{d-d_2} \cdot \gamma_y$
ADD CONSTANT: $\langle x \rangle_e^d + c \rightarrow \langle z \rangle_e^d$ return $\rho_z(X) \leftarrow \rho_x(X) + X^d \cdot c$	ADD CONSTANT: $\langle x \rangle_e^d + c \rightarrow \langle z \rangle_e^d$ return $\gamma_z = \gamma_x + \Delta^d \cdot c$
MULTIPLY CONSTANT: $c \cdot \langle x \rangle_e^d \rightarrow \langle z \rangle_e^d$ $\rho_z(X) \leftarrow c \cdot \rho_x(X)$ return $(z_0, \dots, z_{d-1}, z)$ (coefficients of ρ_z)	MULTIPLY CONSTANT: $c \cdot \langle x \rangle^d \rightarrow \langle z \rangle^d$ return $\gamma_z = c \cdot \gamma_x$
MULTIPLY: $\langle x \rangle_{e_1}^{d_1} \cdot \langle y \rangle_{e_2}^{d_2} \rightarrow \langle z \rangle_e^d$ $\rho_z(X) \leftarrow \rho_x(X) \rho_y(X)$ return $(z_0, \dots, z_{d_1+d_2-1}, xy)$ (coeffs of ρ_z)	MULTIPLY: $\langle x \rangle^{d_1} \cdot \langle y \rangle^{d_2} \rightarrow \langle z \rangle^d$ return $\gamma_z = \gamma_x \cdot \gamma_y$

Fig. 1: Operations upon VOLE commitments for the prover and verifier

Prover: ConstantToVOLE ($\mathbf{x} \in \mathbb{F}_2^n$)	Verifier: ConstantToVOLE ($\mathbf{x} \in \mathbb{F}_2^n$)
1: return $(\langle x_0 \rangle, \dots, \langle x_{n-1} \rangle) \leftarrow ((0, x_0), \dots, (0, x_{n-1})) \in (\mathbb{F}_{2^\lambda}, \mathbb{F}_2)^n$	1: <i>// Holds $\Delta \in \mathbb{F}_2^\lambda$</i> 2: return $(\langle x_0 \rangle, \dots, \langle x_{n-1} \rangle) \leftarrow \mathbf{x} \cdot \Delta \in \mathbb{F}_2^n$

Fig. 2: ConstantToVOLE for the signer and verifier

2 polynomial constraints, while the non-EM one uses norm-based technique to avoid committing all S-box outputs but with degree-3 polynomial constraints.

6.1 Zero-Knowledge Building Blocks for SM4th

This section presents several VOLEitH-based ZKP building blocks for SM4 operations.

ByteCombine. The [ByteCombine](#) function combines a vector of committed 8 bits into one committed field element belonging to $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, which is SM4-compatible as defined in Definition 3.

[ByteCombine](#)($\langle x_0 \rangle, \dots, \langle x_7 \rangle; \lambda_{\mathbb{F}})$

INPUT: Commitment $\langle x_i \rangle$ where $x_i \in \mathbb{F}_2$, for $i \in [0..7]$; the proof-field parameter $\lambda_{\mathbb{F}}$

OUTPUT: Commitment $\langle x \rangle_{\lambda_{\mathbb{F}}}$ to $x \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, containing an embedded $\mathbb{F}_{2^8}$ element

- 1: let $\alpha_8 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ // SM4-compatible basis element of $\mathbb{F}_{2^8}$ within $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
 - 2: **return** $\langle x \rangle_{\lambda_{\mathbb{F}}} = \sum_{i=0}^7 \langle x_i \rangle \cdot \alpha_8^i$
-

Definition 3 (SM4-Compatible Embedding Element). Assume that $8 \mid \lambda_{\mathbb{F}}$. A field element $\alpha_8 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ is SM4-compatible if its minimal polynomial over $\mathbb{F}_2$ equals the SM4 irreducible polynomial as in Equation (6). That is to say, α_8 satisfies $m(\alpha_8) = 0$ in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$.

By Definition 3, we instantly have $\mathbb{F}_2[\alpha_8] = \left\{ \sum_{i=0}^7 c_i \alpha_8^i \mid c_i \in \mathbb{F}_2 \right\}$ is a subfield of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ isomorphic to $\text{GF}(2^8)$ under the map $\phi : \text{GF}(2^8) \rightarrow \mathbb{F}_2[\alpha_8]$ defined by $\phi \left(\sum_{i=0}^7 b_i x^i \right) = \sum_{i=0}^7 b_i \alpha_8^i$. This is because $\mathbb{F}_2[t]/(m(t)) \cong \text{GF}(2^8)$ and the evaluation homomorphism $t \mapsto \alpha_8$ induces an isomorphism onto $\mathbb{F}_2[\alpha_8] \subseteq \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ by the first isomorphism theorem.

SM4-compatible elements have the operation compatibility among $\mathbb{F}_2[\alpha_8]$, $\text{GF}(2^8)$ and $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ as described in Lemma 5. That is to say, for any $a, b, c \in \mathbb{F}_2[\alpha_8]$ and $A = \phi^{-1}(a), B = \phi^{-1}(b), C = \phi^{-1}(c) \in \text{GF}(2^8)$, we have

$$a \cdot_{\lambda_{\mathbb{F}}} b = c \iff A \cdot_8 B = C,$$

where $\cdot_8$ and $\cdot_{\lambda_{\mathbb{F}}}$ denote multiplications in $\text{GF}(2^8)$ modulo $m(t)$ and $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Hence, to prove $A \cdot_8 B = C$ (similar to the form of patched inversion), it suffices to prove $a \cdot_{\lambda_{\mathbb{F}}} b = c$ using the QuickSilver proof given the VOLE commitments $\langle a \rangle_{\lambda_{\mathbb{F}}}, \langle b \rangle_{\lambda_{\mathbb{F}}}$ and $\langle c \rangle_{\lambda_{\mathbb{F}}}$.

Lemma 5 (Operation Compatibility of [ByteCombine](#)). Let $a, b \in \mathbb{F}_2[\alpha_8] \subset \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ with representations $a = \sum_{i=0}^7 a_i \alpha_8^i$ and $b = \sum_{j=0}^7 b_j \alpha_8^j$. Then:

- (1) Closure. The product $a \cdot b$ computed in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ satisfies $a \cdot b \in \mathbb{F}_2[\alpha_8]$.
- (2) Consistency. $\phi^{-1}(a \cdot b) = \phi^{-1}(a) \cdot_8 \phi^{-1}(b)$.

Proof. For (1): Since $\mathbb{F}_2[\alpha_8]$ is a subfield, it is closed under $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ -multiplication.

For (2): Multiplication in $\mathbb{F}_2[\alpha_8]$ is governed by the relation $m(\alpha_8) = 0$, identical to the reduction rule in $\text{GF}(2^8) = \mathbb{F}_2[x]/(m(x))$. The isomorphism ϕ preserves ring operations, hence the induced multiplications coincide. $\square$

Table 10 specifies the SM4-compatible generator α_8 for the proof field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ used in SM4th. The value is presented in: (i) hexadecimal little-endian byte representation, and (ii) expanded polynomial form in the standard basis $\{1, z, \dots, z^{\lambda_{\mathbb{F}}-1}\}$. Since $8 \mid \lambda_{\mathbb{F}}$, there exist 8 such generators, and we pick one with the lowest Hamming weight among them.

SquareAndCombine. The [SquareAndCombine](#) function takes a vector of committed bits as inputs, computes the vector of committed squared bits under the SM4 irreducible polynomial, and runs [ByteCombine](#) to transform these bits into a field element over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Specifically, given the generator $\beta \in \text{GF}(2^8)$ such that $m(\beta) = 0$ as in Equation (6) and the input commitment $\langle x_0 \rangle, \dots, \langle x_7 \rangle$ of bits $x_0, \dots, x_7$, [SquareAndCombine](#) computes the commitments $\langle y_0 \rangle, \dots, \langle y_7 \rangle$ of bits $y_0, \dots, y_7$ such

Table 10: SM4-compatible generators α_8 for [ByteCombine](#)

$\lambda_{\mathbb{F}}$	Hexadecimal (LSB)	Polynomial Form in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
160	0b d7 82 9e 1c c5 6e 4a a4 52 ca 47 5a 3a 72 e0 70 13 a2 00	$z^{151} + z^{149} + z^{145} + z^{140} + z^{137} + z^{136} + z^{134} + z^{133} + z^{132} + z^{127} +$ $z^{126} + z^{125} + z^{118} + z^{117} + z^{116} + z^{113} + z^{109} + z^{108} + z^{107} + z^{105} +$ $z^{102} + z^{100} + z^{99} + z^{97} + z^{94} + z^{90} + z^{89} + z^{88} + z^{87} + z^{86} + z^{83} +$ $z^{81} + z^{78} + z^{76} + z^{73} + z^{71} + z^{69} + z^{66} + z^{62} + z^{59} + z^{57} + z^{54} + z^{53} +$ $z^{51} + z^{50} + z^{49} + z^{47} + z^{46} + z^{42} + z^{40} + z^{36} + z^{35} + z^{34} + z^{31} + z^{28} +$ $z^{27} + z^{26} + z^{25} + z^{23} + z^{17} + z^{15} + z^{14} + z^{12} + z^{10} + z^9 + z^8 + z^3 + z + 1$

[SquareAndCombine](#)($\langle x_0 \rangle, \dots, \langle x_7 \rangle$)

INPUT: Committed bits $\langle x_i \rangle$ for $i \in [0..7]$

OUTPUT: Committed squared field element $\langle y \rangle_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

- 1: $\langle y_0 \rangle \leftarrow \langle x_0 \rangle + \langle x_4 \rangle$
- 2: $\langle y_1 \rangle \leftarrow \langle x_5 \rangle + \langle x_7 \rangle$
- 3: $\langle y_2 \rangle \leftarrow \langle x_1 \rangle + \langle x_4 \rangle + \langle x_5 \rangle$
- 4: $\langle y_3 \rangle \leftarrow \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
- 5: $\langle y_4 \rangle \leftarrow \langle x_2 \rangle + \langle x_4 \rangle + \langle x_5 \rangle + \langle x_6 \rangle$
- 6: $\langle y_5 \rangle \leftarrow \langle x_4 \rangle + \langle x_5 \rangle + \langle x_6 \rangle$
- 7: $\langle y_6 \rangle \leftarrow \langle x_3 \rangle + \langle x_4 \rangle + \langle x_6 \rangle$
- 8: $\langle y_7 \rangle \leftarrow \langle x_4 \rangle + \langle x_6 \rangle$
- 9: **return** [ByteCombine](#)($\langle y_0 \rangle, \dots, \langle y_7 \rangle$)

that $\sum_{i=0}^7 y_i \beta^i = \sum_{i=0}^7 x_i \beta^{2i}$. [SquareAndCombine](#) then runs [ByteCombine](#) with $\langle y_0 \rangle, \dots, \langle y_7 \rangle$ as inputs. Using the SM4 irreducible polynomial as in Equation (6), $\langle y_0 \rangle, \dots, \langle y_7 \rangle$ can be computed. For example, since $\beta^{10} = \beta^5 + \beta^4 + \beta^3 + \beta^2 + \beta$, $\langle x_5 \rangle$ would contribute to $\{\langle y_i \rangle\}_{i \in [1,5]}$.

Affine and InvAffine for T or T' functions. The [T'.Affine](#) function takes a quintuple of commitments $\{\langle \bar{k}_{i,j,k} \rangle, \langle \bar{k}_{i+1,j,k} \rangle, \langle \bar{k}_{i+2,j,k} \rangle, \langle \bar{k}_{i+3,j,k} \rangle, \langle \bar{k}_{i+4,j,k} \rangle\}_{j \in [0,4], k \in [0,32]}$ and the fixed parameter $CK_i \in \{0,1\}^{32}$ as inputs. It outputs the commitments to the bits $\{x_{j,0}, \dots, x_{j,7}\}_{j \in [0,4]}$. For each j , $(x_{j,0}, \dots, x_{j,7})$ works as the inputs of the patched inversion function in S-boxes during the key expansion routine. Although $\{\langle \bar{k}_{i,j,k} \rangle, \langle \bar{k}_{i+4,j,k} \rangle\}$ are not used in [T'.Affine](#), we see it as inputs for the convenience when applying this algorithm in the upper algorithm. For ease of notations, denote

$$\langle \bar{k}_{i,0,0} \rangle, \dots, \langle \bar{k}_{i,0,31} \rangle, \langle \bar{k}_{i,1,0} \rangle, \dots, \langle \bar{k}_{i,1,31} \rangle, \dots, \langle \bar{k}_{i,3,0} \rangle, \dots, \langle \bar{k}_{i,3,31} \rangle, \langle \bar{k}_{i+1,0,0} \rangle, \dots, \langle \bar{k}_{i+4,3,31} \rangle.$$

as $\langle \bar{\mathbf{k}}_{\text{quin}} \rangle$. Use $\langle \bar{\mathbf{k}}_{\text{quin}}[m..n] \rangle$ to represent the entries from the m -th position to the n -th one. Denote

$$\langle x_{0,0} \rangle, \dots, \langle x_{0,7} \rangle, \langle x_{1,0} \rangle, \dots, \langle x_{3,7} \rangle$$

as $\langle \mathbf{x} \rangle$.

Similar to [T'.Affine](#), [T'.InvAffine](#) takes $\langle \bar{\mathbf{k}}_{\text{quin}} \rangle$ as inputs. It outputs the commitments to the bits $\{y_{j,0}, \dots, y_{j,7}\}_{j \in [0,4]}$. For each j , $(y_{j,0}, \dots, y_{j,7})$ works as the outputs of the patched inversion function in S-boxes during the key expansion routine.

The [T.Affine](#) function takes a quintuple of commitments $\{\langle \bar{z}_{i,j,k} \rangle, \langle \bar{z}_{i+1,j,k} \rangle, \langle \bar{z}_{i+2,j,k} \rangle, \langle \bar{z}_{i+3,j,k} \rangle, \langle \bar{z}_{i+4,j,k} \rangle\}_{j \in [0,4], k \in [0,32]}$ and $\langle \bar{\mathbf{k}}_{\text{quin}} \rangle$ as inputs. It outputs the commitments to bits $\{x_{j,0}, \dots, x_{j,7}\}_{j \in [0,4]}$. For each j , $(x_{j,0}, \dots, x_{j,7})$ works as the inputs of the patched inversion function in S-boxes during the encryption routine. Similar to $\langle \bar{\mathbf{k}}_{\text{quin}} \rangle$, denote

$$\langle z_{i,0,0} \rangle, \dots, \langle z_{i,0,31} \rangle, \langle z_{i,1,0} \rangle, \dots, \langle z_{i,1,31} \rangle, \dots, \langle z_{i,3,0} \rangle, \dots, \langle z_{i,3,31} \rangle, \langle z_{i+1,0,0} \rangle, \dots, \langle z_{i+4,3,31} \rangle.$$

as $\langle \mathbf{z}_{\text{quin}} \rangle$.

The [T.InvAffine](#) algorithm takes $\langle \mathbf{z}_{\text{quin}} \rangle$ as inputs. It outputs the commitments to the bits $\{y_{j,0}, \dots, y_{j,7}\}_{j \in [0,4]}$. For each j , $(y_{j,0}, \dots, y_{j,7})$ works as the outputs of the patched inversion function in S-boxes during the encryption routine.

T'.Affine($\langle \bar{k}_{\text{quin}} \rangle, CK_i$)

INPUT: For some $i \in [0, 32)$, a commitment vector to bits $(\bar{k}_{i,j,k}, \bar{k}_{i+1,j,k}, \bar{k}_{i+2,j,k}, \bar{k}_{i+3,j,k}, \bar{k}_{i+4,j,k})$ for $j \in [0, 4)$ and $k \in [0, 8)$; the i -th fixed parameter CK_i

OUTPUT: $\langle \mathbf{x} \rangle$, commitments to $x_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as inputs of $I(\cdot)$ during the key expansion routine

- 1: **new** $\langle \mathbf{x} \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$
 - 2: **new** $\langle \mathbf{t} \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$ // temporary values
 - 3: **new** $\langle \mathbf{ck} \rangle = (\langle ck_{0,0} \rangle, \dots, \langle ck_{0,7} \rangle, \dots, \langle ck_{3,0} \rangle, \dots, \langle ck_{3,7} \rangle) \leftarrow \text{ConstantToVOLE}(CK_i)$, where $ck_{j,k} \in \mathbb{F}_2$
 - 4: $\langle \mathbf{t} \rangle \leftarrow \langle \mathbf{k}_{\text{quin}}[N_{\text{word}} \dots 2N_{\text{word}} - 1] \rangle + \langle \mathbf{k}_{\text{quin}}[2N_{\text{word}} \dots 3N_{\text{word}} - 1] \rangle + \langle \mathbf{k}_{\text{quin}}[3N_{\text{word}} \dots 4N_{\text{word}} - 1] \rangle + \langle \mathbf{ck}[0 \dots N_{\text{word}} - 1] \rangle$
 - 5: **for** $j \in [0 \dots 4)$ **do**
 - 6: $\langle \mathbf{x}[8j \dots 8j + 7] \rangle \leftarrow \langle \mathbf{t}[8j \dots 8j + 7] \rangle \cdot \mathbf{A} + \text{ConstantToVOLE}(\mathbf{c})$, with $\mathbf{A}$ and $\mathbf{c}$ as defined in Equation (5), and $\cdot$ means multiplications in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ after elements in $\mathbf{A}$ are embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
 - 7: **end for**
 - 8: **return** $\{\langle x_{j,0} \rangle, \dots, \langle x_{j,7} \rangle\}_{j \in [0,4)}$
-

T'.InvAffine($\langle \bar{k}_{\text{quin}} \rangle$)

INPUT: For some $i \in [0, 32)$, a commitment vector to bits $(\bar{k}_{i,j,k}, \bar{k}_{i+1,j,k}, \bar{k}_{i+2,j,k}, \bar{k}_{i+3,j,k}, \bar{k}_{i+4,j,k})$ for $j \in [0, 4)$ and $k \in [0, 8)$

OUTPUT: $\langle \mathbf{y} \rangle$, commitments to $y_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as outputs of $I(\cdot)$ during the key expansion routine

- 1: **new** $\langle \mathbf{y} \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$
 - 2: **new** $\langle \mathbf{t} \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$ // temporary values
 - 3: $\langle \mathbf{t} \rangle \leftarrow \langle \mathbf{k}_{\text{quin}}[0 \dots N_{\text{word}} - 1] \rangle + \langle \mathbf{k}_{\text{quin}}[4N_{\text{word}} \dots 5N_{\text{word}} - 1] \rangle$
 - 4: $\langle \mathbf{t} \rangle \leftarrow L'^{-1}(\langle \mathbf{t} \rangle)$, which has the same operations over commitments as how L'^{-1} operates over vectors in $\mathbb{F}_2$ defined in Equation (11), i.e., $L'^{-1}(\langle \mathbf{t} \rangle) = \langle \mathbf{t} \rangle \oplus (\langle \mathbf{t} \rangle \lll 2) \oplus (\langle \mathbf{t} \rangle \lll 4) \oplus (\langle \mathbf{t} \rangle \lll 8) \oplus (\langle \mathbf{t} \rangle \lll 11) \oplus (\langle \mathbf{t} \rangle \lll 12) \oplus (\langle \mathbf{t} \rangle \lll 14) \oplus (\langle \mathbf{t} \rangle \lll 17) \oplus (\langle \mathbf{t} \rangle \lll 22) \oplus (\langle \mathbf{t} \rangle \lll 23) \oplus (\langle \mathbf{t} \rangle \lll 24) \oplus (\langle \mathbf{t} \rangle \lll 30) \oplus (\langle \mathbf{t} \rangle \lll 31)$
 - 5: **for** $j \in [0 \dots 4)$ **do**
 - 6: $\langle \mathbf{t}[8j \dots 8j + 7] \rangle \leftarrow \langle \mathbf{t}[8j \dots 8j + 7] \rangle + \text{ConstantToVOLE}(\mathbf{c})$
 - 7: $\langle \mathbf{y}[8j \dots 8j + 7] \rangle \leftarrow \langle \mathbf{t}[8j \dots 8j + 7] \rangle \cdot \mathbf{A}^{-1}$, where $\mathbf{A}^{-1}$ is defined in Equation (7).
 - 8: **end for**
 - 9: **return** $\langle \mathbf{y} \rangle$
-

T.Affine($\langle \mathbf{z}_{\text{quin}} \rangle, \langle \bar{k}_{\text{quin}} \rangle$)

INPUT: For some $i \in [0, 32)$, a commitment vector to bits $(z_{i,j,k}, z_{i+1,j,k}, z_{i+2,j,k}, z_{i+3,j,k}, z_{i+4,j,k})$ and $(\bar{k}_{i,j,k}, \bar{k}_{i+1,j,k}, \bar{k}_{i+2,j,k}, \bar{k}_{i+3,j,k}, \bar{k}_{i+4,j,k})$, for $j \in [0, 4)$ and $k \in [0, 8)$

OUTPUT: $\langle \mathbf{x} \rangle$, commitments to $x_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as inputs of $I(\cdot)$ during the encryption routine

- 1: **new** $\langle \mathbf{x} \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$
 - 2: **new** $\langle \mathbf{t} \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$ // temporary values
 - 3: $\langle \mathbf{t} \rangle \leftarrow \langle \mathbf{z}_{\text{quin}}[N_{\text{word}} \dots 2N_{\text{word}} - 1] \rangle + \langle \mathbf{z}_{\text{quin}}[2N_{\text{word}} \dots 3N_{\text{word}} - 1] \rangle + \langle \mathbf{z}_{\text{quin}}[3N_{\text{word}} \dots 4N_{\text{word}} - 1] \rangle + \langle \mathbf{k}_{\text{quin}}[4N_{\text{word}} \dots 5N_{\text{word}} - 1] \rangle$
 - 4: **for** $j \in [0 \dots 4)$ **do**
 - 5: $\langle \mathbf{x}[8j \dots 8j + 7] \rangle \leftarrow \langle \mathbf{t}[8j \dots 8j + 7] \rangle \cdot \mathbf{A} + \text{ConstantToVOLE}(\mathbf{c})$, with $\mathbf{A}$ and $\mathbf{c}$ as defined in Equation (5), and $\cdot$ means multiplications in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ after elements in $\mathbf{A}$ are embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
 - 6: **end for**
 - 7: **return** $\langle \mathbf{x} \rangle$
-

T.InvAffine($\langle z_{\text{quin}} \rangle$)

INPUT: For some $i \in [0, 32)$, a commitment vector to bits $(z_{i,j,k}, z_{i+1,j,k}, z_{i+2,j,k}, z_{i+3,j,k}, z_{i+4,j,k})$

OUTPUT: $\langle y \rangle$, commitments to $y_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as outputs of $I(\cdot)$ during the encryption routine

```

1: new  $\langle y \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$ 
2: new  $\langle t \rangle \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{32}$  // temporary values
3:  $\langle t \rangle \leftarrow \langle z_{\text{quin}}[0 \dots N_{\text{word}} - 1] \rangle + \langle z_{\text{quin}}[4N_{\text{word}} \dots 5N_{\text{word}} - 1] \rangle$ 
4:  $\langle t \rangle \leftarrow L^{-1}(\langle t \rangle)$ , which has the same operations over commitments as how  $L'^{-1}$  operates over vectors
   in  $\mathbb{F}_2$  defined in Equation (11), i.e.,  $L^{-1}(\langle t \rangle) = \langle t \rangle \oplus (\langle t \rangle \lll 2) \oplus (\langle t \rangle \lll 4) \oplus (\langle t \rangle \lll 8) \oplus (\langle t \rangle \lll 12) \oplus (\langle t \rangle \lll 14) \oplus (\langle t \rangle \lll 16) \oplus (\langle t \rangle \lll 18) \oplus (\langle t \rangle \lll 22) \oplus (\langle t \rangle \lll 24) \oplus (\langle t \rangle \lll 30)$ 
5: for  $j \in [0 \dots 4)$  do
6:    $\langle t[8j \dots 8j + 7] \rangle \leftarrow \langle t[8j \dots 8j + 7] \rangle + \text{ConstantToVOLE}(\mathbf{c})$ 
7:    $\langle y[8j \dots 8j + 7] \rangle \leftarrow \langle t[8j \dots 8j + 7] \rangle \cdot \mathbf{A}^{-1}$ , where  $\mathbf{A}^{-1}$  is defined in Equation (7).
8: end for
9: return  $\langle y \rangle$ 

```

6.1.1 Building Blocks for Non-EM Variants

This section gives more building blocks specified for the non-EM variant of SM4th.

InvNorm. For a finite field extension $\mathbb{F}_{q^m}/\mathbb{F}_q$, the field norm is the map

$$N_{\mathbb{F}_{q^m}/\mathbb{F}_q}(u) = u^{1+q+q^2+\dots+q^{m-1}}.$$

Its output always lies in the base field $\mathbb{F}_q$.

The function **SM4th.InvNorm** extracts a 4-bit representation of the norm of the inverse of an 8-bit SM4 field element. This operation is performed entirely inside the SM4 byte field and is independent of the proof field size $\lambda_{\mathbb{F}}$. More precisely, the input $x \in \{0, 1\}^8$ is first interpreted as an element x of $\mathbb{F}_{2^8}$ via **ToField**($x, 8$), where $\mathbb{F}_{2^8}$ is viewed as a degree-2 extension of the subfield $\mathbb{F}_{2^4}$. If $x = 0$, then **SM4th.InvNorm** returns $(0, 0, 0, 0)$. Otherwise, it computes the norm of x^{-1} , i.e., x^{-17} , from $\mathbb{F}_{2^8}$ down to $\mathbb{F}_{2^4}$. Recall that for an element $u \in \mathbb{F}_{2^8}$, the field norm from $\mathbb{F}_{2^8}$ to $\mathbb{F}_{2^4}$ is defined by

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(u) = u^{1+2^4} = u^{17}.$$

Hence, for any nonzero $x \in \mathbb{F}_{2^8}$,

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(x^{-1}) = (x^{-1})^{17} = x^{-17} = x^{238},$$

where the last equality follows from $x^{255} = 1$ in $\mathbb{F}_{2^8}^\times$. Since the norm always lies in the subfield $\mathbb{F}_{2^4}$, x^{-17} can be represented using only four bits.

To obtain these four bits, we fix a basis $b = (1, \beta, \beta^2, \beta^3)$ of $\mathbb{F}_{2^4}$, where $\beta = \alpha_{\text{SM4}}^3 + \alpha_{\text{SM4}}^2$ and α_{SM4} is the polynomial basis element of the SM4 field

$$\mathbb{F}_{2^8} = \mathbb{F}_2[\alpha_{\text{SM4}}]/(\alpha_{\text{SM4}}^8 + \alpha_{\text{SM4}}^7 + \alpha_{\text{SM4}}^6 + \alpha_{\text{SM4}}^5 + \alpha_{\text{SM4}}^4 + \alpha_{\text{SM4}}^2 + 1).$$

Concretely, this basis is

$$(1, \beta, \beta^2, \beta^3) = (0x01, 0x0c, 0x50, 0x2a).$$

For an SM4 S-box input u , let $x = A_{\text{SM4}}(u)$ be the affine input to the inversion layer. The norm witness is

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(x^{-1}) = x^{-17} = x^{238}.$$

Since this value lies in $\mathbb{F}_{2^4}$, it has a unique coordinate representation

$$x^{-17} = y_0 + y_1\beta + y_2\beta^2 + y_3\beta^3, \quad y_i \in \mathbb{F}_2.$$

Thus **SM4th.InvNorm** outputs the four-bit vector

$$\mathbf{y} = (y_0, y_1, y_2, y_3).$$

Equivalently, if $\mathbf{y}' = \text{ToBits}(x^{-17})$ in the SM4 polynomial basis, then

$$\mathbf{y} = (y'[0], y'[2], y'[4], y'[1]).$$

These four bits are Boolean witness values; when they are used in the VOLEitH proof, they are committed as bit commitments and then embedded into the proof field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ only through the surrounding VOLE/QuickSilver operations.

SM4th.InvNorm($\mathbf{x}$)

INPUT: $\mathbf{x} \in \{0, 1\}^8$
 OUTPUT: $\mathbf{y} \in \{0, 1\}^4$

- 1: $x \leftarrow \text{ToField}(\mathbf{x}, 8)$
- 2: **if** $x = 0$ **then**
- 3: **return** $(0, 0, 0, 0)$
- 4: **end if**
- 5: $\mathbf{y}' \leftarrow \text{ToBits}(x^{238})$
- 6: $\mathbf{y} \leftarrow (y'[0], y'[2], y'[4], y'[1])$
- 7: **return** $\mathbf{y}$

AffineToConjugates. The [SM4th.AffineToConjugates](#) algorithm converts a committed SM4 S-box input byte vector into the conjugate representation used by the norm constraints. Given a commitment $\langle \mathbf{u} \rangle$ to an 8-bit vector, it first applies the SM4 affine map and obtains the committed byte vector $\langle \mathbf{x}^{(0)} \rangle = \mathbf{A} \cdot \langle \mathbf{u} \rangle + \mathbf{c}$. It then repeatedly applies the SM4 byte-field squaring map at the bit-vector level, producing committed byte vectors $\langle \mathbf{x}^{(r)} \rangle$ representing $\langle (\mathbf{A}\mathbf{u} + \mathbf{c})^{2^r} \rangle$ for $r \in [0..8)$. For each r , the algorithm embeds $\langle \mathbf{x}^{(r)} \rangle$ into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ using [ByteCombine](#) and outputs $\langle \hat{x}^{2^r} \rangle$. This embedding uses the fixed SM4-compatible copy of $\mathbb{F}_{2^8}$ inside $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, so the bit-level squaring map is still the SM4 byte-field squaring map and is independent of the value of $\lambda_{\mathbb{F}}$. The resulting tuple $(\langle \hat{x} \rangle, \langle \hat{x}^2 \rangle, \dots, \langle \hat{x}^{128} \rangle)$ allows the protocol to access the Frobenius conjugates required by the norm check, in particular $\langle \hat{x} \rangle$, $\langle \hat{x}^2 \rangle$, and $\langle \hat{x}^{16} \rangle$. The full tuple is also used when reconstructing the full output bytes of S-boxes from the norm witness through subsequent affine SM4 transformations (Recall that one S-box includes 2 affine transformations and 1 inverse operation).

SM4th.AffineToConjugates($\langle \mathbf{u} \rangle$)

INPUT: Commitment to one byte $\mathbf{u} \in \{0, 1\}^8$
 OUTPUT: $(\langle x \rangle, \langle x^2 \rangle, \dots, \langle x^{128} \rangle)$, where $\text{ToBits}(x) = \mathbf{A}\mathbf{u} + \mathbf{c}$ and x is embedded in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

- 1: $\langle \mathbf{a}^{(0)} \rangle \leftarrow \mathbf{A} \cdot \langle \mathbf{u} \rangle + \mathbf{c}$ // $\mathbf{A}$ and $\mathbf{c}$ are defined as in Equation (4)
- 2: **for** $r \in [0..8)$ **do**
- 3: $\langle x^{2^r} \rangle \leftarrow \text{ByteCombine}(\langle a_0^{(r)} \rangle, \dots, \langle a_7^{(r)} \rangle)$
- 4: **if** $r < 7$ **then**
- 5: // Square in the SM4 byte field
- 6: $\langle a_0^{(r+1)} \rangle \leftarrow \langle a_0^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle$
- 7: $\langle a_1^{(r+1)} \rangle \leftarrow \langle a_5^{(r)} \rangle \oplus \langle a_7^{(r)} \rangle$
- 8: $\langle a_2^{(r+1)} \rangle \leftarrow \langle a_1^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle \oplus \langle a_5^{(r)} \rangle$
- 9: $\langle a_3^{(r+1)} \rangle \leftarrow \langle a_5^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle \oplus \langle a_7^{(r)} \rangle$
- 10: $\langle a_4^{(r+1)} \rangle \leftarrow \langle a_2^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle \oplus \langle a_5^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
- 11: $\langle a_5^{(r+1)} \rangle \leftarrow \langle a_4^{(r)} \rangle \oplus \langle a_5^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
- 12: $\langle a_6^{(r+1)} \rangle \leftarrow \langle a_3^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
- 13: $\langle a_7^{(r+1)} \rangle \leftarrow \langle a_4^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
- 14: **end if**
- 15: **end for**
- 16: **return** $(\langle x \rangle, \langle x^2 \rangle, \dots, \langle x^{128} \rangle)$

InvNormToConjugates. The [SM4th.InvNormToConjugates](#) algorithm expands a committed 4-bit norm coordinate vector into the conjugates of the corresponding subfield value. Given $\langle \mathbf{n} \rangle = (\langle n_0 \rangle, \langle n_1 \rangle, \langle n_2 \rangle, \langle n_3 \rangle)$, it interprets the 4 bits as coordinates with respect to the basis $(1, \beta, \beta^2, \beta^3)$ of the SM4 subfield $\mathbb{F}_{2^4} \subseteq \mathbb{F}_{2^8}$. Thus the represented SM4 subfield value is $h = n_0 + n_1\beta + n_2\beta^2 + n_3\beta^3$. In the proof, these basis constants are embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ using the fixed SM4-compatible embedding from Definition 3. Equivalently, if $\hat{\beta} = \alpha_8^3 + \alpha_8^2 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, then the embedded value is $\hat{h} = n_0 + n_1\hat{\beta} + n_2\hat{\beta}^2 + n_3\hat{\beta}^3$. The algorithm then computes the Frobenius conjugates $\hat{h}^{2^r}$ for $r \in [0..4)$ by squaring the embedded basis constants and keeping the committed coordinates unchanged. The output $(\langle \hat{h} \rangle, \langle \hat{h}^2 \rangle, \langle \hat{h}^4 \rangle, \langle \hat{h}^8 \rangle)$ is used in the norm check and in the reconstruction of the inverse byte from the norm witness.

SM4th.InvNormToConjugates($\langle \mathbf{n} \rangle$)

INPUT: Commitment to norm coordinate $\mathbf{n} = (n_0, n_1, n_2, n_3) \in \{0, 1\}^4$

OUTPUT: $(\langle \hat{h} \rangle, \langle \hat{h}^2 \rangle, \langle \hat{h}^4 \rangle, \langle \hat{h}^8 \rangle)$

- 1: $(\hat{b}_0, \hat{b}_1, \hat{b}_2, \hat{b}_3) \leftarrow (1, \hat{\beta}, \hat{\beta}^2, \hat{\beta}^3)$, where $\hat{\beta} = \alpha_8^3 + \alpha_8^2 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
 - 2: **for** $r \in [0..4)$ **do**
 - 3: $\langle \hat{h}^{2^r} \rangle \leftarrow \sum_{t=0}^3 \langle n_t \rangle \cdot \hat{b}_t^{2^r}$
 - 4: **end for**
 - 5: **return** $(\langle \hat{h} \rangle, \langle \hat{h}^2 \rangle, \langle \hat{h}^4 \rangle, \langle \hat{h}^8 \rangle)$
-

AffineOut. The [SM4th.AffineOut](#) algorithm applies the output affine layer of the SM4 S-box to a value already represented by its Frobenius conjugates. It receives $(\langle q \rangle, \langle q^2 \rangle, \dots, \langle q^{128} \rangle)$, where q is the patched inverse output in the SM4 byte field, embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ through the fixed SM4-compatible embedding. Using the affine map $\mathbf{s} = \mathbf{A} \cdot \mathbf{q} + \mathbf{c}$ from Equation (5), the algorithm evaluates the same affine transformation in its linearized-polynomial form over the embedded copy of $\mathbb{F}_{2^8}$. This avoids converting the committed field representation back into committed bits. More precisely, let

$$\mathcal{A}_{\text{out}}(X) = \hat{c} + \sum_{j=0}^7 \hat{a}_j X^{2^j}$$

be the unique linearized-polynomial representation of the SM4 output affine map after embedding its constants into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Then $s = \mathcal{A}_{\text{out}}(q)$, and its Frobenius conjugates are computed as

$$s^{2^r} = \hat{c}^{2^r} + \sum_{j=0}^7 \hat{a}_j^{2^r} q^{2^{j+r}},$$

where the exponent index is reduced modulo 8 because q belongs to the embedded copy of $\mathbb{F}_{2^8}$. For each $r \in [0..8)$, the algorithm computes the conjugate $\langle s^{2^r} \rangle$ directly from the conjugates of q . The output $(\langle s \rangle, \langle s^2 \rangle, \dots, \langle s^{128} \rangle)$ gives the committed conjugate representation of the SM4 S-box output byte, which can then be used in the key expansion or encryption routines.

SM4th.AffineOut($(\langle q^{2^r} \rangle)_{r=0}^7$)

INPUT: Committed conjugates $(\langle q \rangle, \langle q^2 \rangle, \dots, \langle q^{128} \rangle)$ of the patched inverse output

OUTPUT: Committed conjugates $(\langle s \rangle, \langle s^2 \rangle, \dots, \langle s^{128} \rangle)$, where $\mathbf{s} = \mathbf{A} \cdot \mathbf{q} + \mathbf{c}$

- 1: Let $\mathcal{A}_{\text{out}}(X) = \hat{c} + \sum_{j=0}^7 \hat{a}_j X^{2^j}$ be the linearized-polynomial form of the SM4 output affine map over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
 - 2: **for** $r \in [0..8)$ **do**
 - 3: $\langle s^{2^r} \rangle \leftarrow \hat{c}^{2^r} + \sum_{j=0}^7 \hat{a}_j^{2^r} \cdot \langle q^{2^{(j+r) \bmod 8}} \rangle$
 - 4: **end for**
 - 5: **return** $(\langle s \rangle, \langle s^2 \rangle, \dots, \langle s^{128} \rangle)$
-

Lprime. The [SM4th.L'](#) algorithm applies the linear diffusion layer used in the SM4 key expansion. Given a committed 32-bit word $\langle s \rangle$, it computes the committed word $\langle t \rangle = \langle s \rangle \oplus \text{RotlWord}(\langle s \rangle, 13) \oplus \text{RotlWord}(\langle s \rangle, 23)$. This corresponds to the SM4 key-schedule linear map $L'(s) = s \oplus (s \lll 13) \oplus (s \lll 23)$. The output is used to reconstruct a key-schedule word as $\langle \bar{k}[i] \rangle = \langle \bar{k}[i - 4] \rangle \oplus \text{SM4th.L}'(\langle s \rangle)$ after the S-box output has been obtained.

[SM4th.L'\(\$\langle s \rangle\$ \)](#)

INPUT: Commitment to a 32-bit word s

OUTPUT: Commitment to $L'(s)$

- 1: $\langle t \rangle \leftarrow \langle s \rangle \oplus \text{RotlWord}(\langle s \rangle, 13) \oplus \text{RotlWord}(\langle s \rangle, 23)$
 - 2: **return** $\langle t \rangle$
-

LprimeInv. The [SM4th.L'Inv](#) algorithm applies the inverse of the SM4 key expansion linear map L' . Given a committed 32-bit word $\langle v \rangle$, it computes $\langle L'^{-1}(v) \rangle$ using the fixed XOR-of-rotations expression for the inverse map. This routine is used for full non-anchor key words. In that case, the witness contains the full word $\bar{k}[i]$, so the protocol first computes $\langle v \rangle = \langle \bar{k}[i - 4] \rangle \oplus \langle \bar{k}[i] \rangle$ and then applies [SM4th.L'Inv](#) to recover the committed S-box output candidate. The recovered value is then checked against the forward S-box input using the ordinary inverse-S-box input and output constraints.

[SM4th.L'Inv\(\$\langle v \rangle\$ \)](#)

INPUT: Commitment to a 32-bit word v

OUTPUT: Commitment to $L'^{-1}(v)$

- 1: $\langle t \rangle \leftarrow \langle v \rangle \oplus \text{RotlWord}(\langle v \rangle, 2) \oplus \text{RotlWord}(\langle v \rangle, 4) \oplus \text{RotlWord}(\langle v \rangle, 8) \oplus \text{RotlWord}(\langle v \rangle, 11) \oplus \text{RotlWord}(\langle v \rangle, 12) \oplus \text{RotlWord}(\langle v \rangle, 14) \oplus \text{RotlWord}(\langle v \rangle, 17) \oplus \text{RotlWord}(\langle v \rangle, 22) \oplus \text{RotlWord}(\langle v \rangle, 23) \oplus \text{RotlWord}(\langle v \rangle, 24) \oplus \text{RotlWord}(\langle v \rangle, 30) \oplus \text{RotlWord}(\langle v \rangle, 31)$
 - 2: **return** $\langle t \rangle$
-

T'AffineByte. The [SM4th.T'AffineByte](#) algorithm computes the affine input representation used by one SM4 S-box constraint. Given a committed byte vector $\langle u \rangle$, it first applies the SM4 affine map and obtains $\langle x \rangle = \mathbf{A} \cdot \langle u \rangle + \mathbf{c}$. It then embeds this committed byte vector into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ using [ByteCombine](#) to obtain $\langle x \rangle$. The algorithm also computes $\langle x^2 \rangle$ by applying [SquareAndCombine](#) to the same committed byte vector. The pair $(\langle x \rangle, \langle x^2 \rangle)$ is used in the ordinary inverse-S-box input and output constraints for full non-anchor key words.

[SM4th.T'AffineByte\(\$\langle u \rangle\$ \)](#)

INPUT: Commitment to one S-box input byte vector u

OUTPUT: $(\langle x \rangle, \langle x^2 \rangle)$ where $x = \mathbf{A} \cdot u + \mathbf{c}$

- 1: $\langle x \rangle \leftarrow \mathbf{A} \cdot \langle u \rangle + \mathbf{c}$
 - 2: $\langle x \rangle \leftarrow \text{ByteCombine}(\langle x_0 \rangle, \dots, \langle x_7 \rangle)$
 - 3: $\langle x^2 \rangle \leftarrow \text{SquareAndCombine}(\langle x_0 \rangle, \dots, \langle x_7 \rangle)$
 - 4: **return** $(\langle x \rangle, \langle x^2 \rangle)$
-

T'InvAffineByte. The [SM4th.T'InvAffineByte](#) algorithm computes the inverse affine representation used by the ordinary inverse-S-box input and output constraints. Given a committed byte vector $\langle v \rangle$ corresponding to an SM4 S-box output, it applies the inverse of the output affine layer and obtains $\langle y \rangle = \mathbf{A}^{-1} \cdot (\langle v \rangle + \mathbf{c})$. It then embeds this committed byte vector into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ using [ByteCombine](#) to obtain $\langle y \rangle$. The algorithm also computes $\langle y^2 \rangle$ by applying [SquareAndCombine](#) to

the same committed byte vector. The resulting pair $(\langle y \rangle, \langle y^2 \rangle)$ is used together with $(\langle x \rangle, \langle x^2 \rangle)$ from [SM4th.T'AffineByte](#) to enforce the inverse relation $y = x^{-1}$.

[SM4th.T'InvAffineByte](#)($\langle v \rangle$)

INPUT: Commitment to one byte vector v , where $v = S(u)$

OUTPUT: $(\langle y \rangle, \langle y^2 \rangle)$ where $y = A^{-1} \cdot (v + c)$

- 1: $\langle y \rangle \leftarrow A^{-1} \cdot (\langle v \rangle + c)$
 - 2: $\langle y \rangle \leftarrow \text{ByteCombine}(\langle y_0 \rangle, \dots, \langle y_7 \rangle)$
 - 3: $\langle y^2 \rangle \leftarrow \text{SquareAndCombine}(\langle y_0 \rangle, \dots, \langle y_7 \rangle)$
 - 4: **return** $(\langle y \rangle, \langle y^2 \rangle)$
-

L. The [SM4th.L](#) algorithm applies the SM4 encryption linear layer to a committed 32-bit word. Given $\langle s \rangle$, it computes $\langle t \rangle = \langle s \rangle \oplus \text{RotlWord}(\langle s \rangle, 2) \oplus \text{RotlWord}(\langle s \rangle, 10) \oplus \text{RotlWord}(\langle s \rangle, 18) \oplus \text{RotlWord}(\langle s \rangle, 24)$. This is the diffusion layer used after the four bitwise SM4 S-box outputs in each encryption round. The output is used to reconstruct the next state word as $\langle z[i + 4] \rangle = \langle z[i] \rangle \oplus \text{SM4th.L}(\langle s \rangle)$.

[SM4th.L](#)($\langle s \rangle$)

INPUT: Commitment to a 32-bit word s

OUTPUT: Commitment to $L(s)$

- 1: $\langle t \rangle \leftarrow \langle s \rangle \oplus \text{RotlWord}(\langle s \rangle, 2) \oplus \text{RotlWord}(\langle s \rangle, 10) \oplus \text{RotlWord}(\langle s \rangle, 18) \oplus \text{RotlWord}(\langle s \rangle, 24)$
 - 2: **return** $\langle t \rangle$
-

LInv. The [SM4th.LInv](#) algorithm applies the inverse of the SM4 encryption linear layer. Given a committed 32-bit word $\langle t \rangle$, it computes the committed preimage $\langle s \rangle = L^{-1}(\langle t \rangle)$ using the fixed XOR-of-rotations expression for L^{-1} . This routine is used for full non-anchor encryption state words. In that case, the witness contains the full word $z[i + 4]$, so the protocol first computes $\langle t \rangle = \langle z[i] \rangle \oplus \langle z[i + 4] \rangle$ and then applies [SM4th.LInv](#) to recover the claimed S-box output vector. The recovered vector is then checked against the forward S-box input using the ordinary inverse-S-box input and output constraints.

[SM4th.LInv](#)($\langle t \rangle$)

INPUT: Commitment to a 32-bit word t

OUTPUT: Commitment to $L^{-1}(t)$

- 1: $\langle s \rangle \leftarrow \langle t \rangle \oplus \text{RotlWord}(\langle t \rangle, 2) \oplus \text{RotlWord}(\langle t \rangle, 4) \oplus \text{RotlWord}(\langle t \rangle, 8) \oplus \text{RotlWord}(\langle t \rangle, 12) \oplus \text{RotlWord}(\langle t \rangle, 14) \oplus \text{RotlWord}(\langle t \rangle, 16) \oplus \text{RotlWord}(\langle t \rangle, 18) \oplus \text{RotlWord}(\langle t \rangle, 22) \oplus \text{RotlWord}(\langle t \rangle, 24) \oplus \text{RotlWord}(\langle t \rangle, 30)$
 - 2: **return** $\langle s \rangle$
-

TAffineByte. The [SM4th.TAffineByte](#) algorithm computes the affine input representation used by one SM4 encryption S-box constraint. Given a committed byte vector $\langle u \rangle$, it applies the SM4 affine map and obtains $\langle x \rangle = A \cdot \langle u \rangle + c$. It then embeds this committed byte vector into $\mathbb{F}_{2^{\lambda_F}}$ using [ByteCombine](#) to obtain $\langle x \rangle_{\lambda_F}$. The algorithm also computes $\langle x^2 \rangle_{\lambda_F}$ by applying [SquareAndCombine](#) to the same committed byte vector. The pair $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F})$ is used in the ordinary inverse-S-box input and output constraints for full non-anchor encryption rounds.

TInvAffineByte. The [SM4th.TInvAffineByte](#) algorithm computes the inverse affine output representation used by one SM4 encryption S-box constraint. Given a committed byte vector $\langle v \rangle$ corresponding to an SM4 S-box output, it applies the inverse of the output affine layer and obtains

SM4th.TAffineByte($\langle \mathbf{u} \rangle$)

INPUT: Commitment to one encryption S-box input byte vector $\mathbf{u}$ OUTPUT: $(\langle x \rangle_{\lambda_{\mathbb{F}}}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}})$ where $\mathbf{x} = \mathbf{A} \cdot \mathbf{u} + \mathbf{c}$

- 1: $\langle \mathbf{x} \rangle \leftarrow \mathbf{A} \cdot \langle \mathbf{u} \rangle + \mathbf{c}$
 - 2: $\langle x \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{ByteCombine}(\langle x_0 \rangle, \dots, \langle x_7 \rangle)$
 - 3: $\langle x^2 \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{SquareAndCombine}(\langle x_0 \rangle, \dots, \langle x_7 \rangle)$
 - 4: **return** $(\langle x \rangle_{\lambda_{\mathbb{F}}}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}})$
-

$\langle \mathbf{y} \rangle = \mathbf{A}^{-1} \cdot (\langle \mathbf{v} \rangle + \mathbf{c})$. It then embeds this committed byte vector into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ using **ByteCombine** to obtain $\langle y \rangle_{\lambda_{\mathbb{F}}}$. The algorithm also computes $\langle y^2 \rangle_{\lambda_{\mathbb{F}}}$ by applying **SquareAndCombine** to the same committed byte vector. The resulting pair $(\langle y \rangle_{\lambda_{\mathbb{F}}}, \langle y^2 \rangle_{\lambda_{\mathbb{F}}})$ is used together with $(\langle x \rangle_{\lambda_{\mathbb{F}}}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}})$ from **SM4th.TAffineByte** to enforce the patched inverse relation between the S-box input and output.

SM4th.TInvAffineByte($\langle \mathbf{v} \rangle$)

INPUT: Commitment to one encryption S-box output byte vector $\mathbf{v}$ OUTPUT: $(\langle y \rangle_{\lambda_{\mathbb{F}}}, \langle y^2 \rangle_{\lambda_{\mathbb{F}}})$ where $\mathbf{y} = \mathbf{A}^{-1} \cdot (\mathbf{v} + \mathbf{c})$

- 1: $\langle \mathbf{y} \rangle \leftarrow \mathbf{A}^{-1} \cdot (\langle \mathbf{v} \rangle + \mathbf{c})$
 - 2: $\langle y \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{ByteCombine}(\langle y_0 \rangle, \dots, \langle y_7 \rangle)$
 - 3: $\langle y^2 \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{SquareAndCombine}(\langle y_0 \rangle, \dots, \langle y_7 \rangle)$
 - 4: **return** $(\langle y \rangle_{\lambda_{\mathbb{F}}}, \langle y^2 \rangle_{\lambda_{\mathbb{F}}})$
-

6.2 Witness Extension

This section gives two witness extension algorithms for SM4th and SM4th-EM, which are named as **SM4th.ExtendWitness** and **SM4th.EM.ExtendWitness**, respectively. For SM4th, the algorithm takes the SM4 key $\mathbf{k}$, the SM4 plaintext $\mathbf{in}$, and the instance parameters $\mathbf{param}$, returning an extended witness $\mathbf{w} \in \{0, 1\}^{\ell}$ for later use. For SM4th-EM, whose OWF is $\text{SM4}_{\mathbf{x}}(\mathbf{k}) \oplus \mathbf{k}$, the algorithm takes the secret input block $\mathbf{k}$ and the public SM4 key $\mathbf{x}$. The witness length ℓ is a bit length determined by the SM4 trace and is independent of the proof field size $\lambda_{\mathbb{F}}$.

Choice of witness representation. The non-EM variant of SM4th proves knowledge of an SM4 secret key for the relation $\text{SM4}_{\mathbf{k}}(\mathbf{in}) = \mathbf{out}$. Therefore, the zero-knowledge circuit must cover both the SM4 key expansion and the SM4 encryption routine. If all intermediate key words and encryption state words were stored as full bit witnesses, the resulting signature would be larger. For this reason, the non-EM construction replaces selected key-expansion and encryption state words by compact norm witnesses. Each anchor word stores only four 4-bit norm values, one for each S-box input byte, and the omitted 32-bit word is reconstructed virtually from the norm witness and the neighboring state words inside the constraint system. This reduces the witness length and the amount of data opened in the VOLEitH proof, at the cost of using degree-3 norm constraints and a more involved conjugate representation.

The role of the norm witness is similar in spirit to the norm-based compression used in FAEST for AES, but the construction is SM4-specific. Further, the saving obtained from this norm representation is smaller than in FAEST. This is mainly due to the word-oriented structure of SM4. Both the SM4 key expansion and the SM4 encryption routine update one 32-bit word at a time using a four-word feedback relation. An omitted anchor word must still be reconstructed from neighboring words, the corresponding S-box input, and the output of the SM4 linear layer. Consequently, anchor words cannot be placed too densely: enough adjacent full words must remain available to define later S-box inputs and to support the ordinary patched-inversion checks on non-anchor rounds. In addition, each SM4 round contains only four byte S-boxes before the word-level linear layer L or L' , so replacing one anchor word saves only one 32-bit word while still requiring four 4-bit norm witnesses and several conjugate-based reconstruction constraints. In FAEST, the AES state and

round structure allow the norm technique to be aligned more directly with the byte-oriented S-box layer and the AES linear layer, giving a larger relative reduction in the stored witness. Therefore, the SM4 norm technique is still beneficial, but its witness-size reduction is more modest than the corresponding FAEST optimization.

For the EM variants, they prove the relation $\text{SM4}_x(\mathbf{k}) \oplus \mathbf{k} = \mathbf{y}$, where the SM4 key x is public and the secret witness is the input block $\mathbf{k}$. The round keys are derived from public data and do not need to be stored as private witness bits. Thus the private trace consists only of the SM4 input block and the intermediate encryption states. For this smaller witness, the ordinary patched-inversion constraints are preferable: they use direct bit commitments to the relevant intermediate states and give degree-2 checks over $\mathbb{F}_{2^{\lambda_F}}$. Applying the norm technique to the EM variant would lead to a larger signature size than not applying it.

Algorithms. The extended witness of [SM4th.ExtendWitness](#) consists of three components, corresponding to the initial phase, the key expansion phase and the encryption phase. It uses norm witnesses to replace some full state words. We call such compressed words anchor words.

- **Initial key bits:** the first 4 words obtained from the SM4 key expansion, namely $\bar{\mathbf{k}}[0..4)$, are stored as full 32-bit words. This contributes $32 \cdot 4$ bits.
- **Norm key bits:** instead of storing all expanded key words as full 32-bit words, we replace the words $\bar{\mathbf{k}}[4], \bar{\mathbf{k}}[8], \dots, \bar{\mathbf{k}}[32]$ by norm witnesses, which we call anchor words. For each such anchor word, the algorithm stores one 4-bit value for each of the four S-box input bytes. Since there are 8 anchor words, this contributes $4 \cdot 4 \cdot 8$ bits, *i.e.*, $w_{\text{ke},N}$ in [SM4th.ExtendWitness](#).
- **Full expanded key bits:** the remaining expanded key words are stored as ordinary full 32-bit words. These are the R expanded key words excluding the 8 norm anchor words. Hence this contributes $32 \cdot (R - 8)$ bits, *i.e.*, $w_{\text{ke},F}$ in [SM4th.ExtendWitness](#).
- **Norm encryption bits:** for the encryption trace, we similarly replace the anchor words $\mathbf{z}[5], \mathbf{z}[9], \dots, \mathbf{z}[29]$ by norm witnesses. For each such anchor state, the algorithm stores one 4-bit value for each of the four S-box input bytes. Since there are 7 encryption anchor states, this contributes $4 \cdot 4 \cdot 7$ bits, *i.e.*, $w_{\text{enc},N}$ in [SM4th.ExtendWitness](#).
- **Full encryption bits:** the remaining private intermediate encryption states are stored as full 32-bit words. In the Non-EM SM4th setting, the private encryption witness consists of $\mathbf{z}[4..32)$, excluding the 7 norm anchor states. Therefore, this contributes $32 \cdot ((R - 4) - 7)$ bits, *i.e.*, $w_{\text{enc},F}$ in [SM4th.ExtendWitness](#).

The extended witness of [SM4th.EM.ExtendWitness](#) consists of two components, corresponding to the initial input block and the encryption phase.

- **Initial input bits:** the secret EM input block $\mathbf{k}$, viewed as four 32-bit words.
- **Encryption bits:** the intermediate encryption states $\mathbf{z}[4..32)$, each of size 32 bits, corresponding to the outputs of the round function. This contributes $32 \cdot (R - 4)$ bits.

Therefore, the total size of the extended witness for EM variants is $32 \cdot 32 = 1024$ bits.

6.3 Deriving Constraints for the Key Expansion

This section gives the [SM4th.KeyExpCstrnts](#) algorithm, which derives the constraint values required for the QuickSilver proof for the SM4 circuit corresponding to the key expansion routine. Note that this only applies to the non-EM variant.

The [SM4th.KeyExpCstrnts](#) algorithm checks the key expansion routine of the norm-based SM4th witness. The key-expansion witness is parsed according to the layout $K_0, \dots, K_3$, followed by the norm coordinates for the anchor words, followed by the full bits of the non-anchor words. The set $\mathcal{A}_{\text{ke}} = \{4, 8, 12, 16, 20, 24, 28, 32\}$ specifies the key-expansion words that are represented by norm witnesses instead of full 32-bit words. The algorithm first loads the initial four key words from the witness, and then reconstructs the remaining key words sequentially. For each word index i , it forms the SM4 key-expansion S-box input $\mathbf{u} = \bar{\mathbf{k}}[i - 3] \oplus \bar{\mathbf{k}}[i - 2] \oplus \bar{\mathbf{k}}[i - 1] \oplus CK_{i-4}$. If i is an anchor index, the algorithm reads only the 4-bit norm coordinate for each byte of $\mathbf{u}$. For each

SM4th.ExtendWitness(k, in ; param)

INPUT: $k \in \{0, 1\}^{128}$, $in = in[0, \dots, 3] \in (\{0, 1\}^{32})^4$ OUTPUT: $w \in \{0, 1\}^\ell$: extended witness

```
1: // Initialize empty extended witness arrays, where  $w_N$  stores norm bits and  $w_F$  stores full bits
2: new  $w \in \{0, 1\}^{\leq \ell}$ 
3: new  $w_{ke,N} \in \{0, 1\}^{\leq 128}$ ; new  $w_{ke,F} \in \{0, 1\}^{\leq 768}$ 
4: new  $w_{enc,N} \in \{0, 1\}^{\leq 112}$ ; new  $w_{enc,F} \in \{0, 1\}^{\leq 672}$ 
5: // Flags for using norms in the key expansion and encryption routines, respectively
6:  $\mathcal{A}_{ke} \leftarrow \{4, 8, 12, 16, 20, 24, 28, 32\}$ 
7:  $\mathcal{A}_{enc} \leftarrow \{5, 9, 13, 17, 21, 25, 29\}$ 
8: // Store round keys as array of 32-bit words
9: new  $\bar{k} \in \{0, 1\}^{32; 4 + R}$ 
10:  $\bar{k} \leftarrow \text{SM4.KeyExpansion}(k, \text{param})$ 
11: // Key expansion routine
12: for  $i \in [0..4)$  do
13:    $w \leftarrow w \parallel \bar{k}[i]$ 
14: end for
15: for  $i \in [4..R + 4)$  do
16:   if  $i \in \mathcal{A}_{ke}$  then
17:     for  $j \in [0..4)$  do
18:        $u \leftarrow \bar{k}[i-3][j] \oplus \bar{k}[i-2][j] \oplus \bar{k}[i-1][j] \oplus CK_{i-4}[j]$ 
19:        $w_{ke,N} \leftarrow w_{ke,N} \parallel \text{SM4th.InvNorm}(u)$ 
20:     end for
21:   else
22:      $w_{ke,F} \leftarrow w_{ke,F} \parallel \bar{k}[i]$ 
23:   end if
24: end for
25:  $w \leftarrow w \parallel w_{ke,N} \parallel w_{ke,F}$ 
26: // Encryption routine
27: new  $tmp \in \{0, 1\}^{32}$ 
28: new  $z \in \{0, 1\}^{32; 4 + R}$ 
29: for  $i \in [0..4)$  do
30:    $z[i] \leftarrow in[i]$ 
31: end for
32: for  $i \in [0..R)$  do
33:    $tmp \leftarrow z[i+1] \oplus z[i+2] \oplus z[i+3] \oplus \bar{k}[i+4]$ 
34:    $tmp \leftarrow \text{SubWord}(tmp)$ 
35:    $tmp \leftarrow tmp \oplus \text{RotlWord}(tmp, 2) \oplus \text{RotlWord}(tmp, 10) \oplus \text{RotlWord}(tmp, 18) \oplus \text{RotlWord}(tmp, 24)$ 
36:    $z[i+4] \leftarrow z[i] \oplus tmp$ 
37: end for
38: for  $i \in [4..R)$  do
39:   if  $i \in \mathcal{A}_{enc}$  then
40:     for  $j \in [0..4)$  do
41:        $u \leftarrow z[i-3][j] \oplus z[i-2][j] \oplus z[i-1][j] \oplus \bar{k}[i][j]$ 
42:        $w_{enc,N} \leftarrow w_{enc,N} \parallel \text{SM4th.InvNorm}(u)$ 
43:     end for
44:   else
45:      $w_{enc,F} \leftarrow w_{enc,F} \parallel z[i]$ 
46:   end if
47: end for
48:  $w \leftarrow w \parallel w_{enc,N} \parallel w_{enc,F}$ 
49: return  $w \in \{0, 1\}^\ell$ 
```

SM4th.EM.ExtendWitness($\mathbf{k}, \mathbf{x}; \text{param}$)

INPUT: $\mathbf{k} = \mathbf{k}[0..3] \in (\{0, 1\}^{32})^4$, $\mathbf{x} \in \{0, 1\}^{128}$ OUTPUT: $\mathbf{w} \in \{0, 1\}^\ell$: extended witness

```
1: // Initialize empty extended witness array, eventually of length  $\ell$ 
2: new  $\mathbf{w} \in \{0, 1\}^{\leq \ell}$ 
3: // In EM mode,  $\mathbf{x}$  is the public SM4 key and  $\mathbf{k}$  is the secret input block
4: new  $\bar{\mathbf{x}} \in [\{0, 1\}^{32}; 4 + R]$ 
5:  $\bar{\mathbf{x}} \leftarrow \text{SM4.KeyExpansion}(\mathbf{x}, \text{param})$ 
6:  $\mathbf{w} \leftarrow \mathbf{w} \parallel \mathbf{k}$ 
7: // Encryption routine and saving witness bits
8: new  $\text{tmp} \in \{0, 1\}^{32}$ 
9: new  $\mathbf{z} \in [\{0, 1\}^{32}; 4 + R]$ 
10: for  $i \in [0..4)$  do
11:    $\mathbf{z}[i] \leftarrow \mathbf{k}[i]$ 
12: end for
13: for  $i \in [0..R)$  do
14:    $\text{tmp} \leftarrow \mathbf{z}[i+1] \oplus \mathbf{z}[i+2] \oplus \mathbf{z}[i+3] \oplus \bar{\mathbf{x}}[i+4]$ 
15:    $\text{tmp} \leftarrow \text{SubWord}(\text{tmp})$ 
16:    $\text{tmp} \leftarrow \text{tmp} \oplus \text{RotlWord}(\text{tmp}, 2) \oplus \text{RotlWord}(\text{tmp}, 10) \oplus \text{RotlWord}(\text{tmp}, 18) \oplus \text{RotlWord}(\text{tmp}, 24)$ 
17:    $\mathbf{z}[i+4] \leftarrow \mathbf{z}[i] \oplus \text{tmp}$ 
18: end for
19: for  $i \in [4..R)$  do
20:    $\mathbf{w} \leftarrow \mathbf{w} \parallel \mathbf{z}[i]$ 
21: end for
22: return  $\mathbf{w} \in \{0, 1\}^\ell$ 
```

byte, [SM4th.AffineToConjugates](#) computes the conjugates of the embedded value x corresponding to $\mathbf{A}\mathbf{u} + \mathbf{c}$, and [SM4th.InvNormToConjugates](#) computes the conjugates of the committed norm value h . These embedded values live in the proof field $\mathbb{F}_{2^{\lambda_F}}$, while the byte arithmetic itself is still the SM4 byte-field arithmetic. The norm correctness constraint $\langle o_N \rangle_{\lambda_F}^3 = \langle h \rangle_{\lambda_F} \cdot \langle x^{16} \rangle_{\lambda_F} \cdot \langle x^2 \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ is appended to the output constraint vector. When this constraint is zero, the norm witness is consistent with $h = x^{-17}$. The algorithm then reconstructs the conjugates of the inverse value by setting $\langle q^{2^r} \rangle_{\lambda_F} = \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_F} \cdot \langle h^{2^{r \bmod 4}} \rangle_{\lambda_F}$ for $r \in [0..8)$. Applying [SM4th.AffineOut](#) to these conjugates gives the committed conjugates of the S-box output byte. The key-schedule linear layer L' is then applied in its linearized-polynomial form to the four embedded S-box output bytes. Thus the anchor word $\bar{\mathbf{k}}[i]$ is reconstructed virtually as $\bar{\mathbf{k}}[i-4] \oplus L'(\mathbf{s})$, without storing its full 32 bits in the witness. If i is not an anchor index, the algorithm reads the full word $\bar{\mathbf{k}}[i]$ from the witness. It then applies [SM4th.L'Inv](#) to $\bar{\mathbf{k}}[i-4] \oplus \bar{\mathbf{k}}[i]$ to recover the claimed S-box output $\mathbf{v}$. For each byte, [SM4th.T'AffineByte](#) computes the affine input representation $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F})$, while [SM4th.T'InvAffineByte](#) computes the inverse affine output representation $(\langle y \rangle_{\lambda_F}, \langle y^2 \rangle_{\lambda_F})$. The two ordinary inverse-S-box input and output constraints $\langle o_0 \rangle_{\lambda_F}^3 = \langle x^2 \rangle_{\lambda_F} \cdot \langle y \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ and $\langle o_1 \rangle_{\lambda_F}^3 = \langle x \rangle_{\lambda_F} \cdot \langle y^2 \rangle_{\lambda_F} + \langle y \rangle_{\lambda_F}$ are appended to the constraint vector. All entries of the returned vector $\langle \mathbf{o} \rangle_{\lambda_F}^3$ must evaluate to zero for the committed key-expansion witness to be valid.

6.4 Deriving Constraints for the Encryption

This section gives two algorithms for deriving the OWF constraints in the encryption routine, *i.e.*, [SM4th.EncCstrnts](#) for SM4th and [SM4th.EM.EncCstrnts](#) for SM4th-EM.

The [SM4th.EncCstrnts](#) algorithm checks the encryption routine of the norm-based non-EM SM4th witness. The encryption witness is parsed according to the layout consisting of norm coordinates for the anchor state words, followed by the full bits of the non-anchor state words. The set $\mathcal{A}_{\text{enc}} = \{5, 9, 13, 17, 21, 25, 29\}$ specifies the encryption state words that are represented by norm witnesses instead of full 32-bit words. The algorithm first loads the public input words as $\mathbf{z}[0], \dots, \mathbf{z}[3]$ and the public output words as $\mathbf{z}[R], \dots, \mathbf{z}[R+3]$ in the SM4 final-output order. For each round i , it forms the SM4 encryption S-box input $\mathbf{u} = \mathbf{z}[i+1] \oplus \mathbf{z}[i+2] \oplus \mathbf{z}[i+3] \oplus \bar{\mathbf{k}}[i+4]$.

SM4th.KeyExpCstrnts($\langle w \rangle$; param)

INPUT: Commitment to the key-expansion part of the extended witness $w \in \{0, 1\}^{\ell_{ke}}$

OUTPUT: $\langle o \rangle_{\lambda_F}^3$: constraint vector for degree-3 checks

```

1: // Extended witness layout:  $K_0, \dots, K_3$  full bits, then norm bits, then full non-anchor bits
2:  $\mathcal{A}_{ke} \leftarrow \{4, 8, 12, 16, 20, 24, 28, 32\}$ 
3:  $\langle w_{ke,N} \rangle \leftarrow \langle w[128..256] \rangle$ 
4:  $\langle w_{ke,F} \rangle \leftarrow \langle w[256.. \ell_{ke}] \rangle$ 
5: new  $\langle \bar{k}[0] \rangle, \dots, \langle \bar{k}[R+3] \rangle$ 
6: new  $\langle o \rangle_{\lambda_F}^3$ 
7:  $p_N \leftarrow 0$ 
8:  $p_F \leftarrow 0$ 
9: for  $i \in [0..4)$  do
10:    $\langle \bar{k}[i] \rangle \leftarrow \langle w[32i..32(i+1)] \rangle$ 
11: end for
12: for  $i \in [4..R+4)$  do
13:    $\langle u \rangle \leftarrow \langle \bar{k}[i-3] \rangle \oplus \langle \bar{k}[i-2] \rangle \oplus \langle \bar{k}[i-1] \rangle \oplus CK_{i-4}$ 
14:   if  $i \in \mathcal{A}_{ke}$  then
15:     new  $(\langle s_j \rangle_{\lambda_F}, \langle s_j^2 \rangle_{\lambda_F}, \dots, \langle s_j^{128} \rangle_{\lambda_F})$  for  $j \in [0..4)$ 
16:     for  $j \in [0..4)$  do
17:        $\langle n \rangle \leftarrow \langle w_{ke,N}[4p_N..4p_N+4] \rangle$ 
18:        $p_N \leftarrow p_N + 1$ 
19:        $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F}, \dots, \langle x^{128} \rangle_{\lambda_F}) \leftarrow \text{SM4th.AffineToConjugates}(\langle u[j] \rangle)$ 
20:        $(\langle h \rangle_{\lambda_F}, \langle h^2 \rangle_{\lambda_F}, \langle h^4 \rangle_{\lambda_F}, \langle h^8 \rangle_{\lambda_F}) \leftarrow \text{SM4th.InvNormToConjugates}(\langle n \rangle)$ 
21:        $\langle o_N \rangle_{\lambda_F}^3 \leftarrow \langle h \rangle_{\lambda_F} \cdot \langle x^{16} \rangle_{\lambda_F} \cdot \langle x^2 \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ 
22:        $\langle o \rangle_{\lambda_F}^3 \leftarrow \langle o \rangle_{\lambda_F}^3 \parallel \langle o_N \rangle_{\lambda_F}^3$ 
23:       for  $r \in [0..8)$  do
24:          $\langle q^{2^r} \rangle_{\lambda_F} \leftarrow \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_F} \cdot \langle h^{2^r \bmod 4} \rangle_{\lambda_F}$ 
25:       end for
26:        $(\langle s_j \rangle_{\lambda_F}, \langle s_j^2 \rangle_{\lambda_F}, \dots, \langle s_j^{128} \rangle_{\lambda_F}) \leftarrow \text{SM4th.AffineOut}(\langle q \rangle_{\lambda_F}, \langle q^2 \rangle_{\lambda_F}, \dots, \langle q^{128} \rangle_{\lambda_F})$ 
27:     end for
28:      $\langle t \rangle \leftarrow \text{SM4th.L}'((\langle s_j \rangle_{\lambda_F}, \langle s_j^2 \rangle_{\lambda_F}, \dots, \langle s_j^{128} \rangle_{\lambda_F})_{j=0}^3)$ 
29:      $\langle \bar{k}[i] \rangle \leftarrow \langle \bar{k}[i-4] \rangle \oplus \langle t \rangle$ 
30:   else
31:      $\langle \bar{k}[i] \rangle \leftarrow \langle w_{ke,F}[32p_F..32(p_F+1)] \rangle$ 
32:      $p_F \leftarrow p_F + 1$ 
33:      $\langle v \rangle \leftarrow \text{SM4th.L}'\text{Inv}(\langle \bar{k}[i-4] \rangle \oplus \langle \bar{k}[i] \rangle)$ 
34:     for  $j \in [0..4)$  do
35:        $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F}) \leftarrow \text{SM4th.T}'\text{AffineByte}(\langle u[j] \rangle)$ 
36:        $(\langle y \rangle_{\lambda_F}, \langle y^2 \rangle_{\lambda_F}) \leftarrow \text{SM4th.T}'\text{InvAffineByte}(\langle v[j] \rangle)$ 
37:        $\langle o_0 \rangle_{\lambda_F}^3 \leftarrow \langle x^2 \rangle_{\lambda_F} \cdot \langle y \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ 
38:        $\langle o_1 \rangle_{\lambda_F}^3 \leftarrow \langle x \rangle_{\lambda_F} \cdot \langle y^2 \rangle_{\lambda_F} + \langle y \rangle_{\lambda_F}$ 
39:        $\langle o \rangle_{\lambda_F}^3 \leftarrow \langle o \rangle_{\lambda_F}^3 \parallel \langle o_0 \rangle_{\lambda_F}^3$ 
40:        $\langle o \rangle_{\lambda_F}^3 \leftarrow \langle o \rangle_{\lambda_F}^3 \parallel \langle o_1 \rangle_{\lambda_F}^3$ 
41:     end for
42:   end if
43: end for
44: return  $\langle o \rangle_{\lambda_F}^3$ 

```

If the target state index $a = i + 4$ is an anchor index, the algorithm reads only the 4-bit norm coordinate for each byte of $\mathbf{u}$. For each byte, [SM4th.AffineToConjugates](#) computes the conjugates of the embedded value x corresponding to $\mathbf{A}\mathbf{u} + \mathbf{c}$, and [SM4th.InvNormToConjugates](#) computes the conjugates of the committed norm value h . These embedded values live in the proof field $\mathbb{F}_{2^{\lambda_F}}$, while the byte arithmetic itself is still the SM4 byte-field arithmetic. The norm correctness constraint $\langle o_N \rangle_{\lambda_F}^3 = \langle h \rangle_{\lambda_F} \cdot \langle x^{16} \rangle_{\lambda_F} \cdot \langle x^2 \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ is appended to the output constraint vector. When this constraint is zero, the norm witness is consistent with $h = x^{-17}$. The algorithm then reconstructs the conjugates of the inverse value by setting $\langle q^{2^r} \rangle_{\lambda_F} = \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_F} \cdot \langle h^{2^{r \bmod 4}} \rangle_{\lambda_F}$ for $r \in [0..8)$. Applying [SM4th.AffineOut](#) to these conjugates gives the committed conjugates of the S-box output byte. The SM4 encryption linear layer L is then applied in its linearized-polynomial form to the four embedded S-box output bytes. Thus the anchor state word $\mathbf{z}[a]$ is reconstructed virtually as $\mathbf{z}[i] \oplus L(\mathbf{s})$, without storing its full 32 bits in the witness. If a is not an anchor index, the algorithm uses the full state word $\mathbf{z}[a]$. For internal state words $a < R$, this word is read from the full encryption witness, while for the final boundary words it has already been loaded from the public output. The algorithm then applies [SM4th.LInv](#) to $\mathbf{z}[i] \oplus \mathbf{z}[a]$ to recover the claimed S-box output vector $\mathbf{v}$. For each byte, [SM4th.TAffineByte](#) computes the affine input representation $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F})$, while [SM4th.TInvAffineByte](#) computes the inverse affine output representation $(\langle y \rangle_{\lambda_F}, \langle y^2 \rangle_{\lambda_F})$. The two ordinary inverse-S-box input and output constraints $\langle o_0 \rangle_{\lambda_F}^3 = \langle x^2 \rangle_{\lambda_F} \cdot \langle y \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ and $\langle o_1 \rangle_{\lambda_F}^3 = \langle x \rangle_{\lambda_F} \cdot \langle y^2 \rangle_{\lambda_F} + \langle y \rangle_{\lambda_F}$ are appended to the constraint vector. All entries of the returned vector $\langle \mathbf{o} \rangle_{\lambda_F}^3$ must evaluate to zero for the committed encryption witness to be valid.

The [SM4th.EM.EncCstrnts](#) algorithm checks the encryption routine of the SM4th-EM witness. It derives the commitments to the inputs and outputs of the patched inversion function in S-boxes via the [T.Affine](#) and [T.InvAffine](#) algorithms. The witness words are first expanded by concatenating the committed SM4 input boundary words, the committed intermediate state words, and the committed SM4 output boundary words before invoking [T.Affine](#) and [T.InvAffine](#). It then extracts the commitments to each 8 bits of $\langle \mathbf{x} \rangle$ and $\langle \mathbf{y} \rangle$ and applies [ByteCombine](#) and [SquareAndCombine](#) functions to them, obtaining the combined SM4-compatible commitment in $\mathbb{F}_{2^{\lambda_F}}$ and the combined commitment to its square in $\mathbb{F}_{2^{\lambda_F}}$. The square can be obtained by the $\mathbb{F}_2$ -linearity of the squaring map. For example, let the commitments to the combined input be $\langle \hat{x} \rangle \in \mathbb{F}_{2^{\lambda_F}}$ and its combined square be $\langle \hat{x}_{sq} \rangle \in \mathbb{F}_{2^{\lambda_F}}$. Let the commitments to the output be $\langle \hat{y} \rangle \in \mathbb{F}_{2^{\lambda_F}}$ and its square be $\langle \hat{y}_{sq} \rangle \in \mathbb{F}_{2^{\lambda_F}}$. Finally, the algorithm generates two degree-2 VOLE commitments, of the form

$$\langle \hat{o}_1 \rangle^2 = \langle \hat{x} \rangle \cdot \langle \hat{y}_{sq} \rangle + \langle \hat{y} \rangle, \quad \langle \hat{o}_2 \rangle^2 = \langle \hat{y} \rangle \cdot \langle \hat{x}_{sq} \rangle + \langle \hat{x} \rangle.$$

By the correctness, $\langle \hat{o}_1 \rangle^2$ and $\langle \hat{o}_2 \rangle^2$ should be commitments to zero. This helps to check

$$\hat{x} \cdot \hat{y}^2 + \hat{y} = 0, \quad \hat{y} \cdot \hat{x}^2 + \hat{x} = 0.$$

6.5 Constraints for the OWF

This section describes the remaining steps needed to prove with the QuickSilver zero-knowledge proof system that $\mathbf{pk} = (\mathbf{pk}_1, \mathbf{pk}_2)$ is consistent with the witness $\mathbf{w}$ under the SM4-based one-way function. There are three algorithms:

1. the [SM4th.OWFConstraints](#) algorithm that combines both the key expansion and encryption constraints;
2. the signer-side procedure [SM4th.OWFProve](#); and
3. the verifier-side procedure [SM4th.OWFVerify](#).

[SM4th.OWFConstraints](#)($\langle \mathbf{w} \rangle, \mathbf{pk}; \text{param}$). This is the main algorithm that combines all sub-constraints from the key schedule and encryption routines. Concretely:

1. It parses $\mathbf{pk} = (\mathbf{pk}_1, \mathbf{pk}_2)$ and allocates a new $\langle \mathbf{z} \rangle_{\lambda_F}^d$ for the accumulated constraints.

SM4th.EncCstrnts($\langle in \rangle, \langle out \rangle, \langle \bar{k} \rangle, \langle w \rangle; \text{param}$)

INPUT: Commitment to the encryption part of the extended witness $w \in \{0, 1\}^{\ell_{\text{enc}}}$, committed round-key expressions $\langle \bar{k}[4] \rangle, \dots, \langle \bar{k}[R+3] \rangle$, and public SM4 input/output words $in, out \in (\{0, 1\}^{32})^4$

OUTPUT: $\langle o \rangle_{\lambda_F}^3$: constraint vector for degree-3 checks

```

1: // Encryption witness layout: norm bits first, then full non-anchor state bits
2:  $\mathcal{A}_{\text{enc}} \leftarrow \{5, 9, 13, 17, 21, 25, 29\}$ 
3:  $\langle w_{\text{enc}, N} \rangle \leftarrow \langle w[0..112] \rangle$ 
4:  $\langle w_{\text{enc}, F} \rangle \leftarrow \langle w[112..\ell_{\text{enc}}] \rangle$ 
5: new  $\langle z[0] \rangle, \dots, \langle z[R+3] \rangle$ 
6: new  $\langle o \rangle_{\lambda_F}^3$ 
7:  $p_N \leftarrow 0$ 
8:  $p_F \leftarrow 0$ 
9: // Load public input and output boundary words
10: for  $i \in [0..4)$  do
11:    $\langle z[i] \rangle \leftarrow \langle in[i] \rangle$ 
12:    $\langle z[R+i] \rangle \leftarrow \langle out[3-i] \rangle$ 
13: end for
14: for  $i \in [0..R)$  do
15:    $a \leftarrow i + 4$ 
16:    $\langle u \rangle \leftarrow \langle z[i+1] \rangle \oplus \langle z[i+2] \rangle \oplus \langle z[i+3] \rangle \oplus \langle \bar{k}[i+4] \rangle$ 
17:   if  $a \in \mathcal{A}_{\text{enc}}$  then
18:     new  $(\langle s_j \rangle_{\lambda_F}, \langle s_j^2 \rangle_{\lambda_F}, \dots, \langle s_j^{128} \rangle_{\lambda_F})$  for  $j \in [0..4)$ 
19:     for  $j \in [0..4)$  do
20:        $\langle n \rangle \leftarrow \langle w_{\text{enc}, N}[4p_N..4p_N+4] \rangle$ 
21:        $p_N \leftarrow p_N + 1$ 
22:        $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F}, \dots, \langle x^{128} \rangle_{\lambda_F}) \leftarrow \text{SM4th.AffineToConjugates}(\langle u[j] \rangle)$ 
23:        $(\langle h \rangle_{\lambda_F}, \langle h^2 \rangle_{\lambda_F}, \langle h^4 \rangle_{\lambda_F}, \langle h^8 \rangle_{\lambda_F}) \leftarrow \text{SM4th.InvNormToConjugates}(\langle n \rangle)$ 
24:        $\langle o_N \rangle_{\lambda_F}^3 \leftarrow \langle h \rangle_{\lambda_F} \cdot \langle x^{16} \rangle_{\lambda_F} \cdot \langle x^2 \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ 
25:        $\langle o \rangle_{\lambda_F}^3 \leftarrow \langle o \rangle_{\lambda_F}^3 \parallel \langle o_N \rangle_{\lambda_F}^3$ 
26:       for  $r \in [0..8)$  do
27:          $\langle q^{2^r} \rangle_{\lambda_F} \leftarrow \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_F} \cdot \langle h^{2^r \bmod 4} \rangle_{\lambda_F}$ 
28:       end for
29:        $(\langle s_j \rangle_{\lambda_F}, \langle s_j^2 \rangle_{\lambda_F}, \dots, \langle s_j^{128} \rangle_{\lambda_F}) \leftarrow \text{SM4th.AffineOut}(\langle q \rangle_{\lambda_F}, \langle q^2 \rangle_{\lambda_F}, \dots, \langle q^{128} \rangle_{\lambda_F})$ 
30:     end for
31:      $\langle t \rangle \leftarrow \text{SM4th.L}((\langle s_j \rangle_{\lambda_F}, \langle s_j^2 \rangle_{\lambda_F}, \dots, \langle s_j^{128} \rangle_{\lambda_F})_{j=0}^3)$ 
32:      $\langle z[a] \rangle \leftarrow \langle z[i] \rangle \oplus \langle t \rangle$ 
33:   else
34:     if  $a < R$  then
35:        $\langle z[a] \rangle \leftarrow \langle w_{\text{enc}, F}[32p_F..32(p_F+1)] \rangle$ 
36:        $p_F \leftarrow p_F + 1$ 
37:     end if
38:      $\langle v \rangle \leftarrow \text{SM4th.LInv}(\langle z[i] \rangle \oplus \langle z[a] \rangle)$ 
39:     for  $j \in [0..4)$  do
40:        $(\langle x \rangle_{\lambda_F}, \langle x^2 \rangle_{\lambda_F}) \leftarrow \text{SM4th.TAffineByte}(\langle u[j] \rangle)$ 
41:        $(\langle y \rangle_{\lambda_F}, \langle y^2 \rangle_{\lambda_F}) \leftarrow \text{SM4th.TInvAffineByte}(\langle v[j] \rangle)$ 
42:        $\langle o_0 \rangle_{\lambda_F}^3 \leftarrow \langle x^2 \rangle_{\lambda_F} \cdot \langle y \rangle_{\lambda_F} + \langle x \rangle_{\lambda_F}$ 
43:        $\langle o_1 \rangle_{\lambda_F}^3 \leftarrow \langle x \rangle_{\lambda_F} \cdot \langle y^2 \rangle_{\lambda_F} + \langle y \rangle_{\lambda_F}$ 
44:        $\langle o \rangle_{\lambda_F}^3 \leftarrow \langle o \rangle_{\lambda_F}^3 \parallel \langle o_0 \rangle_{\lambda_F}^3$ 
45:        $\langle o \rangle_{\lambda_F}^3 \leftarrow \langle o \rangle_{\lambda_F}^3 \parallel \langle o_1 \rangle_{\lambda_F}^3$ 
46:     end for
47:   end if
48: end for
49: return  $\langle o \rangle_{\lambda_F}^3$ 

```

SM4th-EM.EncCstrnts($\langle \mathbf{in} \rangle, \langle \mathbf{out} \rangle, \langle \bar{\mathbf{k}} \rangle, \langle \mathbf{w} \rangle; \text{param}$)

INPUT: Commitments to SM4 input/output blocks $\mathbf{in}, \mathbf{out} \in \{0, 1\}^{128}$, middle encryption witness words $\mathbf{w} \in \mathbb{F}_2^{32(R-4)}$, and expanded keys $\bar{\mathbf{k}} \in (\mathbb{F}_2^{32})^{4+R}$

OUTPUT: $\langle \mathbf{o} \rangle_{\lambda_{\mathbb{F}}}^2$: constraint vector $\mathbf{o} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{2S_{\text{enc}}}$

```

1: new  $\langle \mathbf{x} \rangle \in \mathbb{F}_2^{8S_{\text{enc}}}$  // Patched inverse input bits
2: new  $\langle \mathbf{y} \rangle \in \mathbb{F}_2^{8S_{\text{enc}}}$  // Patched inverse output bits
3: let  $\langle \mathbf{w} \rangle \leftarrow \langle \mathbf{in} \rangle \| \langle \mathbf{w} \rangle \| \langle \mathbf{out} \rangle$ 
4: for  $i \in [0, R)$  do
5:    $\langle \mathbf{x}[iN_{\text{word}} \dots (i+1)N_{\text{word}} - 1] \rangle \leftarrow \text{T.Affine}(\langle \mathbf{w}[iN_{\text{word}} \dots (i+5)N_{\text{word}} - 1] \rangle, \langle \bar{\mathbf{k}}[iN_{\text{word}} \dots (i+5)N_{\text{word}} - 1] \rangle)$ 
6:    $\langle \mathbf{y}[iN_{\text{word}} \dots (i+1)N_{\text{word}} - 1] \rangle \leftarrow \text{T.InvAffine}(\langle \mathbf{w}[iN_{\text{word}} \dots (i+5)N_{\text{word}} - 1] \rangle)$ 
7: end for
8: for  $j \in [0 \dots S_{\text{enc}})$  do
9:   new  $\langle \hat{\mathbf{x}} \rangle_{\lambda_{\mathbb{F}}}, \langle \hat{\mathbf{x}}_{\text{sq}} \rangle_{\lambda_{\mathbb{F}}}, \langle \hat{\mathbf{y}} \rangle_{\lambda_{\mathbb{F}}}, \langle \hat{\mathbf{y}}_{\text{sq}} \rangle_{\lambda_{\mathbb{F}}}$ , each a commitment to an element in  $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ 
10:   $\langle \hat{\mathbf{x}} \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{ByteCombine}(\langle \mathbf{x}[8j \dots 8j+7] \rangle)$ 
11:   $\langle \hat{\mathbf{x}}_{\text{sq}} \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{SquareAndCombine}(\langle \mathbf{x}[8j \dots 8j+7] \rangle)$ 
12:   $\langle \hat{\mathbf{y}} \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{ByteCombine}(\langle \mathbf{y}[8j \dots 8j+7] \rangle)$ 
13:   $\langle \hat{\mathbf{y}}_{\text{sq}} \rangle_{\lambda_{\mathbb{F}}} \leftarrow \text{SquareAndCombine}(\langle \mathbf{y}[8j \dots 8j+7] \rangle)$ 
14:   $\langle \mathbf{o}_{2j} \rangle_{\lambda_{\mathbb{F}}}^2 \leftarrow \langle \hat{\mathbf{x}}_{\text{sq}} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle \hat{\mathbf{y}} \rangle_{\lambda_{\mathbb{F}}} + \langle \hat{\mathbf{x}} \rangle_{\lambda_{\mathbb{F}}}$ 
15:   $\langle \mathbf{o}_{2j+1} \rangle_{\lambda_{\mathbb{F}}}^2 \leftarrow \langle \hat{\mathbf{x}} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle \hat{\mathbf{y}}_{\text{sq}} \rangle_{\lambda_{\mathbb{F}}} + \langle \hat{\mathbf{y}} \rangle_{\lambda_{\mathbb{F}}}$ 
16: end for
17: return  $(\langle \mathbf{o}_0 \rangle_{\lambda_{\mathbb{F}}}^2, \dots, \langle \mathbf{o}_{2S_{\text{enc}}-1} \rangle_{\lambda_{\mathbb{F}}}^2)$ 

```

2. It first appends a small keyspace-reduction constraint $\langle \mathbf{w}[0] \rangle \times \langle \mathbf{w}[1] \rangle$ for the secret witness. In the EM variant, these two bits are the first two bits of the secret input block $\mathbf{k}$; in the non-EM variant, they are the first two bits of the secret SM4 key.
3. If $\text{EM} = \text{true}$, it interprets pk_1 as the public SM4 key, runs [SM4.KeyExpansion](#), and then sets $\langle \bar{\mathbf{k}} \rangle$ using [ConstantToVOLE](#). It also sets $\langle \mathbf{w} \rangle[0 \dots 128)$ as the commitment to the secret SM4 input block, and sets $\langle \mathbf{w} \rangle[0 \dots 128) + \text{ConstantToVOLE}(\text{pk}_2)$ as the commitment to the corresponding SM4 encryption output block.
If $\text{EM} = \text{false}$, the algorithm calls [ConstantToVOLE](#)(pk_1) to get a bitwise-committed input block $\langle \mathbf{in} \rangle$ and calls [ConstantToVOLE](#)(pk_2) to get the committed output block $\langle \mathbf{out} \rangle$ accordingly. For the key schedule, it calls [SM4th.KeyExpCstrnts](#)($\langle \mathbf{w} \rangle; \text{param}$), obtaining both round-key commitments $\langle \bar{\mathbf{k}} \rangle$ and a degree- d constraint vector $\langle \tilde{\mathbf{z}} \rangle_{\lambda_{\mathbb{F}}}^d$. This is later appended to the main buffer.
4. In the encryption routine, it extracts the portion of $\langle \mathbf{w} \rangle$ dedicated to the encryption bits, and calls [SM4th.EncCstrnts](#)($\langle \mathbf{in} \rangle, \langle \mathbf{out} \rangle, \dots$) or [SM4th-EM.EncCstrnts](#)($\langle \mathbf{in} \rangle, \langle \mathbf{out} \rangle, \dots$) to produce the encryption constraints. These are all collected and appended into the same $\langle \mathbf{z} \rangle_{\lambda_{\mathbb{F}}}^d$.
5. Finally, the algorithm returns $\langle \mathbf{z} \rangle_{\lambda_{\mathbb{F}}}^d$.

SM4th.OWFProve and **SM4th.OWFVerify**. The [SM4th.OWFProve](#) describes the signer's algorithm:

1. It converts each bit $\mathbf{w}[i]$ into a VOLE commitment, using a row in $\mathbf{V}$ as tags. It also parses $\mathbf{u}$ into $(d-1)$ proof-response mask elements and parses the last $2\lambda_{\mathbb{F}}$ rows of $\mathbf{V}$ into the two ZK-mask elements v_0^* and v_1^* .
2. Then, it invokes [SM4th.OWFConstraints](#)($\langle \mathbf{w} \rangle, \text{pk}$), which returns a polynomial vector $\langle \mathbf{z} \rangle_{\lambda_{\mathbb{F}}}^d$ of degree d . This vector is then parsed as a vector $(\mathbf{a}_0, \mathbf{a}_1, \dots, \mathbf{a}_{d-1}) \in (\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^C)^d$.
3. Finally, it generates the QuickSilver response $(\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{d-1})$ by hashing $\mathbf{a}_j \| (\text{masks})$ with chall_2 via [ZKHash](#). The response $(\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{d-1})$ is appended as the final proof message.

The [SM4th.OWFVerify](#) algorithm describes the corresponding verifier procedure. It reconstructs the commitment values $\langle \mathbf{w}[i] \rangle$ from $\mathbf{d}[i]$ and $\mathbf{Q}$. It then, from the verifier's side, recomputes the constraints $\langle \mathbf{z} \rangle_{\lambda_{\mathbb{F}}}^d$ using [SM4th.OWFConstraints](#)($\langle \mathbf{w} \rangle, \text{pk}$), and checks consistency of the hashes with the prover's $\{\tilde{a}_i\}_{i \in [1 \dots d]}$. In particular, it computes $\tilde{q} = \text{ZKHash}(\text{chall}_2, \mathbf{b} \| \mathbf{q}^*)$, then returns $\tilde{q} + \sum_{i \in [1 \dots d]} \tilde{a}_i \cdot \Delta^i$. If the proof is correct, this final expression should be the same as $\tilde{a}_0$.

SM4th.OWFConstraints($\langle w \rangle$, pk; param)

INPUT: Committed witness $w \in \mathbb{F}_2^\ell$ OUTPUT: Committed constraints $\langle z \rangle_{\lambda_F}^d$, for $z \in \mathbb{F}_{2^{\lambda_F}}^C$ (will check that $z = 0$)

```
1: Parse  $pk = (pk_1, pk_2) \in \{0, 1\}^{128} \times \{0, 1\}^{128}$ 
2: new  $\langle z \rangle_{\lambda_F}^d, z \in \mathbb{F}_{2^{\lambda_F}}^{\leq C}$ 
3:  $\langle z \rangle_{\lambda_F}^d \leftarrow \langle z \rangle_{\lambda_F}^d \parallel \langle w[0] \rangle \cdot \langle w[1] \rangle$  // secret-witness keyspace reduction
4: if EM = true then
5:    $\bar{k} \leftarrow \text{SM4.KeyExpansion}(pk_1, \text{param})$  //  $(R+4) \cdot 32$ -bit round key  $\bar{k} \in (\{0, 1\}^{32})^{R+4}$ 
6:    $\langle \bar{k} \rangle \leftarrow \text{ConstantToVOLE}(\bar{k})$ 
7:    $\langle in \rangle \leftarrow \langle w[0..128] \rangle$  // secret SM4 input block
8:    $\langle out \rangle \leftarrow \langle w[0..128] \rangle + \text{ConstantToVOLE}(pk_2)$  // SM4 output block, since  $pk_2 = \text{SM4}_{pk_1}(in) \oplus in$ 
9: else
10:   $\langle in \rangle \leftarrow \text{ConstantToVOLE}(pk_1)$  // convert each bit separately
11:   $\langle out \rangle \leftarrow \text{ConstantToVOLE}(pk_2)$ 
12:   $(\langle \bar{k} \rangle, \langle \tilde{z} \rangle_{\lambda_F}^d) \leftarrow \text{SM4th.KeyExpCstrnts}(\langle w[0..l_{ke}] \rangle; \text{param})$ 
13:   $\langle z \rangle_{\lambda_F}^d \leftarrow \langle z \rangle_{\lambda_F}^d \parallel \langle \tilde{z} \rangle_{\lambda_F}^d$  // Saving constraints
14: end if
15:  $\langle \tilde{w} \rangle \leftarrow \langle w[l_{ke}..l_{ke} + l_{enc}] \rangle$  // Selecting encryption bits
16: if EM = true then
17:   $\langle \tilde{z} \rangle_{\lambda_F}^d \leftarrow \text{SM4th.EM.EncCstrnts}(\langle in \rangle, \langle out \rangle, \langle \bar{k} \rangle, \langle \tilde{w} \rangle; \text{param})$  //  $d = 2$ 
18: else
19:   $\langle \tilde{z} \rangle_{\lambda_F}^d \leftarrow \text{SM4th.EncCstrnts}(\langle in \rangle, \langle out \rangle, \langle \bar{k} \rangle, \langle \tilde{w} \rangle; \text{param})$  //  $d = 3$ 
20: end if
21:  $\langle z \rangle_{\lambda_F}^d \leftarrow \langle z \rangle_{\lambda_F}^d \parallel \langle \tilde{z} \rangle_{\lambda_F}^d$  // Appending values
22: return  $\langle z \rangle_{\lambda_F}^d$ 
```

SM4th.OWFProve(w, u, V , pk, chall₂; param)

INPUT: $w \in \{0, 1\}^\ell$, $u \in \{0, 1\}^{(d-1)\lambda_F}$, $V \in \{0, 1\}^{(\ell+2\lambda_F) \times \lambda_F} \dots$ OUTPUT: $\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{d-1} \in \mathbb{F}_{2^{\lambda_F}}$

```
1: for  $i \in [0..l]$  do
2:   Let  $\langle w[i] \rangle \leftarrow (w[i], \text{ToField}(V|_i; \lambda_F))$ 
3: end for
4: // embed VOLE masks
5: for  $i \in [0..2\lambda_F]$  do
6:    $v[i] \leftarrow \text{ToField}(V|_{\ell+i}; \lambda_F)$ 
7: end for
8: for  $j \in [0..d-1]$  do
9:    $u_j^* \leftarrow \text{ToField}(u[j\lambda_F..(j+1)\lambda_F]; \lambda_F)$ 
10: end for
11: for  $j \in [0..2]$  do
12:   //  $\alpha_{\lambda_F} \in \mathbb{F}_{2^{\lambda_F}}$ , basis element over  $\mathbb{F}_2$ 
13:    $v_j^* \leftarrow \sum_{i \in [0.. \lambda_F)} v[i + \lambda_F j] \alpha_{\lambda_F}^i$ 
14: end for
15: // Compute the constraints
16: // (polynomial  $\sum_{i \in [0..d-1]} a_i x^i + z x^d; z = 0$ )
17:  $\langle z \rangle_{\lambda_F}^d \leftarrow \text{SM4th.OWFConstraints}(\langle w \rangle, pk)$ 
18: Parse  $\langle z \rangle_{\lambda_F}^d$  as  $(a_0, a_1, \dots, a_{d-1}) \in (\mathbb{F}_{2^{\lambda_F}}^C)^d$ 
19: if  $d = 2$  then
20:    $\tilde{a}_0 \leftarrow \text{ZKHash}(\text{chall}_2, (a_0 \| v_0^*))$ 
21:    $\tilde{a}_1 \leftarrow \text{ZKHash}(\text{chall}_2, (a_1 \| v_1^* + u_0^*))$ 
22: else if  $d = 3$ 
23:    $\tilde{a}_0 \leftarrow \text{ZKHash}(\text{chall}_2, (a_0 \| v_0^*))$ 
24:    $\tilde{a}_1 \leftarrow \text{ZKHash}(\text{chall}_2, (a_1 \| v_1^* + u_0^*))$ 
25:    $\tilde{a}_2 \leftarrow \text{ZKHash}(\text{chall}_2, (a_2 \| u_1^*))$ 
26: end if
27: return  $(\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{d-1})$ 
```

SM4th.OWFVerify($d, Q, \text{pk}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \dots, \tilde{a}_{d-1}; \text{param}, \text{param}_{\text{OWF}}$)

INPUT: $d \in \{0, 1\}^\ell$, $Q \in \{0, 1\}^{(\ell+2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$...

OUTPUT: $\tilde{a}_0 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

```

1:  $\Delta \leftarrow \text{ToField}(\text{chall}_3; \lambda_{\mathbb{F}})$ 
2: for  $i \in [0.. \ell)$  do
3:   Let  $\langle w[i] \rangle \leftarrow \text{ToField}(Q|_i; \lambda_{\mathbb{F}}) + d[i] \cdot \Delta$ 
4: end for
5: // embed VOLE masks
6: for  $i \in [0.. 2\lambda_{\mathbb{F}})$  do
7:    $q[i] \leftarrow \text{ToField}(Q|_{\ell+i}; \lambda_{\mathbb{F}})$ 
8: end for
9: for  $j \in [0.. 2)$  do
10:  //  $\alpha_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ , basis element over  $\mathbb{F}_2$ 
11:   $q_j^* \leftarrow \sum_{i \in [0.. \lambda_{\mathbb{F}})} q[i + \lambda_{\mathbb{F}}j] \alpha_{\lambda_{\mathbb{F}}}^i$ 
12: end for
13:  $q^* \leftarrow q_0^* + q_1^* \cdot \Delta$ 
14:  $\langle z \rangle_{\lambda_{\mathbb{F}}}^d \leftarrow \text{SM4th.OWFConstraints}(\langle w \rangle, \text{pk})$ 
15: Parse  $\langle z \rangle_{\lambda_{\mathbb{F}}}^d$  as  $b \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^C$ 
16:  $\tilde{q} \leftarrow \text{ZKHash}(\text{chall}_2, (b \| q^*))$ 
17: // Reproduce prover's  $\tilde{a}_0$ 
18: return  $\tilde{q} + \sum_{i \in [1.. d)} \tilde{a}_i \cdot \Delta^i$ 

```

6.6 Construction of SM4th

Based on the building blocks and functions described above, this section gives the formal construction of the SM4th and SM4th-EM signature schemes.

6.6.1 Key Generation

The **SM4th.KeyGen** algorithm describes the key generation procedure for the SM4-based schemes. All SM4 keys, inputs, and outputs in this algorithm have length 128 bits.

The output depends on whether the encryption mode flag **EM** is set to **true**. If **EM** = **true**, the algorithm samples a public SM4 key $x \in \{0, 1\}^{128}$ uniformly at random and samples a secret input block $k \in \{0, 1\}^{128}$ subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It computes $y \leftarrow \text{SM4}_x(k) \oplus k$, sets the secret key to $\text{sk} \leftarrow (x, k)$, and defines the public key as $\text{pk} \leftarrow (x, y)$. In this case, x serves as the public SM4 key and k serves as the secret input block. Otherwise, when **EM** = **false**, the algorithm samples a secret SM4 key $k \in \{0, 1\}^{128}$ uniformly at random, subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It then samples a public input block $x \in \{0, 1\}^{128}$ uniformly at random, computes $y \leftarrow \text{SM4}_k(x)$, sets $\text{sk} \leftarrow (x, k)$, and outputs $\text{pk} \leftarrow (x, y)$. In this case, k is the secret SM4 key and x is the public input block. Finally, the algorithm returns the pair (sk, pk) .

6.6.2 Signing

The **SM4th.Sign** algorithm proceeds in four conceptually separate phases.

Phase I: message binding and IV derivation. On input a message msg , a secret key $\text{sk} = (\text{sk}_1, \text{sk}_2)$, a public key $\text{pk} = (\text{pk}_1, \text{pk}_2)$, and the public parameters param , the signer first initializes a 32-bit counter $\text{ctr} \leftarrow 0$ and samples fresh signing randomness $\rho \leftarrow \{0, 1\}^\xi$ uniformly at random. It then computes $\mu \leftarrow H(\text{pk} \| \text{msg}; 2\lambda)$, which binds the signing transcript to both the message and the public key. Next, it derives $(r, \text{iv}^{\text{pre}}) \leftarrow H_3(\text{sk}_2 \| \mu \| \rho)$ and $\text{iv} \leftarrow H_4(\text{iv}^{\text{pre}}; \lambda_{\text{iv}})$. Here, $r \in \{0, 1\}^{N_{\text{block}}}$ is the root seed used to seed the subsequent BAVC/VOLE commitment generation, while $\text{iv} \in \{0, 1\}^{\lambda_{\text{iv}}}$ serves as the initialization value for the derived pseudorandom process.

SM4th.KeyGen(param)

INPUT: Public parameters **param**OUTPUT: Secret key $\mathbf{sk} \in (\{0, 1\}^{128}, \{0, 1\}^{128})$ and public key $\mathbf{pk} \in (\{0, 1\}^{128}, \{0, 1\}^{128})$

```
1: if EM = true then
2:    $\mathbf{x} \leftarrow \{0, 1\}^{128}$  // public SM4 key
3:   repeat
4:      $\mathbf{k} \leftarrow \{0, 1\}^{128}$  // secret input block
5:   until  $\mathbf{k}[0] \wedge \mathbf{k}[1] = 0$ 
6:    $\mathbf{y} \leftarrow \text{SM4}_{\mathbf{x}}(\mathbf{k}) \oplus \mathbf{k}$ 
7:    $\mathbf{sk} \leftarrow (\mathbf{x}, \mathbf{k})$  // public SM4 key, secret input block
8:    $\mathbf{pk} \leftarrow (\mathbf{x}, \mathbf{y})$  // public SM4 key, public output
9: else
10:  repeat
11:     $\mathbf{k} \leftarrow \{0, 1\}^{128}$  // secret SM4 key
12:  until  $\mathbf{k}[0] \wedge \mathbf{k}[1] = 0$ 
13:   $\mathbf{x} \leftarrow \{0, 1\}^{128}$  // public input block
14:   $\mathbf{y} \leftarrow \text{SM4}_{\mathbf{k}}(\mathbf{x})$ 
15:   $\mathbf{sk} \leftarrow (\mathbf{x}, \mathbf{k})$  // public input block, secret SM4 key
16:   $\mathbf{pk} \leftarrow (\mathbf{x}, \mathbf{y})$  // public input, public output
17: end if
18: return ( $\mathbf{sk}, \mathbf{pk}$ )
```

SM4th.Sign(msg, sk, pk; param)

INPUT: Message **msg**, secret key $\mathbf{sk} = (\mathbf{sk}_1, \mathbf{sk}_2)$, public key $\mathbf{pk} = (\mathbf{pk}_1, \mathbf{pk}_2)$, public parameters **param**OUTPUT: Signature σ

```
1:  $\text{ctr} \leftarrow 0$  // 32-bit int
2:  $\rho \leftarrow \{0, 1\}^\xi$  // Signature randomness
3:  $\mu \leftarrow \text{H}(\mathbf{pk} \parallel \text{msg}; 2\lambda)$ 
4:  $(r, \text{iv}^{\text{pre}}) \leftarrow \text{H}_3(\mathbf{sk}_2 \parallel \mu \parallel \rho) \in \{0, 1\}^{N_{\text{block}}} \times \{0, 1\}^{\lambda_{\text{iv}}}$ 
5:  $\text{iv} \leftarrow \text{H}_4(\text{iv}^{\text{pre}}, \lambda_{\text{iv}})$ 
6:  $\hat{\ell} \leftarrow \ell + d\lambda_{\mathbb{F}} + B$ 
7:  $(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(r, \text{iv}, \hat{\ell}; \text{param})$ 
8:  $\text{chall}_1 \leftarrow \text{H}_2^1(\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}; 5\lambda_{\mathbb{F}} + 64)$ 
9: // VOLE consistency check
10:  $\tilde{\mathbf{u}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{u}) \in \{0, 1\}^{\lambda_{\mathbb{F}} + B}$ 
11:  $\tilde{\mathbf{V}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{V}) \in \{0, 1\}^{(\lambda_{\mathbb{F}} + B) \times \lambda_{\mathbb{F}}}$ 
12: if EM = true then
13:    $\mathbf{w} \leftarrow \text{SM4th.EM.ExtendWitness}(\mathbf{sk}_2, \mathbf{sk}_1; \text{param}) \in \{0, 1\}^\ell$ 
14: else
15:    $\mathbf{w} \leftarrow \text{SM4th.ExtendWitness}(\mathbf{sk}_2, \mathbf{sk}_1; \text{param}) \in \{0, 1\}^\ell$ 
16: end if
17:  $\mathbf{d} \leftarrow \mathbf{w} \oplus \mathbf{u}_{[0, \ell]}$  // Mask extended witness
18:  $\text{chall}_2 \leftarrow \text{H}_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}} \parallel \mathbf{d}; 3\lambda_{\mathbb{F}} + 64)$ 
19: // QuickSilver proof
20:  $\mathbf{u}_{\text{zk}} \leftarrow \mathbf{u}_{[\ell \dots \ell + (d-1)\lambda_{\mathbb{F}}]}$ 
21:  $\mathbf{V}_{\text{zk}} \leftarrow \mathbf{V}_{[0 \dots \ell + 2\lambda_{\mathbb{F}}]} \in \{0, 1\}^{(\ell + 2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$ 
22:  $(\tilde{a}_0, \dots, \tilde{a}_{d-1}) \leftarrow \text{SM4th.OWFProve}(\mathbf{w}, \mathbf{u}_{\text{zk}}, \mathbf{V}_{\text{zk}}, \mathbf{pk}, \text{chall}_2; \text{param})$ 
23: repeat
24:    $\text{chall}_3 \leftarrow \text{H}_3^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \dots \parallel \tilde{a}_{d-1} \parallel \text{ctr}; \lambda_{\mathbb{F}})$ 
25:   if  $\text{chall}_3[\lambda_{\mathbb{F}} - w_{\text{grind}}, \lambda_{\mathbb{F}}] \neq 0^{w_{\text{grind}}}$  then
26:      $\text{ctr} \leftarrow \text{ctr} + 1$ 
27:   continue
28: end if
29:  $I \leftarrow \text{DecodeChall}(\text{chall}_3[0, \lambda_{\mathbb{F}} - w_{\text{grind}}]; \text{param})$ 
30:  $\text{decom}_I \leftarrow \text{BAVC.Open}(\text{decom}, I; \text{param})$ 
31: until  $\text{decom}_I \neq \perp$ 
32: return  $\sigma \leftarrow (\{c_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \dots, \tilde{a}_{d-1}, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$ 
```

Phase II: VOLE commitment and first challenge. Using (r, iv) , the signer invokes

$$(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(r, \text{iv}, \hat{\ell}; \text{param}),$$

where $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$. This gives a VOLE commitment, the corresponding decommitment information, and the correlated vectors required later in the proof. We have $d = 3$ for the non-EM variant SM4th and $d = 2$ for the EM variant SM4th-EM.

After that, the signer derives the first Fiat–Shamir challenge

$$\text{chall}_1 \leftarrow H_2^1(\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}; 5\lambda_{\mathbb{F}} + 64).$$

This challenge binds the entire VOLE commitment transcript to the message-dependent digest μ and will be used to compress the VOLE correlations.

Phase III: VOLE consistency check and second challenge. Given chall_1 , the signer performs the VOLE consistency check by hashing the correlated values:

$$\tilde{\mathbf{u}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{u}) \in \{0, 1\}^{\lambda_{\mathbb{F}} + B}, \quad \tilde{\mathbf{V}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{V}) \in \{0, 1\}^{(\lambda_{\mathbb{F}} + B) \times \lambda_{\mathbb{F}}}.$$

These hashed values serve as compressed consistency certificates for VOLE correlations.

Next, the signer derives the extended witness $\mathbf{w} \in \{0, 1\}^{\ell}$ from the secret key. If $\text{EM} = \text{true}$, it sets $\mathbf{w} \leftarrow \text{SM4th.EM.ExtendWitness}(\text{sk}_2, \text{sk}_1; \text{param})$, where sk_1 is the public SM4 key stored in the secret key and sk_2 is the secret SM4 input block. If $\text{EM} = \text{false}$, it sets $\mathbf{w} \leftarrow \text{SM4th.ExtendWitness}(\text{sk}_2, \text{sk}_1; \text{param})$, where sk_1 is the public input block and sk_2 is the secret SM4 key. It then masks the witness using the first ℓ bits of $\mathbf{u}$: $\mathbf{d} \leftarrow \mathbf{w} \oplus \mathbf{u}_{[0..\ell]}$.

Finally, the signer derives the second challenge

$$\text{chall}_2 \leftarrow H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}} \parallel \mathbf{d}; 3\lambda_{\mathbb{F}} + 64).$$

This challenge binds together the VOLE consistency check and the masked witness, and will be used as the challenge for the subsequent ZKP.

Phase IV: ZKP and opening challenge. To prepare the QuickSilver proof, the signer truncates the VOLE correlations to the portion actually needed by the proof system. For the SM4th proof implementation, $\mathbf{u}$ keeps $(d-1)\lambda_{\mathbb{F}}$ bits, while $\mathbf{V}$ keeps two ZK-mask blocks for both EM-d2 and non-EM-d3 variants: $\mathbf{u} \leftarrow \mathbf{u}_{[\ell..\ell+(d-1)\lambda_{\mathbb{F}}]}$ and $\mathbf{V} \leftarrow \mathbf{V}_{[0..\ell+2\lambda_{\mathbb{F}}]} \in \{0, 1\}^{(\ell+2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$. It then computes the proof components

$$(\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{d-1}) \leftarrow \text{SM4th.OWFProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2; \text{param}).$$

After the proof is generated, the signer derives the third challenge

$$\text{chall}_3 \leftarrow H_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \dots \parallel \tilde{a}_{d-1} \parallel \text{ctr}; \lambda_{\mathbb{F}})$$

inside a grinding loop, and checks whether its last w_{grind} bits are all zero. If this condition fails, the signer increments ctr and retries. Otherwise, it decodes $I \leftarrow \text{DecodeChall}(\text{chall}_3[0, \lambda_{\mathbb{F}} - w_{\text{grind}}]; \text{param})$, and computes the corresponding BAVC opening $\text{decom}_I \leftarrow \text{BAVC.Open}(\text{decom}, I; \text{param})$. If $\text{decom}_I = \perp$, the signer continues grinding until a valid opening is obtained.

Once successful, the signer outputs the signature

$$\sigma \leftarrow (\{\mathbf{c}_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \dots, \tilde{a}_{d-1}, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr}),$$

which consists of the partial VOLE commitment openings $\{\mathbf{c}_i\}_{i \in [1, \tau]}$, the hashed VOLE consistency value $\tilde{\mathbf{u}}$, the masked witness $\mathbf{d}$, the ZKP components from $\tilde{a}_1$ to $\tilde{a}_{d-1}$, the BAVC opening decom_I , the opening challenge chall_3 , the preimage iv^{pre} used to derive iv , and the grinding counter ctr .

Note that a conceptually simpler variant would include the full prover transcript, so that the verifier could recompute all Fiat–Shamir challenges directly. Instead, our construction transmits

only a compressed form of the commitment transcript. The omitted parts are not lost: they can be reconstructed from the selective-opening challenge chall_3 together with the decommitment information decom_I . Indeed, chall_3 determines which positions must be opened, while decom_I reveals exactly the seeds and auxiliary commitment data needed to reconstruct those openings. Consequently, the verifier can recover the missing commitment information, rebuild the same transcript, and recompute the corresponding challenges exactly as in the full-transcript variant. This yields a more compact signature without affecting soundness.

6.6.3 Verifying

The verification algorithm **SM4th.Verify** also proceeds in four conceptually separate phases.

Phase I: parsing and preliminary checks. On input a message msg , a public key pk , a signature

$$\sigma = (\{c_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \dots, \tilde{a}_{d-1}, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr}),$$

and the public parameters param , the verifier first recomputes $\mu \leftarrow H(\text{pk} \parallel \text{msg}; 2\lambda)$ and derives $\text{iv} \leftarrow H_4(\text{iv}^{\text{pre}}; \lambda_{\text{iv}})$. It then checks whether the grinding condition is satisfied, namely whether the last w_{grind} bits of chall_3 are all zero. If this condition fails, the verifier immediately rejects.

Phase II: VOLE reconstruction and first challenge. Next, the verifier reconstructs the committed VOLE transcript from the transmitted partial commitment information and the selective opening. Concretely, it invokes

$$(\text{com}, \mathbf{Q}) \leftarrow \text{VOLEReconstruct}(\text{decom}_I, \text{chall}_3[0, \lambda_{\mathbb{F}} - w_{\text{grind}}], c_1, \dots, c_{\tau-1}, \text{iv}; \text{param}).$$

If this reconstruction procedure returns $\perp$, the verifier rejects. Otherwise, it recomputes the first Fiat-Shamir challenge as

$$\text{chall}_1 \leftarrow H_2^1(\mu \parallel \text{com} \parallel c_1 \parallel \dots \parallel c_{\tau-1} \parallel \text{iv}; 5\lambda_{\mathbb{F}} + 64).$$

Thus, the verifier recovers exactly the same first challenge as in signing, provided that the reconstructed commitment transcript is consistent with the transmitted signature.

Phase III: VOLE consistency check and second challenge. Using chall_1 , the verifier computes the hashed VOLE value

$$\tilde{\mathbf{Q}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda_{\mathbb{F}} + B) \times \lambda_{\mathbb{F}}}.$$

It then parses chall_3 as a bit string $(\delta_0, \dots, \delta_{\lambda_{\mathbb{F}}-1}) \in \{0, 1\}^{\lambda_{\mathbb{F}}}$ and reconstructs the consistency matrix

$$\tilde{\mathbf{V}}' \leftarrow \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \parallel \dots \parallel \delta_{\lambda_{\mathbb{F}}-1} \cdot \tilde{\mathbf{u}}].$$

Intuitively, this step re-applies the VOLE correction values determined by chall_3 and the transmitted compressed consistency term $\tilde{\mathbf{u}}$. The verifier then derives the second challenge

$$\text{chall}_2 \leftarrow H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}}' \parallel \mathbf{d}; 3\lambda_{\mathbb{F}} + 64).$$

If the transmitted values are valid, the verifier reconstructs the same second challenge as the signer.

Phase IV: ZKP verification and transcript consistency. In the final phase, the verifier recomputes the missing proof component by running

$$\tilde{a}_0 \leftarrow \text{SM4th.OWFVerify}(\mathbf{d}, \mathbf{Q}_{[0, \ell+2\lambda_{\mathbb{F}}]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \dots, \tilde{a}_{d-1}, \text{pk}; \text{param}).$$

It then recomputes the final Fiat-Shamir challenge

$$\text{chall}_3' \leftarrow H_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \dots \parallel \tilde{a}_{d-1} \parallel \text{ctr}; \lambda_{\mathbb{F}}).$$

The verifier accepts if and only if $\text{chall}_3' = \text{chall}_3$; otherwise, it rejects.

SM4th.Verify(msg, pk, σ ; param)

INPUT: Message msg, public key pk, signature σ , public parameters param

OUTPUT: true or false

```
1: Parse  $\sigma = (\{c_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \dots, \tilde{a}_{d-1}, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$ 
2:  $\mu \leftarrow \text{H}(\text{pk} \parallel \text{msg}; 2\lambda)$ 
3:  $\text{iv} \leftarrow \text{H}_4(\text{iv}^{\text{pre}}; \lambda_{\text{iv}})$ 
4: if  $\text{chall}_3[\lambda_{\mathbb{F}} - w_{\text{grind}}, \lambda_{\mathbb{F}}] \neq 0^{w_{\text{grind}}}$  then
5:   return false
6: end if
7: // Reconstruct VOLEs and check commitment
8:  $(\text{com}, \mathbf{Q}) \leftarrow \text{VOLEReconstruct}(\text{decom}_I, \text{chall}_3[0, \lambda_{\mathbb{F}} - w_{\text{grind}}], c_1, \dots, c_{\tau-1}, \text{iv}; \text{param})$ 
9: if VOLEReconstruct returned  $\perp$  then
10:  return false
11: end if
12:  $\text{chall}_1 \leftarrow \text{H}_2^1(\mu \parallel \text{com} \parallel c_1 \parallel \dots \parallel c_{\tau-1} \parallel \text{iv}; 5\lambda_{\mathbb{F}} + 64)$ 
13: // Apply the VOLE correction values and check consistency
14:  $\tilde{\mathbf{Q}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda_{\mathbb{F}} + B) \times \lambda_{\mathbb{F}}}$ 
15: Parse  $\text{chall}_3 = (\delta_0, \dots, \delta_{\lambda_{\mathbb{F}}-1}) \in \{0, 1\}^{\lambda_{\mathbb{F}}}$ 
16:  $\tilde{\mathbf{V}}' \leftarrow \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda_{\mathbb{F}}-1} \cdot \tilde{\mathbf{u}}]$ 
17:  $\text{chall}_2 \leftarrow \text{H}_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}}' \parallel \mathbf{d}; 3\lambda_{\mathbb{F}} + 64)$ 
18: // Compute  $\tilde{a}_0$ 
19:  $\tilde{a}_0 \leftarrow \text{SM4th.OWFVerify}(\mathbf{d}, \mathbf{Q}_{[0, \ell+2\lambda_{\mathbb{F}}]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \dots, \tilde{a}_{d-1}, \text{pk}; \text{param})$ 
20: Compute  $\text{chall}'_3 \leftarrow \text{H}_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \dots \parallel \tilde{a}_{d-1} \parallel \text{ctr}; \lambda_{\mathbb{F}})$ 
21: return true if  $\text{chall}_3 = \text{chall}'_3$  else false
```

7 Algorithm Description: the uBlockith Signature Scheme

This section presents the whole construction of uBlockith. For uBlockith, the proof field size is $\lambda_{\mathbb{F}} = 256$, which coincides with the classical security parameter λ . Nevertheless, we write the VOLE and QuickSilver operations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ to keep the notation consistent with the parameter separation used throughout the paper.

7.1 Zero-Knowledge Building Blocks for uBlockith

We give several zero-knowledge building blocks in this section.

VOLESubWord and VOLEInvSubWord. Given a vector of degree-1 VOLE commitments for witness $\mathbf{x} \in \{0, 1\}^{4n}$, the **uBlockith.VOLESubWord** algorithm computes the degree-3 VOLE commitments of **SubDWord**($\mathbf{x}$) or **SubQWord**($\mathbf{x}$), depending on $n = 16$ or $n = 32$. Given a vector of degree-1 VOLE commitments for witness $\mathbf{y} \in \{0, 1\}^{4n}$, the **uBlockith.VOLEInvSubWord** algorithm computes the degree-3 VOLE commitments of **SubDWord** $^{-1}(\mathbf{y})$ or **SubQWord** $^{-1}(\mathbf{y})$, depending on $n = 16$ or $n = 32$. The degree increases to 3 because the Boolean ANF of the uBlock S-box and inverse S-box contains cubic monomials.

The VOLE.T function. Given a vector of degree-1 VOLE commitments for witness $\mathbf{x} \in \{0, 1\}^{64}$, the **uBlockith.VOLE.T** algorithm computes degree-1 VOLE commitments for $T(\mathbf{x})$, where the function $T(\cdot)$ multiplies each element in $\mathbf{x}$ by 2 in $\text{GF}(2^4)$. The concrete definition of $T(\cdot)$ is described in Equation (17) and Equation (18). Since T is $\mathbb{F}_2$ -linear, it does not increase the VOLE commitment degree.

7.2 Witness Extension

The algorithm **uBlockith.ExtendWitness** takes as input the uBlock key $\mathbf{k}$, the uBlock plaintext $\mathbf{in} \in \{0, 1\}^{N_{\text{block}}}$, and the instance parameters param, and outputs an extended witness $\mathbf{w} \in \{0, 1\}^{\ell}$ for later use in the proof. For uBlockith, the uBlock block or key length is $N_{\text{block}} = 256$, which

uBlockith.VOLESubWord($\langle \mathbf{x} \rangle, n$)

INPUT: $\langle \mathbf{x} \rangle$: degree-1 commitments to $\mathbf{x} \in \{0, 1\}^{4n}$ OUTPUT: $\langle \mathbf{y} \rangle^3$: degree-3 commitments to $\mathbf{y} \in \{0, 1\}^{4n}$

```
1: Parse  $\langle \mathbf{x} \rangle = (\langle \mathbf{x}[0] \rangle, \dots, \langle \mathbf{x}[n-1] \rangle)$ 
2: new  $(\langle \mathbf{y}[0] \rangle^3, \dots, \langle \mathbf{y}[n-1] \rangle^3) \in (\mathbb{F}_{2^{\lambda_F}}^4)^n$ 
3: for  $i \in [0, n)$  do
4:   new  $(\langle y_0 \rangle^3, \langle y_1 \rangle^3, \langle y_2 \rangle^3, \langle y_3 \rangle^3) \in \mathbb{F}_{2^{\lambda_F}}^4$ 
5:   Parse  $\langle \mathbf{x}[i] \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$ 
6:    $\langle y_0 \rangle^3 \leftarrow \text{ConstantToVOLE}(1) + \langle x_0 \rangle + \langle x_2 \rangle \cdot \langle x_3 \rangle$ 
7:    $\langle y_1 \rangle^3 \leftarrow \text{ConstantToVOLE}(1) + \langle x_0 \rangle + \langle x_1 \rangle + \langle x_0 \rangle \cdot \langle x_1 \rangle + \langle x_0 \rangle \cdot \langle x_2 \rangle + \langle x_0 \rangle \cdot \langle x_1 \rangle \cdot \langle x_2 \rangle + \langle x_0 \rangle \cdot \langle x_3 \rangle + \langle x_2 \rangle \cdot \langle x_3 \rangle$ 
8:    $\langle y_2 \rangle^3 \leftarrow \text{ConstantToVOLE}(1) + \langle x_1 \rangle + \langle x_0 \rangle \cdot \langle x_1 \rangle + \langle x_2 \rangle + \langle x_1 \rangle \cdot \langle x_2 \rangle \cdot \langle x_3 \rangle$ 
9:    $\langle y_3 \rangle^3 \leftarrow \langle x_1 \rangle + \langle x_2 \rangle + \langle x_3 \rangle + \langle x_1 \rangle \cdot \langle x_2 \rangle$ 
10:   $\langle \mathbf{y}[i] \rangle^3 \leftarrow (\langle y_0 \rangle^3, \langle y_1 \rangle^3, \langle y_2 \rangle^3, \langle y_3 \rangle^3)$ 
11: end for
12: return  $(\langle \mathbf{y}[0] \rangle^3, \dots, \langle \mathbf{y}[n-1] \rangle^3)$ 
```

uBlockith.VOLEInvSubWord($\langle \mathbf{y} \rangle, n$)

INPUT: $\langle \mathbf{y} \rangle$: degree-1 commitments to $\mathbf{y} \in \{0, 1\}^{4n}$ OUTPUT: $\langle \mathbf{z} \rangle^3$: degree-3 commitments to $\mathbf{z} \in \{0, 1\}^{4n}$ such that $\mathbf{z} = \text{SubWord}^{-1}(\mathbf{y})$

```
1: Parse  $\langle \mathbf{y} \rangle = (\langle \mathbf{y}[0] \rangle, \dots, \langle \mathbf{y}[n-1] \rangle)$ 
2: new  $(\langle \mathbf{z}[0] \rangle^3, \dots, \langle \mathbf{z}[n-1] \rangle^3) \in (\mathbb{F}_{2^{\lambda_F}}^4)^n$ 
3: for  $i \in [0, n)$  do
4:   new  $(\langle z_0 \rangle^3, \langle z_1 \rangle^3, \langle z_2 \rangle^3, \langle z_3 \rangle^3) \in \mathbb{F}_{2^{\lambda_F}}^4$ 
5:   Parse  $\langle \mathbf{y}[i] \rangle = (\langle y_0 \rangle, \langle y_1 \rangle, \langle y_2 \rangle, \langle y_3 \rangle)$ 
6:    $\langle z_0 \rangle^3 \leftarrow \langle y_0 \rangle \cdot \langle y_1 \rangle + \langle y_2 \rangle + \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_3 \rangle + \langle y_2 \rangle \cdot \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_1 \rangle \cdot \langle y_3 \rangle$ 
7:    $\langle z_1 \rangle^3 \leftarrow \langle y_0 \rangle + \langle y_1 \rangle + \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_3 \rangle$ 
8:    $\langle z_2 \rangle^3 \leftarrow \text{ConstantToVOLE}(1) + \langle y_0 \rangle + \langle y_2 \rangle + \langle y_0 \rangle \cdot \langle y_1 \rangle$ 
9:    $\langle z_3 \rangle^3 \leftarrow \text{ConstantToVOLE}(1) + \langle y_2 \rangle + \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_2 \rangle + \langle y_1 \rangle \cdot \langle y_2 \rangle + \langle y_2 \rangle \cdot \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_2 \rangle \cdot \langle y_3 \rangle$ 
10:   $\langle \mathbf{z}[i] \rangle^3 \leftarrow (\langle z_0 \rangle^3, \langle z_1 \rangle^3, \langle z_2 \rangle^3, \langle z_3 \rangle^3)$ 
11: end for
12: return  $(\langle \mathbf{z}[0] \rangle^3, \dots, \langle \mathbf{z}[n-1] \rangle^3)$ 
```

uBlockith.VOLE.T($\langle \mathbf{x} \rangle$)

INPUT: $\langle \mathbf{x} \rangle$: degree-1 commitments to $\mathbf{x} \in \{0, 1\}^{64}$ OUTPUT: $\langle \mathbf{y} \rangle$: degree-1 commitments to $\mathbf{y} \in \{0, 1\}^{64}$

```
1: Parse  $\langle \mathbf{x} \rangle = (\langle \mathbf{x}[0] \rangle, \dots, \langle \mathbf{x}[15] \rangle)$ 
2: new  $(\langle \mathbf{y}[0] \rangle, \dots, \langle \mathbf{y}[15] \rangle) \in (\mathbb{F}_{2^{\lambda_F}}^4)^{16}$ 
3: for  $i \in [0, 16)$  do
4:   new  $(\langle y_0 \rangle, \langle y_1 \rangle, \langle y_2 \rangle, \langle y_3 \rangle) \in \mathbb{F}_{2^{\lambda_F}}^4$ 
5:   Parse  $\langle \mathbf{x}[i] \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$ 
6:    $\langle y_0 \rangle \leftarrow \langle x_1 \rangle$ 
7:    $\langle y_1 \rangle \leftarrow \langle x_2 \rangle$ 
8:    $\langle y_2 \rangle \leftarrow \langle x_0 \rangle + \langle x_3 \rangle$ 
9:    $\langle y_3 \rangle \leftarrow \langle x_0 \rangle$ 
10:   $\langle \mathbf{y}[i] \rangle \leftarrow (\langle y_0 \rangle, \langle y_1 \rangle, \langle y_2 \rangle, \langle y_3 \rangle)$ 
11: end for
12: return  $(\langle \mathbf{y}[0] \rangle, \dots, \langle \mathbf{y}[15] \rangle)$ 
```

equals to the block or key length used in the Ballet PRGs. Hence, we use N_{block} for the sizes of both primitives for ease of notation.

The construction of $\mathbf{w}$ consists of three main parts: the initial phase, the key expansion phase, and the encryption phase.

- **Initial bits** (`ini_bits`): the first N_{block} bits obtained from the initial key for the Non-EM variant or from the plaintext for the EM variant.
- **Expanded key bits** (`exp_key_bits`): for each round of the Non-EM variant, the algorithm appends the first $N_{\text{block}}/4$ bits of the expanded key to the extended witness, resulting in $N_{\text{block}}/4 \cdot R$ bits. For the EM variant, this part is the empty string and contributes 0 bits, since the round keys are derived from the public key.
- **Encryption bits** (`enc_bits`): the intermediate encryption state pairs $(\mathbf{x}_0, \mathbf{x}_1)$ are stored after every two rounds, except for the final boundary state. Here $\mathbf{x}_0, \mathbf{x}_1 \in \{0, 1\}^{N_{\text{block}}/2}$, so each stored pair has total length N_{block} bits. This contributes $N_{\text{block}} \cdot (R/2 - 1)$ bits for both the Non-EM and EM variants.

Therefore, the extended witness can be written as:

$$\mathbf{w} = \underbrace{\text{ini_bits}}_{N_{\text{block}}} \parallel \underbrace{\text{exp_key_bits}}_{N_{\text{block}}/4 \cdot R} \parallel \underbrace{\text{enc_bits}}_{N_{\text{block}} \cdot (R/2 - 1)}.$$

For the EM variant, the middle component `exp_key_bits` has length 0.

7.3 Deriving Constraints for the Key Expansion

The algorithm `uBlockith.KeyExpCstrnts` derives the constraint values required for the QuickSilver proof for the uBlock circuit corresponding to the key expansion routine. In addition, it simultaneously computes the degree-1 VOLE commitments to all round keys.

The algorithm starts by parsing the initial round key $\langle \bar{\mathbf{k}}[0] \rangle = (\langle \mathbf{k}_0^{(0)} \rangle, \langle \mathbf{k}_1^{(0)} \rangle, \langle \mathbf{k}_2^{(0)} \rangle, \langle \mathbf{k}_3^{(0)} \rangle)$ from the first 256 bits of the committed witness $\langle \mathbf{w} \rangle$. Then, for each round i , it computes two temporary committed words $(\langle \mathbf{t}_0 \rangle, \langle \mathbf{t}_1 \rangle)$ by applying the linear map $PK(\cdot)$ to $(\langle \mathbf{k}_0^{(i)} \rangle, \langle \mathbf{k}_1^{(i)} \rangle)$. Using these values, the algorithm derives the non-linear intermediate value $\langle \mathbf{t}_2 \rangle^3 \leftarrow \langle \mathbf{k}_2^{(i)} \rangle + \text{uBlockith.VOLESubWord}(\langle \mathbf{t}_0 \rangle + \text{ConstantToVOLE}(RC_i), 16)$, as well as the linear update $\langle \mathbf{t}_3 \rangle \leftarrow \langle \mathbf{k}_3^{(i)} \rangle + \text{uBlockith.VOLE.T}(\langle \mathbf{t}_1 \rangle)$. The next-round word $\langle \mathbf{k}_0^{(i+1)} \rangle$ is explicitly parsed from the committed witness $\langle \mathbf{w} \rangle$, rather than being carried forward as $\langle \mathbf{t}_2 \rangle^3$. This keeps the round-key commitments themselves degree-1. The consistency between the witness-provided value and the key expansion logic is enforced by the degree-3 constraint

$$\langle \mathbf{o}[64i \dots 64i + 63] \rangle^3 \leftarrow \langle \mathbf{k}_0^{(i+1)} \rangle + \langle \mathbf{t}_2 \rangle^3,$$

where the degree-1 commitment $\langle \mathbf{k}_0^{(i+1)} \rangle$ is naturally lifted to degree 3 before addition. Finally, the next round key commitment is assembled as

$$\langle \bar{\mathbf{k}}[i + 1] \rangle = (\langle \mathbf{k}_0^{(i+1)} \rangle, \langle \mathbf{t}_3 \rangle, \langle \mathbf{t}_1 \rangle, \langle \mathbf{t}_0 \rangle).$$

As a result, all round-key commitments remain degree-1, while all non-linear relations are isolated into the degree-3 constraint vector $\langle \mathbf{o} \rangle^3$, which is later verified within the QuickSilver proof system.

7.4 Deriving Constraints for the Encryption

The algorithm `uBlockith.EncCstrnts` derives the degree-3 constraint values required by the QuickSilver proof for the uBlock encryption circuit. Given VOLE commitments to the plaintext, ciphertext, witness, and expanded round keys, it first augments the encryption witness with VOLE commitments to the public input and output. It then processes the encryption two rounds at a time. For each even round index i , let $j = i/2$. The algorithm parses the committed state pair $(\langle \mathbf{x}_0 \rangle, \langle \mathbf{x}_1 \rangle)$ from the j -th stored state and loads the corresponding round key from $\langle \bar{\mathbf{k}}[j] \rangle$.

uBlockith.ExtendWitness($\mathbf{k}, \mathbf{in}; \text{param}$)

INPUT: $\mathbf{k} \in \{0, 1\}^{N_{\text{block}}}$, $\mathbf{in} \in \{0, 1\}^{N_{\text{block}}}$ OUTPUT: $\mathbf{w} \in \{0, 1\}^\ell$: extended witness

```
1: // Initialize empty extended witness array, eventually of length  $\ell$ 
2: new  $\mathbf{w} \in \{0, 1\}^{\leq \ell}$ 
3: // Store round keys as array of  $N_{\text{block}}$ -bit words
4: new  $\bar{\mathbf{k}} \in [\{0, 1\}^{N_{\text{block}}}; R + 1]$ 
5:  $\bar{\mathbf{k}} \leftarrow \text{uBlock.KeyExpansion}(\mathbf{k}, \{RC_i\}_{i \in [R]}, PK(\cdot), \text{param})$ 
6: if EM = false then // extract key schedule witness components
7:    $\mathbf{w} \leftarrow \mathbf{w} \parallel \bar{\mathbf{k}}[0]$  // Saving the initial  $N_{\text{block}}$ -bit key word
8:   for  $i \in [1..R]$  do
9:      $\mathbf{w} \leftarrow \mathbf{w} \parallel \bar{\mathbf{k}}[i][0..N_{\text{block}}/4 - 1]$  // Saving the first  $N_{\text{block}}/4$  bits of each expanded key word
10:   end for
11: else
12:    $\mathbf{w} \leftarrow \mathbf{w} \parallel \mathbf{in}$  // For EM,  $\mathbf{in}$  is secret
13: end if
14: // Encryption routine and saving witness bits
15: new  $\mathbf{x}_0 \in \{0, 1\}^{N_{\text{block}}/2}$ , new  $\mathbf{x}_1 \in \{0, 1\}^{N_{\text{block}}/2}$ 
16:  $\mathbf{x}_0 \leftarrow \mathbf{in}[0..N_{\text{block}}/2 - 1]$ ,  $\mathbf{x}_1 \leftarrow \mathbf{in}[N_{\text{block}}/2..N_{\text{block}} - 1]$ 
17: for  $i \in [0..R - 1]$  do
18:   Parse  $\bar{\mathbf{k}}[i] = (\mathbf{k}_0, \mathbf{k}_1)$  with  $\mathbf{k}_0, \mathbf{k}_1 \in \{0, 1\}^{N_{\text{block}}/2}$ 
19:    $\mathbf{x}_0 \leftarrow \text{SubQWord}(\mathbf{x}_0 \oplus \mathbf{k}_0)$  // defined in Equation (15)
20:    $\mathbf{x}_1 \leftarrow \text{SubQWord}(\mathbf{x}_1 \oplus \mathbf{k}_1)$ 
21:    $\mathbf{x}_1 \leftarrow \mathbf{x}_1 \oplus \mathbf{x}_0$ 
22:    $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \oplus (\mathbf{x}_1 \lll 4)_{32}$  // 32-bit word-wise cyclic left shifts
23:    $\mathbf{x}_1 \leftarrow \mathbf{x}_1 \oplus (\mathbf{x}_0 \lll 8)_{32}$ 
24:    $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \oplus (\mathbf{x}_1 \lll 8)_{32}$ 
25:    $\mathbf{x}_1 \leftarrow \mathbf{x}_1 \oplus (\mathbf{x}_0 \lll 20)_{32}$ 
26:    $\mathbf{x}_0 \leftarrow \mathbf{x}_0 \oplus \mathbf{x}_1$ 
27:    $\mathbf{x}_0 \leftarrow PL(\mathbf{x}_0)$  // defined in Table 7
28:    $\mathbf{x}_1 \leftarrow PR(\mathbf{x}_1)$  // defined in Table 7
29:   if  $i \bmod 2 = 1 \wedge i < R - 1$  then
30:      $\mathbf{w} \leftarrow \mathbf{w} \parallel \mathbf{x}_0 \parallel \mathbf{x}_1$  // Saving one  $N_{\text{block}}$ -bit state pair every two rounds
31:   end if
32: end for
33: return  $\mathbf{w} \in \{0, 1\}^\ell$ 
```

uBlockith.KeyExpCstrnts($\langle \mathbf{w} \rangle; \text{param}$)

INPUT: Commitments to the witness $\langle \mathbf{w} \rangle$ where $\mathbf{w} \in \{0, 1\}^{\ell_{\text{ke}}}$ OUTPUT: $\langle \bar{\mathbf{k}} \rangle$, where $\bar{\mathbf{k}} \in (\{0, 1\}^{64}, \{0, 1\}^{64}, \{0, 1\}^{64}, \{0, 1\}^{64})^{R+1}$; $\langle \mathbf{o} \rangle_{\lambda_{\mathbb{F}}}^3$ for the constraint vector $\mathbf{o} \in \mathbb{F}^{4S_{\text{ke}}}$

```
1: new  $\langle \mathbf{t}_0 \rangle, \langle \mathbf{t}_1 \rangle, \langle \mathbf{t}_2 \rangle^3, \langle \mathbf{t}_3 \rangle$  // temporary commitments to keys
2: parse  $\langle \bar{\mathbf{k}}[0] \rangle = (\langle \mathbf{k}_0^{(0)} \rangle, \langle \mathbf{k}_1^{(0)} \rangle, \langle \mathbf{k}_2^{(0)} \rangle, \langle \mathbf{k}_3^{(0)} \rangle)$  from  $\langle \mathbf{w}[0..255] \rangle$ 
3: for  $i \in [0..R - 1]$  do
4:    $(\langle \mathbf{t}_0 \rangle, \langle \mathbf{t}_1 \rangle) \leftarrow PK(\langle \mathbf{k}_0^{(i)} \rangle, \langle \mathbf{k}_1^{(i)} \rangle)$  //  $PK(\cdot)$  is defined in Equation (16) and can be naturally extended to VOLE commitments
5:    $\langle \mathbf{t}_2 \rangle^3 \leftarrow \langle \mathbf{k}_2^{(i)} \rangle + \text{uBlockith.VOLESubWord}(\langle \mathbf{t}_0 \rangle + \text{ConstantToVOLE}(RC_i), 16)$ 
6:    $\langle \mathbf{t}_3 \rangle \leftarrow \langle \mathbf{k}_3^{(i)} \rangle + \text{uBlockith.VOLE.T}(\langle \mathbf{t}_1 \rangle)$ 
7:   parse  $\langle \mathbf{k}_0^{(i+1)} \rangle \leftarrow \langle \mathbf{w}[64 \cdot i + 256..64 \cdot i + 319] \rangle$ 
8:    $\langle \mathbf{o}[64 \cdot i..64 \cdot i + 63] \rangle^3 \leftarrow \langle \mathbf{k}_0^{(i+1)} \rangle + \langle \mathbf{t}_2 \rangle^3$  // lift  $\langle \mathbf{k}_0^{(i+1)} \rangle$  to degree 3
9:    $\langle \bar{\mathbf{k}}[i + 1] \rangle \leftarrow (\langle \mathbf{k}_0^{(i+1)} \rangle, \langle \mathbf{t}_3 \rangle, \langle \mathbf{t}_1 \rangle, \langle \mathbf{t}_0 \rangle)$ 
10: end for
11: return  $(\langle \bar{\mathbf{k}} \rangle, \langle \mathbf{o} \rangle^3)$ 
```

From this state, the algorithm evaluates the forward round computation on VOLE commitments: it applies `uBlockith.VOLESubWord` to the key-added state, followed by the prescribed linear operations, rotations, and permutations, and then adds $\langle \bar{k}[i+1] \rangle$ to obtain $\langle y_0 \rangle^3, \langle y_1 \rangle^3$, which represent the forward-derived inputs to the S-boxes of round $i+1$. To enforce consistency, the algorithm also derives a backward representation $\langle z_0 \rangle^3, \langle z_1 \rangle^3$ from the next committed state by inverting the linear layer and applying `uBlockith.VOLEInvSubWord`. In the final iteration, this next state is reconstructed from the ciphertext after removing the last round key.

Finally, the algorithm appends $\langle y_0 \rangle^3 + \langle z_0 \rangle^3$ and $\langle y_1 \rangle^3 + \langle z_1 \rangle^3$ to the constraint vector $\langle o \rangle^3$. Hence, $\langle o \rangle^3$ collects all bit-level constraints ensuring that the committed intermediate states, expanded keys, and public input/output together describe a valid execution of the uBlock encryption routine. The VOLE tags associated with these commitments are elements of the proof field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$.

7.5 Constraints for the OWF

This section describes the remaining steps needed to prove that $\mathbf{pk} = (\mathbf{pk}_1, \mathbf{pk}_2)$ is consistent with the witness $\mathbf{w}$ under the uBlock-based one-way function. We have three algorithms:

1. `uBlockith.OWFConstraints` that combines both the key expansion and encryption constraints;
2. the signer-side procedure `uBlockith.OWFProve`; and
3. the verifier-side procedure `uBlockith.OWFVerify`.

`uBlockith.OWFConstraints`. This is the main algorithm that combines all sub-constraints from the key-schedule and encryption routines. Concretely:

1. It parses the public key $\mathbf{pk}$ as $(\mathbf{pk}_1, \mathbf{pk}_2)$, where both components are N_{block} -bit strings, and allocates a new $\langle z \rangle^3$ for the accumulated VOLE commitments of bit-level constraints.
2. It appends the secret-witness keyspace-reduction constraint $\langle w[0] \rangle \cdot \langle w[1] \rangle = 0$, viewed as a degree-3 constraint. In the EM variant, these are the first two bits of the secret input block. In the non-EM variant, they are the first two bits of the secret uBlock key.
3. If $\text{EM} = \text{true}$, then $\mathbf{pk}_1$ is interpreted as the public uBlock key. It runs the key expansion algorithm `uBlock.KeyExpansion` on $\mathbf{pk}_1$ and obtains public round keys, which are converted to VOLE commitments by `ConstantToVOLE`. In this case, the first N_{block} bits of the witness are the secret uBlock input block, and the OWF relation is $\mathbf{pk}_2 = E_{\mathbf{pk}_1}(\mathbf{in}) \oplus \mathbf{in}$. Therefore, the committed encryption input is $\langle \mathbf{in} \rangle = \langle w[0..N_{\text{block}}-1] \rangle$, while the committed block-cipher output is $\langle \mathbf{out} \rangle = \langle w[0..N_{\text{block}}-1] \rangle + \text{ConstantToVOLE}(\mathbf{pk}_2)$.
4. If $\text{EM} = \text{false}$, then $\mathbf{pk}_1$ and $\mathbf{pk}_2$ are respectively the public uBlock input and output blocks. The algorithm obtains $\langle \mathbf{in} \rangle$ and $\langle \mathbf{out} \rangle$ by applying `ConstantToVOLE` to $\mathbf{pk}_1$ and $\mathbf{pk}_2$. It also calls `uBlockith.KeyExpCstrnts`($\langle w[0..\ell_{\text{ke}}-1] \rangle$; param) to obtain the round-key commitments $\langle \bar{k} \rangle$ and the key-schedule constraint vector $\langle \bar{z} \rangle^3$. These constraints are appended to the main constraint vector $\langle z \rangle^3$.
5. Next, it extracts the encryption part of the witness and calls `uBlockith.EncCstrnts` to produce the encryption constraints. In the non-EM variant, this slice starts after the key-expansion witness. In the EM variant, since $\ell_{\text{ke}} = 0$, this slice contains the secret initial block as part of the encryption witness layout. The resulting constraints are appended to $\langle z \rangle^3$.
6. Finally, the algorithm returns $\langle z \rangle^3$.

`uBlockith.OWFProve`. The `uBlockith.OWFProve` algorithm describes the signer's procedure. It first converts each witness bit w_i into a VOLE commitment using the corresponding row of $\mathbf{V}$ as the tag. It also parses $\mathbf{u}$ and the last $2\lambda_{\mathbb{F}}$ rows of $\mathbf{V}$ to obtain mask elements u_0^*, u_1^* and v_0^*, v_1^* in the proof field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Then it invokes `uBlockith.OWFConstraints`($\langle \mathbf{w} \rangle, \mathbf{pk}$), which returns a degree-3 polynomial vector $\langle z \rangle^3$. The prover parses the coefficient vectors as $(\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2) \in (\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^C)^3$ and computes the QuickSilver responses by hashing them with the corresponding mask terms via `ZKHash`.

`uBlockith.OWFVerify`. The `uBlockith.OWFVerify` algorithm describes the corresponding verifier procedure. It reconstructs the verifier-side VOLE values from $\mathbf{d}, \mathbf{Q}$, and the challenge $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. It then recomputes the constraints using `uBlockith.OWFConstraints`($\langle \mathbf{w} \rangle, \mathbf{pk}$) and obtains the evaluated constraint vector $\mathbf{b} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^C$. Finally, it computes $\tilde{q} = \text{ZKHash}(\text{chall}_2, \mathbf{b}||q^*)$ and returns $\tilde{q} + \tilde{a}_1\Delta + \tilde{a}_2\Delta^2$, which should equal the prover's $\tilde{a}_0$ for a valid proof.

uBlockith.EncCstrnts($\langle in \rangle, \langle out \rangle, \langle \bar{k} \rangle, \langle w \rangle; \text{param}$)

INPUT: Commitments to uBlock plaintext $in \in \{0, 1\}^{N_{\text{block}}}$, uBlock ciphertext $out \in \{0, 1\}^{N_{\text{block}}}$, witness $w \in \{0, 1\}^{\ell_{\text{enc}}}$, and expanded keys $\bar{k} \in (\{0, 1\}^{N_{\text{block}}})^{R+1}$

OUTPUT: $\langle o \rangle^3$: constraint vector $o \in \mathbb{F}_2^{C_{\text{enc}}}$

```

1: new  $\langle o \rangle^3$ 
2: new  $\langle x_0 \rangle, \langle x_1 \rangle \in (\mathbb{F}_2)^{N_{\text{block}}/2}$  // State of the encryption routine
3: new  $\langle y_0 \rangle^3, \langle y_1 \rangle^3 \in (\mathbb{F}_2)^{N_{\text{block}}/2}$  // Forward-derived input side of S-boxes
4: new  $\langle z_0 \rangle^3, \langle z_1 \rangle^3 \in (\mathbb{F}_2)^{N_{\text{block}}/2}$  // Backward-derived input side of S-boxes
5: new  $\langle k_0 \rangle, \langle k_1 \rangle \in (\mathbb{F}_2)^{N_{\text{block}}/2}$  // Expanded key halves
6:  $\langle w \rangle^* \leftarrow \langle in \rangle \parallel \langle w \rangle \parallel \langle out \rangle$ 
7: for  $i \in [0..R-2]$  with  $i \bmod 2 = 0$  do
8:    $j \leftarrow i/2$ 
9:    $(\langle x_0 \rangle, \langle x_1 \rangle) \leftarrow (\langle w^*[jN_{\text{block}}..jN_{\text{block}} + N_{\text{block}}/2 - 1] \rangle, \langle w^*[jN_{\text{block}} + N_{\text{block}}/2..(j+1)N_{\text{block}} - 1] \rangle)$ 
10:   $(\langle k_0 \rangle, \langle k_1 \rangle) \leftarrow \langle \bar{k}[i] \rangle$ 
11:   $\langle x_0 \rangle^3 \leftarrow \text{uBlockith.VOLESubWord}(\langle x_0 \rangle + \langle k_0 \rangle, 32)$ 
12:   $\langle x_1 \rangle^3 \leftarrow \text{uBlockith.VOLESubWord}(\langle x_1 \rangle + \langle k_1 \rangle, 32)$ 
13:   $\langle x_1 \rangle^3 \leftarrow \langle x_1 \rangle^3 + \langle x_0 \rangle^3$ 
14:   $\langle x_0 \rangle^3 \leftarrow \langle x_0 \rangle^3 + (\langle x_1 \rangle^3 \lll 4)_{32}$ 
15:   $\langle x_1 \rangle^3 \leftarrow \langle x_1 \rangle^3 + (\langle x_0 \rangle^3 \lll 8)_{32}$ 
16:   $\langle x_0 \rangle^3 \leftarrow \langle x_0 \rangle^3 + (\langle x_1 \rangle^3 \lll 8)_{32}$ 
17:   $\langle x_1 \rangle^3 \leftarrow \langle x_1 \rangle^3 + (\langle x_0 \rangle^3 \lll 20)_{32}$ 
18:   $\langle x_0 \rangle^3 \leftarrow \langle x_0 \rangle^3 + \langle x_1 \rangle^3$ 
19:   $\langle x_0 \rangle^3 \leftarrow PL(\langle x_0 \rangle^3)$  //  $PL(\cdot)$  is defined in Table 7, which can be naturally extended to VOLE
    commitments for bits
20:   $\langle x_1 \rangle^3 \leftarrow PR(\langle x_1 \rangle^3)$  //  $PR(\cdot)$  is defined in Table 7, which can be naturally extended to VOLE
    commitments for bits
21:   $(\langle k_0 \rangle, \langle k_1 \rangle) \leftarrow \langle \bar{k}[i+1] \rangle$ 
22:   $\langle y_0 \rangle^3 \leftarrow \langle x_0 \rangle^3 + \langle k_0 \rangle$  // compute the  $i+1$ -th round first inputs of S-boxes
23:   $\langle y_1 \rangle^3 \leftarrow \langle x_1 \rangle^3 + \langle k_1 \rangle$  // compute the  $i+1$ -th round second inputs of S-boxes
24:  if  $i = R-2$  then
25:     $(\langle k_0 \rangle, \langle k_1 \rangle) \leftarrow \langle \bar{k}[i+2] \rangle$ 
26:     $\langle x_0 \rangle \leftarrow \langle w^*[(j+1)N_{\text{block}}..(j+1)N_{\text{block}} + N_{\text{block}}/2 - 1] \rangle + \langle k_0 \rangle$  //  $out = \bar{k}[R] \oplus (x_0, x_1)$ 
27:     $\langle x_1 \rangle \leftarrow \langle w^*[(j+1)N_{\text{block}} + N_{\text{block}}/2..(j+2)N_{\text{block}} - 1] \rangle + \langle k_1 \rangle$  //  $out = \bar{k}[R] \oplus (x_0, x_1)$ 
28:  else
29:     $(\langle x_0 \rangle, \langle x_1 \rangle) \leftarrow (\langle w^*[(j+1)N_{\text{block}}..(j+1)N_{\text{block}} + N_{\text{block}}/2 - 1] \rangle, \langle w^*[(j+1)N_{\text{block}} + N_{\text{block}}/2..(j+2)N_{\text{block}} - 1] \rangle)$ 
30:  end if
31:   $\langle x_1 \rangle \leftarrow PR^{-1}(\langle x_1 \rangle)$ 
32:   $\langle x_0 \rangle \leftarrow PL^{-1}(\langle x_0 \rangle)$ 
33:   $\langle x_0 \rangle \leftarrow \langle x_0 \rangle + \langle x_1 \rangle$ 
34:   $\langle x_1 \rangle \leftarrow \langle x_1 \rangle + (\langle x_0 \rangle \lll 20)_{32}$ 
35:   $\langle x_0 \rangle \leftarrow \langle x_0 \rangle + (\langle x_1 \rangle \lll 8)_{32}$ 
36:   $\langle x_1 \rangle \leftarrow \langle x_1 \rangle + (\langle x_0 \rangle \lll 8)_{32}$ 
37:   $\langle x_0 \rangle \leftarrow \langle x_0 \rangle + (\langle x_1 \rangle \lll 4)_{32}$ 
38:   $\langle x_1 \rangle \leftarrow \langle x_1 \rangle + \langle x_0 \rangle$ 
39:   $\langle z_0 \rangle^3 \leftarrow \text{uBlockith.VOLEInvSubWord}(\langle x_0 \rangle, 32)$ 
40:   $\langle z_1 \rangle^3 \leftarrow \text{uBlockith.VOLEInvSubWord}(\langle x_1 \rangle, 32)$ 
41:   $\langle o \rangle^3 \leftarrow \langle o \rangle^3 \parallel \langle y_0 \rangle^3 + \langle z_0 \rangle^3 \parallel \langle y_1 \rangle^3 + \langle z_1 \rangle^3$ 
42: end for
43: return  $\langle o \rangle^3$ 

```

uBlockith.OWFConstraints($\langle w \rangle$, pk; param)

INPUT: Committed witness $\langle w \rangle$ for $w \in \mathbb{F}_2^\ell$
OUTPUT: Committed constraints $\langle z \rangle^3$, for $z \in \mathbb{F}_2^C$

- 1: Parse $\text{pk} = (\text{pk}_1, \text{pk}_2) \in \{0, 1\}^{N_{\text{block}}} \times \{0, 1\}^{N_{\text{block}}}$
- 2: // Allocate space for constraint commitments
- 3: **new** $\langle z \rangle^3$, $z \in \mathbb{F}_2^{\leq C}$
- 4: $\langle z \rangle^3 \leftarrow \langle z \rangle^3 \parallel \langle w[0] \rangle \cdot \langle w[1] \rangle$ // secret-witness keyspace reduction
- 5: // Define committed input and output blocks for the OWF relation
- 6: **if** EM = true **then**
- 7: $\bar{k} \leftarrow \text{uBlockith.KeyExpansion}(\text{pk}_1, \{RC_i\}_{i \in [R]}, PK(\cdot), \text{param})$
- 8: $\langle \bar{k} \rangle \leftarrow \text{ConstantToVOLE}(\bar{k})$
- 9: $\langle in \rangle \leftarrow \langle w[0 .. N_{\text{block}} - 1] \rangle$
- 10: $\langle out \rangle \leftarrow \langle w[0 .. N_{\text{block}} - 1] \rangle + \text{ConstantToVOLE}(\text{pk}_2)$
- 11: **else**
- 12: $\langle in \rangle \leftarrow \text{ConstantToVOLE}(\text{pk}_1)$
- 13: $\langle out \rangle \leftarrow \text{ConstantToVOLE}(\text{pk}_2)$
- 14: $(\langle \bar{k} \rangle, \langle \tilde{z} \rangle^3) \leftarrow \text{uBlockith.KeyExpCstrnts}(\langle w[0 .. \ell_{\text{ke}} - 1] \rangle; \text{param})$
- 15: $\langle z \rangle^3 \leftarrow \langle z \rangle^3 \parallel \langle \tilde{z} \rangle^3$
- 16: **end if**
- 17: $\langle \tilde{w} \rangle \leftarrow \langle w[\ell_{\text{ke}} .. \ell_{\text{ke}} + \ell_{\text{enc}} - 1] \rangle$
- 18: $\langle \tilde{z} \rangle^3 \leftarrow \text{uBlockith.EncCstrnts}(\langle in \rangle, \langle out \rangle, \langle \bar{k} \rangle, \langle \tilde{w} \rangle; \text{param})$
- 19: $\langle z \rangle^3 \leftarrow \langle z \rangle^3 \parallel \langle \tilde{z} \rangle^3$
- 20: **return** $\langle z \rangle^3$

uBlockith.OWFProve(w, u, V , pk, chall₂; param)

INPUT: $w \in \{0, 1\}^\ell$, $u \in \{0, 1\}^{2\lambda_{\mathbb{F}}}$, $V \in \{0, 1\}^{(\ell+2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$
OUTPUT: $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

- 1: **for** $i \in [0 .. \ell - 1]$ **do**
- 2: Let $\langle w_i \rangle \leftarrow (w_i, \text{ToField}(V|_i; \lambda_{\mathbb{F}}))$
- 3: **end for**
- 4: // Embed VOLE masks
- 5: **for** $i \in [0 .. 2\lambda_{\mathbb{F}} - 1]$ **do**
- 6: $v[i] \leftarrow \text{ToField}(V|_{\ell+i}; \lambda_{\mathbb{F}})$
- 7: **end for**
- 8: **for** $j \in [0 .. 1]$ **do**
- 9: $u_j^* \leftarrow \text{ToField}(u[j\lambda_{\mathbb{F}} .. (j+1)\lambda_{\mathbb{F}} - 1]; \lambda_{\mathbb{F}})$
- 10: // $\alpha_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, basis element over $\mathbb{F}_2$
- 11: $v_j^* \leftarrow \sum_{i \in [0 .. \lambda_{\mathbb{F}} - 1]} v[i + \lambda_{\mathbb{F}}j] \cdot \alpha_{\lambda_{\mathbb{F}}}^i$
- 12: **end for**
- 13: // Compute the constraints
- 14: // (poly $a_0 + a_1X + a_2X^2 + zX^3$; $z = 0$)
- 15: $\langle z \rangle^3 \leftarrow \text{uBlockith.OWFConstraints}(\langle w \rangle, \text{pk})$
- 16: Parse $\langle z \rangle^3$ as $(a_0, a_1, a_2) \in (\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^C)^3$
- 17: $\tilde{a}_0 \leftarrow \text{ZKHash}(\text{chall}_2, (a_0 || u_0^*))$
- 18: $\tilde{a}_1 \leftarrow \text{ZKHash}(\text{chall}_2, (a_1 || u_1^* + v_0^*))$
- 19: $\tilde{a}_2 \leftarrow \text{ZKHash}(\text{chall}_2, (a_2 || v_1^*))$
- 20: **return** $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$

uBlockith.OWFVerify($d, Q, pk, chall_2, chall_3, \tilde{a}_1, \tilde{a}_2; param, param_{OWF}$)

INPUT: $d \in \{0, 1\}^\ell$, $Q \in \{0, 1\}^{(\ell+2\lambda_F) \times \lambda_F}$

OUTPUT: $\tilde{a}_0 \in \mathbb{F}_{2^{\lambda_F}}$

```

1:  $\Delta \leftarrow \text{ToField}(chall_3; \lambda_F)$ 
2: for  $i \in [0.. \ell - 1]$  do
3:   Let  $\langle w_i \rangle \leftarrow \text{ToField}(Q|_i; \lambda_F) + d_i \cdot \Delta$ 
4: end for
5: // Embed VOLE masks
6: for  $i \in [0.. 2\lambda_F - 1]$  do
7:    $q[i] \leftarrow \text{ToField}(Q|_{\ell+i}; \lambda_F)$ 
8: end for
9: for  $j \in [0.. 1]$  do
10:  //  $\alpha_{\lambda_F} \in \mathbb{F}_{2^{\lambda_F}}$ , basis element over  $\mathbb{F}_2$ 
11:   $q_j^* \leftarrow \sum_{i \in [0.. \lambda_F-1]} q[i + \lambda_F j] \cdot \alpha_{\lambda_F}^i$ 
12: end for
13:  $q^* \leftarrow q_0^* + \Delta \cdot q_1^*$ 
14: // Compute the evaluated constraints
15:  $\langle z \rangle^3 \leftarrow \text{uBlockith.OWFConstraints}(\langle w \rangle, pk)$ 
16: Parse  $\langle z \rangle^3$  as  $b \in \mathbb{F}_{2^{\lambda_F}}^C$ 
17:  $\tilde{q} \leftarrow \text{ZKHash}(chall_2, (b || q^*))$ 
18: // Reproduce prover's  $\tilde{a}_0$ 
19: return  $\tilde{q} + \tilde{a}_1 \cdot \Delta + \tilde{a}_2 \cdot \Delta^2$ 

```

7.6 Construction of uBlockith

Based on the building blocks and functions described above, this section gives the formal construction of the uBlockith and uBlockith-EM signature schemes.

7.6.1 Key Generation

The **uBlockith.KeyGen** algorithm describes the key generation procedure for the uBlock-based schemes. On input the public parameters **param**, the algorithm first samples a secret vector $k \in \{0, 1\}^{N_{\text{block}}}$ uniformly at random, subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It then samples an independent vector $x \in \{0, 1\}^{N_{\text{block}}}$ uniformly at random. The roles of these two strings depend on whether the encryption mode flag **EM** is true.

If **EM** = true, the algorithm computes $y \leftarrow \text{uBlock}_x(k) \oplus k$, sets the secret key to $sk \leftarrow (x, k)$, and defines the public key as $pk \leftarrow (x, y)$. In this case, k is the secret input block, while x is the public uBlock key. Otherwise, when **EM** = false, the algorithm computes $y \leftarrow \text{uBlock}_k(x)$, sets $sk \leftarrow (x, k)$, and outputs $pk \leftarrow (x, y)$. In this case, k is the secret uBlock key, while x is the public input block. Finally, the algorithm returns the pair (sk, pk) .

7.6.2 Signing

Compared with SM4th, the signing procedures of uBlockith follow exactly the same high-level structure, including message binding and IV derivation, VOLE commitment and consistency checking, OWF proof generation, and the final grinding-based opening step. Therefore, we only highlight the differences below.

The first difference lies in the extension degree and the resulting VOLE length. In uBlockith, the extension degree is $d = 3$ rather than $d = 2$, and accordingly the signer invokes

$$(\text{com}, \text{decom}, c_1, \dots, c_{\tau-1}, u, V) \leftarrow \text{VOLECommit}(r, iv, \ell + 3\lambda_F + B; \text{param}),$$

instead of using length $\ell + 2\lambda_F + B$. Equivalently, the corresponding parameter $\hat{\ell}$ is instantiated as $\hat{\ell} = \ell + 3\lambda_F + B$ for uBlockith.

The second difference is in the witness extension procedure. In place of **SM4th.ExtendWitness**, the signer uses the uBlock-specific extension algorithm **uBlockith.ExtendWitness**.

uBlockith.KeyGen(param)

INPUT: Public parameters **param**OUTPUT: Secret key $\text{sk} \in (\{0, 1\}^{N_{\text{block}}}, \{0, 1\}^{N_{\text{block}}})$ and public key $\text{pk} \in (\{0, 1\}^{N_{\text{block}}}, \{0, 1\}^{N_{\text{block}}})$

```
1: repeat
2:    $\mathbf{k} \leftarrow \{0, 1\}^{N_{\text{block}}}$ 
3: until  $\mathbf{k}[0] \wedge \mathbf{k}[1] = 0$ 
4:  $\mathbf{x} \leftarrow \{0, 1\}^{N_{\text{block}}}$ 
5: if EM = true then
6:    $\mathbf{y} \leftarrow \text{uBlock}_{\mathbf{x}}(\mathbf{k}) \oplus \mathbf{k}$ 
7:    $\text{sk} \leftarrow (\mathbf{x}, \mathbf{k})$  // public uBlock key, secret input block
8:    $\text{pk} \leftarrow (\mathbf{x}, \mathbf{y})$  // public key block, OWF output
9: else
10:   $\mathbf{y} \leftarrow \text{uBlock}_{\mathbf{k}}(\mathbf{x})$ 
11:   $\text{sk} \leftarrow (\mathbf{x}, \mathbf{k})$  // public input block, secret uBlock key
12:   $\text{pk} \leftarrow (\mathbf{x}, \mathbf{y})$  // public input block, OWF output
13: end if
14: return (sk, pk)
```

uBlockith.Sign(msg, sk, pk; param)

INPUT: Message **msg**, secret key $\text{sk} = (\text{sk}_1, \text{sk}_2)$, public key $\text{pk} = (\text{pk}_1, \text{pk}_2)$, public parameters **param**OUTPUT: Signature σ

```
1:  $\text{ctr} \leftarrow 0$  // 32-bit int
2:  $\rho \leftarrow \{0, 1\}^{\xi}$  // Signature randomness
3:  $\mu \leftarrow H_2^0(\text{pk} \parallel \text{msg}; 2\lambda)$ 
4:  $(r, \text{iv}^{\text{pre}}) \leftarrow H_3(\text{sk}_2 \parallel \mu \parallel \rho; N_{\text{block}} + \lambda_{\text{iv}})$ 
5:  $\text{iv} \leftarrow H_4(\text{iv}^{\text{pre}}; \lambda_{\text{iv}})$ 
6:  $(\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(r, \text{iv}, \ell + 3\lambda_{\mathbb{F}} + B; \text{param})$  //  $d = 3$  for uBlockith
7:  $\text{chall}_1 \leftarrow H_2^1(\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}; 5\lambda_{\mathbb{F}} + 64)$ 
8: // VOLE consistency check
9:  $\tilde{\mathbf{u}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{u}) \in \{0, 1\}^{\lambda_{\mathbb{F}} + B}$ 
10:  $\tilde{\mathbf{V}} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{V}) \in \{0, 1\}^{(\lambda_{\mathbb{F}} + B) \times \lambda_{\mathbb{F}}}$ 
11: if EM = true then
12:    $\mathbf{w} \leftarrow \text{uBlockith.ExtendWitness}(\text{sk}_2, \text{sk}_1; \text{param}) \in \{0, 1\}^{\ell}$ 
13: else
14:    $\mathbf{w} \leftarrow \text{uBlockith.ExtendWitness}(\text{sk}_2, \text{sk}_1; \text{param}) \in \{0, 1\}^{\ell}$ 
15: end if
16:  $\mathbf{d} \leftarrow \mathbf{w} \oplus \mathbf{u}_{[0, \ell]}$  // Mask extended witness
17:  $\text{chall}_2 \leftarrow H_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}} \parallel \mathbf{d}; 3\lambda_{\mathbb{F}} + 64)$ 
18: // QuickSilver proof
19:  $\mathbf{u} \leftarrow \mathbf{u}_{[\ell, \ell + 2\lambda_{\mathbb{F}}]}$  // keep the two mask blocks
20:  $\mathbf{V} \leftarrow \mathbf{V}_{[0, \ell + 2\lambda_{\mathbb{F}}]} \in \{0, 1\}^{(\ell + 2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$  // drop the last  $\lambda_{\mathbb{F}} + B$  rows
21:  $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2) \leftarrow \text{uBlockith.OWFProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2; \text{param})$ 
22: repeat
23:    $\text{chall}_3 \leftarrow H_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}; \lambda_{\mathbb{F}})$ 
24:   if  $\text{chall}_3[\lambda_{\mathbb{F}} - w_{\text{grind}}, \lambda_{\mathbb{F}}] \neq 0^{w_{\text{grind}}}$  then
25:      $\text{ctr} \leftarrow \text{ctr} + 1$ 
26:     continue
27:   end if
28:    $I \leftarrow \text{DecodeChall}(\text{chall}_3[0, \lambda_{\mathbb{F}} - w_{\text{grind}}]; \text{param})$ 
29:    $\text{decom}_I \leftarrow \text{BAVC.Open}(\text{decom}, I; \text{param})$ 
30: until  $\text{decom}_I \neq \perp$ 
31: return  $\sigma \leftarrow (\{\mathbf{c}_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_2, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$ 
```

The third difference appears in the OWF proof phase. Since the uBlock-based OWF requires two transmitted proof components rather than one, the signer truncates the VOLE correlations to $\mathbf{u} \leftarrow \mathbf{u}_{[\ell, \ell+2\lambda_{\mathbb{F}}]}$ and $\mathbf{V} \leftarrow \mathbf{V}_{[0, \ell+2\lambda_{\mathbb{F}}]} \in \{0, 1\}^{(\ell+2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$, and computes $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2) \leftarrow \text{uBlockith.OWFProve}(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2; \text{param})$. Accordingly, the third Fiat–Shamir challenge is derived as $\text{chall}_3 \leftarrow \text{H}_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}; \lambda_{\mathbb{F}})$, and the resulting signature becomes $\sigma \leftarrow (\{\mathbf{c}_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_2, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$.

7.6.3 Verifying

The verification algorithm of uBlockith differs from that of SM4th in a fully analogous way. First, the verifier reconstructs the VOLE transcript under the parameter choice $\tilde{\ell} = \ell + 3\lambda_{\mathbb{F}} + B$. Second, after reconstructing $\mathbf{Q}$, the verifier invokes the uBlock-specific OWF verification algorithm

$$\tilde{a}_0 \leftarrow \text{uBlockith.OWFVerify}(\mathbf{d}, \mathbf{Q}_{[0, \ell+2\lambda_{\mathbb{F}}]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \tilde{a}_2, \text{pk}; \text{param}),$$

which now takes both transmitted proof components $\tilde{a}_1$ and $\tilde{a}_2$ as input. Finally, it recomputes

$$\text{chall}'_3 \leftarrow \text{H}_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}; \lambda_{\mathbb{F}}),$$

and accepts if and only if $\text{chall}'_3 = \text{chall}_3$.

uBlockith.Verify(msg, pk, σ ; param)

INPUT: Message msg, public key pk, signature σ , public parameters param

OUTPUT: true or false

- 1: Parse $\sigma = (\{\mathbf{c}_i\}_{i \in [1, \tau]}, \tilde{\mathbf{u}}, \mathbf{d}, \tilde{a}_1, \tilde{a}_2, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$
 - 2: $\mu \leftarrow \text{H}_2^0(\text{pk} \parallel \text{msg}; 2\lambda)$
 - 3: $\text{iv} \leftarrow \text{H}_4(\text{iv}^{\text{pre}}; \lambda_{\text{iv}})$
 - 4: **if** $\text{chall}_3[\lambda_{\mathbb{F}} - w_{\text{grind}}, \lambda_{\mathbb{F}}] \neq 0^{w_{\text{grind}}}$ **then**
 - 5: **return** false
 - 6: **end if**
 - 7: // Reconstruct VOLEs and check commitment
 - 8: $(\text{com}, \mathbf{Q}) \leftarrow \text{VOLEReconstruct}(\text{decom}_I, \text{chall}_3[0, \lambda_{\mathbb{F}} - w_{\text{grind}}], \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}; \text{param})$
 - 9: **if** VOLEReconstruct returned $\perp$ **then**
 - 10: **return** false
 - 11: **end if**
 - 12: $\text{chall}_1 \leftarrow \text{H}_2^1(\mu \parallel \text{com} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_{\tau-1} \parallel \text{iv}; 5\lambda_{\mathbb{F}} + 64)$
 - 13: // Apply the VOLE correction values and check consistency
 - 14: $\mathbf{Q} \leftarrow \text{VOLEHash}(\text{chall}_1, \mathbf{Q}) \in \{0, 1\}^{(\lambda_{\mathbb{F}} + B) \times \lambda_{\mathbb{F}}}$
 - 15: Parse $\text{chall}_3 = (\delta_0, \dots, \delta_{\lambda_{\mathbb{F}}-1}) \in \{0, 1\}^{\lambda_{\mathbb{F}}}$
 - 16: $\tilde{\mathbf{V}}' \leftarrow \tilde{\mathbf{Q}} \oplus [\delta_0 \cdot \tilde{\mathbf{u}} \dots \delta_{\lambda_{\mathbb{F}}-1} \cdot \tilde{\mathbf{u}}]$
 - 17: $\text{chall}_2 \leftarrow \text{H}_2^2(\text{chall}_1 \parallel \tilde{\mathbf{u}} \parallel \tilde{\mathbf{V}}' \parallel \mathbf{d}; 3\lambda_{\mathbb{F}} + 64)$
 - 18: // Compute $\tilde{a}_0$
 - 19: $\tilde{a}_0 \leftarrow \text{uBlockith.OWFVerify}(\mathbf{d}, \mathbf{Q}_{[0, \ell+2\lambda_{\mathbb{F}}]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \tilde{a}_2, \text{pk}; \text{param})$
 - 20: Compute $\text{chall}'_3 \leftarrow \text{H}_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}; \lambda_{\mathbb{F}})$
 - 21: **return** true if $\text{chall}_3 = \text{chall}'_3$ else false
-

In summary, relative to SM4th, the uBlockith variant differs only in the concrete OWF instantiation and the resulting parameter adjustments: the extension degree becomes $d = 3$, the VOLE length is increased from $\ell + 2\lambda_{\mathbb{F}} + B$ to $\ell + 3\lambda_{\mathbb{F}} + B$, the proof stage uses $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$ instead of $(\tilde{a}_0, \tilde{a}_1)$, and the signature correspondingly contains an additional transmitted component $\tilde{a}_2$. All remaining steps are identical.

8 Algorithm Description: the Vistrutith Signature Scheme

We present the construction of Vistrutith in this section.

8.1 Zero-Knowledge Building Blocks for Vistrutith

This section presents several VOLEitH-based ZKP building blocks for Vistrutah operations. For Vistrutith, we have $\lambda = \lambda_{\mathbb{F}} = N_{\text{block}} = 512$, so we use λ to represent all of them for ease of notation.

ByteCombine. The [Vistrutith.ByteCombine](#) function combines a vector of committed 8 bits into one committed field element belonging to $\mathbb{F}_{2^\lambda}$, which is Vistrutah-compatible similar to SM4-compatible as in Definition 3.

[Vistrutith.ByteCombine](#)($\langle x_0 \rangle, \dots, \langle x_7 \rangle; \lambda$)

INPUT: Commitment $\langle x_i \rangle$ where $x_i \in \mathbb{F}_2$, for $i \in [0..7]$; the security parameter λ

OUTPUT: Commitment $\langle x \rangle_\lambda$ to $x \in \mathbb{F}_{2^\lambda}$, containing an embedded $\mathbb{F}_{2^8}$ element

- 1: let $\beta_8 \in \mathbb{F}_{2^\lambda}$ // Vistrutah-compatible basis element of $\mathbb{F}_{2^8}$ within $\mathbb{F}_{2^\lambda}$
 - 2: **return** $\langle x \rangle_\lambda = \sum_{i=0}^7 \langle x_i \rangle_2 \cdot \beta_8^i$
-

Let the map $\phi : \text{GF}(2^8) \rightarrow \mathbb{F}_2[\beta_8]$ defined by $\phi\left(\sum_{i=0}^7 b_i x^i\right) = \sum_{i=0}^7 b_i \beta_8^i$. Similar to Lemma 5, for any $a, b, c \in \mathbb{F}_2[\beta_8]$ and $A = \phi^{-1}(a), B = \phi^{-1}(b), C = \phi^{-1}(c) \in \text{GF}(2^8)$, we have

$$a \cdot_\lambda b = c \iff A \cdot_8 B = C,$$

where $\cdot_8$ and $\cdot_\lambda$ denote multiplications in $\text{GF}(2^8)$ modulo $m(t)$ as in Equation (20) and $\mathbb{F}_{2^\lambda}$. Hence, to prove $A \cdot_8 B = C$ (similar to the form of patched inversion), it suffices to prove $a \cdot_\lambda b = c$ using the QuickSilver proof given the VOLE commitments $\langle a \rangle_\lambda, \langle b \rangle_\lambda$ and $\langle c \rangle_\lambda$. Table 11 gives the Vistrutah-compatible generator β_8 in the similar form with Table 10.

SquareBits. The function [Vistrutith.SquareBits](#) takes a vector of committed bits as inputs, and computes the vector of committed squared bits under the Vistrutah irreducible polynomial. It uses the 2-linearity property of $\mathbb{F}_{2^8}$.

[Vistrutah.SquareBits](#)($\langle x_0 \rangle, \dots, \langle x_7 \rangle$)

INPUT: Committed bits $\langle x_i \rangle$ for $i \in [0..7]$

OUTPUT: Committed squared bits $\langle y_i \rangle$ for $i \in [0..7]$

- 1: $\langle y_0 \rangle \leftarrow \langle x_0 \rangle + \langle x_4 \rangle + \langle x_6 \rangle$
 - 2: $\langle y_1 \rangle \leftarrow \langle x_4 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
 - 3: $\langle y_2 \rangle \leftarrow \langle x_1 \rangle + \langle x_5 \rangle$
 - 4: $\langle y_3 \rangle \leftarrow \langle x_4 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
 - 5: $\langle y_4 \rangle \leftarrow \langle x_2 \rangle + \langle x_4 \rangle + \langle x_7 \rangle$
 - 6: $\langle y_5 \rangle \leftarrow \langle x_5 \rangle + \langle x_6 \rangle$
 - 7: $\langle y_6 \rangle \leftarrow \langle x_3 \rangle + \langle x_5 \rangle$
 - 8: $\langle y_7 \rangle \leftarrow \langle x_6 \rangle + \langle x_7 \rangle$
 - 9: **return** $\langle y_0 \rangle, \dots, \langle y_7 \rangle$
-

SquareAndCombine. The [Vistrutith.SquareAndCombine](#) function takes a vector of committed bits as inputs, runs [Vistrutith.SquareBits](#), and runs [Vistrutith.ByteCombine](#) to transform these bits into a field element over $\mathbb{F}_{2^\lambda}$. This is similar to [SquareAndCombine](#) but with different irreducible polynomials defined in Equation (20) and finite fields $\mathbb{F}_{2^\lambda}$.

StateToBytes. The function [Vistrutith.StateToBytes](#) converts a committed state represented as an array of 128 committed bits into an array of 16 committed bytes. More precisely, the input $\langle s \rangle$ is partitioned into 16 consecutive blocks of 8 committed bits. For each $i \in [0..16)$, the chunk $\langle s[8i] \rangle, \dots, \langle s[8i+7] \rangle$ is combined into one committed byte by applying [ByteCombine](#). The resulting committed bytes are appended in order to form the output array $\langle \text{out} \rangle$. Hence, [Vistrutith.StateToBytes](#) maps a committed 128-bit state to a committed 16-byte array.

Table 11: Vistrutah-Compatible Generator β_8 for [Vistrutith.ByteCombine](#) when $\lambda = 512$

Hexadecimal (LSB)	Polynomial Form in $\mathbb{F}_{2^\lambda}$
48 ab f0 fb db 90 0a d1 86 e2 84 b7 31	$z^{511} + z^{510} + z^{509} + z^{505} + z^{504} + z^{501} + z^{500} + z^{498}$
8a 91 c6 02 e6 ad 2a 69 70 be fe 70 d1	$z^{497} + z^{496} + z^{495} + z^{492} + z^{491} + z^{487} + z^{486} + z^{485}$
92 ee 0a d8 66 50 6f a1 f0 f5 80 10 f5	$z^{484} + z^{482} + z^{479} + z^{478} + z^{477} + z^{475} + z^{474} + z^{468}$
98 1f d7 06 e4 0a 54 b1 40 cb 76 c7 0f	$z^{467} + z^{466} + z^{463} + z^{461} + z^{460} + z^{459} + z^{457} + z^{456}$
21 42 e0 3b d9 bb 1c ec f4 98 37 e3	$z^{455} + z^{454} + z^{452} + z^{451} + z^{448} + z^{445} + z^{444} + z^{443}$
	$z^{441} + z^{440} + z^{439} + z^{438} + z^{437} + z^{430} + z^{425} + z^{421}$
	$z^{416} + z^{411} + z^{410} + z^{409} + z^{408} + z^{407} + z^{406} + z^{402}$
	$z^{401} + z^{400} + z^{398} + z^{397} + z^{396} + z^{394} + z^{393} + z^{391}$
	$z^{390} + z^{387} + z^{385} + z^{384} + z^{382} + z^{375} + z^{373} + z^{372}$
	$z^{368} + z^{366} + z^{364} + z^{362} + z^{355} + z^{353} + z^{351} + z^{350}$
	$z^{349} + z^{346} + z^{338} + z^{337} + z^{335} + z^{334} + z^{332} + z^{330}$
	$z^{329} + z^{328} + z^{324} + z^{323} + z^{322} + z^{321} + z^{320} + z^{319}$
	$z^{316} + z^{315} + z^{311} + z^{310} + z^{309} + z^{308} + z^{306} + z^{304}$
	$z^{300} + z^{295} + z^{287} + z^{286} + z^{285} + z^{284} + z^{282} + z^{280}$
	$z^{279} + z^{278} + z^{277} + z^{276} + z^{271} + z^{269} + z^{264} + z^{262}$
	$z^{261} + z^{259} + z^{258} + z^{257} + z^{256} + z^{254} + z^{252} + z^{246}$
	$z^{245} + z^{242} + z^{241} + z^{239} + z^{238} + z^{236} + z^{235} + z^{227}$
	$z^{225} + z^{223} + z^{222} + z^{221} + z^{219} + z^{218} + z^{217} + z^{215}$
	$z^{212} + z^{209} + z^{207} + z^{206} + z^{204} + z^{200} + z^{198} + z^{197}$
	$z^{196} + z^{191} + z^{190} + z^{189} + z^{188} + z^{187} + z^{186} + z^{185}$
	$z^{183} + z^{181} + z^{180} + z^{179} + z^{178} + z^{177} + z^{174} + z^{173}$
	$z^{172} + z^{166} + z^{165} + z^{163} + z^{160} + z^{157} + z^{155} + z^{153}$
	$z^{151} + z^{149} + z^{147} + z^{146} + z^{144} + z^{143} + z^{142} + z^{141}$
	$z^{138} + z^{137} + z^{129} + z^{127} + z^{126} + z^{122} + z^{121} + z^{119}$
	$z^{116} + z^{112} + z^{111} + z^{107} + z^{105} + z^{101} + z^{100} + z^{96}$
	$z^{95} + z^{93} + z^{92} + z^{90} + z^{89} + z^{88} + z^{87} + z^{82}$
	$z^{79} + z^{78} + z^{77} + z^{73} + z^{71} + z^{66} + z^{65} + z^{63}$
	$z^{62} + z^{60} + z^{56} + z^{51} + z^{49} + z^{47} + z^{44} + z^{39}$
	$z^{38} + z^{36} + z^{35} + z^{33} + z^{32} + z^{31} + z^{30} + z^{29}$
	$z^{28} + z^{27} + z^{25} + z^{24} + z^{23} + z^{22} + z^{21} + z^{20}$
	$z^{15} + z^{13} + z^{11} + z^9 + z^8 + z^6 + z^3$

[Vistrutah.SquareAndCombine](#)($\langle x_0 \rangle, \dots, \langle x_7 \rangle$)

INPUT: Committed bits $\langle x_i \rangle$ for $i \in [0..7]$

OUTPUT: Committed squared field element $\langle y \rangle_\lambda \in \mathbb{F}_{2^\lambda}$

- 1: $\langle y_0 \rangle, \dots, \langle y_7 \rangle \leftarrow \text{Vistrutith.SquareBits}(\langle x_0 \rangle, \dots, \langle x_7 \rangle)$
 - 2: **return** [Vistrutith.ByteCombine](#)($\langle y_0 \rangle, \dots, \langle y_7 \rangle$)
-

[Vistrutith.StateToBytes](#)($\langle s \rangle$)

INPUT: $\langle s \rangle$, a state array of 128 committed bits

OUTPUT: $\langle out \rangle_\lambda$, an array of 16 committed bytes

- 1: **new** $\langle out \rangle_\lambda \in \mathbb{F}_{2^\lambda}^{16}$
 - 2: **for** $i \in [0..16)$ **do**
 - 3: $\langle bt \rangle \leftarrow \text{Vistrutith.ByteCombine}(\langle s[8i] \rangle, \dots, \langle s[8i+7] \rangle)$
 - 4: $\langle out \rangle \leftarrow \langle out \parallel bt \rangle$
 - 5: **end for**
 - 6: **return** $\langle out \rangle$
-

InvNorm. For a finite field extension $\mathbb{F}_{q^m}/\mathbb{F}_q$, the field norm is the map

$$N_{\mathbb{F}_{q^m}/\mathbb{F}_q}(u) = u^{1+q+q^2+\dots+q^{m-1}}.$$

Its output always lies in the base field $\mathbb{F}_q$.

The function [Vistruth.InvNorm](#) extracts a 4-bit representation of the norm of the inverse of an 8-bit field element. More precisely, the input $\mathbf{x} \in \{0, 1\}^8$ is first interpreted as an element x of $\mathbb{F}_{2^8}$ via [ToField](#)($\mathbf{x}, 8$), where $\mathbb{F}_{2^8}$ is viewed as a degree-2 extension of the subfield $\mathbb{F}_{2^4}$. If $x = 0$, then [Vistruth.InvNorm](#) returns $(0, 0, 0, 0)$. Otherwise, it computes the norm of x^{-1} , *i.e.*, x^{-17} , from $\mathbb{F}_{2^8}$ down to $\mathbb{F}_{2^4}$. Recall that for an element $u \in \mathbb{F}_{2^8}$, the field norm from $\mathbb{F}_{2^8}$ to $\mathbb{F}_{2^4}$ is defined by

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(u) = u^{1+2^4} = u^{17}.$$

Hence, for any nonzero $x \in \mathbb{F}_{2^8}$,

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(x^{-1}) = (x^{-1})^{17} = x^{-17}.$$

Since the norm always lies in the subfield $\mathbb{F}_{2^4}$, x^{-17} can be represented using only four bits.

To obtain these four bits, we fix a basis $b = (1, \beta, \beta^2, \beta^3)$ of $\mathbb{F}_{2^4}$, where $\beta = \alpha^6 + \alpha^4$ and α is the basis element of $\mathbb{F}_{2^8}$. The role of this basis is to provide a coordinate system for elements of the subfield $\mathbb{F}_{2^4}$. That is, every element of $\mathbb{F}_{2^4}$ can be written uniquely as

$$y_0 \cdot 1 + y_1 \cdot \beta + y_2 \cdot \beta^2 + y_3 \cdot \beta^3, \quad y_i \in \mathbb{F}_2.$$

Thus, if $\mathbf{y} = (y_0, y_1, y_2, y_3) = \text{Vistruth.InvNorm}(\mathbf{x})$, then $\mathbf{y}$ should be understood as the coordinate vector of $N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(x^{-1})$ with respect to the basis b .

Concretely, after computing the bit decomposition $\mathbf{y}' = \text{ToBits}(1/x^{17})$, the algorithm outputs

$$\mathbf{y} = (y'[0], y'[6], y'[7], y'[2]).$$

These four selected bits are exactly the coordinates of the norm value with respect to the chosen basis b . In this way, one can compute $x^{-17} \in \mathbb{F}_{2^4} \subseteq \mathbb{F}_{2^8}$.

[Vistruth.InvNorm](#)($\mathbf{x}$)

INPUT: $\mathbf{x} \in \{0, 1\}^8$

OUTPUT: $\mathbf{y} \in \{0, 1\}^4$

```

1:  $x \leftarrow \text{ToField}(\mathbf{x}, 8)$ 
2: if  $x = 0$  then
3:   return  $(0, 0, 0, 0)$ 
4: end if
5:  $\mathbf{y}' \leftarrow \text{ToBits}(1/x^{17})$ 
6:  $\mathbf{y} \leftarrow (\mathbf{y}'[0], \mathbf{y}'[6], \mathbf{y}'[7], \mathbf{y}'[2])$ 
7: return  $\mathbf{y}$ 
```

InvNormCombine. The function [Vistruth.InvNormCombine](#) lifts a 4-bit VOLE commitment representing an element of the subfield $\mathbb{F}_{2^4}$ to a commitment in $\mathbb{F}_{2^\lambda}$. Let $\beta_8 \in \mathbb{F}_{2^\lambda}$ be the generator used in [Vistruth.ByteCombine](#), and define $\beta_4 = \beta_8^6 + \beta_8^4$. Then $(1, \beta_4, \beta_4^2, \beta_4^3)$ forms a basis of the embedded copy of $\mathbb{F}_{2^4}$ inside $\mathbb{F}_{2^\lambda}$. Recall that this commitment is Vistruth-compatible. When the input is the output of [Vistruth.InvNorm](#), the resulting commitment corresponds to the element x^{-17} embedded in $\mathbb{F}_{2^\lambda}$.

InvNormToConjugates. Let $x \in \mathbb{F}_{2^m}$. The Frobenius map over $\mathbb{F}_2$ is the field automorphism $x \mapsto x^2$. The *Frobenius conjugates* of x are the elements obtained by repeated applications of this map, namely $x, x^2, x^{2^2}, \dots, x^{2^{m-1}}$. In particular, for $x \in \mathbb{F}_{2^4}$, its conjugates are x, x^2, x^4, x^8 , and for $x \in \mathbb{F}_{2^8}$, its conjugates are $x, x^2, x^4, x^8, x^{16}, x^{32}, x^{64}, x^{128}$.

Vistruth.InvNormCombine($\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle$)

INPUT: $\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle$, where $x_i \in \mathbb{F}_2$

OUTPUT: $\langle y \rangle_\lambda$, where $y \in \mathbb{F}_{2^\lambda}$

- 1: Let $\beta_8 \in \mathbb{F}_{2^\lambda}$ be the generator used in **Vistruth.ByteCombine**
 - 2: Let $\beta_4 \leftarrow \beta_8^6 + \beta_8^4$
 - 3: **return** $\langle y \rangle_\lambda \leftarrow \sum_{i=0}^3 \langle x_i \rangle_2 \cdot \beta_4^i$
-

The function **Vistruth.InvNormToConjugates** lifts a 4-bit VOLE commitment, representing an element of the subfield $\mathbb{F}_{2^4}$, to $\mathbb{F}_{2^\lambda}$ and computes all of its Frobenius conjugates. The input $\langle \mathbf{x} \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$ is interpreted as the coordinate vector of an element $y = x_0 + x_1\beta_4 + x_2\beta_4^2 + x_3\beta_4^3 \in \mathbb{F}_{2^4}$, where $\beta_4 = \beta_8^6 + \beta_8^4$ and $\beta_8 \in \mathbb{F}_{2^\lambda}$ is the generator used in **Vistruth.ByteCombine**. Note that y is exactly **Vistruth.InvNormCombine**($\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle$).

Beyond y , the function returns the whole four conjugates $y_0 = y, y_1 = y^2, y_2 = y^4, y_3 = y^8$, all embedded in $\mathbb{F}_{2^\lambda}$. This follows from the fact that in characteristic 2, the Frobenius map $u \mapsto u^2$ acts coefficient-wise on the basis expansion, so repeated squaring transforms the basis elements as

$$1, \beta_4, \beta_4^2, \beta_4^3 \mapsto 1, \beta_4^{2^j}, \beta_4^{2^{j+1}}, \beta_4^{3 \cdot 2^j}.$$

Vistruth.InvNormToConjugates($\langle \mathbf{x} \rangle$)

INPUT: $\langle \mathbf{x} \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$, where $x_i \in \mathbb{F}_2$

OUTPUT: $(\langle y_0 \rangle_\lambda, \langle y_1 \rangle_\lambda, \langle y_2 \rangle_\lambda, \langle y_3 \rangle_\lambda)$, where $y_j \in \mathbb{F}_{2^\lambda}$

- 1: Let $\beta_8 \in \mathbb{F}_{2^\lambda}$ be the generator used in **Vistruth.ByteCombine**
 - 2: Let $\beta_4 \leftarrow \beta_8^6 + \beta_8^4$
 - 3: **for** $j \in [0..4)$ **do**
 - 4: $\langle y_j \rangle_\lambda \leftarrow \sum_{i=0}^3 \langle x_i \rangle \cdot \beta_4^{i \cdot 2^j}$
 - 5: **end for**
 - 6: **return** $(\langle y_0 \rangle_\lambda, \langle y_1 \rangle_\lambda, \langle y_2 \rangle_\lambda, \langle y_3 \rangle_\lambda)$
-

StateToConjugates. The function **Vistruth.StateToConjugates** computes the Frobenius conjugates of every byte in a committed 128-bit state. The input $\langle \mathbf{s} \rangle$ is viewed as 16 consecutive committed bytes, each represented by 8 committed bits. For the i -th byte, the algorithm first extracts $\langle \mathbf{x}_0 \rangle = (\langle \mathbf{s}[8i] \rangle, \dots, \langle \mathbf{s}[8i+7] \rangle)$, which is interpreted as an element $x_i \in \mathbb{F}_{2^8}$.

It then computes the sequence

$$x_i, x_i^2, x_i^4, x_i^8, x_i^{16}, x_i^{32}, x_i^{64}, x_i^{128}$$

by repeated squaring in $\mathbb{F}_{2^8}$. At each stage, the current 8-bit representation is lifted to a VOLE commitment in $\mathbb{F}_{2^\lambda}$ using **Vistruth.ByteCombine**. Thus, for each byte index i , the output consists of the 8 commitments $\langle y_{i,0} \rangle_\lambda, \langle y_{i,1} \rangle_\lambda, \dots, \langle y_{i,7} \rangle_\lambda$, where $y_{i,j} = x_i^{2^j}$.

SBoxAffine. The function **Vistruth.SBoxAffine** applies the affine component of the AES S-box to a collection of committed byte values. Its input is not given as raw bytes, but as a collection of Frobenius conjugates. More precisely, for each position $i \in [0..n)$, the input contains a tuple

$$\langle \mathbf{x}_i \rangle_\lambda^2 = (\langle x_{i,0} \rangle_\lambda^2, \dots, \langle x_{i,7} \rangle_\lambda^2),$$

where $x_{i,j} = x_i^{2^j}$. Thus, each input byte is represented by the full conjugate family

$$x_{i,0}, x_{i,0}^2, x_{i,0}^{2^2}, \dots, x_{i,0}^{2^7}.$$

Note that each $x_{i,j} \in \mathbb{F}_{2^\lambda}$, which embeds a field element belonging to $\mathbb{F}_{2^8}$.

Vistruth.StateToConjugates($\langle s \rangle$)

INPUT: $\langle s \rangle$, an array of 128 VOLE commitments to the bits of a stateOUTPUT: $\langle y_{i,j} \rangle_\lambda$ for $i \in [0..16)$ and $j \in [0..8)$

```
1: for  $i \in [0..16)$  do
2:    $\langle x_0 \rangle \leftarrow (\langle s[8i] \rangle, \dots, \langle s[8i+7] \rangle)$ 
3:   for  $j \in [0..7)$  do
4:      $\langle y_{i,j} \rangle_\lambda \leftarrow \text{Vistruth.ByteCombine}(\langle x_j \rangle)$ 
5:     if  $j < 7$  then
6:        $\langle x_{j+1} \rangle \leftarrow \text{Vistruth.SquareBits}(\langle x_j \rangle)$ 
7:     end if
8:   end for
9: end for
10: return  $(\langle y_{0,0} \rangle_\lambda, \dots, \langle y_{15,7} \rangle_\lambda)$ 
```

The reason for using such a conjugate representation is that the affine part of the AES S-box is a byte-wise affine transformation over $\mathbb{F}_2$. Equivalently, when the byte is viewed as an element z of $\mathbb{F}_{2^8}$, every $\mathbb{F}_2$ -linear map on $\mathbb{F}_{2^8}$ can be written as a *linearized polynomial* of the form

$$L(z) = \sum_{k=0}^7 \chi_k \cdot z^{2^k},$$

where $\chi_k \in \mathbb{F}_{2^k}$. Hence, every the S-box affine map can be written as

$$A_{\text{aff}}(z) = \chi_8 + \sum_{k=0}^7 \chi_k \cdot z^{2^k}. \quad (26)$$

According to Equation (19), one can compute $(\chi_0, \dots, \chi_8)$ via solving a Moore-matrix linear system such that

$$(\chi_0, \dots, \chi_7) = (05, 09, \text{f9}, 25, \text{f4}, 01, \text{b5}, 8\text{f}).$$

When the input element is z^2 , due to the 2-linearity, we have

$$A_{\text{aff}}(z^2) = (A_{\text{aff}}(z))^2 = \chi_8^2 + \sum_{k=0}^7 \chi_k^2 \cdot z^{2^{k+1}}.$$

Here, $(\chi_0^2, \dots, \chi_8^2)$ can be computed as

$$(\chi_0^2, \dots, \chi_7^2) = (11, 41, 07, 7\text{d}, 56, 01, \text{fc}, \text{c2}).$$

When z is embedded into an element of $\mathbb{F}_{2^\lambda}$ via the Vistruth-compatible transformation, Equation (26) stills holds but $(\chi_0, \dots, \chi_7)$ will be embedded into

$$(\zeta_0, \dots, \zeta_7) = (\text{Vistruth.ByteCombine}(\text{ToBits}(\chi_0)), \dots, \text{Vistruth.ByteCombine}(\text{ToBits}(\chi_7))). \quad (27)$$

Vistruth.SBoxAffine computes the affine layer by taking a linear combination of the committed conjugates with fixed coefficients $\zeta_0, \dots, \zeta_7$, together with a constant term ζ_8 . If the flag *sq* is set to **false**, the ordinary affine map is evaluated. If *sq* is set to **true**, the algorithm instead evaluates the Frobenius square of the affine map. This is achieved by squaring the coefficients and shifting the conjugate index by one, which matches the identity

The purpose of **Vistruth.SBoxAffine** is to compute the next S-box-layer values in $\mathbb{F}_{2^\lambda}$ with the Vistruth-compatibility. The ordinary version produces the affine output for each current S-box input (*i.e.*, the output of the patched inversion), while the square version simultaneously provides its square. These outputs are then used as the input and squared input, respectively, for the next stage of the S-box-related constraints.

Vistrutith.SBoxAffine($\langle \mathbf{x} \rangle_\lambda^2; n, sq$)

INPUT: $\langle \mathbf{x} \rangle_\lambda^2 = (\langle x_0 \rangle_\lambda^2, \dots, \langle x_{n-1} \rangle_\lambda^2)$, where each $\langle x_i \rangle_\lambda^2 = (\langle x_{i,0} \rangle_\lambda^2, \dots, \langle x_{i,7} \rangle_\lambda^2)$ is a tuple of degree-2 commitments to the conjugates of some $x_{i,0} \in \mathbb{F}_{2^8} \subset \mathbb{F}_{2^\lambda}$, namely $x_{i,j} = x_{i,0}^{2^j}$ for $j \in [0..8)$; a flag $sq \in \{\text{false}, \text{true}\}$
OUTPUT: $\langle \mathbf{y} \rangle_\lambda^2 = (\langle y_0 \rangle_\lambda^2, \dots, \langle y_{n-1} \rangle_\lambda^2)$, where each $y_i \in \mathbb{F}_{2^\lambda}$

```
1: if  $sq = \text{true}$  then
2:    $s \leftarrow 2, \quad t \leftarrow 1$ 
3: else
4:    $s \leftarrow 1, \quad t \leftarrow 0$ 
5: end if
6: Let  $\zeta_0, \dots, \zeta_8 \in \mathbb{F}_{2^\lambda}$  be the constants defining the affine map in Equation (27)
7: for  $i \in [0..n)$  do
8:    $\langle y_i \rangle_\lambda^2 \leftarrow \zeta_8^s$ 
9:   for  $k \in [0..8)$  do
10:     $\langle y_i \rangle_\lambda^2 \leftarrow \langle y_i \rangle_\lambda^2 + \zeta_k^s \cdot \langle x_{i,(k+t) \bmod 8} \rangle_\lambda^2$  // shifting 1 means the square by definition
11:   end for
12: end for
13: return  $\langle \mathbf{y} \rangle_\lambda^2$ 
```

SBoxInvAffine. The function [Vistrutith.SBoxInvAffine](#) applies the inverse affine component of the AES S-box to each committed byte of the state. For each byte index i , the input consists of eight degree-1 commitments $\langle x_{i,0} \rangle, \dots, \langle x_{i,7} \rangle$ to the bits of that byte. The output is another tuple of eight committed bits $\langle y_{i,0} \rangle, \dots, \langle y_{i,7} \rangle$, corresponding to the inverse affine transformation.

By Equation (21), the inverse affine map is given bitwise by

$$y_{i,j} = x_{i,(j-1) \bmod 8} \oplus x_{i,(j-3) \bmod 8} \oplus x_{i,(j-6) \bmod 8} \oplus c'_j, \quad 0 \leq j < 8,$$

where $c'_j = 1$ for $j \in \{0, 2\}$ and $c'_j = 0$ otherwise. In the committed setting, the constant bit c'_j is converted into a VOLE commitment using [ConstantToVOLE](#), and the whole transformation is evaluated bitwise over the state.

Vistrutith.SBoxInvAffine($\langle \mathbf{s} \rangle$)

INPUT: For each byte $i \in [0..16)$, commitments $\langle x_{i,0} \rangle, \dots, \langle x_{i,7} \rangle$ to bits in $\mathbb{F}_2$

OUTPUT: For each byte i , commitments $\langle y_{i,0} \rangle, \dots, \langle y_{i,7} \rangle$ to bits in $\mathbb{F}_2$

```
1: for  $i \in [0..16)$  do
2:   for  $j \in [0..8)$  do
3:      $c_j \leftarrow 1$  if  $j \in \{0, 2\}$ , else 0
4:      $\langle y_{i,j} \rangle \leftarrow \langle x_{i,(j-1) \bmod 8} \rangle + \langle x_{i,(j-3) \bmod 8} \rangle + \langle x_{i,(j-6) \bmod 8} \rangle + \text{ConstantToVOLE}(c_j)$  // defined in
        Equation (21)
5:   end for
6: end for
7: return  $(\langle y_{0,0} \rangle, \dots, \langle y_{15,7} \rangle)$ 
```

MixColumns. The function [Vistrutith.MixColumns](#) applies the [MixColumns](#) transformation to a sequence of committed state bytes. Its input is a vector $\langle \mathbf{x} \rangle_\lambda^2 = (\langle x_0 \rangle_\lambda^2, \dots, \langle x_{15} \rangle_\lambda^2)$, where each entry is a degree-2 commitment to an element of $\mathbb{F}_{2^8}$, embedded in $\mathbb{F}_{2^\lambda}$. The output is another vector of degree-2 commitments $\langle \mathbf{y} \rangle_\lambda^2$, obtained by applying the column-wise linear transformation. In the function, the constants 01, 02, and 03 are first lifted to $\mathbb{F}_{2^\lambda}$ via [Vistrutith.ByteCombine](#), yielding the field elements ν_1, ν_2 , and ν_3 .

The flag sq determines whether the ordinary or squared version of the transformation is applied. If $sq = \text{false}$, the constants ν_1, ν_2 , and ν_3 are used as they are. If $sq = \text{true}$, they are replaced by their Frobenius squares ν_1^2, ν_2^2 , and ν_3^2 . This corresponds to applying the squared form of the same linear transformation in characteristic 2.

Vistruth.MixColumns($\langle x \rangle_\lambda^2, sq$)

INPUT: $\langle x \rangle_\lambda^2 = (\langle x_0 \rangle_\lambda^2, \dots, \langle x_{15} \rangle_\lambda^2)$, where each $\langle x_i \rangle_\lambda^2$ is a degree-2 commitment to $x_i \in \mathbb{F}_{2^8} \subseteq \mathbb{F}_{2^\lambda}$; a flag $sq \in \{\text{true}, \text{false}\}$

OUTPUT: $\langle y \rangle_\lambda^2 = (\langle y_0 \rangle_\lambda^2, \dots, \langle y_{15} \rangle_\lambda^2)$, where each $\langle y_i \rangle_\lambda^2$ is a degree-2 commitment to $y_i \in \mathbb{F}_{2^8} \subseteq \mathbb{F}_{2^\lambda}$

```
1: Let  $\nu_1 \leftarrow \text{Vistruth.ByteCombine}(\text{ToBits}(01))$ 
2: Let  $\nu_2 \leftarrow \text{Vistruth.ByteCombine}(\text{ToBits}(02))$ 
3: Let  $\nu_3 \leftarrow \text{Vistruth.ByteCombine}(\text{ToBits}(03))$ 
4: if  $sq = 1$  then
5:    $\nu_1 \leftarrow \nu_1^2, \nu_2 \leftarrow \nu_2^2, \nu_3 \leftarrow \nu_3^2$ 
6: end if
7: for  $c \in [0..4)$  do
8:    $i_0 \leftarrow 4c, i_1 \leftarrow 4c+1, i_2 \leftarrow 4c+2, i_3 \leftarrow 4c+3$ 
9:    $\langle y_{i_0} \rangle_\lambda^2 \leftarrow \nu_2 \cdot \langle x_{i_0} \rangle_\lambda^2 + \nu_3 \cdot \langle x_{i_1} \rangle_\lambda^2 + \nu_1 \cdot \langle x_{i_2} \rangle_\lambda^2 + \nu_1 \cdot \langle x_{i_3} \rangle_\lambda^2$ 
10:   $\langle y_{i_1} \rangle_\lambda^2 \leftarrow \nu_1 \cdot \langle x_{i_0} \rangle_\lambda^2 + \nu_2 \cdot \langle x_{i_1} \rangle_\lambda^2 + \nu_3 \cdot \langle x_{i_2} \rangle_\lambda^2 + \nu_1 \cdot \langle x_{i_3} \rangle_\lambda^2$ 
11:   $\langle y_{i_2} \rangle_\lambda^2 \leftarrow \nu_1 \cdot \langle x_{i_0} \rangle_\lambda^2 + \nu_1 \cdot \langle x_{i_1} \rangle_\lambda^2 + \nu_2 \cdot \langle x_{i_2} \rangle_\lambda^2 + \nu_3 \cdot \langle x_{i_3} \rangle_\lambda^2$ 
12:   $\langle y_{i_3} \rangle_\lambda^2 \leftarrow \nu_3 \cdot \langle x_{i_0} \rangle_\lambda^2 + \nu_1 \cdot \langle x_{i_1} \rangle_\lambda^2 + \nu_1 \cdot \langle x_{i_2} \rangle_\lambda^2 + \nu_2 \cdot \langle x_{i_3} \rangle_\lambda^2$ 
13: end for
14: return  $\langle y \rangle_\lambda^2$ 
```

BitInvMixColumns. The function [Vistruth.BitInvMixColumns](#) implements the [InvMixColumns](#) transformation directly on committed state bits. The input is a committed 128-bit state $\langle s \rangle$, viewed as 16 committed bytes arranged into 4 columns. For each column $c \in [0..4)$, the algorithm first extracts the four byte vectors $\langle a^{(0)} \rangle, \dots, \langle a^{(3)} \rangle$, each consisting of 8 committed bits.

For each such byte $\langle a^{(r)} \rangle$, the [InvMixColumns](#) step requires multiplication by the AES-field constants 09, 0b, 0d, and 0e. These multiplications are represented by fixed $\mathbb{F}_2$ -linear maps $\Gamma_{09}, \Gamma_{0b}, \Gamma_{0d}, \Gamma_{0e} : \mathbb{F}_2^8 \rightarrow \mathbb{F}_2^8$. Concretely, for each column, the output bytes $\langle o_{4c} \rangle, \dots, \langle o_{4c+3} \rangle$ are computed as linear combinations of the four input bytes after applying the corresponding maps Γ_η , according to the [InvMixColumns](#) matrix.

Vistruth.BitInvMixColumns($\langle s \rangle$)

INPUT: $\langle s \rangle = (\langle s_0 \rangle, \dots, \langle s_{127} \rangle)$, a tuple of committed state bits

OUTPUT: $\langle o \rangle = (\langle o_0 \rangle, \dots, \langle o_{127} \rangle)$, the committed bits after inverse MixColumns

```
1: for  $c \in [0..4)$  do
2:   for  $r \in [0..4)$  do
3:      $\langle a^{(r)} \rangle \leftarrow (\langle s_{32c+8r} \rangle, \dots, \langle s_{32c+8r+7} \rangle)$ 
4:   end for
5:    $\langle o_{4c+0} \rangle \leftarrow \Gamma_{0e}(\langle a^{(0)} \rangle) + \Gamma_{0b}(\langle a^{(1)} \rangle) + \Gamma_{0d}(\langle a^{(2)} \rangle) + \Gamma_{09}(\langle a^{(3)} \rangle)$ 
6:    $\langle o_{4c+1} \rangle \leftarrow \Gamma_{09}(\langle a^{(0)} \rangle) + \Gamma_{0e}(\langle a^{(1)} \rangle) + \Gamma_{0b}(\langle a^{(2)} \rangle) + \Gamma_{0d}(\langle a^{(3)} \rangle)$ 
7:    $\langle o_{4c+2} \rangle \leftarrow \Gamma_{0d}(\langle a^{(0)} \rangle) + \Gamma_{09}(\langle a^{(1)} \rangle) + \Gamma_{0e}(\langle a^{(2)} \rangle) + \Gamma_{0b}(\langle a^{(3)} \rangle)$ 
8:    $\langle o_{4c+3} \rangle \leftarrow \Gamma_{0b}(\langle a^{(0)} \rangle) + \Gamma_{0d}(\langle a^{(1)} \rangle) + \Gamma_{09}(\langle a^{(2)} \rangle) + \Gamma_{0e}(\langle a^{(3)} \rangle)$ 
9: end for
10: return  $\langle o_0 \rangle \parallel \langle o_1 \rangle \parallel \dots \parallel \langle o_{15} \rangle$ 
```

The concrete constructions of $\Gamma_{09}, \Gamma_{0b}, \Gamma_{0d}, \Gamma_{0e}$ are given as below.

ShiftRows. The function [Vistruth.ShiftRows](#) applies the AES [ShiftRows](#) transformation to a committed state represented as an ordered list of 16 committed bytes. The input is a tuple $\langle s \rangle_\lambda^2$ of degree-2 VOLE commitments, where each committed value lies in $\mathbb{F}_{2^8} \subseteq \mathbb{F}_{2^\lambda}$. Internally, the state is viewed as a 4×4 byte matrix in column-major order, so that the byte $\langle s_{4c+r} \rangle_\lambda^2$ corresponds to the entry in row r and column c .

The transformation [ShiftRows](#) rotates each row r to the left by exactly r positions. In terms of the byte ordering, this means that for each row index $r \in [0..4)$ and column index $c \in [0..4)$, the

$$Q_{\text{max}} = Q_{\text{max}}(y) = 11.5 - 0.5y, \quad y \in [0, 23]$$

output byte is given by

$$\langle s'_{4c+r} \rangle_{\lambda}^2 = \langle s_{4((c+r) \bmod 4)+r} \rangle_{\lambda}^2.$$

Vistruthith.ShiftRows($\langle \mathbf{s} \rangle_{\lambda}^2$)

INPUT: $\langle \mathbf{s} \rangle_{\lambda}^2 = (\langle s_0 \rangle_{\lambda}^2, \dots, \langle s_{15} \rangle_{\lambda}^2)$ in byte order, where each $\langle s_i \rangle_{\lambda}^2$ is a degree-2 commitment to $s_i \in \mathbb{F}_{2^8}$

OUTPUT: $\langle \mathbf{s}' \rangle_{\lambda}^2 = (\langle s'_0 \rangle_{\lambda}^2, \dots, \langle s'_{15} \rangle_{\lambda}^2)$

```

1: for  $r \in [0..4)$  do
2:   for  $c \in [0..4)$  do
3:      $\langle s'_{4c+r} \rangle_{\lambda}^2 \leftarrow \langle s_{4((c+r) \bmod 4)+r} \rangle_{\lambda}^2$ 
4:   end for
5: end for
6: return  $(\langle s'_0 \rangle_{\lambda}^2, \dots, \langle s'_{15} \rangle_{\lambda}^2)$ 

```

BitInvShiftRows. The function [Vistruthith.BitInvShiftRows](#) reverses the AES [ShiftRows](#) transformation on a committed bit-level state. Its input is a committed 128-bit state $\langle \mathbf{s} \rangle$. The output is another 128-bit committed state $\langle \mathbf{s}' \rangle$, obtained by undoing the row shifts in [ShiftRows](#).

The forward [ShiftRows](#) transformation rotates row r to the left by r positions. Accordingly, the inverse transformation rotates row r to the right by r positions, which is equivalently expressed as a left rotation by $-r \bmod 4$. Thus, for each row index $r \in [0..4)$ and column index $c \in [0..4)$, the byte occupying position (r, c) in the inverse-shifted state is copied from position $i = 4((c - r) \bmod 4) + r$ in the input state.

Vistruthith.BitInvShiftRows($\langle \mathbf{s} \rangle$)

INPUT: $\langle \mathbf{s} \rangle = (\langle s_0 \rangle, \dots, \langle s_{127} \rangle)$, a tuple of VOLE commitments to state bits

OUTPUT: $\langle \mathbf{s}' \rangle = (\langle s'_0 \rangle, \dots, \langle s'_{127} \rangle)$

```

1: for  $r \in [0..4)$  do
2:   for  $c \in [0..4)$  do
3:      $i \leftarrow 4((c - r) \bmod 4) + r$ 
4:      $(\langle s'_{8(4c+r)} \rangle, \dots, \langle s'_{8(4c+r)+7} \rangle) \leftarrow (\langle s_{8i} \rangle, \dots, \langle s_{8i+7} \rangle)$ 
5:   end for
6: end for
7: return  $(\langle s'_0 \rangle, \dots, \langle s'_{127} \rangle)$ 

```

BitAddRoundKey. The function [Vistruthith.BitAddRoundKey](#) implements the AES AddRoundKey step directly on committed state bits. Its input consists of two 128-bit tuples of VOLE commitments, namely the committed state $\langle \mathbf{s} \rangle$ and the committed round key $\langle \mathbf{k} \rangle$. The output is a new tuple $\langle \mathbf{o} \rangle$ of committed bits representing the updated state.

For each bit position $i \in [0..128)$, the output bit is computed as

$$\langle o_i \rangle = \langle s_i \rangle + \langle k_i \rangle,$$

where the addition is over $\mathbb{F}_2$, i.e., it corresponds to bitwise XOR. Thus, [Vistruthith.BitAddRoundKey](#) performs the key addition independently on every committed bit of the state.

AddRoundKey. The function [Vistruthith.AddRoundKey](#) implements the AES AddRoundKey step at the byte level. Its input consists of a tuple $\langle \mathbf{s} \rangle_{\lambda}^2$ of degree-2 commitments to the 16 state bytes, together with a tuple $\langle \mathbf{k} \rangle_{\lambda}^d$ of commitments to the corresponding round-key bytes, where the key commitments have degree $d \in \{1, 2\}$. The output is a tuple $\langle \mathbf{o} \rangle_{\lambda}^d$ of committed bytes representing the updated state after key addition.

For each byte position $i \in [0..16)$, the output byte is computed as $\langle o_i \rangle_{\lambda}^d = \langle s_i \rangle_{\lambda}^2 + \langle k_i \rangle_{\lambda}^d$, where the addition is the field addition in $\mathbb{F}_{2^8}$, which corresponds to bitwise XOR on the embedded $\mathbb{F}_{2^8}$ values.

Vistruth.BitAddRoundKey($\langle s \rangle, \langle k \rangle$)

INPUT: $\langle s \rangle = (\langle s_0 \rangle, \dots, \langle s_{127} \rangle)$ and $\langle k \rangle = (\langle k_0 \rangle, \dots, \langle k_{127} \rangle)$, tuples of VOLE commitments to state bits and round-key bits

OUTPUT: $\langle o \rangle = (\langle o_0 \rangle, \dots, \langle o_{127} \rangle)$, a tuple of VOLE commitments to state bits after the AddRoundKey step

```
1: for  $i \in [0..128)$  do
2:    $\langle o_i \rangle \leftarrow \langle s_i \rangle + \langle k_i \rangle$ 
3: end for
4: return  $(\langle o_0 \rangle, \dots, \langle o_{127} \rangle)$ 
```

Vistruth.AddRoundKey($\langle s \rangle_\lambda^2, \langle k \rangle_\lambda^d$)

INPUT: $\langle s \rangle_\lambda^2 = (\langle s_0 \rangle_\lambda^2, \dots, \langle s_{15} \rangle_\lambda^2)$, a tuple of degree-2 commitments to state bytes; $\langle k \rangle_\lambda^d = (\langle k_0 \rangle_\lambda^d, \dots, \langle k_{15} \rangle_\lambda^d)$, a tuple of degree- d commitments to round-key bytes, where $d \in \{1, 2\}$

OUTPUT: $\langle o \rangle_\lambda^d = (\langle o_0 \rangle_\lambda^d, \dots, \langle o_{15} \rangle_\lambda^d)$, a tuple of degree- d commitments to state bytes after AddRoundKey

```
1: for  $i \in [0..16)$  do
2:    $\langle o_i \rangle_\lambda^d \leftarrow \langle s_i \rangle_\lambda^2 + \langle k_i \rangle_\lambda^d$ 
3: end for
4: return  $(\langle o_0 \rangle_\lambda^d, \dots, \langle o_{15} \rangle_\lambda^d)$ 
```

BitInvMix. The function [Vistruth.BitInvMix](#) implements the bit-level inverse of the public byte-wise inter-branch permutation [Vistrutah.Mix](#) on the 512-bit state. Its input is a tuple $\langle s \rangle$ of committed state bits, which is viewed as 64 consecutive committed bytes $\langle a_0 \rangle, \dots, \langle a_{63} \rangle$, each consisting of 8 committed bits.

Recall that [Vistrutah.Mix](#) takes four 128-bit branches, each containing 16 bytes, and rearranges them according to the fixed permutation $\pi_{\text{mix}} = (0, 1, 2, 3, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13, 14, 15)$, together with the interleaving of the four branches. Accordingly, [Vistruth.BitInvMix](#) undoes this rearrangement by moving the byte $\langle a_{4j+b} \rangle$ back to position $16b + \pi_{\text{mix}}[j]$, for each $j \in [0..16)$ and branch index $b \in \{0, 1, 2, 3\}$.

Vistruth.BitInvMix($\langle s \rangle$)

INPUT: $\langle s \rangle = (\langle s_0 \rangle, \dots, \langle s_{511} \rangle)$, a tuple of VOLE commitments to state bits

OUTPUT: $\langle o \rangle = (\langle o_0 \rangle, \dots, \langle o_{511} \rangle)$, a tuple of VOLE commitments to state bits after [Vistrutah.Mix](#)⁻¹

```
1: Let  $\pi_{\text{mix}} = (0, 1, 2, 3, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13, 14, 15)$ .
2: Parse  $\langle s \rangle$  as 64 committed bytes  $\langle a_0 \rangle, \langle a_1 \rangle, \dots, \langle a_{63} \rangle$ ,  $\langle a_i \rangle \in \{0, 1\}^8$ .
3: for  $j \in [0..16)$  do
4:    $u \leftarrow \pi_{\text{mix}}[j]$ 
5:    $\langle o_{16 \cdot 0 + u} \rangle \leftarrow \langle a_{4j+0} \rangle$ 
6:    $\langle o_{16 \cdot 1 + u} \rangle \leftarrow \langle a_{4j+1} \rangle$ 
7:    $\langle o_{16 \cdot 2 + u} \rangle \leftarrow \langle a_{4j+2} \rangle$ 
8:    $\langle o_{16 \cdot 3 + u} \rangle \leftarrow \langle a_{4j+3} \rangle$ 
9: end for
10: return  $\langle o_0 \rangle \parallel \langle o_1 \rangle \parallel \dots \parallel \langle o_{63} \rangle$ 
```

8.2 Witness Extension

The [Vistruth.ExtendWitness](#) algorithm takes the Vistrutah master key k , the plaintext in , and the instance parameters $param$, and returns an extended witness $w \in \{0, 1\}^\ell$ for later use. The extended witness consists of three components, corresponding to the key part, the initial intermediate phase, and the intermediate encryption phases.

- **Key bits** (`key_bits`): the full 512-bit master key k . This contributes 512 bits.

- **Initial norm bits** (`init_norm_bits`): after the initial whitening step, the algorithm obtains the 512-bit state $P = P_0 \parallel P_1 \parallel P_2 \parallel P_3$, and computes the inverse norm of each of its 64 bytes by applying [VistruthInvNorm](#). Since each inverse norm contributes 4 bits, this part contributes $64 \cdot 4 = 256$ bits.
- **Intermediate encryption bits** (`enc_bits`): for each intermediate step $i \in [1..R/2)$, the algorithm first computes the 512-bit state

$$\mathbf{M}_i = M_{i,0} \parallel M_{i,1} \parallel M_{i,2} \parallel M_{i,3}$$

obtained after the zero-key [AESRound](#) layer, the mixing layer, the round-key update, and the round-key addition. It then appends both the full state $\mathbf{M}_i$ and the packed inverse norms $\mathbf{N}_i$, where $\mathbf{N}_i$ contains the 4-bit output of [VistruthInvNorm](#) for each of the 64 bytes of $\mathbf{M}_i$. Thus, each intermediate step contributes $512 + 64 \cdot 4 = 768$ bits, and over all intermediate steps this contributes $768 \cdot (R/2 - 1)$ bits.

Therefore, the extended witness can be written as

$$\mathbf{w} = \underbrace{\text{key_bits}}_{512} \parallel \underbrace{\text{init_norm_bits}}_{256} \parallel \underbrace{\text{enc_bits}}_{768 \cdot (R/2 - 1)}.$$

For Vistrutah where $R = 18$, we have $\ell = 512 + 256 + 768 \cdot 8 = 6912$.

8.3 Deriving Constraints for the Encryption

This section describes how to derive constraints for the encryption routine in [VistruthEncCstrnts](#). Below, we first introduce several sub-functions.

ForwardByNorm. The function [VistruthForwardByNorm](#) reconstructs one fixed-key AES-like round from a committed 512-bit state together with the corresponding inverse-norm witness bits. Its input consists of a committed state $\langle \mathbf{S} \rangle$, the committed inverse-norm bits $\langle \mathbf{N} \rangle$, and the committed fixed-key branches $\langle FK_0 \rangle, \dots, \langle FK_3 \rangle$. Its output is the committed byte state $\langle \mathbf{A} \rangle_\lambda$ after the fixed-key AESRound, its bitwise Frobenius square $\langle \mathbf{A}^{[2]} \rangle_\lambda$, and a commitment set $\mathcal{Z}_{\text{norm}}$ where every element should be a commitment to zero.

The algorithm proceeds branch by branch. For each 128-bit branch $\langle \mathbf{S}_b \rangle$, it first computes, for every byte, all Frobenius conjugates of the input byte by calling [VistruthStateToConjugates](#). Thus, for each byte position u , it obtains the committed values $S_{b,u}, S_{b,u}^2, S_{b,u}^4, \dots, S_{b,u}^{128}$. In parallel, the corresponding 4-bit inverse-norm witness is lifted by [VistruthInvNormToConjugates](#), yielding the conjugates of the subfield element $S_{b,u}^{-17}$.

Using these two families of conjugates, the algorithm reconstructs the conjugates of the inverse byte. Since $S_{b,u}^{-1} = S_{b,u}^{16} \cdot S_{b,u}^{-17}$, multiplying the appropriate conjugates of $S_{b,u}^{16}$ and $S_{b,u}^{-17}$ yields the full sequence $S_{b,u}^{-1}, S_{b,u}^{-2}, \dots, S_{b,u}^{-128}$. At the same time, the algorithm appends to $\mathcal{Z}_{\text{norm}}$ the constraint $S_{b,u}^{-17} \cdot S_{b,u}^{16} \cdot S_{b,u}^2 + S_{b,u}$, which is zero when the inverse-norm witness is correct. These relations certify that the supplied norm bits indeed match the corresponding byte inverse.

After reconstructing the inverse-byte conjugates, the algorithm applies [VistruthSBoxAffine](#) twice: once in the ordinary mode and once in the squared mode. This produces, respectively, computes the affine S-box output and its bitwise Frobenius square. Each of these two byte states is then processed through the committed versions of [ShiftRows](#) and [MixColumns](#). The appropriate branch round key is then added: the ordinary branch key for the unsquared state, and the squared branch key for the squared state.

At the end, the four branch outputs are concatenated to form the committed 512-bit state $\langle \mathbf{A} \rangle_\lambda$ after the fixed-key AESRound, together with its bitwise square $\langle \mathbf{A}^{[2]} \rangle_\lambda$.

[VistruthForwardByNorm](#) serves as the main routine that turns the compressed inverse-norm witness into the full committed forward computation of one AES-like round, while simultaneously producing the algebraic checks needed to validate the reconstruction.

BackwardZeroRndByBits. The function [VistruthBackwardZeroRndByBits](#) reconstructs, at the bit level, the inverse-S-box outputs of the zero-key AESRound preceding the committed state $\langle \mathbf{M}_i \rangle$.

Vistrutith.ExtendWitness(*in*, *k*, param)

INPUT: $\mathbf{in} \in \{0, 1\}^{512}$, $\mathbf{k} \in \{0, 1\}^{512}$ OUTPUT: $\mathbf{w} \in \{0, 1\}^\ell$

```
1: Parse  $\mathbf{in} = \mathbf{in}_0 \parallel \mathbf{in}_1 \parallel \mathbf{in}_2 \parallel \mathbf{in}_3$ ,  $\mathbf{k} = \mathbf{k}_0 \parallel \mathbf{k}_1 \parallel \mathbf{k}_2 \parallel \mathbf{k}_3$  // all branches are 128 bits
2:  $FK_0 \leftarrow \mathbf{k}_0$ ,  $FK_1 \leftarrow \mathbf{k}_1$ 
3:  $(FK_2 \parallel FK_3) \leftarrow \text{Vistrutah.Shuffle}(\mathbf{k}_2 \parallel \mathbf{k}_3)$ 
4:  $RK_0 \leftarrow FK_1$ ,  $RK_1 \leftarrow FK_0$ ,  $RK_2 \leftarrow FK_3$ ,  $RK_3 \leftarrow FK_2$ 
5:  $\mathbf{w} \leftarrow \mathbf{k}$ 
6: // Initial whitening state
7:  $P_0 \leftarrow \mathbf{in}_0 \oplus RK_0$ 
8:  $P_1 \leftarrow \mathbf{in}_1 \oplus RK_1$ 
9:  $P_2 \leftarrow \mathbf{in}_2 \oplus RK_2$ 
10:  $P_3 \leftarrow \mathbf{in}_3 \oplus RK_3$ 
11:  $P \leftarrow P_0 \parallel P_1 \parallel P_2 \parallel P_3$ 
12: Parse  $P$  as 64 bytes  $(P_0^{\text{byte}}, \dots, P_{63}^{\text{byte}})$ .
13: new  $\mathbf{Q}_0 \in \{0, 1\}^{256}$ 
14: for  $t \in [0..64)$  do
15:    $\mathbf{Q}_0[4t..4t+3] \leftarrow \text{Vistrutith.InvNorm}(P_t^{\text{byte}})$  // Store the inverse norm of  $P$ 
16: end for
17:  $\mathbf{w} \leftarrow \mathbf{w} \parallel \mathbf{Q}_0$ 
18: // Compute the first fixed-key AESRound, but do not store its output
19:  $A_0 \leftarrow \text{AESRound}(P_0, FK_0)$ 
20:  $A_1 \leftarrow \text{AESRound}(P_1, FK_1)$ 
21:  $A_2 \leftarrow \text{AESRound}(P_2, FK_2)$ 
22:  $A_3 \leftarrow \text{AESRound}(P_3, FK_3)$ 
23: // Intermediate Vistrutah steps
24: for  $i \in [1..R/2)$  do
25:    $S_0 \leftarrow \text{AESRound}(A_0, \mathbf{0}^{128})$ 
26:    $S_1 \leftarrow \text{AESRound}(A_1, \mathbf{0}^{128})$ 
27:    $S_2 \leftarrow \text{AESRound}(A_2, \mathbf{0}^{128})$ 
28:    $S_3 \leftarrow \text{AESRound}(A_3, \mathbf{0}^{128})$ 
29:    $(S_0, S_1, S_2, S_3) \leftarrow \text{Vistrutah.Mix}(S_0, S_1, S_2, S_3)$ 
30:    $RK_0 \leftarrow \text{Vistrutah.RotByte}_5(RK_0)$ 
31:    $RK_1 \leftarrow \text{Vistrutah.RotByte}_{10}(RK_1)$ 
32:    $RK_2 \leftarrow \text{Vistrutah.RotByte}_5(RK_2)$ 
33:    $RK_3 \leftarrow \text{Vistrutah.RotByte}_{10}(RK_3)$ 
34:    $M_{i,0} \leftarrow S_0 \oplus RK_0 \oplus RC_{i-1}$ 
35:    $M_{i,1} \leftarrow S_1 \oplus RK_1$ 
36:    $M_{i,2} \leftarrow S_2 \oplus RK_2$ 
37:    $M_{i,3} \leftarrow S_3 \oplus RK_3$ 
38:    $\mathbf{M}_i \leftarrow M_{i,0} \parallel M_{i,1} \parallel M_{i,2} \parallel M_{i,3}$ 
39:   Parse  $\mathbf{M}_i$  as 64 bytes  $(M_{i,0}^{\text{byte}}, \dots, M_{i,63}^{\text{byte}})$ .
40:   new  $\mathbf{N}_i \in \{0, 1\}^{256}$ 
41:   for  $t \in [0..64)$  do
42:      $\mathbf{N}_i[4t..4t+3] \leftarrow \text{Vistrutith.InvNorm}(M_{i,t}^{\text{byte}})$ 
43:   end for
44:    $\mathbf{w} \leftarrow \mathbf{w} \parallel \mathbf{M}_i \parallel \mathbf{N}_i$ 
45:    $A_0 \leftarrow \text{AESRound}(M_{i,0}, FK_0)$ 
46:    $A_1 \leftarrow \text{AESRound}(M_{i,1}, FK_1)$ 
47:    $A_2 \leftarrow \text{AESRound}(M_{i,2}, FK_2)$ 
48:    $A_3 \leftarrow \text{AESRound}(M_{i,3}, FK_3)$ 
49: end for
50: //The final output is not appended
51: return  $\mathbf{w}$ 
```

Vistrutith.ForwardByNorm($\langle \mathbf{S} \rangle, \langle \mathbf{N} \rangle, \langle FK_0 \rangle, \dots, \langle FK_3 \rangle$)

INPUT: $\langle \mathbf{S} \rangle$, a committed 512-bit state; $\langle \mathbf{N} \rangle \in \mathbb{F}_2^{256}$, four InvNorm bits per byte; committed fixed-key branches $\langle FK_0 \rangle, \dots, \langle FK_3 \rangle$

OUTPUT: $(\langle \mathbf{A} \rangle_\lambda, \langle \mathbf{A}^{[2]} \rangle_\lambda, \mathcal{Z}_{\text{norm}})$, where $\mathbf{A}$ is the byte state after the fixed-key AESRound, and $\mathbf{A}^{[2]}$ is its bitwise Frobenius square

```

1: new  $\mathcal{Z}_{\text{norm}}$ 
2: for  $b \in [0..4)$  do
3:   Let  $\langle \mathbf{S}_b \rangle$  be the  $b$ -th 128-bit branch of  $\langle \mathbf{S} \rangle$ 
4:    $\langle C_{b,u,0} \rangle_\lambda, \dots, \langle C_{b,u,7} \rangle_\lambda \leftarrow \text{Vistrutith.StateToConjugates}(\langle \mathbf{S}_b \rangle) \parallel C_{b,u,j} = S_{b,u}^{2^j}$  for byte-indices  $u \in [0, 16)$ , i.e.,  $S_{b,u}$  is the  $u$ -th byte in  $\mathbf{S}_b$ 
5:   for  $u \in [0..16)$  do
6:      $t \leftarrow 16b + u$ 
7:      $\langle \langle H_{b,u,0} \rangle_\lambda, \dots, \langle H_{b,u,3} \rangle_\lambda \rangle \leftarrow \text{Vistrutith.InvNormToConjugates}(\langle N[4t] \rangle, \dots, \langle N[4t+3] \rangle)$ 
8:      $\langle z \rangle \leftarrow \langle H_{b,u,0} \rangle_\lambda \cdot \langle C_{b,u,4} \rangle_\lambda \cdot \langle C_{b,u,1} \rangle_\lambda + \langle C_{b,u,0} \rangle_\lambda \parallel x^{-17} \cdot x^{16} \cdot x^2 + x = 0$ 
9:     Append  $z$  to  $\mathcal{Z}_{\text{norm}}$ 
10:    for  $j \in [0..8)$  do
11:       $\langle P_{b,u,j} \rangle_\lambda \leftarrow \langle C_{b,u,(j+4) \bmod 8} \rangle_\lambda \cdot \langle H_{b,u,j \bmod 4} \rangle_\lambda \parallel x^{-1}, x^{-2}, \dots, x^{-128}$ 
12:    end for
13:  end for
14:  for  $sq \in \{\text{false}, \text{true}\}$  do
15:     $\langle \mathbf{A}_b^{sq} \rangle_\lambda \leftarrow \text{Vistrutith.SBoxAffine}(\langle P_{b,0,0} \rangle_\lambda, \dots, \langle P_{b,0,7} \rangle_\lambda, \dots, \langle P_{b,15,0} \rangle_\lambda, \dots, \langle P_{b,15,7} \rangle_\lambda; 16, sq)$ 
16:     $\langle \mathbf{A}_b^{sq} \rangle_\lambda \leftarrow \text{Vistrutith.ShiftRows}(\langle \mathbf{A}_b^{sq} \rangle_\lambda)$ 
17:     $\langle \mathbf{A}_b^{sq} \rangle_\lambda \leftarrow \text{Vistrutith.MixColumns}(\langle \mathbf{A}_b^{sq} \rangle_\lambda, sq)$ 
18:  end for
19:   $\langle FK_b^{(0)} \rangle_\lambda \leftarrow \text{Vistrutith.StateToBytes}(\text{ConstantToVOLE}(FK_b))$ 
20:   $\langle FK_b^{(1)} \rangle_\lambda \leftarrow \text{Vistrutith.StateToBytes}(\text{Vistrutith.SquareBits}(\text{ConstantToVOLE}(FK_b)))$ 
21:   $\langle \mathbf{A}_b^0 \rangle_\lambda \leftarrow \text{Vistrutith.AddRoundKey}(\langle \mathbf{A}_b^{\text{false}} \rangle_\lambda, \langle FK_b^{(0)} \rangle_\lambda)$ 
22:   $\langle \mathbf{A}_b^1 \rangle_\lambda \leftarrow \text{Vistrutith.AddRoundKey}(\langle \mathbf{A}_b^{\text{true}} \rangle_\lambda, \langle FK_b^{(1)} \rangle_\lambda)$ 
23: end for
24:  $\langle \mathbf{A} \rangle_\lambda \leftarrow \langle \mathbf{A}_0^0 \rangle_\lambda \parallel \langle \mathbf{A}_1^0 \rangle_\lambda \parallel \langle \mathbf{A}_2^0 \rangle_\lambda \parallel \langle \mathbf{A}_3^0 \rangle_\lambda$ 
25:  $\langle \mathbf{A}^{[2]} \rangle_\lambda \leftarrow \langle \mathbf{A}_0^1 \rangle_\lambda \parallel \langle \mathbf{A}_1^1 \rangle_\lambda \parallel \langle \mathbf{A}_2^1 \rangle_\lambda \parallel \langle \mathbf{A}_3^1 \rangle_\lambda$ 
26: return  $(\langle \mathbf{A} \rangle_\lambda, \langle \mathbf{A}^{[2]} \rangle_\lambda, \mathcal{Z}_{\text{norm}})$ 

```

Its input is the committed 512-bit state after the application of [Vistrutah.Mix](#) and the subsequent round-key injection, together with the committed round-key branches $\langle RK_0^{(i)} \rangle, \dots, \langle RK_3^{(i)} \rangle$ and the public round constant RC_{i-1} . Its output is a committed 512-bit bitstring $\langle Y_i \rangle$, representing the state immediately before the SubBytes layer of that zero-key AESRound.

The algorithm first parses $\langle M_i \rangle$ into four 128-bit branches $\langle M_{i,0} \rangle, \dots, \langle M_{i,3} \rangle$. It then removes the round-key injection: for the first branch, this means xoring both $\langle RK_0^{(i)} \rangle$ and the round constant RC_{i-1} , while for the remaining three branches it xors only the corresponding round-key branch. This yields the intermediate branches $\langle U_0 \rangle, \dots, \langle U_3 \rangle$.

Next, the algorithm inverts the public linear layers of the zero-key AESRound in reverse order. It first applies [Vistrutith.BitInvMix](#) to undo the inter-branch mixing step [Vistrutah.Mix](#). Then, on each 128-bit branch separately, it applies [Vistrutith.BitInvMixColumns](#) to undo [MixColumns](#), followed by [Vistrutith.BitInvShiftRows](#) to undo [ShiftRows](#). At this point, each branch is in the bit representation of the affine S-box output. Finally, the algorithm applies [Vistrutith.SBoxInvAffine](#) to each branch, thereby recovering the committed bits of the patched inversions in S-boxes.

[Vistrutith.BackwardZeroRndByBits](#)($\langle M_i \rangle, \langle RK_0^{(i)} \rangle, \dots, \langle RK_3^{(i)} \rangle, RC_{i-1}$)

INPUT: $\langle M_i \rangle$, the committed 512-bit state after [Vistrutah.Mix](#) and round-key injection; round-key branches $\langle RK_0^{(i)} \rangle, \dots, \langle RK_3^{(i)} \rangle$; round constant RC_{i-1}

OUTPUT: $\langle Y_i \rangle$, the committed bit representation of the S-box inverse outputs of the preceding zero-key AESRound

```

1: Parse  $\langle M_i \rangle = \langle M_{i,0} \rangle \parallel \langle M_{i,1} \rangle \parallel \langle M_{i,2} \rangle \parallel \langle M_{i,3} \rangle$ .
2:  $\langle U_0 \rangle \leftarrow \langle M_{i,0} \rangle + \langle RK_0^{(i)} \rangle + \langle RC_{i-1} \rangle$ .
3: for  $b \in \{1, 2, 3\}$  do
4:    $\langle U_b \rangle \leftarrow \langle M_{i,b} \rangle + \langle RK_b^{(i)} \rangle$ .
5: end for
6:  $(\langle U_0 \rangle, \langle U_1 \rangle, \langle U_2 \rangle, \langle U_3 \rangle) \leftarrow \text{Vistrutith.BitInvMix}(\langle U_0 \rangle, \langle U_1 \rangle, \langle U_2 \rangle, \langle U_3 \rangle)$ .
7: for  $b \in [0..4)$  do
8:    $\langle U_b \rangle \leftarrow \text{Vistrutith.BitInvMixColumns}(\langle U_b \rangle)$ .
9:    $\langle U_b \rangle \leftarrow \text{Vistrutith.BitInvShiftRows}(\langle U_b \rangle)$ .
10:   $\langle Y_b \rangle \leftarrow \text{Vistrutith.SBoxInvAffine}(\langle U_b \rangle)$ .
11: end for
12: return  $\langle Y_i \rangle = \langle Y_0 \rangle \parallel \langle Y_1 \rangle \parallel \langle Y_2 \rangle \parallel \langle Y_3 \rangle$ .
```

AppendCstrntsByBits. The function [Vistrutith.AppendCstrntsByBits](#) links a committed byte state $\langle X \rangle_\lambda$ and its bitwise square $\langle X^{[2]} \rangle_\lambda$ to a committed bit state $\langle Y \rangle$. It parses $\langle Y \rangle$ into 64 bytes, and for each byte position t , reconstructs both the byte value $\langle Y_t \rangle_\lambda$ and its bitwise square $\langle Y_t^{[2]} \rangle_\lambda$ using [Vistrutith.ByteCombine](#) and [Vistrutith.SquareAndCombine](#), respectively. It then appends two degree-at-most-3 constraints, $\langle X_t^{[2]} \rangle_\lambda \cdot \langle Y_t \rangle_\lambda + \langle X_t \rangle_\lambda$ and $\langle X_t \rangle_\lambda \cdot \langle Y_t^{[2]} \rangle_\lambda + \langle Y_t \rangle_\lambda$, which enforce consistency between the byte-level and bit-level representations.

BackwardFinalRndByBits. The function [Vistrutith.BackwardFinalRndByBits](#) reconstructs, at the bit level, the inverse-S-box outputs of the final AES round from the committed ciphertext and the committed final round keys. Its input consists of the committed 512-bit ciphertext $\langle out \rangle$, together with the final round-key branches $\langle RK_0^{(T)} \rangle, \dots, \langle RK_3^{(T)} \rangle$. Its output is a committed 512-bit bitstring $\langle Y_T \rangle$, representing the state immediately before [SubBytes](#) of the final AES round.

The algorithm first parses the ciphertext into four 128-bit branches $\langle out_0 \rangle, \dots, \langle out_3 \rangle$. For each branch b , it removes the final AddRoundKey operation by xoring the corresponding committed round key $\langle RK_b^{(T)} \rangle$. Since the final AES round does not contain a [MixColumns](#) layer, the algorithm then only applies the inverse row permutation [Vistrutith.BitInvShiftRows](#), followed by the inverse affine map [Vistrutith.SBoxInvAffine](#). This yields the committed bit representation of the inverse-S-box outputs for each branch. Finally, the four branches are concatenated to obtain $\langle Y_T \rangle$.

Vistruthith.AppendCstrntsByBits($\langle \mathbf{X} \rangle_\lambda, \langle \mathbf{X}^{[2]} \rangle_\lambda, \langle \mathbf{Y} \rangle$)

INPUT: $\langle \mathbf{X} \rangle_\lambda$, a committed 512-bit byte state; $\langle \mathbf{X}^{[2]} \rangle_\lambda$, its bitwise Frobenius square; $\langle \mathbf{Y} \rangle$, a committed 512-bit bit state

OUTPUT: $\mathcal{Z}_{io}$, a list of degree-at-most-3 constraints

```

1: new  $\mathcal{Z}_{io}$ 
2: for  $t \in [0..64]$  do
3:    $\langle Y_t \rangle_\lambda \leftarrow \text{Vistruthith.ByteCombine}(\langle Y[8t] \rangle, \dots, \langle Y[8t+7] \rangle)$ 
4:    $\langle Y_t^{[2]} \rangle_\lambda \leftarrow \text{Vistruthith.SquareAndCombine}(\langle Y[8t] \rangle, \dots, \langle Y[8t+7] \rangle)$ 
5:    $\langle z_t^{(0)} \rangle \leftarrow \langle X_t^{[2]} \rangle_\lambda \cdot \langle Y_t \rangle_\lambda + \langle X_t \rangle_\lambda$ 
6:    $\langle z_t^{(1)} \rangle \leftarrow \langle X_t \rangle_\lambda \cdot \langle Y_t^{[2]} \rangle_\lambda + \langle Y_t \rangle_\lambda$ 
7:   Append  $\langle z_t^{(0)} \rangle$  and  $\langle z_t^{(1)} \rangle$  to  $\mathcal{Z}_{io}$ 
8: end for
9: return  $\mathcal{Z}_{io}$ 

```

Vistruthith.BackwardFinalRndByBits($\langle \mathbf{out} \rangle, \langle RK_0^{(T)} \rangle, \dots, \langle RK_3^{(T)} \rangle$)

INPUT: $\langle \mathbf{out} \rangle$, the committed 512-bit ciphertext; final round-key branches $\langle RK_0^{(T)} \rangle, \dots, \langle RK_3^{(T)} \rangle$

OUTPUT: $\langle \mathbf{Y}_T \rangle$, the committed bit representation of the S-box inverse outputs of the final AES round

```

1: Parse  $\langle \mathbf{out} \rangle = \langle out_0 \rangle \parallel \langle out_1 \rangle \parallel \langle out_2 \rangle \parallel \langle out_3 \rangle$ .
2: for  $b \in [0..4]$  do
3:    $\langle U_b \rangle \leftarrow \langle out_b \rangle + \langle RK_b^{(T)} \rangle$ .
4:    $\langle U_b \rangle \leftarrow \text{Vistruthith.BitInvShiftRows}(\langle U_b \rangle)$ .
5:    $\langle Y_b \rangle \leftarrow \text{Vistruthith.SBoxInvAffine}(\langle U_b \rangle)$ .
6: end for
7: return  $\langle \mathbf{Y}_T \rangle = \langle Y_0 \rangle \parallel \langle Y_1 \rangle \parallel \langle Y_2 \rangle \parallel \langle Y_3 \rangle$ .

```

EncCstrnts. Based on the building blocks above, the function **Vistruthith.EncCstrnts** constructs the full set of algebraic constraints certifying the correctness of the Vistrutah encryption relation. Its input consists of commitments to the plaintext $\mathbf{in}$, the ciphertext $\mathbf{out}$, and the extended witness $\mathbf{w}$. Its output is a collection of degree-3 constraint commitments, organized into three lists $\mathcal{Z}_{\text{norm}}$, $\mathcal{Z}_{io,0}$, and $\mathcal{Z}_{io,1}$, all of which are required to open to zero.

The witness $\mathbf{w}$ is first parsed into the committed master key $\langle \mathbf{K} \rangle$, the initial inverse-norm bits $\langle \mathbf{Q}_0 \rangle$, and the intermediate witness blocks $(\langle \mathbf{M}_i \rangle, \langle \mathbf{N}_i \rangle)$ for $i = 1, \dots, T-1$, where $T = R/2$. From the committed key, the algorithm reconstructs the fixed-key branches $FK_0, \dots, FK_3$ and all rotated round-key branches $RK_0^{(j)}, \dots, RK_3^{(j)}$ for $j = 0, \dots, T$. Thus, all round keys used by the encryption circuit are derived deterministically from the committed witness key.

The first phase checks the initial fixed-key AESRound. Starting from the committed plaintext, the algorithm first applies the committed bit-level AddRoundKey using the initial round keys $RK_b^{(0)}$, obtaining the whitened state $\langle \mathbf{P} \rangle$. It then invokes **Vistruthith.ForwardByNorm** on $\langle \mathbf{P} \rangle$ and the committed inverse-norm bits $\langle \mathbf{Q}_0 \rangle$. This produces the committed byte state after the first fixed-key AESRound, together with its bitwise Frobenius square and a collection of norm-consistency constraints. These constraints are appended to $\mathcal{Z}_{\text{norm}}$.

The second phase iterates through the intermediate Vistrutah steps. For each $i \in [1..T]$, the algorithm first calls **Vistruthith.BackwardZeroRndByBits** on the committed intermediate state $\langle \mathbf{M}_i \rangle$, the corresponding rotated round keys, and the public round constant RC_{i-1} . This reconstructs the committed bit-level S-box inputs of the preceding zero-key AESRound. The procedure **Vistruthith.AppendCstrntsByBits** is then used to enforce consistency between these reconstructed bit-level values and the previously computed byte-level forward state $\langle \mathbf{A} \rangle_\lambda$ and its square $\langle \mathbf{A}^{[2]} \rangle_\lambda$. The resulting constraints are appended to $\mathcal{Z}_{io,0}$ and $\mathcal{Z}_{io,1}$.

After this backward check, the algorithm performs the next forward fixed-key AESRound by calling **Vistruthith.ForwardByNorm** on $\langle \mathbf{M}_i \rangle$ and the committed inverse-norm witness $\langle \mathbf{N}_i \rangle$. This again yields a new byte-level forward state, its square, and additional norm-consistency constraints,

which are appended to $\mathcal{Z}_{\text{norm}}$. In this way, each intermediate witness block is checked both backward and forward, thereby linking consecutive rounds of the encryption computation.

The final phase checks the last AES round against the committed ciphertext. The algorithm invokes [Vistruth.BackwardFinalRndByBits](#) on $\langle \text{out} \rangle$ and the final rotated round keys, obtaining the committed bit-level S-box inputs of the last AES round. It then calls [Vistruth.AppendCstrntsByBits](#) once more to enforce consistency between this reconstructed final-round input and the last forward state $\langle \mathbf{A} \rangle_\lambda$ together with its square $\langle \mathbf{A}^{[2]} \rangle_\lambda$. The constraints are appended to $\mathcal{Z}_{\text{io},0}$ and $\mathcal{Z}_{\text{io},1}$.

Overall, [Vistruth.EncCstrnts](#) combines three kinds of checks: the norm constraints validating the inverse-norm witness, the input/output consistency constraints linking byte-level and bit-level representations, and the final consistency check against the ciphertext. Hence, the concatenation $\mathcal{Z}_{\text{norm}} \parallel \mathcal{Z}_{\text{io},0} \parallel \mathcal{Z}_{\text{io},1}$ vanishes exactly when the committed witness $\langle \mathbf{w} \rangle$ is consistent with a correct Vistruth encryption from *in* to *out*.

Vistruth.EncCstrnts($\langle \text{in} \rangle, \langle \text{out} \rangle, \langle \mathbf{w} \rangle$; param)

INPUT: Commitments to $\text{in} \in \{0, 1\}^{512}$, $\text{out} \in \{0, 1\}^{512}$, and extended witness $\mathbf{w}$

OUTPUT: Three lists of degree-3 commitments $\mathcal{Z}_{\text{norm}}, \mathcal{Z}_{\text{io},0}, \mathcal{Z}_{\text{io},1}$, all of which must open to zero

```

1:  $T \leftarrow R/2$ 
2: new  $\mathcal{Z}_{\text{norm}}, \mathcal{Z}_{\text{io},0}, \mathcal{Z}_{\text{io},1}$ 
3: Parse  $\langle \mathbf{w} \rangle = \langle \mathbf{K} \rangle \parallel \langle \mathbf{Q}_0 \rangle \parallel (\langle \mathbf{M}_1 \rangle \parallel \langle \mathbf{N}_1 \rangle) \parallel \dots \parallel (\langle \mathbf{M}_{T-1} \rangle \parallel \langle \mathbf{N}_{T-1} \rangle)$ 
4:  $// |\mathbf{K}| = 512, |\mathbf{Q}_0| = 256, |\mathbf{M}_i| = 512, |\mathbf{N}_i| = 256$ 
5: Parse  $\langle \mathbf{K} \rangle = \langle \mathbf{k}_0 \rangle \parallel \langle \mathbf{k}_1 \rangle \parallel \langle \mathbf{k}_2 \rangle \parallel \langle \mathbf{k}_3 \rangle$ 
6:  $//$  Compute the rotated round-key branches
7:  $\langle FK_0 \rangle \leftarrow \langle \mathbf{k}_0 \rangle, \quad \langle FK_1 \rangle \leftarrow \langle \mathbf{k}_1 \rangle$ 
8:  $(\langle FK_2 \rangle \parallel \langle FK_3 \rangle) \leftarrow \text{Vistruth.Shuffle}(\langle \mathbf{k}_2 \rangle \parallel \langle \mathbf{k}_3 \rangle)$ 
9: for  $j \in [0, T]$  do
10:    $\langle RK_0^{(j)} \rangle \leftarrow \text{Vistruth.RotByte}_5^j(\langle FK_1 \rangle)$ 
11:    $\langle RK_1^{(j)} \rangle \leftarrow \text{Vistruth.RotByte}_{10}^j(\langle FK_0 \rangle)$ 
12:    $\langle RK_2^{(j)} \rangle \leftarrow \text{Vistruth.RotByte}_5^j(\langle FK_3 \rangle)$ 
13:    $\langle RK_3^{(j)} \rangle \leftarrow \text{Vistruth.RotByte}_{10}^j(\langle FK_2 \rangle)$ 
14: end for
15:  $//$  Forward-check the initial fixed-key AESRound using  $\mathbf{Q}_0$ 
16: Parse  $\langle \text{in} \rangle = \langle \text{in}_0 \rangle \parallel \langle \text{in}_1 \rangle \parallel \langle \text{in}_2 \rangle \parallel \langle \text{in}_3 \rangle$ .
17: for  $b \in [0..4)$  do
18:    $\langle P_b \rangle \leftarrow \text{Vistruth.BitAddRoundKey}(\langle \text{in}_b \rangle, \langle RK_b^{(0)} \rangle)$ 
19: end for
20:  $\langle \mathbf{P} \rangle \leftarrow \langle P_0 \rangle \parallel \langle P_1 \rangle \parallel \langle P_2 \rangle \parallel \langle P_3 \rangle$ .
21:  $(\langle \mathbf{A} \rangle_\lambda, \langle \mathbf{A}^{[2]} \rangle_\lambda, \mathcal{Z}) \leftarrow \text{Vistruth.ForwardByNorm}(\langle \mathbf{P} \rangle, \langle \mathbf{Q}_0 \rangle, \langle FK_0 \rangle, \dots, \langle FK_3 \rangle)$ 
22:  $\mathcal{Z}_{\text{norm}} \leftarrow \mathcal{Z}_{\text{norm}} \parallel \mathcal{Z}$ 
23:  $//$  For each intermediate step, check the zero-key AESRound backwards and the following fixed-key
    AESRound forwards
24: for  $i \in [1..T)$  do
25:    $\langle \mathbf{Y}_i \rangle \leftarrow \text{Vistruth.BackwardZeroRndByBits}(\langle \mathbf{M}_i \rangle, \langle RK_0^{(i)} \rangle, \dots, \langle RK_3^{(i)} \rangle, \langle RK_{i-1} \rangle)$ 
26:   Append Vistruth.AppendCstrntsByBits( $\langle \mathbf{A} \rangle_\lambda, \langle \mathbf{A}^{[2]} \rangle_\lambda, \langle \mathbf{Y}_i \rangle$ ) to  $\mathcal{Z}_{\text{io},0}$  and  $\mathcal{Z}_{\text{io},1}$ .
27:    $(\langle \mathbf{A} \rangle_\lambda, \langle \mathbf{A}^{[2]} \rangle_\lambda, \mathcal{Z}) \leftarrow \text{Vistruth.ForwardByNorm}(\langle \mathbf{M}_i \rangle, \langle \mathbf{N}_i \rangle, \langle FK_0 \rangle, \dots, \langle FK_3 \rangle)$ 
28:    $\mathcal{Z}_{\text{norm}} \leftarrow \mathcal{Z}_{\text{norm}} \parallel \mathcal{Z}$ 
29: end for
30:  $//$  Check the final AES round backwards from the ciphertext
31:  $\langle \mathbf{Y}_T \rangle \leftarrow \text{Vistruth.BackwardFinalRndByBits}(\langle \text{out} \rangle, \langle RK_0^{(T)} \rangle, \dots, \langle RK_3^{(T)} \rangle)$ 
32: Append Vistruth.AppendCstrntsByBits( $\langle \mathbf{A} \rangle_\lambda, \langle \mathbf{A}^{[2]} \rangle_\lambda, \langle \mathbf{Y}_T \rangle$ ) to  $\mathcal{Z}_{\text{io},0}$  and  $\mathcal{Z}_{\text{io},1}$ 
33: return  $\mathcal{Z}_{\text{norm}} \parallel \mathcal{Z}_{\text{io},0} \parallel \mathcal{Z}_{\text{io},1}$ 

```

8.4 Complete OWF Constraints

This section describes the final QuickSilver relation for the Vistrutah one-way function. We write the public key as $\text{pk} = (\text{pk}_1, \text{pk}_2) \in \{0, 1\}^{512} \times \{0, 1\}^{512}$, where pk_1 is the public 512-bit input block and pk_2 is the public 512-bit output block. The extended witness is obtained from [Vistrutith.ExtendWitness](#), namely $\mathbf{w} = \mathbf{K} \parallel \mathbf{Q}_0 \parallel (\mathbf{M}_1 \parallel \mathbf{N}_1) \parallel \dots \parallel (\mathbf{M}_{T-1} \parallel \mathbf{N}_{T-1})$ with $T = R/2$.

Unlike the SM4 and Vistrutah-based variants, [Vistrutith](#) does not require a separate nonlinear key-expansion constraint routine. The first 512 witness bits already contain the master key, and all branch keys and rotated round-key branches used by the cipher are public $\mathbb{F}_2$ -linear functions of it. Therefore the complete OWF relation consists only of two parts: the standard one-bit key-space-reduction term and the encryption constraints returned by [Vistrutith.EncCstrnts](#). Since the reduction term has degree 2 while [Vistrutith.EncCstrnts](#) returns degree-3 commitments, we first define a small lifting routine [Deg2To3](#).

Deg2To3. The function [Deg2To3](#) takes a scalar or vector of degree-2 commitments and lifts it to degree 3. It does so by multiplying each entry with a degree-1 commitment to the constant 1, obtained through [ConstantToVOLE](#). Since 1 is the multiplicative identity in the embedded copy of $\mathbb{F}_2 \subseteq \mathbb{F}_{2^\lambda}$, the committed value is unchanged.

[Deg2To3](#)($\langle \mathbf{x} \rangle_\lambda^2$)

INPUT: $\langle \mathbf{x} \rangle_\lambda^2 = (\langle x_0 \rangle_\lambda^2, \dots, \langle x_{m-1} \rangle_\lambda^2)$, where $m \geq 1$; a scalar input is treated as the case $m = 1$

OUTPUT: $\langle \mathbf{y} \rangle_\lambda^3 = (\langle y_0 \rangle_\lambda^3, \dots, \langle y_{m-1} \rangle_\lambda^3)$, where $y_i = x_i$ for all i

- 1: $\langle 1 \rangle \leftarrow \text{ConstantToVOLE}(1)$
 - 2: **for** $i \in [0..m)$ **do**
 - 3: $\langle y_i \rangle_\lambda^3 \leftarrow \langle x_i \rangle_\lambda^2 \cdot \langle 1 \rangle$
 - 4: **end for**
 - 5: **return** $\langle \mathbf{y} \rangle_\lambda^3$
-

Vistrutith.OWFConstraints. This is the main routine that builds the complete Vistrutith OWF relation. It first parses the public key and allocates a fresh buffer $\langle \mathbf{z} \rangle_\lambda^3$ for the accumulated constraints. It then appends the standard key-space-reduction term $\langle w[0] \rangle \cdot \langle w[1] \rangle$, lifted to degree 3 using [Deg2To3](#). After that, it turns the public input and output blocks into bitwise VOLE commitments by calling [ConstantToVOLE](#). Finally, it selects the Vistrutith witness slice and calls [Vistrutith.EncCstrnts](#). Since [Vistrutith.EncCstrnts](#) already returns degree-3 commitments, no further lifting is needed. The output is the concatenation of the lifted key-space-reduction term and the encryption constraints.

[Vistrutith.OWFConstraints](#)($\langle \mathbf{w} \rangle$, pk ; param)

INPUT: Committed witness $\langle \mathbf{w} \rangle$, where $\mathbf{w} \in \mathbb{F}_2^\ell$

OUTPUT: Committed constraints $\langle \mathbf{z} \rangle_\lambda^3$, where $\mathbf{z} \in \mathbb{F}_{2^\lambda}^{\leq C}$ and the verifier checks that $\mathbf{z} = \mathbf{0}$

- 1: Parse $\text{pk} = (\text{pk}_1, \text{pk}_2) \in \{0, 1\}^{512} \times \{0, 1\}^{512}$
 - 2: **new** $\langle \mathbf{z} \rangle_\lambda^3$, where $\mathbf{z} \in \mathbb{F}_{2^\lambda}^{\leq C}$
 - 3: $\langle \tilde{\mathbf{z}} \rangle_\lambda^3 \leftarrow \text{Deg2To3}(\langle w[0] \rangle \cdot \langle w[1] \rangle)$ // reduce the key-space by one bit
 - 4: $\langle \mathbf{z} \rangle_\lambda^3 \leftarrow \langle \mathbf{z} \rangle_\lambda^3 \parallel \langle \tilde{\mathbf{z}} \rangle_\lambda^3$
 - 5: $\langle \mathbf{in} \rangle \leftarrow \text{ConstantToVOLE}(\text{pk}_1)$
 - 6: $\langle \mathbf{out} \rangle \leftarrow \text{ConstantToVOLE}(\text{pk}_2)$
 - 7: $\langle \tilde{\mathbf{w}} \rangle \leftarrow \langle w[0..\ell] \rangle$ // Vistrutith uses no separate key-expansion witness
 - 8: $\langle \tilde{\mathbf{z}} \rangle_\lambda^3 \leftarrow \text{Vistrutith.EncCstrnts}(\langle \mathbf{in} \rangle, \langle \mathbf{out} \rangle, \langle \tilde{\mathbf{w}} \rangle; \text{param}, \text{param}_{\text{OWF}})$
 - 9: $\langle \mathbf{z} \rangle_\lambda^3 \leftarrow \langle \mathbf{z} \rangle_\lambda^3 \parallel \langle \tilde{\mathbf{z}} \rangle_\lambda^3$
 - 10: **return** $\langle \mathbf{z} \rangle_\lambda^3$
-

Vistrutith.OWFProve and **Vistrutith.OWFVerify**. The prover and verifier follow the standard degree-3 QuickSilver interface. The prover first turns every witness bit into a VOLE commitment using one row of $\mathbf{V}$ as the tag. It then embeds the additional 2λ mask rows into field elements, computes **Vistrutith.OWFConstraints**, and parses the resulting degree-3 vector as coefficient vectors $(\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2)$. These are hashed together with the corresponding masks to produce $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$.

The verifier reconstructs the committed witness values from $\mathbf{d}$, $\mathbf{Q}$, and the challenge $\Delta = \text{ToField}(\text{chall}_3; \lambda)$. It also reconstructs the mask contribution $q^* = q_0^* + \Delta \cdot q_1^*$, recomputes **Vistrutith.OWFConstraints** $(\langle \mathbf{w} \rangle, \text{pk})$, and hashes the evaluated constraint vector. Finally, it subtracts the terms involving $\tilde{a}_1$ and $\tilde{a}_2$ to recover $\tilde{a}_0$. If the proof is valid, the returned value must match the prover's $\tilde{a}_0$.

Vistrutith.OWFProve $(\mathbf{w}, \mathbf{u}, \mathbf{V}, \text{pk}, \text{chall}_2; \text{param}, \text{param}_{\text{OWF}})$

INPUT: $\mathbf{w} \in \{0, 1\}^\ell$, $\mathbf{u} \in \{0, 1\}^{2\lambda}$, $\mathbf{V} \in \{0, 1\}^{(\ell+2\lambda) \times \lambda}$, pk , and chall_2

OUTPUT: $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2 \in \mathbb{F}_{2^\lambda}$

```

1: for  $i \in [0, \ell)$  do
2:   Let  $\langle w[i] \rangle \leftarrow (\mathbf{w}[i], \text{ToField}(\mathbf{V}|_i; \lambda))$ 
3: end for
4: for  $i \in [0, 2\lambda)$  do
5:    $v[i] \leftarrow \text{ToField}(\mathbf{V}|_{\ell+i}; \lambda)$ 
6: end for
7: for  $j \in [0, 2)$  do
8:    $u_j^* \leftarrow \text{ToField}(\mathbf{u}[j\lambda..(j+1)\lambda]; \lambda)$ 
9:    $v_j^* \leftarrow \sum_{i \in [0.. \lambda)} v[i + \lambda j] \alpha_\lambda^i$ 
10: end for
11: // compute a degree-3 constraint vector
12:  $\langle \mathbf{z} \rangle_\lambda^3 \leftarrow \text{Vistrutith.OWFConstraints}(\langle \mathbf{w} \rangle, \text{pk}; \text{param}, \text{param}_{\text{OWF}})$ 
13: Parse  $\langle \mathbf{z} \rangle_\lambda^3$  as  $(\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2) \in (\mathbb{F}_{2^\lambda}^C)^3$ 
14:  $\tilde{a}_0 \leftarrow \text{ZKHash}(\text{chall}_2, (\mathbf{a}_0 \| v_0^*))$ 
15:  $\tilde{a}_1 \leftarrow \text{ZKHash}(\text{chall}_2, (\mathbf{a}_1 \| (u_0^* + v_1^*)))$ 
16:  $\tilde{a}_2 \leftarrow \text{ZKHash}(\text{chall}_2, (\mathbf{a}_2 \| u_1^*))$ 
17: return  $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$ 

```

Vistrutith.OWFVerify $(\mathbf{d}, \mathbf{Q}, \text{pk}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \tilde{a}_2; \text{param})$

INPUT: $\mathbf{d} \in \{0, 1\}^\ell$, $\mathbf{Q} \in \{0, 1\}^{(\ell+2\lambda) \times \lambda}$, pk , chall_2 , chall_3 , $\tilde{a}_1$, and $\tilde{a}_2$

OUTPUT: $\tilde{a}_0 \in \mathbb{F}_{2^\lambda}$

```

1:  $\Delta \leftarrow \text{ToField}(\text{chall}_3; \lambda)$ 
2: for  $i \in [0.. \ell)$  do
3:   Let  $\langle w[i] \rangle \leftarrow \text{ToField}(\mathbf{Q}|_i; \lambda) + \mathbf{d}[i] \cdot \Delta$ 
4: end for
5: for  $i \in [0.. 2\lambda)$  do
6:    $q[i] \leftarrow \text{ToField}(\mathbf{Q}|_{\ell+i}; \lambda)$ 
7: end for
8: for  $j \in [0.. 2)$  do
9:    $q_j^* \leftarrow \sum_{i \in [0.. \lambda)} q[i + \lambda j] \alpha_\lambda^i$ 
10: end for
11:  $q^* \leftarrow q_0^* + \Delta \cdot q_1^*$ 
12: // compute the evaluated constraint vector
13:  $\langle \mathbf{z} \rangle_\lambda^3 \leftarrow \text{Vistrutith.OWFConstraints}(\langle \mathbf{w} \rangle, \text{pk}; \text{param}, \text{param}_{\text{OWF}})$ 
14: Parse  $\langle \mathbf{z} \rangle_\lambda^3$  as  $\mathbf{b} \in \mathbb{F}_{2^\lambda}^C$  //  $\mathbf{b} = \mathbf{a}_0 + \Delta \mathbf{a}_1 + \Delta^2 \mathbf{a}_2$ 
15:  $\tilde{q} \leftarrow \text{ZKHash}(\text{chall}_2, (\mathbf{b} \| q^*))$ 
16: // recover the prover's  $\tilde{a}_0$ 
17: return  $\tilde{q} - \tilde{a}_1 \cdot \Delta - \tilde{a}_2 \cdot \Delta^2$ 

```

8.5 Construction of Vistrutith

Based on the building blocks and functions described above, this section gives the formal construction of the Vistrutith signature scheme.

8.5.1 Key Generation

The [Vistrutith.KeyGen](#) algorithm describes the key generation procedure for Vistrutith based on Vistrutah. On input the public parameters **param**, the algorithm first samples a key vector $\mathbf{k} \in \{0, 1\}^\lambda$ uniformly at random, subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It then samples an input vector $\mathbf{x} \in \{0, 1\}^\lambda$ uniformly at random.

The algorithm computes $\mathbf{y} \leftarrow \text{Vistrutah}_{\mathbf{k}}(\mathbf{x})$, sets $\text{sk} \leftarrow (\mathbf{x}, \mathbf{k})$, and outputs $\text{pk} \leftarrow (\mathbf{x}, \mathbf{y})$, where $\mathbf{k}$ is treated as the secret Vistrutah key and $\mathbf{x}$ as the public input. The algorithm then returns (sk, pk) .

[Vistrutith.KeyGen](#)(param)

INPUT: Public parameters **param**

OUTPUT: Secret key $\text{sk} \in (\{0, 1\}^\lambda, \{0, 1\}^\lambda)$ and public key $\text{pk} \in (\{0, 1\}^\lambda, \{0, 1\}^\lambda)$

```
1: repeat
2:    $\mathbf{k} \leftarrow \{0, 1\}^\lambda$ 
3: until  $\mathbf{k}[0] \wedge \mathbf{k}[1] = 0$ 
4:  $\mathbf{x} \leftarrow \{0, 1\}^\lambda$ 
5:  $\mathbf{y} \leftarrow \text{Vistrutah}_{\mathbf{k}}(\mathbf{x})$ 
6:  $\text{sk} \leftarrow (\mathbf{x}, \mathbf{k})$  // public input, secret Vistrutah key
7:  $\text{pk} \leftarrow (\mathbf{x}, \mathbf{y})$  // public input, public output
8: return  $(\text{sk}, \text{pk})$ 
```

8.5.2 Signing

Vistrutith.Sign(msg, sk, pk; param)

INPUT: Message msg, secret key $sk = (sk_1, sk_2)$, public key $pk = (pk_1, pk_2)$, public parameters param

OUTPUT: Signature σ

```

1: ctr  $\leftarrow 0$  // 32-bit int
2:  $\rho \leftarrow \{0, 1\}^\xi$  // Signature randomness
3:  $\mu \leftarrow H_2^0(pk \parallel msg; 2\lambda)$ 
4:  $(r, iv^{pre}) \leftarrow H_3(sk_2 \parallel \mu \parallel \rho; \lambda + \lambda_{iv})$ 
5:  $iv \leftarrow H_4(iv^{pre}; \lambda_{iv})$ 
6:  $(com, decom, c_1, \dots, c_{\tau-1}, u, V) \leftarrow \text{VOLECommit}(r, iv, \ell + 3\lambda + B; param)$  //  $d = 3$  for Vistrutith
7:  $chall_1 \leftarrow H_2^1(\mu \parallel com \parallel c_1 \parallel \dots \parallel c_{\tau-1} \parallel iv; 5\lambda + 64)$ 
8: // VOLE consistency check
9:  $\tilde{u} \leftarrow \text{VOLEHash}(chall_1, u) \in \{0, 1\}^{\lambda+B}$ 
10:  $\tilde{V} \leftarrow \text{VOLEHash}(chall_1, V) \in \{0, 1\}^{(\lambda+B) \times \lambda}$ 
11:  $w \leftarrow \text{Vistrutith.ExtendWitness}(sk_2, sk_1; param) \in \{0, 1\}^\ell$ 
12:  $d \leftarrow w \oplus u_{[0, \ell]}$  // Mask extended witness
13:  $chall_2 \leftarrow H_2^2(chall_1 \parallel \tilde{u} \parallel \tilde{V} \parallel d; 3\lambda + 64)$ 
14: // QuickSilver proof
15:  $u \leftarrow u_{[\ell, \ell+2\lambda]}$  // drop last  $\lambda + B$  bits
16:  $V \leftarrow V_{[0, \ell+2\lambda]} \in \{0, 1\}^{(\ell+2\lambda) \times \lambda}$  // drop last  $\lambda + B$  rows
17:  $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2) \leftarrow \text{Vistrutith.OWFProve}(w, u, V, pk, chall_2; param)$ 
18: repeat
19:    $chall_3 \leftarrow H_2^3(chall_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel ctr; \lambda)$ 
20:   if  $chall_3[\lambda - w_{grind}, \lambda] \neq 0^{w_{grind}}$  then
21:     ctr  $\leftarrow ctr + 1$ 
22:     continue
23:   end if
24:    $I \leftarrow \text{DecodeChall}(chall_3[0, \lambda - w_{grind}]; param)$ 
25:    $decom_I \leftarrow \text{BAVC.Open}(decom, I; param)$ 
26: until  $decom_I \neq \perp$ 
27: return  $\sigma \leftarrow (\{c_i\}_{i \in [1, \tau]}, \tilde{u}, d, \tilde{a}_1, \tilde{a}_2, decom_I, chall_3, iv^{pre}, ctr)$ 

```

Compared with uBlockith, the signing procedures of Vistrutith follow exactly the same high-level structure, including message binding and IV derivation, VOLE commitment and consistency checking, OWF proof generation, and the final grinding-based opening step. The degrees of constraints are both 3. The only difference is that Vistrutith invokes the OWF functions corresponding to Vistrutah instead of uBlock. Further, as the key schedule routine of Vistrutah does not involve any non-linear S-boxes, we do not introduce any EM variants.

8.5.3 Verifying

Vistrutith.Verify(msg, pk, σ ; param)

INPUT: Message msg, public key pk, signature σ , public parameters param

OUTPUT: true or false

```

1: Parse  $\sigma = (\{c_i\}_{i \in [1, \tau]}, \tilde{u}, d, \tilde{a}_1, \tilde{a}_2, \text{decom}_I, \text{chall}_3, \text{iv}^{\text{pre}}, \text{ctr})$ 
2:  $\mu \leftarrow H_2^0(\text{pk} \parallel \text{msg}; 2\lambda)$ 
3:  $\text{iv} \leftarrow H_4(\text{iv}^{\text{pre}}; \lambda_{\text{iv}})$ 
4: if  $\text{chall}_3[\lambda - w_{\text{grind}}, \lambda] \neq 0^{w_{\text{grind}}}$  then
5:   return false
6: end if
7: // Reconstruct VOLEs and check commitment
8:  $(\text{com}, Q) \leftarrow \text{VOLEReconstruct}(\text{decom}_I, \text{chall}_3[0, \lambda - w_{\text{grind}}], c_1, \dots, c_{\tau-1}, \text{iv}; \text{param})$ 
9: if VOLEReconstruct returned  $\perp$  then
10:  return false
11: end if
12:  $\text{chall}_1 \leftarrow H_2^1(\mu \parallel \text{com} \parallel c_1 \parallel \dots \parallel c_{\tau-1} \parallel \text{iv}; 5\lambda + 64)$ 
13: // Apply the VOLE correction values and check consistency
14:  $Q \leftarrow \text{VOLEHash}(\text{chall}_1, Q) \in \{0, 1\}^{(\lambda+B) \times \lambda}$ 
15: Parse  $\text{chall}_3 = (\delta_0, \dots, \delta_{\lambda-1}) \in \{0, 1\}^\lambda$ 
16:  $\tilde{V}' \leftarrow Q \oplus [\delta_0 \cdot \tilde{u} \dots \delta_{\lambda-1} \cdot \tilde{u}]$ 
17:  $\text{chall}_2 \leftarrow H_2^2(\text{chall}_1 \parallel \tilde{u} \parallel \tilde{V}' \parallel d; 3\lambda + 64)$ 
18: // Compute  $\tilde{a}_0$ 
19:  $\tilde{a}_0 \leftarrow \text{Vistrutith.OWFVerify}(d, Q_{[0, \ell+2\lambda]}, \text{chall}_2, \text{chall}_3, \tilde{a}_1, \tilde{a}_2, \text{pk}; \text{param})$ 
20: Compute  $\text{chall}'_3 \leftarrow H_2^3(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \tilde{a}_2 \parallel \text{ctr}; \lambda)$ 
21: return true if  $\text{chall}_3 = \text{chall}'_3$  else false

```

Relative to uBlockith, the Vistrutith differs only in the concrete OWF instantiation, *i.e.*, using **Vistrutith.OWFVerify** instead of **uBlockith.OWFVerify**. The remaining steps are identical.

9 Security Statements and Analysis

This section presents the security statements and analyses of our signature schemes. Since our schemes preserve the protocol structure of FAEST and only replace the concrete instantiations of the underlying cryptographic primitives, such as using a larger field and replacing the PRGs and hash functions, our security proofs essentially follow those of FAEST, with the corresponding assumptions and parameters adjusted accordingly.

9.1 Theoretical Security

9.1.1 Security Targets and Model

We prove the *existential unforgeability against chosen-message attacks* (EUF-CMA) security of our signature schemes in both the ROM and the QROM. We first establish *existential unforgeability against key-only attacks* (EUF-KO), where the adversary is given only the public key and must output a valid forgery. We then extend this notion to EUF-CMA, where the adversary is additionally given access to a signing oracle, which returns a valid signature given a message.

Definition 4 (EUF-CMA and EUF-KO Security in the ROM or QROM). *Let $M \in \{\text{ROM}, \text{QROM}\}$. A signature scheme*

$$\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$$

$\boxed{\text{ExpEUF-CMA}}$ ExpEUFKO	OSign(msg)
1 : $\mathcal{M} := \emptyset$	1 : $\mathcal{M} := \mathcal{M} \cup \{\text{msg}\}$
2 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{param})$	2 : return $\text{Sign}^{\text{H}_M}(\text{sk}, \text{msg})$
3 : $(\text{msg}, \sigma) \leftarrow \mathcal{A}^{\text{H}_M, \boxed{\text{OSign}}}(\text{param}, \text{pk})$	
4 : return $\text{Verify}^{\text{H}_M}(\text{msg}, \text{pk}, \sigma; \text{param}) \wedge \text{msg} \notin \mathcal{M}$	

Fig. 3: The $\boxed{\text{EUF-CMA}}$ and EUF-KO security experiments in the ROM/QROM. Here $M \in \{\text{ROM}, \text{QROM}\}$, and H_M denotes classical access to the random oracle(s) in the ROM and quantum access in the QROM. Only in EUF-CMA does $\mathcal{A}$ get access to OSign.

is EUF-CMA (resp., EUF-KO) secure in model M if every efficient adversary has at most negligible advantage of winning the EUF-CMA (resp., EUF-KO) security experiment in Figure 3.

In the ROM, the adversary is a PPT algorithm with classical oracle access to the random oracle(s). In the QROM, the adversary is a QPT algorithm with quantum access to the random oracle(s). In both models, the signing oracle OSign is classical. We write

$$\text{AdvEUF-CMA}_{\mathcal{A}, M}^{\text{SIG}} = \Pr[\text{ExpEUF-CMA}_{\mathcal{A}, M}^{\text{SIG}} = 1],$$

and analogously

$$\text{AdvEUF-KO}_{\mathcal{A}, M}^{\text{SIG}} = \Pr[\text{ExpEUF-KO}_{\mathcal{A}, M}^{\text{SIG}} = 1].$$

The security proofs follow the same high-level strategy as FAEST. In the ROM, we first prove EUF-KO security by viewing a valid signature forgery as an accepting non-interactive ZKP. The proof proceeds through a sequence of games: the public key is first replaced by a lossy and random-looking instance under the pseudo-randomness of block ciphers, and then any accepting forgery is reduced to breaking the soundness of the corresponding interactive ZKP, except with negligible probability due to random-oracle collisions, preimage attacks, or incorrect guesses of Fiat–Shamir challenges. The EUF-KO result is then lifted to EUF-CMA security by simulating the signing oracle using the honest-verifier zero-knowledge property of the proof system and programming the Fiat–Shamir random oracles. Intuitively, the honest-verifier zero-knowledge property guarantees that such transcripts can be generated without knowing the witness, namely the secret key, while remaining indistinguishable from honestly generated transcripts. Therefore, in the security reduction, the simulator can answer signing queries even though it only receives the public key and does not know the corresponding secret key. To make the simulated transcript pass verification after applying the Fiat–Shamir transform, the simulator programs the random oracles so that the hash outputs coincide with the simulated challenges. The unpredictability of the commitments ensures that the adversary has not queried these random-oracle inputs in advance, except with negligible probability, and hence the programming is not detected.

In the QROM, the same conceptual reduction from EUF-CMA to EUF-KO is used, but the proof must account for quantum superposition queries to the random oracles. Hence, ordinary random-oracle programming is replaced by standard QROM reprogramming tools, such as adaptive reprogramming and compressed-oracle arguments. Following the FAEST proof, the non-interactive proof is interpreted as a Fiat–Shamir transform of a suitable multi-round interactive proof, so that round-by-round (RBR) soundness can be applied in the presence of quantum-accessible random oracles. The resulting bounds differ from the ROM bounds mainly by the usual quantum query losses and by replacing classical unpredictability arguments with their quantum analogues.

Multi-key EUF-CMA. We additionally define the multi-key EUF-CMA security, which captures the security of a signature scheme when an adversary is given several independently generated public keys and may adaptively request signatures under any of them. The adversary wins if it outputs a valid signature on a fresh message under one of these public keys.

This notion is needed due to the limitation of FAEST’s proof to meet the NGGC’s signing-query requirement. NGGC requires that a single key pair should support signing and verification

ExpMK-EUF-CMA _{A,M} ^{SIG} (U)	OSign(i, msg)
1 : $\mathcal{M} := \emptyset$	1 : if $i \notin [U]$ then return $\perp$
2 : for $i \in [U]$ do $(pk_i, sk_i) \leftarrow \text{KeyGen}(\text{param})$	2 : $\mathcal{M} := \mathcal{M} \cup \{(i, \text{msg})\}$
3 : $(i^*, \text{msg}^*, \sigma^*) \leftarrow \mathcal{A}^{\text{H}_M, \text{OSign}}(\text{param}, pk_1, \dots, pk_U)$	3 : return $\text{Sign}^{\text{H}_M}(sk_i, \text{msg})$
4 : return $i^* \in [U] \wedge \text{Verify}^{\text{H}_M}(\text{msg}^*, pk_{i^*}, \sigma^*; \text{param})$ $\wedge (i^*, \text{msg}^*) \notin \mathcal{M}$	

Fig. 4: The multi-key EUF-CMA security experiment in the ROM/QROM for U keys, where $[U] = \{1, \dots, U\}$. Here $M \in \{\text{ROM}, \text{QROM}\}$, and H_M denotes classical access to the random oracle(s) in the ROM and quantum access in the QROM. The signing oracle is classical and records queried pairs (i, msg) .

for at least 2^{64} distinct messages, and the overall EUF-CMA experiment should allow $Q_{\text{sig}} = 2^{80}$ signing queries. However, for our signature schemes and FAEST, the EUF-CMA proof relies on the PRGs using a 128-bit IV, and the hiding and simulation arguments use a divergence-based analysis that is bounded for at most 2^{64} signing queries under a fixed key. Although this can be handled by enlarging the size of IV to 256 bits, at the 128-bit security, the maximum size of IV is 128 bits when using PRGs with a block size of 128 bits. Then, requiring one key pair to answer 2^{80} signing queries would fall outside the range covered by the existing proof.

We address this mismatch by interpreting the NGGC bound in a multi-key setting. Concretely, we consider $U = 2^{16}$ independently generated key pairs, and each key is used for at most $q = 2^{64}$ signing queries. The total number of signing queries is then $Q_{\text{sig}} = U \cdot q = 2^{16} \cdot 2^{64} = 2^{80}$. This preserves the per-key bound as in the FAEST proof while matching the global signing-query bound and single-key-signing bound required by NGGC.

Definition 5 (Multi-key EUF-CMA Security in the ROM or QROM). Let $U = U(\lambda)$ be the number of keys and $M \in \{\text{ROM}, \text{QROM}\}$. A signature scheme

$$\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$$

is U -key EUF-CMA secure in model M if every efficient adversary has at most negligible advantage of winning $\text{ExpMK-EUF-CMA}_{A,M}^{\text{SIG}}(U)$.

In the ROM, the adversary is a PPT algorithm with classical oracle access to the random oracle(s). In the QROM, the adversary is a QPT algorithm with quantum access to the random oracle(s). In both models, the signing oracle is classical.

We write

$$\text{AdvMK-EUF-CMA}_{A,M}^{\text{SIG}}(U) = \Pr[\text{ExpMK-EUF-CMA}_{A,M}^{\text{SIG}}(U) = 1].$$

According to the union bound, the advantage of any multi-key adversary $\mathcal{A}$ can be bounded by the sum of its success probabilities for forging under each individual public key. Therefore, for suitable single-key adversaries $\mathcal{B}_i$ derived from $\mathcal{A}$, we have

$$\text{AdvMK-EUF-CMA}_{A,M}^{\text{SIG}}(U) \leq \sum_{i \in [U]} \text{AdvEUF-CMA}_{\mathcal{B}_i, M}^{\text{SIG}} \leq U \cdot \max_{i \in [U]} \text{AdvEUF-CMA}_{\mathcal{B}_i, M}^{\text{SIG}}. \quad (28)$$

9.1.2 Building Blocks

This section gives basic notations and preliminaries that will be used in the following context.

PRP advantage. We use the standard PRP advantage notation. Let $P : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$ be a block cipher, where for every key $k \in \mathcal{K}$, the map $P(k, \cdot)$ is a permutation over $\mathcal{M}$. A PRP distinguisher $\mathcal{D}$ is an oracle algorithm that is given oracle access to a pair of permutations (O, O^{-1}) and outputs

a bit. Its goal is to distinguish whether $O = P(k, \cdot)$ for a uniformly random key $k \leftarrow \mathcal{K}$, or whether $O = \pi$ for a uniformly random permutation $\pi \leftarrow \text{Perm}(\mathcal{M})$, where $\text{Perm}(\mathcal{M})$ denotes the set of all permutations over $\mathcal{M}$. We define the PRP advantage of $\mathcal{D}$ against P as

$$\text{AdvPRP}_{\mathcal{D}}^P = \left| \Pr \left[\mathcal{D}^{P(k, \cdot), P^{-1}(k, \cdot)} = 1 : k \leftarrow \mathcal{K} \right] - \Pr \left[\mathcal{D}^{\pi, \pi^{-1}} = 1 : \pi \leftarrow \text{Perm}(\mathcal{M}) \right] \right|. \quad (29)$$

When we want to make the query budget explicit, we write $\text{AdvPRP}_{\mathcal{D}}^P[q]$ for the same advantage, under the additional restriction that $\mathcal{D}$ makes at most q oracle queries in total to its challenge oracles. Here the query count includes both forward and inverse oracle queries. We also write

$$\text{AdvPRP}^P[q] = \max_{\mathcal{D}: Q_{\mathcal{D}} \leq q} \text{AdvPRP}_{\mathcal{D}}^P$$

for the maximum advantage over all distinguishers making at most q oracle queries.

OWF advantage. We also use the standard OWF advantage notation. Let $F : \mathcal{X} \rightarrow \mathcal{Y}$ be a deterministic function. An OWF adversary $\mathcal{A}$ is given $y = F(x)$ for a uniformly random $x \leftarrow \mathcal{X}$, and outputs a candidate preimage $x' \in \mathcal{X}$. We define the OWF advantage of $\mathcal{A}$ against F as

$$\text{AdvOWF}_{\mathcal{A}}^F = \Pr[F(x') = y : x \leftarrow \mathcal{X}, y := F(x), x' \leftarrow \mathcal{A}(y)]. \quad (30)$$

Almost-injective PRG. We use the almost-injective property of PRGs [21]. For a fixed IV and tweak, consider the function that maps a PRG key k to the output $\text{PRG}(k, \text{iv}, \text{twk})$. The property says that it should be hard to find an output value y that can be produced by two distinct keys. Equivalently, except with negligible probability, there should not exist $k \neq k'$ such that

$$\text{PRG}(k, \text{iv}, \text{twk}) = \text{PRG}(k', \text{iv}, \text{twk}) = y.$$

This property is stronger than ordinary collision resistance in the following sense: the adversary does not need to output two colliding keys, but only an output value that has at least two distinct preimages. In our security proof, this property is used to rule out extraction failures in PRG-based leaf commitments. We next recall the definitions.

Definition 6 (Almost-injective PRG [21]). Let $\text{PRG}(k, \text{iv}, \text{twk})$ be an IV-based tweakable PRG as defined in the [PRG](#) algorithm, where $k \in \mathcal{K}_{\lambda} = \{0, 1\}^{\lambda}$, $\text{iv} \in \mathcal{I}_{\lambda}$, and $\text{twk} \in \mathcal{T}_{\lambda}$. Consider the following game, parameterized by Q_{out} .

1. The adversary $\mathcal{A}$ outputs an IV $\text{iv} \in \mathcal{I}_{\lambda}$ together with Q_{out} tuples $(\text{twk}_j, y_j)_{j \in [1..Q_{\text{out}}]}$
2. If, for some $j \in [1..Q_{\text{out}}]$, there exist two distinct keys $k \neq k'$ in $\mathcal{K}_{\lambda}$ such that $\text{PRG}(k, \text{iv}, \text{twk}_j) = y_j = \text{PRG}(k', \text{iv}, \text{twk}_j)$, then the game outputs 1; otherwise, it outputs 0.

We define the advantage of $\mathcal{A}$ in this game as

$$\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{out}}] = \Pr[\text{the game outputs 1}].$$

We say that [PRG](#) is almost injective if, for every polynomial-time adversary $\mathcal{A}$ and every polynomial Q_{out} , the advantage $\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{out}}]$ is negligible.

We assume our PRGs are almost-injective if modeling them as ideal ciphers. Following the heuristic analysis of FAEST, the almost-injectivity advantage is estimated as as

$$\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{out}}] \lesssim \frac{Q_{\mathcal{C}}(Q_{\mathcal{C}} - Q_{\text{out}})}{2^{2\lambda}} + \frac{Q_{\text{out}}}{2^{2\lambda}}, \quad (31)$$

where we model the block cipher $\mathcal{C}$ as an ideal cipher and assume a bound $Q_{\mathcal{C}}$ on ideal cipher queries. Basically, the adversary gets non-negligible advantages only when the query numbers are larger than 2^{λ} .

Definition 6 describes the case of a single IV, which can be naturally extended to the case of multiple IVs as described in Definition 7.

Definition 7 (Multi-IV almost-injective PRG). Let $\text{PRG}(k, \text{iv}, \text{twk})$ be an IV-based tweakable PRG as in Definition 6. Consider the following game, parameterized by Q_{iv} and Q_{out} .

1. The adversary $\mathcal{A}$ outputs Q_{iv} IVs

$$\text{iv}_i \in \mathcal{I}_\lambda \quad \text{for } i \in [1..Q_{\text{iv}}],$$

together with Q_{out} tuples for each IV,

$$(\text{twk}_{i,j}, y_{i,j})_{j \in [1..Q_{\text{out}}]}.$$

2. If there exist $i \in [1..Q_{\text{iv}}]$, $j \in [1..Q_{\text{out}}]$, and two distinct keys $k \neq k'$ in $\mathcal{K}_\lambda$ such that

$$\text{PRG}(k, \text{iv}_i, \text{twk}_{i,j}) = y_{i,j} = \text{PRG}(k', \text{iv}_i, \text{twk}_{i,j}),$$

then the game outputs 1; otherwise, it outputs 0.

We define the advantage of $\mathcal{A}$ in this game as

$$\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{iv}}, Q_{\text{out}}] = \Pr[\text{the game outputs 1}].$$

We say that PRG is multi-IV almost injective if, for every polynomial-time adversary $\mathcal{A}$ and every polynomials Q_{iv} and Q_{out} , the advantage $\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{iv}}, Q_{\text{out}}]$ is negligible.

The single-IV notion of Definition 6 corresponds to the special case that $Q_{\text{iv}} = 1$. By a union-bound argument, if the single-IV advantage is bounded by $\epsilon(Q_{\text{out}})$, then the multi-IV advantage is bounded by $Q_{\text{iv}} \cdot \epsilon(Q_{\text{out}})$.

Keyed-hash PRF advantage. Let $\mathcal{K} = \{0, 1\}^\lambda$, let $\mathcal{X}$ be an input domain, and let $\mathcal{Y}$ be an output range. For a random oracle $H : \{0, 1\}^* \rightarrow \mathcal{Y}$, define the keyed-hash function

$$F_k^H(x) := H(x \| k).$$

A PRF distinguisher $\mathcal{A}$ is given oracle access to the public random oracle H and to a challenge oracle $O : \mathcal{X} \rightarrow \mathcal{Y}$, and outputs a bit. Its goal is to distinguish whether $O = F_k^H$ for a uniformly random key $k \leftarrow \mathcal{K}$, or whether $O = R$ for a uniformly random function $R \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})$ independent of H . We define

$$\begin{aligned} \text{AdvPRF}_{\mathcal{A}}^{\text{KH}} &= \left| \Pr[\mathcal{A}^{H, F_k^H} = 1 : H \leftarrow \text{Func}(\{0, 1\}^*, \mathcal{Y}), k \leftarrow \mathcal{K}] \right. \\ &\quad \left. - \Pr[\mathcal{A}^{H, R} = 1 : H \leftarrow \text{Func}(\{0, 1\}^*, \mathcal{Y}), R \leftarrow \text{Func}(\mathcal{X}, \mathcal{Y})] \right|. \end{aligned} \quad (32)$$

When the query budget is explicit, we write $\text{AdvPRF}_{\mathcal{A}}^{\text{KH}}[q_H, q_F]$ for adversaries making at most q_H queries to H and at most q_F queries to the challenge oracle O .

Let $\epsilon := \max_{x \in \mathcal{X}, u \in \{0, 1\}^*} \Pr_{k \leftarrow \mathcal{K}}[\text{enc}(x, k) = u]$. By the standard split-key PRF bound for hash-based constructions in the ROM, we have $\text{AdvPRF}_{\mathcal{A}}^{\text{KH}}[q_H, q_F] \leq q_H 2^{-\lambda}$ [44]. A looser per-query hybrid bound, useful when each of the q_F invocations uses an independent hidden seed, is

$$\text{AdvPRF}_{\mathcal{A}}^{\text{KH}}[q_H, q_F] \leq q_H q_F 2^{-\lambda}.$$

In the QROM, the split-key PRF bound of Don, Fehr, and Huang [45] gives

$$\text{AdvQPRF}_{\mathcal{A}}^{\text{KH}}[q_H, q_F] \leq 4\sqrt{2q_F^2 q_H \epsilon} + 4\sqrt{2q_H^2 q_F \epsilon}.$$

Thus, for a uniform λ -bit hidden key,

$$\text{AdvQPRF}_{\mathcal{A}}^{\text{KH}}[q_H, q_F] \leq 4\sqrt{\frac{2q_F^2 q_H}{2^\lambda}} + 4\sqrt{\frac{2q_H^2 q_F}{2^\lambda}}. \quad (33)$$

9.1.3 Security Analysis of Block Ciphers as OWFs

The formal security proof of our signature schemes relies on the PRG security, which seems stronger than the one-wayness. However, from the perspective of practical attacks, it is easy to break the security of signature schemes by breaking the one-wayness of OWFs to recover the secret key, and we are not aware of any attack strategy for forging a signature without actually inverting the OWFs. Hence, we provide some formal analysis of the one-wayness of our two OWF instantiations.

Standard block ciphers. As in Lemma 6, Chase *et al.* [37] show that it is feasible to use a block cipher, with key size equal to the block size and viewed as a PRP, to instantiate an OWF.

Lemma 6 (Security of block-cipher-based OWFs [37]). *Let $P : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$ be a PRP. For any fixed $x \in \mathcal{M}$, define*

$$F_x(k) = P(k, x).$$

Then F_x is an OWF. Concretely, any adversary $\mathcal{A}$ against the OWF F_x can be turned into an adversary $\mathcal{A}'$ against the PRP P , such that

$$\text{AdvOWF}_{\mathcal{A}}^{F_x} \leq \left(1 + \frac{|\mathcal{K}|}{|\mathcal{M}|}\right) \text{AdvPRP}_{\mathcal{A}'}^P + \frac{|\mathcal{K}|}{|\mathcal{M}|(|\mathcal{M}| - 1)}.$$

The reduction is tight such that the adversary $\mathcal{A}'$ makes at most 2 queries to the PRP oracle and takes computation similar to $\mathcal{A}$ plus one extra PRP evaluation.

In Chinitz, we use block ciphers with $|\mathcal{K}| = |\mathcal{M}| = 2^\lambda$ as OWFs and assume that they are PRP-secure. By Lemma 6, for every OWF adversary $\mathcal{A}$, there exists a PRP adversary $\mathcal{A}'$ such that

$$\text{AdvOWF}_{\mathcal{A}}^{F_x} \leq 2 \cdot \text{AdvPRP}_{\mathcal{A}'}^P + \frac{1}{2^\lambda - 1} \leq 2 \cdot \text{AdvPRP}_{\mathcal{A}'}^P + 2^{-\lambda+1}. \quad (34)$$

Even-Mansour Constructions. The Even-Mansour construction [36] is a way to construct a block cipher F from a random permutation π . We follow the proof of [38] and give a lemma below.

Lemma 7 (Security of Even-Mansour-based OWFs [38]). *Let $\mathcal{M} = \{0, 1\}^\lambda$ and let $P : \mathcal{X} \times \mathcal{M} \rightarrow \mathcal{M}$ be a block cipher. For a fixed public value $x \in \mathcal{X}$, write $\pi_x(\cdot) = P(x, \cdot)$. In the ideal-permutation model, π_x is modeled as a uniformly random permutation over $\mathcal{M}$. Define the Even-Mansour OWF*

$$F_{\pi_x}(k) = \pi_x(k) \oplus k = P(x, k) \oplus k.$$

Then F_{π_x} is an OWF in the ideal-permutation model. Concretely, for any adversary $\mathcal{A}$ making at most q adaptive queries to π_x and π_x^{-1} , we have

$$\text{AdvOWF}_{\mathcal{A}}^{F_{\pi_x}} \leq \frac{2(q+1)}{2^\lambda - 1} \leq 4(q+1) \cdot 2^{-\lambda}.$$

The running time is that of $\mathcal{A}$ plus one extra ideal-permutation query.

Accounting for the reduced key space. For the modular proof in the QROM, we restrict the valid secret-key space to be 3/4 of the standard λ -bit key space, as presented in the [SM4th.KeyGen](#), [uBlockith.KeyGen](#), and [Vistruthith.KeyGen](#) algorithms. This restriction is used in the lossy-key argument. After replacing an honestly generated public key by a uniformly random instance, a constant fraction of public keys no longer have any valid preimage under the OWF. For such public keys, the statement proved by the signature proof system is false. Therefore, any accepting forgery can be reduced to ordinary soundness of the underlying proof system, rather than to knowledge soundness.

This distinction is useful because knowledge soundness would require an extractor that recovers a valid secret key from an accepting Fiat-Shamir transcript. Such extraction is delicate in the QROM, since the adversary may query the random oracles in superposition and ordinary rewinding or random-oracle programming techniques do not apply directly. By contrast, ordinary soundness only needs to show that no prover can make the verifier accept a false statement, which can be

handled through the round-by-round soundness analysis used in the FAEST proof. The price of this argument is a small loss in OWF security: an attacker can discard the 1/4 of the key space that violates the constraint, so the remaining search space has size $3 \cdot 2^{\lambda-2}$, corresponding to a loss of about $\log_2(4/3) \approx 0.42$ bits. For simplicity, we upper bound this loss by 0.5 bits.

Note that the full key space could be recovered if one only required a modular proof in the ROM, where the corresponding extraction and programming arguments are easier to carry out.

9.1.4 Security Analysis of VOLE Commitments

This section claims the security properties of **VOLECommit** as proved in FAEST [21].

Binding. We argue that the VOLE commitments as in **VOLECommit** are binding, by showing that there is an extractor algorithm that is allowed to observe random oracle queries, and will successfully extract the adversary's committed message with high probability.

Lemma 8 (VOLE commitment is extractable binding [21]). *Consider the extractable binding game below for (**VOLECommit**, **VOLEReconstruct**), a stateful PPT adversary $\mathcal{A}$ and a straightline extractor $\mathcal{E}$:*

1. *Commit-Phase: Let $\mathcal{C} = \{\}$ be an empty associative array.*
 - (a) $\mathcal{C} = (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}}) \leftarrow \mathcal{A}^{\text{H}_1, \text{H}_2, \text{H}_4}(\text{commit})$.
 - (b) $\mathcal{D} = (\mathbf{u}_i^*, \mathbf{V}_i^*)_{i \in [0, \tau)} \leftarrow \mathcal{E}^{\text{H}_4}(\mathcal{L}_1, \mathcal{L}_2, \text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$, where $\mathcal{L}_b$ is a set $\{(x_i, \text{H}_b(x_i))\}$ of query-response pairs from queries $\mathcal{A}$ made to the random oracle H_b .
 - (c) Set $\mathcal{C}[\mathcal{C}] = \mathcal{D}$. Note that $\mathcal{D} = \perp$ is allowed.
 - (d) $\mathcal{A}$ can choose to go to Step 1a or it outputs $\mathcal{C}$ and the game continues below.
2. *Decommitment-Phase: Parse $\mathcal{C} = (\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}^{\text{pre}})$.*
3. $[\text{chall}_3, \text{decom}_I] \leftarrow \mathcal{A}^{\text{H}_1, \text{H}_2, \text{H}_4}(\text{open})$, where $I = (\Delta_0, \dots, \Delta_{\tau-1}) \leftarrow \text{DecodeChall}(\text{chall}_3)$.
4. *If $I = \perp$ then output 0 (failure).*
5. *Let $\text{iv} = \text{H}_4(\text{iv}^{\text{pre}})$.*
6. *For $i \in [0.. \tau)$: $(\delta_{i,0}, \dots, \delta_{i,k_i}) \leftarrow \text{BitDec}(\Delta_i, k_i)$.*
7. $(\overline{\text{com}}, Q) \leftarrow \text{VOLEReconstruct}^{\text{H}_1, \text{H}_2}(\text{chall}_3, \text{decom}_I, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \text{iv}, \hat{\ell}, \text{param})$.
8. *Parse $[Q_0 \cdots Q_{\tau-1}]_{\hat{\ell}, w} = Q$, where $Q_i \in \mathbb{F}_2^{\hat{\ell} \times k_i}$.*
9. *Output 0 (failure) if*
 - (a) $\overline{\text{com}} = \perp$ or $\text{com} \neq \overline{\text{com}}$ (i.e., verifier rejects), or
 - (b) $Q_i = \mathbf{V}_i^* \oplus [\delta_{i,0} \cdot (\mathbf{u}_i^* \oplus \mathbf{c}_i), \dots, \delta_{i,k_i} \cdot (\mathbf{u}_i^* \oplus \mathbf{c}_i)]$ for $i \in [0, \tau)$ (i.e., extraction correct).
10. *Output 1 (success) otherwise.*

Suppose $\mathcal{A}$ makes at most Q_1, Q_2 queries to the oracles H_1, H_2 , respectively. Moreover, suppose that $\mathcal{A}$ checks whether **VOLEReconstruct** returns $\perp$ for the purported opening decom_I , and outputs $\perp$ if that happens. There is a deterministic (unbounded) straightline extractor $\mathcal{E}$ which is allowed to program H_4 , such that the advantage of $\mathcal{A}$ in the above game is at most

$$\frac{(Q_1 + 1)^2}{2^{2\lambda}} + \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[L, Q_4],$$

which is no more than

$$\frac{(Q_1 + 1)^2}{2^{2\lambda}} + L \cdot \text{AdvInj}_{\mathcal{B}}^{\text{PRG}_{2\lambda}}[Q_4] \quad (35)$$

according to Definition 7. The running time of $\mathcal{B}$ is roughly that of $\mathcal{A}$.

Lemma 8 shows that, after switching iv to an injective mode, a valid commitment determines a unique value that can be extracted. Hence, any accepted opening that disagrees with the extracted value can only occur through a bad event, such as a random-oracle collision, an unexpected preimage, or a violation of the injectivity of the underlying PRG. The probability of these events gives the advantage bound in the lemma. The extractor may be inefficient, since it can brute-force preimages. This is not an issue here, because the extractor is used only in the security proof, and the resulting binding property is statistical.

Hiding. We claim that **VOLECommit** is hiding, i.e., any PPT adversary can not distinguish the VOLE-related messages of a signer $(\mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u})$ with random bit vectors.

Lemma 9 (VOLECommit is multi-hiding [21]). Let H_1 be random oracles. Consider the following multi-hiding experiment for VOLECommit^{H_1} and a Q -query PPT adversary $\mathcal{A}$, defined for some arbitrary $\hat{\ell}$ and param .

1. Sample $b^* \leftarrow \{0, 1\}$.
2. For $j \in [1, Q]$:
 - $r^j \leftarrow \{0, 1\}^\lambda$, $\text{iv}^j \leftarrow \{0, 1\}^{\lambda_{\text{iv}}}$.
 - $(\text{com}^j, \text{decom}^j, (\mathbf{c}_i^j)_{i \in [1, \tau]}, \mathbf{u}^j, \mathbf{V}^j) := \text{VOLECommit}(r^j, \text{iv}^j, \hat{\ell}, \text{param})$.
 - Sample $\text{chall}_3^j \leftarrow \{0, 1\}^{\lambda - w_{\text{grind}}} \times 0^{w_{\text{grind}}}$ uniformly, conditioned on BAVC.Open not aborting for $I^j = (\Delta_0^j, \dots, \Delta_{\tau-1}^j) := \text{DecodeChall}_3(\text{chall}_3^j; \text{param})$.
 - $\text{decom}_{I^j} := \text{BAVC.Open}(\text{decom}^j, I^j)$.
 - Let $\mathbf{u}^j \leftarrow \mathbb{F}_2^{\hat{\ell}}$ and define

$$(\bar{\mathbf{u}}^j, \bar{\mathbf{c}}_1^j, \dots, \bar{\mathbf{c}}_{\tau-1}^j) = \begin{cases} (\mathbf{u}^j, \mathbf{c}_1^j, \dots, \mathbf{c}_{\tau-1}^j) & \text{if } b^* = 0, \\ \text{random from } (\mathbb{F}_2^{\hat{\ell}})^\tau & \text{if } b^* = 1. \end{cases}$$

3. $b \leftarrow \mathcal{A}^{H_1} \left(((\bar{\mathbf{u}}^j, (\bar{\mathbf{c}}_i^j)_{i \in [1, \tau]}), (\text{decom}^j)_{i \in [0.., \tau]}, \text{chall}_3^j, \text{com}^j)_{j \in [1, N]} \right)$.
4. Output 1 (success) if $b = b^*$. Else output 0 (failure).

Write $\text{PRG}_{\text{len}(s)} = \text{PRG}(s; \text{len})$ and $\beta = \lceil \frac{\lambda}{N_{\text{block}}} \rceil$. Let G_i^* be the hiding game with $b^* = i$. Then there exist PRP adversaries $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$, whose running times are roughly that of $\mathcal{A}$, such that

$$\Pr[G_0^* = 1] \leq e^{Q\tau(4\lceil \log(L) \rceil \beta^2 + 4\beta^2 N_0^2 + \lceil \hat{\ell}/N_{\text{block}} \rceil^2)/2^{N_{\text{block}}}} \cdot (\Pr[G_1^* = 1] + Q\tau \cdot (\lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}_1}^{\text{PRG}_{2^{N_{\text{block}}}}} [1] + \text{AdvPRP}_{\mathcal{B}_2}^{\text{PRG}_{2^{N_{\text{block}}}}} [N_0] + \text{AdvPRP}_{\mathcal{B}_3}^{\text{PRG}_{\hat{\ell}}} [1])). \quad (36)$$

The same holds against quantum adversaries, in which case the advantage terms are with respect to quantum adversaries.

Unpredictability. We recall that VOLECommit has the unpredictability property, i.e., any PPT adversary cannot predict, before the commitment is generated, the commitment string together with its associated correction vectors.

Lemma 10 (VOLECommit is unpredictable [21]). Fix some $\hat{\ell} \in \mathbb{N}$ and admissible param . Let $\mathcal{A}$ be a PPT adversary in the following game $\text{ExpUnpred}_{b, \mathcal{A}}^{\text{VOLECommit}, \hat{\ell}, \text{param}}(\lambda)$:

1. $(\mathcal{L}_{\text{com}}, \text{state}) \leftarrow \mathcal{A}^{H_1}(1^\lambda)$. Here $\mathcal{L}_{\text{com}}$ is a list of commitment guesses.
2. $r \leftarrow \{0, 1\}^\lambda$, $\text{iv} \leftarrow \{0, 1\}^{\lambda_{\text{iv}}}$.
3. $C = (\text{com}, \text{decom}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(r, \text{iv}, \hat{\ell}; \text{param})$.
4. Let $b' \leftarrow \mathcal{A}^{H_1}(\text{state}, C)$.
5. If $(\text{com}, \mathbf{c}_1, \dots, \mathbf{c}_{\tau-1}) \in \mathcal{L}_{\text{com}}$ and $b = 1$, output 0. Else output b' .

For brevity, let G_b denote $\text{ExpUnpred}_{b, \mathcal{A}}^{\text{VOLECommit}, \hat{\ell}, \text{param}}$ for $b \in \{0, 1\}$. Also let $\beta = \lceil \frac{\lambda}{N_{\text{block}}} \rceil$. For any adversary $\mathcal{A}$ which makes at most Q_1 queries to H_1 , and outputs lists $\mathcal{L}$ of size at most Q_{com} , we have

$$\Pr[G_0 = 1] \leq e^{(4\lceil \log(L) \rceil \beta^2 + 4\beta^2)/2^{N_{\text{block}}}} \cdot (\Pr[G_1 = 1] + \lceil \log(L) \rceil \cdot \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2^\lambda}} [1] + \text{AdvPRP}_{\mathcal{B}}^{\text{PRG}_{2^\lambda}} [1] + (2Q_1 + Q_{\text{com}}) \cdot 2^{-2\lambda}).$$

9.1.5 Existentially Unforgeable against Chosen Message Attacks in the ROM

This section presents the EUF-CMA security of our signature schemes. We first claim the EUF-KO property in Theorem 1, which is later extended into EUF-CMA in Theorem 2.

Theorem 1 (EUF-KO in the ROM [21]). *Let $H_1, \{H_2^i\}_{i \in [0,3]}$ and H_4 be modeled as random oracles and param be the parameters of our signature schemes. Let $\mathcal{A}$ be an adversary which, for simplicity, runs `Verify` on its purported forgery and outputs $\perp$ if the verification fails. Let $Q_1, Q_{2,i}, Q_3$ and Q_4 denote the upper bounds on the number of queries of $\mathcal{A}$ to H_1, H_2^i, H_3 , and H_4 , respectively, and let $Q_2 = Q_{2,1} + Q_{2,2} + Q_{2,3}$. Suppose that $\hat{\ell} \leq 2^{13}$ and $2 \log \tau \leq \min\{B, 50\}$. Then, we have*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}} \leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot \left(\text{AdvSnd}_{\mathcal{B}_2}^{\text{Sig}} + \text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}} \right).$$

where OWF is the OWF (more precisely, PRG) defined by param , and

$$\begin{aligned} \text{AdvSnd}_{\mathcal{B}_2}^{\text{Sig}} &\leq \frac{(Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2}{2^{2\lambda}} \\ &\quad + \frac{(Q_1 + 1)^2 + (Q_C + 1)^2}{2^{2\lambda}} \\ &\quad + \frac{(Q_{2,1} + Q_{2,2} + Q_{2,3} + 9) \cdot d}{2^{\lambda_{\mathbb{F}}}}. \end{aligned} \tag{37}$$

Recall that the concrete value of $\text{AdvPRG}_{\mathcal{B}_1}^{\text{OWF}}$ is defined in Equation (34).

In our signature schemes, the key generation algorithm `KeyGen` excludes keys with bits $k[0] = k[1] = 1$. Thus, we have

$$\frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})} = \frac{3}{4} \quad \text{and} \quad \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} = 4.$$

Intuitively, in Equation (37), the first term comes from the soundness of the underlying interactive proof. The second term comes from the transformation of the state-restoration soundness for interactive proof to the soundness for interactive oracle proof. The third term comes from the transformation of the state-restoration soundness for interactive oracle proof to the round-by-round soundness. Such transformations are adapted from [46].

We next state the EUF-CMA property as in Theorem 2.

Theorem 2 (EUF-CMA in the ROM [21]). *Let $H_0, H_1, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as random oracles. Let H_3 be a PRF. For any PPT adversary $\mathcal{A}$ which makes Q_{sig} queries to $\mathcal{O}_{\text{sig}}$ and $Q_0, Q_1, Q_{2,1}, Q_{2,2}$ and $Q_{2,3}$ queries to H_0, H_1, H_2^1, H_2^2 , and H_2^3 , respectively. Then, we have*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{Sig}}[Q_{\text{sig}}] &\leq \rho \cdot \text{AdvEUF-KO}_{\mathcal{A}}^{\text{Sig}} \\ &\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2^{N_{\text{block}}}}} [1] + \text{AdvPRP}^{\text{PRG}_{2^{N_{\text{block}}}}} [1] \right) \\ &\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda_{\mathbb{F}} + 64}} + \frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda_{\mathbb{F}} + 64}} \right), \end{aligned} \tag{38}$$

where we have

$$\rho = e^{Q_{\text{sig}} \cdot (4 \lceil \log(L) \rceil \beta^2 + 4\beta^2) / 2^{N_{\text{block}}} + Q\tau (4 \lceil \log(L) \rceil \beta^2 + 4\beta^2 N_0^2 + \lceil \hat{\ell} / N_{\text{block}} \rceil^2) / 2^{N_{\text{block}}}}$$

is bounded by 2 for all proposed parameter sets. The reduction and bound hold for both non-EM and EM variants.

The reduction in Equation (38) combines Theorem 1, Lemma 9, Lemma 10, and random oracles.

According to Theorem 2, the EUF-CMA security of our signature schemes follows from the EUF-KO security of the underlying construction, the PRF security of H_3 , the pseudorandomness of the PRG calls, and the unpredictability and multi-hiding properties of `VOLECommit`. The reduction incurs only small constant multiplicative factors in the main terms, while the remaining additive losses are proportional to the number of signing queries and random-oracle queries.

9.1.6 Existentially Unforgeable against Chosen Message Attacks in the QROM

This section claims the EUF-CMA security in the QROM, which, similar to the variant in the ROM, is later extended into the EUF-CMA security.

Theorem 3 (EUF-KO in the QROM [21]). *Let $\mathcal{A}^{H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_3, H_4}$ be a QROM EUF-KO adversary that makes a total of $Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_3, Q_4$ queries to its oracles. Then there exist a quantum PRG adversary $\mathcal{B}_2$ against OWF, a QROM adversary $\mathcal{B}_1^{H_1, H_2^0, H_2^1, H_2^2, H_2^3, H_4}$ against the $\text{Sig}_r^{\text{elc}}$ that makes at most a total of $Q_1 + \tau + 1, Q_{2,0} + 1, Q_{2,1} + 1, Q_{2,2} + 1, Q_{2,3} + 1$, and $Q_4 + 1$ queries to its oracles, and an adversary $\mathcal{B}_3$ against the almost injectivity of PRG (as in Equation (31)) such that*

$$\text{AdvEUFKO}_{\mathcal{A}}^{\text{Sig}} \leq \frac{1}{1 - \frac{\text{dom}(\text{OWF})}{\text{cod}(\text{OWF})}} \cdot \left(\text{AdvSnd}_{\mathcal{B}_1}^{\text{Sig}_r^{\text{elc}}} + \text{AdvPRG}_{\mathcal{B}_2}^{\text{OWF}} + \text{AdvInj}_{\mathcal{B}_3}^{\text{PRG}_{2\lambda}}[L] \right),$$

where $\text{Sig}_r^{\text{elc}}$ is a variant of our signatures with “explicit leaf commitment” for ease of security proof. Specifically, we have

$$\text{AdvSnd}_{\mathcal{B}}^{\text{Sig}_r^{\text{elc}}} \leq 10(\tau + 1) \cdot Q^3 \cdot 2^{-2\lambda} + 10 \cdot Q^2 \cdot d \cdot 2^{-\lambda_{\mathbb{F}}}.$$

Here, we have

$$Q = Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4 + 3\tau + 19.$$

We give a slightly different quantum generalization of Lemma 10.

Lemma 11 (VOLECommit is quantum-guess unpredictable [21]). *Fix some $\hat{\ell} \in \mathbb{N}$ and admissible param. Let $\mathcal{A}$ be an adversary in the following game $\text{ExpQUnpred}_{b, \mathcal{A}}^{\text{VOLECommit}, \hat{\ell}, \text{param}}$.*

1. $(L, \text{state}) \leftarrow \mathcal{A}^{H_1}(1^\lambda)$, where L is a quantum register with the right number of qubits to hold a superposition of lists $\mathcal{L}_{\text{com}}$ of commitment guesses.
2. $r \leftarrow \{0, 1\}^\lambda$, $\text{iv} \leftarrow \{0, 1\}^{\lambda_{\text{iv}}}$.
3. $C = (\text{com}, \text{decom}, c_1, \dots, c_{\tau-1}, \mathbf{u}, \mathbf{V}) \leftarrow \text{VOLECommit}(r, \text{iv}, \hat{\ell}; \text{param})$.
4. Set $b' = 1$. If $b = 1$, apply the binary measurement to L that tests membership of com in the list $\mathcal{L}_{\text{com}}$ contained in the register L in superposition. If the measurement outputs 1, set $b' = 0$.
5. If $b' = 1$, set $b' \leftarrow \mathcal{A}^{H_1}(\text{state}, L, C)$.
6. Output b' .

For brevity, let G_b denote $\text{ExpQUnpred}_{b, \mathcal{A}}^{\text{VOLECommit}, \hat{\ell}, \text{param}}$ for $b \in \{0, 1\}$. Also let $\beta = \lceil \frac{\lambda}{N_{\text{block}}} \rceil$. For any adversary $\mathcal{A}$ which makes at most Q_1 queries to H_1 , and outputs lists $\mathcal{L}$ of size at most Q_{com} , we have

$$\begin{aligned} \Pr[G_0 = 1] &\leq e^{(4\lceil \log(L) \rceil \beta^2 + 16\beta^2)/2^{N_{\text{block}}}} \cdot \left(\Pr[G_1 = 1] + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2N_{\text{block}}}}[1] \right. \\ &\quad \left. + \text{AdvPRP}^{\text{PRG}_{2N_{\text{block}}}}[1] + (3\sqrt{Q_1} + 2\sqrt{Q_{\text{com}}}) \cdot 2^{-\lambda} + Q_{\text{com}} \cdot 2^{-2\lambda} \right). \end{aligned}$$

We finally give the security claim of EUF-CMA.

Theorem 4 (EUF-CMA in the QROM [21]). *Let $H_1, H_2^0, H_2^1, H_2^2, H_2^3$ and H_4 be modeled as quantum-accessible random oracles, and let H_3 be a PRF. Then, for any quantum polynomial-time adversary $\mathcal{A}$ which makes Q_{sig} queries to $\mathcal{O}_{\text{sig}}$ and $Q_1, Q_{2,1}, Q_{2,2}, Q_{2,3}$ queries to H_1, H_2^1, H_2^2, H_2^3 , respectively, we have*

$$\begin{aligned} \text{AdvEUF-CMA}_{\mathcal{A}}^{\text{Sig}}[Q_{\text{sig}}] &\leq \rho \cdot \text{AdvEUFKO}_{\mathcal{A}}^{\text{Sig}} \\ &\quad + \rho \cdot Q_{\text{sig}} \cdot \left(\text{AdvPRF}^{H_3} + \lceil \log(L) \rceil \cdot \text{AdvPRP}^{\text{PRG}_{2N_{\text{block}}}}[1] + \text{AdvPRP}^{\text{PRG}_{2N_{\text{block}}}}[1] \right) \\ &\quad + \rho \cdot Q_{\text{sig}} \cdot \left((3\sqrt{Q_1} + 2\sqrt{Q_{2,1}}) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right. \\ &\quad \left. + \frac{3}{2} \sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda_{\mathbb{F}}} + 64}} + \frac{3}{2} \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda_{\mathbb{F}}} + 64}} \right), \end{aligned}$$

where ρ is bounded by 2 for all proposed parameter sets.

The reduction in Theorem 4 shows that the EUF-CMA security of Sig in the QROM follows from the corresponding EUF-KO security, the PRF security of H_3 , the pseudorandomness of the PRG calls used in the VOLE commitment, and the bounds for quantum random-oracle bad events. In particular, the first term captures the reduction to key-only unforgeability, while the second term accounts for the cost of replacing the signing randomness and PRG-generated values by random ones in the simulation of signing queries. The remaining terms bound the probability that a quantum adversary guesses or hits certain random-oracle values involved in the signing simulation. Overall, the theorem states that chosen-message security is preserved up to a linear loss in the number of signing queries and additional QROM query-dependent error terms.

9.1.7 Query Complexity Analysis in Bits

This section gives a security analysis in bits for our signature schemes in the ROM and QROM according to Section 9.1.5 and 9.1.6. Specifically, we estimate the query complexity to random oracles or block ciphers needed for achieving the target security. Results show that the query costs of all our schemes are close to the target security bits of ICCS. Since each query also incurs a nontrivial computational cost, these estimates provide conservative evidence that our schemes meet the requirements of ICCS. We also estimate the concrete security bits from the best-known concrete attacks in Section 9.2.

Methodology, assumptions, and estimations. We adapt the methodology in SPHINCS⁺ [30]. That is, given fixed signing-query parameters, we set the total advantage to one and solve the number of queries to random oracles, block ciphers, and/or hash functions as the results.

For the SM4-based loose variants, namely SM4th-d3-128s-lo, SM4th-d3-128f-lo, SM4th-EM-d2-128s-lo, and SM4th-EM-d2-128f-lo, we use the multi-key interpretation with $U = 2^{16}$ independently generated key pairs and $q_{\text{sig}} = 2^{64}$ signing queries per key. For all other parameter sets, we use the single-key interpretation with $U = 1$ and $q_{\text{sig}} = 2^{80}$. Equivalently, we define

$$(\mu_{\text{mk}}, q_{\text{sig}}) = \begin{cases} (16, 2^{64}) & \text{for the SM4-based loose variants,} \\ (0, 2^{80}) & \text{for all other variants,} \end{cases}$$

where $\mu_{\text{mk}} = \log_2 U$. The multi-key union bound multiplies the single-key advantage by U , and hence subtracts μ_{mk} bits from linear advantage terms and $\mu_{\text{mk}}/2$ bits from quadratic collision-type query bounds. To compute the concrete security in bits as described in Theorem 2 and Theorem 4, we make the following assumptions and estimations.

1. We estimate that $\text{AdvPRP}^{\text{PRG}_{k, N_{\text{block}}}}[1] \approx 2^{-N_{\text{block}}-15}$ in the ROM, and $\text{AdvPRP}^{\text{PRG}_{k, N_{\text{block}}}}[1] \leq 2^{(-N_{\text{block}}-15)/2}$ in the QROM by the direct upper bound of Grover's algorithm, where $k = 1$ or 2. When $N_{\text{block}} = 128$ for SM4 in SM4th, we estimate that $\text{AdvPRP}^{\text{PRG}_{N_{\text{block}}}}[1] \leq 2^{-143}$ in the ROM and $\text{AdvPRP}^{\text{PRG}_{N_{\text{block}}}}[1] \leq 2^{-112}$ in the QROM according to [34, 35].
2. We estimate the OWF advantages for the non-EM constructions according to Lemma 6. That is, in the ROM, it loses 1-2 bits based on the PRG advantage. We assume that it also loses 1-2 bits in the QROM based on the PRG advantage in the QROM. We also assume that the advantages of OWFs AdvOWF for Even-Mansour constructions in the ROM and QROM are close to those of the non-EM ones in the same model.
3. We assume that H_3 is modeled as a keyed-hash PRF as described in Section 9.1.2 with a randomness of ξ bits as keys [21]. Then, we have $\text{AdvPRF}^{H_3} = Q_3/2^\xi$ in the ROM and $\text{AdvPRF}^{H_3} = \sqrt{Q_3}/2^\xi$ in the QROM, where Q_3 is the quantum or classical random-oracle query number to H_3 .
4. We assume that the concrete cost to query SM3 one time in the quantum model is 2^{30} according to the estimation of SM3 quantum circuits [39].

We give several explanations about our estimations and assumptions above.

Concrete security of PRGs in the classical model. The NIST post-quantum standardization call evaluates concrete security using the gate complexity of classical circuits. In the classical model, NIST's reference estimates assign exhaustive key recovery against λ -bit-key AES a cost of $2^\lambda \cdot 2^{C_{\text{AES}}}$,

where C_{AES} is the base-2 logarithm of the classical gate cost of one AES evaluation in NIST's metric. From NIST's estimates, $C_{\text{AES}} \approx 15$ for AES-128 and AES-192, and $C_{\text{AES}} \approx 16$ for AES-256.

Since exact gate-level cost estimates for SM4, Ballet, uBlock, and Vistrutah are not available, we use the AES-128 constant as a conservative lower-bound normalization. Namely, we set $C_{\text{SM4}} = C_{\text{Ballet}} = C_{\text{uBlock}} = C_{\text{Vistrutah}} = 15$. This is justified by the fact that these primitives have circuit sizes at least comparable to AES-128, so using the AES-128 constant does not overestimate the brute-force cost of the corresponding PRG instances.

Concrete security of PRGs in the quantum model. NIST considers the concrete costs of attacking AES-128 by key recovery as $2^{170}/\text{MAXDEPTH}$, where the term 2^{170} is drawn from [40], and the parameter MAXDEPTH means the maximum allowed serial quantum circuit depth, which is set as 2^{40} , 2^{64} or 2^{96} in NIST's proposal [34] and later updated into 2^{48} or 2^{56} [35].

Let $k = 128$ be the AES key size, let D_{AES} denote the effective logical depth of one AES Grover iteration (*e.g.*, T-depth), and let W_{AES} denote the number of logical qubits in the AES circuit. For the AES-128 circuit used in the NIST accounting, the parameters are reproduced as

$$D_{\text{AES}} = 110799, \quad W_{\text{AES}} = 984$$

A serial Grover search would require approximately $R_{\text{AES}} = \frac{\pi}{4} \cdot 2^{k/2} = \frac{\pi}{4} \cdot 2^{64}$ iterations. Under a maximum-depth restriction D_{max} , one Grover instance can perform only $N_{\text{iter}} = \frac{D_{\text{max}}}{D_{\text{AES}}}$ iterations. Therefore, the attack must be parallelized. Using the standard parallel Grover relation $N_{\text{iter}} = \frac{(\pi/4) \cdot 2^{k/2}}{\sqrt{S}}$, the required number of parallel Grover instances is

$$S = \left(\frac{(\pi/4) \cdot 2^{k/2} \cdot D_{\text{AES}}}{D_{\text{max}}} \right)^2.$$

The depth-limited logical circuit-size, or logical qubit-cycle cost, is then

$$C_{\text{AES}} = D_{\text{max}} \cdot S \cdot W_{\text{AES}} = \frac{(\pi/4)^2 2^k D_{\text{AES}}^2 W_{\text{AES}}}{D_{\text{max}}}.$$

Substituting $k = 128$, $D_{\text{AES}} = 110799$, and $W_{\text{AES}} = 984$, one obtains

$$\log_2((\pi/4)^2 2^{128} \cdot 110799^2 \cdot 984) = 170.76.$$

Thus, up to rounding and constant-factor conventions,

$$C_{\text{AES-128}} \approx \frac{2^{170.8}}{D_{\text{max}}} \simeq \frac{2^{170}}{\text{MAXDEPTH}},$$

which is the origin of the NIST AES-128 reference number.

The same normalization can be transferred to SM4 by comparing the implementation cost of SM4 and AES-128 under the same depth-limited Grover metric. However, we failed to find a quantum circuit analysis for SM4 using exactly the same gate-counting convention as the NIST AES baseline. We then use Lin *et al.* [47] to estimate the relative implementation gap between SM4 and AES-128 under a common Toffoli-depth and width metric, and anchor this relative gap to the NIST AES-128 baseline. This assumes that the relative depth gap measured in Toffoli depth is a reasonable proxy for the corresponding relative logical-depth gap under the NIST normalization.

Lin *et al.* report that their lowest-depth SM4 implementation, corresponding to 8 parallel S-boxes, has Toffoli depth $\Delta_{\text{SM4}} = 2244$ and width $Q_{\text{SM4}} = 368$. For the AES-128 implementation of Grassl *et al.* used in the same table, they report Toffoli depth $\Delta_{\text{AES}} = 12672$ and width $Q_{\text{AES}} = 984$. In the Grover key-search circuit, Lin *et al.* use two plaintext-ciphertext pairs and implement the oracle by running two encryptions and two inverse encryptions as in Equations (10) and (11) [47], therefore giving the per-iteration Toffoli depth and width as $D_{\text{iter}} = 2\Delta$ and $W_{\text{iter}} = 2Q + 1$. Hence, we have $D_{\text{SM4}} = 4488$, $W_{\text{SM4}} = 737$, $D_{\text{AES}} = 25344$, and $W_{\text{AES}} = 1969$.

Under a maximum-depth bound $D_{\max}$, the depth-limited Grover cost scales as $R^2 \cdot D_{\text{iter}}^2 \cdot W_{\text{iter}}/D_{\max}$, where $R = (\pi/4)2^{64}$ for 128-bit key search. Since SM4 and AES-128 have the same key size, the Grover factor R^2 cancels in the relative comparison. The relevant gap is therefore

$$\Delta_{\text{SM4/AES}} = \log_2 \left(\frac{D_{\text{SM4}}^2 W_{\text{SM4}}}{D_{\text{AES}}^2 W_{\text{AES}}} \right) = \log_2 \left(\frac{4488^2 \cdot 737}{25344^2 \cdot 1969} \right) \approx -6.41.$$

Using the unrounded AES-128 normalization derived from the NIST baseline, $C_{\text{AES-128}} \approx 2^{170.76}/\text{MAXDEPTH}$, this gives

$$C_{\text{SM4}} \approx \frac{2^{170.76-6.41}}{\text{MAXDEPTH}} = \frac{2^{164.35}}{\text{MAXDEPTH}}.$$

For $\text{MAXDEPTH} = 2^{56}$, which corresponds to the number of 200ns cycles that can be completed in roughly 500 years [35], this gives a cost of approximately 2^{108} . This satisfies the ICCS security requirement of 80 bits in the quantum model.

We failed to find the implementations of quantum circuits for uBlock, Ballet, and Vistrutah. Hence, we use the upper bound of Grover’s algorithm. That is, for a N_{block} -bit block OWF, we estimate that $\text{AdvOWF} \leq 2^{(-N_{\text{block}}-15)/2+2}$. However, we note that the above analysis could be naturally generalized if such quantum circuits are given in the future.

Concrete security of guessing a bad challenge in the quantum model. We now consider a different task: not breaking SM3 as a hash function, but finding an input transcript or randomness value for which the Fiat–Shamir challenge generated by SM3 satisfies a target event of probability $p = d \cdot 2^{-128}$, which is at most $p = 2^{-126}$ where $d = 2$ or 3 in our schemes. This probability is an assumption of the Fiat–Shamir search problem, not a property of SM3 itself. We are not aware of any better attacks than Grover’s search in the quantum model. The Grover search for an event of probability p requires approximately

$$R_{\text{FS}} = \frac{\pi}{4} \frac{1}{\sqrt{p}} = \frac{\pi}{4} \cdot 2^{63} \approx 2^{62.651}$$

iterations. For the SM3 circuit, we use the low- T -depth implementation of [39], which reports

$$D_{T,\text{SM3}} = 4528, \quad W_{\text{SM3}} = 1369, \quad T_{\text{SM3}} = 977920.$$

Here $D_{T,\text{SM3}}$ is the T -depth, and W_{SM3} is the number of logical qubits. It gives a total T -count $T_{\text{SM3}} = 977920$, obtained by adding the 965632 T -gates from the adders and the 12288 T -gates from the Boolean components.

Using a very coarse estimate, we charge one SM3 evaluation per Grover iteration and set $D_{T,\text{iter}} = D_{T,\text{SM3}}$, $W_{\text{iter}} = W_{\text{SM3}}$, and $T_{\text{iter}} = T_{\text{SM3}}$. This ignores the extra cost of checking the challenge predicate, diffusion, and possible uncomputation, and therefore gives a synchronized baseline estimate in terms of the reported SM3 circuit itself. Thus $D_{T,\text{iter}} = 4528 \approx 2^{12.145}$, $W_{\text{iter}} = 1369 \approx 2^{10.419}$, and $T_{\text{iter}} = 977920 \approx 2^{19.899}$.

If one additionally imposes a maximum-depth bound $D_{\max}$, then the same parallel-Grover normalization gives

$$C_{\text{FS}}(D_{\max}) = \frac{R_{\text{FS}}^2 D_{T,\text{iter}}^2 W_{\text{iter}}}{D_{\max}} \approx \frac{2^{160.011}}{D_{\max}},$$

as long as $D_{\max} < R_{\text{FS}} \cdot D_{T,\text{iter}} \approx 2^{74.796}$. For $D_{\max} = 2^{56}$, this gives

$$C_{\text{FS}}(2^{56}) \approx 2^{104.011}.$$

Thus, even under this simplified one-SM3-per-iteration accounting and using the larger bad-challenge probability bound $p \leq 2^{-126}$, the estimated depth-limited quantum cost is about 2^{104} at $\text{MAXDEPTH} = 2^{56}$.

ROM query-cost estimates. We write the three terms in Equation (38) as

$$\begin{aligned} T_1 &:= \rho \cdot \text{AdvEUFKO}^{\text{Sig}}, \\ T_2 &:= \rho \cdot Q_{\text{sig}} \left(\text{AdvPRF}^{\text{H}_3} + (\lceil \log L \rceil + 1) \text{AdvPRP}^{\text{PRG}_{2\lambda}}[1] \right), \\ T_3 &:= \rho \cdot Q_{\text{sig}} \left(\frac{2Q_1 + Q_{2,1}}{2^{2\lambda}} + \frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda_{\mathbb{F}}+64}} + \frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda_{\mathbb{F}}+64}} \right). \end{aligned}$$

We estimate the query cost by setting each term equal to one and solving for the corresponding query budgets. We ignore small additive constants in the following estimates; the repetition factor satisfies $\rho \leq 2$, and hence only costs at most one bit.

For the first term, using the EUF-KO bound and the fact that $\text{dom}(\text{OWF})/\text{cod}(\text{OWF}) = 3/4$ in Theorem 1, we get

$$T_1 \lesssim 4\rho \left(\frac{Q_{2,\Sigma}^2 + Q_1^2 + Q_{\text{C}}^2}{2^{2\lambda}} + \frac{dQ_{2,\Sigma}}{2^{\lambda_{\mathbb{F}}}} + \text{AdvOWF} \right),$$

where $Q_{2,\Sigma} = Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3}$. In the ROM, we estimate $\text{AdvOWF} \approx 2^{-N_{\text{block}}-15}$. After applying the possible multi-key loss $2^{\mu_{\text{mk}}}$, the corresponding query complexities are

$$Q_1, Q_{2,\Sigma} \approx 2^{\lambda-4-\mu_{\text{mk}}/2}, \quad Q_{2,\Sigma} \approx \frac{2^{\lambda_{\mathbb{F}}-\mu_{\text{mk}}}}{d}, \quad Q_{\text{OWF}} \approx 2^{N_{\text{block}}+15-\mu_{\text{mk}}}.$$

Considering that $4\rho \leq 8$, we have

$$Q_{\text{ROM,ti}}^{(1)} \approx \min \left\{ 2^{\lambda-2-\mu_{\text{mk}}/2}, \frac{2^{\lambda_{\mathbb{F}}-3-\mu_{\text{mk}}}}{d}, 2^{N_{\text{block}}+12-\mu_{\text{mk}}} \right\} \quad (39)$$

after considering the loss of constant terms.

For the second term, since H_3 is instantiated as a keyed random-oracle-style hash with a hidden seed of $\xi = \lambda + 128$ bits, we use

$$\text{AdvPRF}^{\text{H}_3}[Q_{\text{H}_3}] \lesssim \frac{Q_{\text{H}_3}}{2^{\lambda+128}}.$$

Setting $T_2 = 1$ and $\rho = 2$ gives

$$Q_{\text{H}_3}^{(2)} \approx \frac{2^{\lambda+127}}{Q_{\text{sig}}}. \quad (40)$$

Note that the AdvPRP term gives roughly

$$\lambda - \log_2 Q_{\text{sig}} - \log_2(\lceil \log L \rceil + 1) - 1$$

bits of security, which is not related to the queries of random oracles and block ciphers.

For the third term, solving the three summands separately gives

$$\begin{aligned} Q_1^{(3)}, Q_{2,1}^{(3)} &\approx \frac{2^{2\lambda-1-\mu_{\text{mk}}}}{Q_{\text{sig}}}, \\ Q_{2,2}^{(3)} &\approx \frac{2^{5\lambda+63}}{Q_{\text{sig}}}, \quad Q_{2,3}^{(3)} \approx \frac{2^{3\lambda+63}}{Q_{\text{sig}}}. \end{aligned}$$

The first summand is therefore dominant, so

$$Q_{\text{ROM}}^{(3)} \approx \frac{2^{2\lambda-1-\mu_{\text{mk}}}}{Q_{\text{sig}}}. \quad (41)$$

Combining the three estimates and applying the multi-key union bound when needed, the ROM query complexity is bounded by

$$Q_{\text{ROM}} \approx \min \left\{ 2^{\lambda-2-\mu_{\text{mk}}/2}, \frac{2^{\lambda_{\mathbb{F}}-3-\mu_{\text{mk}}}}{d}, \frac{2^{\lambda+127-\mu_{\text{mk}}}}{Q_{\text{sig}}}, \frac{2^{2\lambda-1-\mu_{\text{mk}}}}{Q_{\text{sig}}}, 2^{N_{\text{block}}+12-\mu_{\text{mk}}} \right\}. \quad (42)$$

Table 12: Query costs for the proposed signature schemes in the ROM

Scheme	$\log_2 Q_{\text{ROM}}$	Dominant term
SM4th-d3-128s-lo	107.4	$(2^{\lambda_{\mathbb{F}}-3-\mu_{\text{mk}}})/d$
SM4th-d3-128f-lo	107.4	$(2^{\lambda_{\mathbb{F}}-3-\mu_{\text{mk}}})/d$
SM4th-EM-d2-128s-lo	108	$(2^{\lambda_{\mathbb{F}}-3-\mu_{\text{mk}}})/d$
SM4th-EM-d2-128f-lo	108	$(2^{\lambda_{\mathbb{F}}-3-\mu_{\text{mk}}})/d$
SM4th-d3-128s-ti	126.0	$2^{\lambda-2}$
SM4th-d3-128f-ti	126.0	$2^{\lambda-2}$
SM4th-EM-d2-128s-ti	126.0	$2^{\lambda-2}$
SM4th-EM-d2-128f-ti	126.0	$2^{\lambda-2}$
uBlockith-d3-256s	251.4	$2^{\lambda_{\mathbb{F}}-3}/d$
uBlockith-d3-256f	251.4	$2^{\lambda_{\mathbb{F}}-3}/d$
uBlockith-EM-d3-256s	251.4	$2^{\lambda_{\mathbb{F}}-3}/d$
uBlockith-EM-d3-256f	251.4	$2^{\lambda_{\mathbb{F}}-3}/d$
Vistrutith-d3-512s	507.4	$2^{\lambda_{\mathbb{F}}-3}/d$
Vistrutith-d3-512f	507.4	$2^{\lambda_{\mathbb{F}}-3}/d$

Table 12 reports the query exponents obtained by solving the conservative ROM proof bound of Equation (42). These values should be interpreted as proof-bound random-oracle query thresholds, rather than as the final NGGC attack-cost security strengths. Consequently, the entries can be a few bits below the nominal NGGC security targets even when no known concrete attack reaches the corresponding cost. For the SM4-based loose variants, the bound should be interpreted in the multi-key EUF-CMA sense. Concretely, these variants support $q_{\text{sig}} = 2^{64}$ signing queries per key and use 2^{16} independent key pairs to meet the global 2^{80} -query requirement. All other parameter sets can directly support $Q_{\text{sig}} = q_{\text{sig}} = 2^{80}$ signing queries under a single key.

In Table 12, we omit the AdvPRP contribution from the final query-cost estimate. This is because the AdvPRP term is not governed by the random-oracle or block-cipher query budgets solved in Equation (42); under our concrete PRP estimates, its security contribution remains negative in the advantage exponent independently of the query-complexity term. However, our SM4th-ti variants meet the strict security requirements due to the 256-bit PRGs, where the PRP term satisfies $Q_{\text{sig}} \cdot \text{AdvPRP} \leq 2^{80} \cdot 2^{-256} \cdot C_{\text{Ballet}}^{-1} = 2^{-193}$ and is smaller than 2^{-128} . It is also natural to use larger block ciphers as PRGs for uBlockith and Vistrutith if such a strict requirement is preferred.

Note that the actual computational cost of an adversary is expected to be higher than the reported query-complexity exponent. This is because each query corresponds to at least one concrete evaluation of the underlying hash function, random-oracle instantiation, or block cipher, and collision-type attacks may also require substantial memory or time-space tradeoffs.

QROM query-cost estimates. We follow the same strategy as in the ROM case. Let

$$Q_{\Sigma} := Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4.$$

By Theorem 3, the QROM proof uses $Q = Q_{\Sigma} + 3\tau + 19$. In the following concrete estimates we absorb the additive term $3\tau + 19$ into the constants. We follow the notations of q_{sig} and μ_{mk} as in the ROM case. We write the three terms in Theorem 4 as

$$\begin{aligned}
T_1^{\text{Q}} &:= \rho \cdot \text{AdvEUFKO}^{\text{Sig}}, \\
T_2^{\text{Q}} &:= \rho \cdot Q_{\text{sig}} \left(\text{AdvPRF}^{\text{H}_3} + (\lceil \log L \rceil + 1) \text{AdvPRP}^{\text{PRG}_{2^{\lambda_{\text{block}}}}} [1] \right), \\
T_3^{\text{Q}} &:= \rho \cdot Q_{\text{sig}} \left((3\sqrt{Q_1} + 2\sqrt{Q_{2,1}}) \cdot 2^{-\lambda} + Q_{2,1} \cdot 2^{-2\lambda} \right. \\
&\quad \left. + \frac{3}{2} \sqrt{\frac{Q_{\text{sig}} + Q_{2,2}}{2^{5\lambda_{\mathbb{F}}+64}}} + \frac{3}{2} \sqrt{\frac{Q_{\text{sig}} + Q_{2,3}}{2^{3\lambda_{\mathbb{F}}+64}}} \right).
\end{aligned}$$

For the first term, using $\text{dom}(\text{OWF})/\text{cod}(\text{OWF}) = 3/4$ and $\rho \leq 2$, we get

$$T_1^Q \lesssim 8 \left(10(\tau+1)Q_\Sigma^3 \cdot 2^{-2\lambda} + 10dQ_\Sigma^2 \cdot 2^{-\lambda_{\mathbb{F}}} + \text{AdvPRG}^{\text{OWF}} + \text{AdvInj}^{\text{PRG}_{2\lambda}}[L] \right).$$

Solving the two soundness terms gives

$$Q_\Sigma \approx \left(\frac{2^{2\lambda-\mu_{\text{mk}}}}{80(\tau+1)} \right)^{1/3}, \quad Q_\Sigma \approx \left(\frac{2^{\lambda_{\mathbb{F}}-\mu_{\text{mk}}}}{80d} \right)^{1/2}.$$

For the almost-injectivity term, Equation (31) gives

$$\text{AdvInj}_{\mathcal{A}}^{\text{PRG}}[Q_{\text{out}}] \lesssim \frac{Q_{\text{C}}(Q_{\text{C}} - Q_{\text{out}})}{2^{2\lambda}} + \frac{Q_{\text{out}}}{2^{2\lambda}}.$$

Thus this term only becomes relevant when the ideal-cipher query budget is close to 2^λ . We denote the concrete quantum cost of the OWF or PRG term by

$$\kappa_{\text{OWF}}^Q := -\log_2(\text{AdvPRG}^{\text{OWF}}).$$

Therefore,

$$Q_{\text{QROM}}^{(1)} \approx \min \left\{ \left(\frac{2^{2\lambda-\mu_{\text{mk}}}}{80(\tau+1)} \right)^{1/3}, \left(\frac{2^{\lambda_{\mathbb{F}}-\mu_{\text{mk}}}}{80d} \right)^{1/2}, 2^{\kappa_{\text{OWF}}^Q - \mu_{\text{mk}}}, 2^{\lambda-\mu_{\text{mk}}/2} \right\}. \quad (43)$$

For the loose variants of SM4th, we use the concrete term $2^{160-\mu_{\text{mk}}}/\text{MAXDEPTH}$ to substitute the term $\left(\frac{2^{\lambda_{\mathbb{F}}-\mu_{\text{mk}}}}{80d} \right)^{1/2}$ as described in Section 9.1.7. Then we have

$$Q_{\text{QROM},\text{lo}}^{(1)} \approx \min \left\{ \left(\frac{2^{2\lambda-\mu_{\text{mk}}}}{80(\tau+1)} \right)^{1/3}, \frac{2^{160-\mu_{\text{mk}}}}{\text{MAXDEPTH}}, 2^{\kappa_{\text{OWF}}^Q - \mu_{\text{mk}}}, 2^{\lambda-\mu_{\text{mk}}/2} \right\}. \quad (44)$$

For the second term, following the estimate in Section 9.1.2, we use

$$\text{AdvPRF}^{\text{H}_3}[Q_{\text{H}_3}] \lesssim \sqrt{\frac{Q_{\text{H}_3}}{2^\xi}}, \quad \xi = \lambda + 128.$$

Setting $T_2^Q = 1$ gives

$$Q_{\text{H}_3}^{(2)} \approx \frac{2^{\lambda+128-2\mu_{\text{mk}}}}{\rho^2 q_{\text{sig}}^2} \gtrsim \frac{2^{\lambda+126-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}. \quad (45)$$

The PRP term is checked independently by the concrete quantum PRP estimate for the underlying block cipher. For the third term, solving the square-root summands gives

$$Q_1^{(3)}, Q_{2,1}^{(3)} \approx \frac{2^{2\lambda-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}.$$

The linear term $Q_{2,1} \cdot 2^{-2\lambda}$ gives the weaker condition $Q_{2,1} \approx 2^{2\lambda-\mu_{\text{mk}}}/q_{\text{sig}}$. The remaining two terms give

$$Q_{2,2}^{(3)} \approx \frac{2^{5\lambda_{\mathbb{F}}+64-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}, \quad Q_{2,3}^{(3)} \approx \frac{2^{3\lambda_{\mathbb{F}}+64-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}.$$

Hence the first square-root term is dominant:

$$Q_{\text{QROM}}^{(3)} \approx \frac{2^{2\lambda-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}. \quad (46)$$

Table 13: Query costs for the proposed signature schemes in the QROM

Scheme	$\log_2 Q_{\text{QROM}}$	Dominant term
SM4th-d3-128s-lo	76.7	$(2^{2\lambda-\mu_{\text{mk}}}/(80(\tau+1)))^{1/3}$
SM4th-d3-128f-lo	76.5	$(2^{2\lambda-\mu_{\text{mk}}}/(80(\tau+1)))^{1/3}$
SM4th-EM-d2-128s-lo	76.7	$(2^{2\lambda-\mu_{\text{mk}}}/(80(\tau+1)))^{1/3}$
SM4th-EM-d2-128f-lo	76.5	$(2^{2\lambda-\mu_{\text{mk}}}/(80(\tau+1)))^{1/3}$
SM4th-d3-128s-ti	76.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
SM4th-d3-128f-ti	76.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
SM4th-EM-d2-128s-ti	76.3	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
SM4th-EM-d2-128f-ti	76.3	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
uBlockith-d3-256s	124.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
uBlockith-d3-256f	124.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
uBlockith-EM-d3-256s	124.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
uBlockith-EM-d3-256f	124.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
Vistrutith-d3-512s	252.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$
Vistrutith-d3-512f	252.0	$(2^{\lambda_{\mathbb{F}}}/(80d))^{1/2}$

Combining the above estimates, the QROM query complexity, after applying the possible multi-key loss, is

$$Q_{\text{QROM}} \approx \min \left\{ \left(\frac{2^{2\lambda-\mu_{\text{mk}}}}{80(\tau+1)} \right)^{1/3}, \left(\frac{2^{\lambda_{\mathbb{F}}-\mu_{\text{mk}}}}{80d} \right)^{1/2}, 2^{\kappa_{\text{OWF}}^{\text{Q}}-\mu_{\text{mk}}}, 2^{\lambda-\mu_{\text{mk}}/2}, \frac{2^{\lambda+126-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}, \frac{2^{2\lambda-2\mu_{\text{mk}}}}{q_{\text{sig}}^2} \right\}, \quad (47)$$

and for the loose variants, the query complexity is

$$Q_{\text{QROM,lo}} \approx \min \left\{ \left(\frac{2^{2\lambda-\mu_{\text{mk}}}}{80(\tau+1)} \right)^{1/3}, \frac{2^{160-\mu_{\text{mk}}}}{\text{MAXDEPTH}}, 2^{\kappa_{\text{OWF}}^{\text{Q}}-\mu_{\text{mk}}}, 2^{\lambda-\mu_{\text{mk}}/2}, \frac{2^{\lambda+126-2\mu_{\text{mk}}}}{q_{\text{sig}}^2}, \frac{2^{2\lambda-2\mu_{\text{mk}}}}{q_{\text{sig}}^2} \right\}, \quad (48)$$

Table 13 reports the query exponents obtained by solving the conservative QROM proof bound of Theorem 4. These values should be interpreted as proof-bound random-oracle query thresholds, rather than as the final NGGC quantum attack-cost security strengths. The loss comes mainly from the explicit constants in the round-by-round QROM soundness analysis. Consequently, the entries can be a few bits below the nominal NGGC quantum-security targets even when no known concrete quantum attack reaches the corresponding cost.

As in the ROM case, the SM4-based loose variants are interpreted in the multi-key EUF-CMA setting, where each key supports $q_{\text{sig}} = 2^{64}$ signing queries and 2^{16} independent keys are used to match the global $Q_{\text{sig}} = 2^{80}$ -query requirement. The remaining parameter sets are evaluated with a direct single-key bound for $Q_{\text{sig}} = q_{\text{sig}} = 2^{80}$. We again omit the AdvPRP contribution from the table, since it is not related to the random-oracle or block-cipher query budget solved in the QROM query-complexity calculation.

For the SM4-based tight variants and the larger-block variants, the dominant loss comes from the square-root field-size term in the QROM soundness bound. For the SM4-based loose variants, after replacing the field-size term by the concrete Grover's cost of guessing a bad Fiat-Shamir challenge, the dominant term becomes the cubic-root soundness term.

We again note that the actual computational cost of an adversary is expected to be higher than the reported query-complexity exponent.

9.2 Practical Security against Known Attacks

Section 9.1.7 and Section 9.1.7 estimate the concrete query-costs for our signature schemes in the ROM and QROM. This section describes the best-known concrete attacks against our schemes, which also guides us to give the concrete estimated security bits. Results show that all our schemes satisfy the security-bit requirements of ICCS.

9.2.1 Brute Force the Public Key

The simplest attack strategy for VOLEith-based signatures is to invert the OWF value contained in the public key and thereby recover a valid secret key. For our schemes, this corresponds to a key-recovery attack on the underlying block-cipher-based OWF. In the standard variants, the public key contains values of $E_{\mathbf{k}}(\mathbf{x})$ for fixed public inputs $\mathbf{x}$, so the attack is essentially exhaustive search over the block-cipher key $\mathbf{k}$. In the EM variants, the public key contains values of $E_{\mathbf{x}}(\mathbf{k}) \oplus \mathbf{k}$, and the attack similarly amounts to finding a preimage key $\mathbf{k}$ for this public OWF output.

There are two minor differences from ordinary block-cipher key recovery. First, our key-generation algorithm rejects keys whose first two bits are both 1, so the effective secret-key space has size $3 \cdot 2^{N_{\text{block}}-2}$, which causes a loss of only $\log_2(4/3) \approx 0.42$ bits. Second, because the public key contains only a small number of block-cipher output blocks, there may exist collisions, so an attacker may find a different key $\mathbf{k}'$ that is still a valid preimage of the OWF. As discussed in the OWF analysis in Section 9.1.3, this collision effect has at most a small impact (*i.e.*, 2-3 bits) on the concrete security compared with the security of PRGs.

Based on the above two differences, for the non-EM variants, we estimate the costs of brute-forcing the public key to be about $2^{N_{\text{block}}-1-2-0.42}$ block-cipher evaluations, while for the EM variants we conservatively estimate it as about $2^{N_{\text{block}}-1-3-0.42}$ evaluations. Thus, compared with brute-forcing key recovery on the underlying block cipher, our OWF instantiations lose only a small constant number of bits of security. According to the estimation in Section 9.1.3, we obtain Table 14. For the QROM estimate, we use a quantum-search cost model rather than simply dividing the classical ROM exponent by two. For SM4-based variants, Section 9.1.3 gives a dedicated depth-limited Grover estimate for full 128-bit SM4 key recovery. Under the bound $\text{MAXDEPTH} = 2^{56}$, this cost is about 2^{108} logical gate-level operations. Therefore, for the SM4th public-key brute-force attack, we start from this baseline and subtract only the constant-bit losses caused by the OWF encoding: the collision/preimage loss and the $\log_2(4/3)$ -bit keyspace-reduction loss. This gives $2^{108-2-\log_2(4/3)}$ for the non-EM SM4th variants and $2^{108-3-\log_2(4/3)}$ for the EM variants.

For uBlockith and Vistrutith, we are not aware of comparable depth-limited quantum circuit estimates for uBlock or Vistrutah. We therefore do not assign a precise gate-level constant to one coherent OWF evaluation. Let C_E^Q denote the base-2 gate-cost exponent of one reversible evaluation of the corresponding OWF, including coherent checking and uncomputation overhead up to constant factors. If the effective classical search space has exponent s , then generic Grover search requires about $2^{s/2}$ OWF evaluations, and hence has gate cost $2^{s/2+C_E^Q}$. Here $s = N_{\text{block}} - 3 - \log_2(4/3)$ for the non-EM variants and $s = N_{\text{block}} - 4 - \log_2(4/3)$ for the EM variants. The table therefore keeps the uBlock and Vistrutah QROM costs symbolic. These estimates already meet the NGGC targets for very small values of C_E^Q , namely $C_E^Q \geq 2.3$ for the uBlock-EM case and $C_E^Q \geq 1.8$ for the Vistrutith case. Any nontrivial reversible implementation of a 256-bit or 512-bit block-cipher evaluation is expected to cost far more than this.

As shown from Table 14, all our signature schemes satisfy the concrete-bit requirements of ICCS in the ROM and QROM.

9.2.2 Brute Force the PRG and Multi-Key Attacks

Another possible attack strategy is to recover the signing key by attacking the PRG calls used inside [BAVC.Commit](#), [VOLECommit](#), and the conversion procedures that derive VOLE-related randomness. In our schemes, these PRGs are instantiated from the underlying block ciphers, namely SM4, Ballet, or Vistrutah, using a per-signature IV and domain-separated tweaks. Given one signature, an attacker may try to guess an unrevealed PRG seed and test whether the expanded output

Table 14: Estimated ROM and QROM gate-level costs of brute-forcing the public key

Scheme	ROM gate-level cost	QROM gate-level cost
SM4th-d3-128s/f-lo/ti	$2^{139.6}$	$2^{105.6}$
SM4th-EM-d2-128s/f-lo/ti	$2^{138.6}$	$2^{104.6}$
uBlockith-d3-256s/f	$2^{267.6}$	$2^{126.3+C_{\text{uBlock}}^Q}$
uBlockith-EM-d3-256s/f	$2^{266.6}$	$2^{125.8+C_{\text{uBlock}}^Q}$
Vistrutith-d3-512s/f	$2^{523.6}$	$2^{254.3+C_{\text{Vistrutah}}^Q}$

is consistent with the public transcript and the revealed openings. If the same (iv, twk) pair were reused across many PRG calls, this could lead to a multi-target attack, where one guessed seed is tested against many possible targets. Such attacks can sometimes reduce the cost compared with attacking each PRG output independently, and are a form of time-space tradeoff attack.

Our parameterization avoids this issue by assigning distinct tweaks to the different PRG invocations. First, consider the length-doubling tweakable PRGs used in the tree-based vector commitment [BAVC.Commit](#). At each tree level, a signature reveals all but one of the node seeds on the authentication path. Given a candidate guess g for a missing seed, the attacker can expand g with the corresponding tweak and compare the resulting child seeds with the known neighboring values. A match would identify the missing seed, except with the negligible probability of a PRG-output collision, and could eventually reveal the hidden leaf information. Second, PRG calls are also used when deriving the VOLE-related masks. There, all but one of the relevant seeds may be exposed, so an attacker could again guess the missing seed, expand it, and test whether the resulting mask is consistent with the transcript or with a candidate signing key. However, these two kinds of PRG calls use different tweaks, and different locations in the commitment tree or VOLE generation also use domain-separated tweaks. Therefore, outputs from different roles cannot be combined into a single multi-target search instance.

Consequently, recovering the signing key through these PRG calls has essentially the same cost as attacking a single target of the underlying block-cipher-based PRG, which satisfies the concrete security-bit requirements of ICCS in both the ROM and QROM.

9.2.3 Attack Soundness of the ZK Protocol

Instead of recovering the secret key from the public key or from PRG seeds, an attacker may try to forge a signature by violating the soundness of the underlying ZKP. In this attack, the adversary uses an invalid witness for the public statement and attempts to produce an accepting Fiat–Shamir transcript. This type of attack is captured by the EUF-KO bound in Theorem 1.

We now explain why the concrete terms in the soundness bound do not give effective attacks when the hash functions are modeled as random oracles. The first two quadratic terms in Equation (37),

$$\frac{(Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2}{2^{2\lambda}} \quad \text{and} \quad \frac{(Q_1 + 1)^2 + (Q_C + 1)^2}{2^{2\lambda}},$$

correspond to finding collisions or bad preimages in random-oracle outputs of length at least 2λ . For a random oracle, such events require about 2^λ queries by a generic birthday attack. In our concrete instantiations, the random oracles are implemented with hash functions such as SM3-based domain-separated hashes, so the corresponding generic attack is a collision search against these hash outputs. The third term in Equation (37),

$$\frac{(Q_{2,1} + Q_{2,2} + Q_{2,3} + 9)d}{2^{\lambda_F}},$$

comes from the round-by-round soundness of the underlying proof system over the field $\mathbb{F}_{2^{\lambda_F}}$. Intuitively, after the adversary fixes a false transcript, it can only pass the verifier if a Fiat–Shamir challenge falls into a small bad set. For a degree- d consistency check, this bad set has size at most

Table 15: Estimated ROM and QROM gate-level costs of attacking ZK soundness

Scheme	ROM gate-level cost	QROM gate-level cost
SM4th-d3-128s/f-lo	$G_{\text{ROM}}(128, 128, 3) = 2^{144.4}$	$G_{\text{QROM}}(128, 128, 3) = 2^{104.4}$
SM4th-EM-d2-128s/f-lo	$G_{\text{ROM}}(128, 128, 2) = 2^{145.0}$	$G_{\text{QROM}}(128, 128, 2) = 2^{105.0}$
SM4th-d3-128s/f-ti	$G_{\text{ROM}}(128, 160, 3) = 2^{146.0}$	$G_{\text{QROM}}(128, 160, 3) = 2^{105.2}$
SM4th-EM-d2-128s/f-ti	$G_{\text{ROM}}(128, 160, 2) = 2^{146.0}$	$G_{\text{QROM}}(128, 160, 2) = 2^{105.2}$
uBlockith-d3-256s/f	$G_{\text{ROM}}(256, 256, 3) = 2^{272.4}$	$G_{\text{QROM}}(256, 256, 3) = 2^{190.6}$
uBlockith-EM-d3-256s/f	$G_{\text{ROM}}(256, 256, 3) = 2^{272.4}$	$G_{\text{QROM}}(256, 256, 3) = 2^{190.6}$
Vistrutith-d3-512s/f	$G_{\text{ROM}}(512, 512, 3) = 2^{528.4}$	$G_{\text{QROM}}(512, 512, 3) = 2^{361.2}$

a $d/2^{\lambda_{\mathbb{F}}}$ fraction of all possible challenges. Thus, for a random oracle, the best generic strategy is exhaustive search over challenges, requiring about $2^{\lambda_{\mathbb{F}}}/d$ trials.

For the EM variants, the soundness reduction also relies on the almost-injectivity of the PRG-based leaf commitments. As discussed in Definition 6 and Equation (31), this assumption rules out the event that a commitment value has two distinct PRG-key preimages. We are not aware of an attack substantially better than a generic collision or exhaustive-search attack against the underlying block-cipher-based PRG. This term therefore becomes relevant only when the ideal-cipher query budget approaches the $2^{N_{\text{block}}}$ scale.

Overall, generic attacks against the soundness of the ZK protocol have two main branches. The first branch searches for collisions or bad preimages in random-oracle outputs of length at least 2λ . The second branch searches for a bad Fiat–Shamir challenge for the degree- d consistency check over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. In the ROM, the first branch requires about 2^{λ} classical random-oracle evaluations, while the second branch requires about $2^{\lambda_{\mathbb{F}}}/d$ evaluations. Let C_{H}^{cl} denote the base-2 logarithm of the classical gate cost of one SM3-based random-oracle evaluation. Following the NIST normalization for SHA3-256 collision search [34], we use $C_{\text{H}}^{\text{cl}} = 18$. Thus the ROM cost estimate is $G_{\text{ROM}}(\lambda, \lambda_{\mathbb{F}}, d) = \min\{2^{\lambda+C_{\text{H}}^{\text{cl}}}, 2^{\lambda_{\mathbb{F}}-\log_2 d+C_{\text{H}}^{\text{cl}}}\}$. In the QROM, according to the Brassard–Hoyer–Tapp quantum collision-finding algorithm [48], the random-oracle collision branch is estimated by the generic quantum collision-search complexity $2^{2\lambda/3}$ for a 2λ -bit output. Let C_{H}^{q} denote the base-2 logarithm of the cost of one quantum-accessible SM3 oracle evaluation. Using the SM3 quantum circuit of [39], which has $T_{\text{SM3}} = 977920$, we set $C_{\text{H}}^{\text{q}} = \log_2(977920) \approx 19.90$. For the bad-challenge branch, we use the same depth-limited parallel Grover model as in Section 9.1.3. With $D_{T, \text{SM3}} = 4528$, $W_{\text{SM3}} = 1369$, and $\text{MAXDEPTH} = 2^{56}$, the base-2 cost exponent is $\lambda_{\mathbb{F}} - \log_2 d + 2\log_2(\pi/4) + 2\log_2(4528) + \log_2(1369) - 56 = \lambda_{\mathbb{F}} - \log_2 d + 34.012 - 56$. Therefore the QROM cost estimate is $G_{\text{QROM}}(\lambda, \lambda_{\mathbb{F}}, d) = \min\{2^{2\lambda/3+C_{\text{H}}^{\text{q}}}, 2^{\lambda_{\mathbb{F}}-\log_2 d+34.012-56}\}$.

Table 15 summarizes the resulting estimates. For the SM4-based tight variants, the larger field size increases the bad-challenge branch, but the collision branch still depends on the 2λ -bit random-oracle output with $\lambda = 128$, and therefore dominates the table entry.

As shown from Table 15, all our signature schemes satisfy the concrete-bit requirements of ICCS in the ROM and QROM.

10 Performance Evaluation

10.1 Performance Analysis

This section gives the performance analysis of transmission costs for our signature schemes. Recall that in Section 3 we describe the main parameter **param** of our signature schemes, which include

$$\text{param} = (\lambda, \lambda_{\text{Q}}, \lambda_{\mathbb{F}}, \tau, w_{\text{grind}}, T_{\text{open}}, d, B, \text{OWF}, \ell, \lambda_{\text{iv}}, N_{\text{block}}).$$

Table 16: Transmission sizes for all parameter sets

Scheme	Sizes in Bytes		
	sk	pk	signature
SM4th-d3-128s-lo	32	32	5 056
SM4th-d3-128f-lo	32	32	6 724
SM4th-EM-d2-128s-lo	32	32	3 818
SM4th-EM-d2-128f-lo	32	32	4 932
SM4th-d3-128s-ti	32	32	9 176
SM4th-d3-128f-ti	32	32	11 864
SM4th-EM-d2-128s-ti	32	32	7 556
SM4th-EM-d2-128f-ti	32	32	9 450
uBlockith-d3-256s	64	64	24 144
uBlockith-d3-256f	64	64	31 556
uBlockith-EM-d3-256s	64	64	19 056
uBlockith-EM-d3-256f	64	64	25 028
Vistrutith-d3-512s	128	128	83 004
Vistrutith-d3-512f	128	128	106 788

The main parameter can determine the public key size, the secret key size, and the signature size (in bits) as below. The size of the public key is

$$\underbrace{\lambda}_{\text{public input size of OWFs}} + \underbrace{\lambda}_{\text{public output size of OWFs}}.$$

The size of the secret key is

$$\underbrace{\lambda}_{\text{public input size of OWFs}} + \underbrace{\lambda}_{\text{secret key size of OWFs}}.$$

As for the signature size, it is

$$\underbrace{\tau \cdot (\ell + (d-1)\lambda_{\mathbb{F}} + \lambda_{\mathbb{F}} + B)}_{\substack{\text{witness/mask commitment} \\ + \text{responses to VOLE/ZK checks}}} + \underbrace{T_{\text{open}} \cdot N_{\text{block}} + 2N_{\text{block}}\tau}_{\text{GGM tree opening}} + \underbrace{\lambda_{\mathbb{F}}}_{\Delta \text{ challenge}} + \underbrace{\lambda_{\text{iv}}}_{\text{iv}^{\text{pre}}} + \underbrace{32}_{\text{counter}}.$$

Table 16 reports the transmission sizes for all signature schemes based on the performance analysis above, including the public key, secret key, and signature size.

10.2 Detailed Configuration of Implementation Platforms

We provide two implementations, as required by NGGC of ICCS. The first is a reference implementation in standard C, designed for portability and clarity. The second is an optimized implementation written in C with architecture-specific extensions, targeting x86-64 platforms equipped with instruction set extensions commonly found in modern Intel and AMD processors. For SM4th-lo, we also provide the optimized implementation using the SM4 instruction set in a Hygon processor. All reference and optimized implementations for the same scheme are identical in signature results, which output the same known-answer tested (KAT) vectors.

We evaluate most implementations on a workstation featuring an Intel Xeon 6982P-C processor at 3.85 GHz and 14 GiB memory. These experiments are performed on Linux 6.12.57, with the implementations compiled using GCC 14.2.0. The SM4th-lo implementation is also run on a server equipped with dual-socket Hygon C86-4G (OPN:7490) processors running at approximately 3.00 GHz and 2.2 TiB of memory. These experiments are performed on Alibaba Cloud Linux 3 (Soaring

Table 17: Instruction sets used by the optimized implementations

Protocol	CPU / Platform	Instruction sets used
SM4th-lo	x86-64 Intel CPU	SSE2/XMM, SSSE3, AVX2, AES-NI, PCLMULQDQ
SM4th-lo	Hygon CPU with CIS support	SSE2/XMM, AVX2, Hygon CIS SM4, PCLMULQDQ
SM4th-ti	x86-64 Intel CPU	SSE2/XMM, AVX2, PCLMULQDQ
uBlockith	x86-64 Intel CPU	SSE2/XMM, AVX2, PCLMULQDQ
Vistrutith	x86-64 Intel CPU	SSE2/XMM, AVX2, AES-NI, PCLMULQDQ

Table 18: Open-source libraries and reference code sources used by implementations

Protocol	Open-source libraries / imported sources
SM4th-lo	SM4; SM4-NI or Hygon CIS SM4 ISA; shared FAEST-style protocol components
SM4th-ti	SM4; Ballet; shared FAEST-style protocol components
uBlockith	uBlock; Ballet; shared FAEST-style protocol components
Vistrutith	Vistrutah; shared FAEST-style protocol components

Falcon) with Linux kernel 6.6.47-004.ant8.x86_64, and the implementations are compiled using GCC 10.2.1 20200825 (Alibaba 10.2.1-3.9 2.32). The implementation is written in C, with routines implemented in assembly. All experiments use only a single CPU core.

Instruction sets. Table 17 reports the ISA used in the optimized implementations of our signature schemes. As shown, all implementations rely on SIMD acceleration through SSE2/XMM and AVX2 instructions. The implementation of SM4th further exploits AES-NI and SSSE3 on Intel x86-64 processors, while a dedicated implementation is provided for Hygon processors with CIS support. Vistrutith additionally leverages AES-NI to accelerate AES-based Vistrutah. PCLMULQDQ is employed whenever carry-less finite-field arithmetic is required.

Crypto libraries. Table 18 summarizes the open-source libraries and reference code sources used in our implementations. For the OWF in SM4th, the implementation uses SM4-related reference code by OpenSSL⁵. For the PRG in SM4th-lo, we use platform-specific acceleration based on either SM4-NI⁶ or the Hygon CIS SM4 ISA⁷. For the PRGs in uBlockith and SM4th-ti, we use Ballet. For OWFs in uBlockith, we use components from uBlock⁸. For Vistrutith, the implementation is based on the Vistrutah reference code in C⁹. All implementations also share FAEST-style protocol components for the common MPC-in-the-head and VOLE-based building blocks¹⁰.

10.3 Acceleration Techniques

This section introduces the acceleration techniques worth mentioning in our implementations.

10.3.1 Acceleration Techniques of PRGs

In the reference implementations of our signature schemes, a major part of the signing and verifying time is spent in PRGs, making instruction-set architecture (ISA) acceleration crucial. We build platform-and-scheme-specific optimized implementations inspired by the following two observations. First, different from FAEST, we can not use the AES-specific ISA AES-NI on modern Intel and AMD CPUs due to using different PRGs. Second, PRGs in our schemes are invoked in several

⁵ <https://github.com/openssl/openssl/blob/master/crypto/sm4/sm4.c>

⁶ <https://github.com/mjosaarinen/sm4ni>

⁷ <https://gitee.com/anolis/cloud-kernel/pulls/6631>

⁸ Ballet and uBlock both at https://sfjs.cacrnet.org.cn/site/term/list_76_1.html

⁹ <https://github.com/jedisct1/vistrutah>

¹⁰ <https://github.com/faest-sign/faest-ref>

specific modes, which inspires us to make scheme-specific optimizations. Below, we first describe general acceleration techniques for these specific modes, and then present several platform-specific and scheme-specific techniques in different schemes, respectively.

General techniques for specific PRG modes. We make the following two observations: (1) PRGs in [BAVC.LeafCommit](#) and [ConvertToVOLE](#) share the same keys, so their key schedule routines only need to run once. (2) In [ConvertToVOLE](#), there are τ sub-GGM trees. For a fixed sub-tree with index i , PRGs are evaluated in parallel for different keys but the tweak/counter base derived from i is shared. This creates a fixed-tweak, multiple-keys workload, *e.g.*, $\text{PRG}(k_0, \text{iv}, \text{twk}_i), \dots, \text{PRG}(k_{m-1}, \text{iv}, \text{twk}_i)$. To achieve acceleration in the second case, we implement a fixed-tweak batched PRG core with variable output length. The key idea is to separate key-dependent work from tweak-dependent work: for a fixed index i , the counter base derived from $\text{iv} + \text{twk}_i$ is prepared once and reused across all lanes, while each lane derives its own round-key or prepared-key state only once and then reuses it across all counter blocks required by that PRG instance.

PRG design in SM4th. We design SM4-CTR-style PRGs that combine existing instruction sets and scheme properties. In particular, we use either SM4-NI (computing S-boxes of SM4 via AES-NI) on Intel CPUs or the SM4 ISA on Hygon CPUs. We also exploit the three output regimes of PRGs in SM4th-lo: 2λ bits, and VOLE-related 276 bytes for non-EM variants, and 162 bytes for EM variants. Below, we describe the techniques on Intel and Hygon CPUs, respectively.

On Intel CPUs. For the 2λ -bit PRG in the seed expansion procedure of [BAVC.Commit](#) (*i.e.*, $\text{PRG}(k_\alpha, \text{iv}, \alpha; 2\lambda)$), there is no available open-source dedicated 2-block SM4 kernel. The open-source SM4-NI backend provides a 4-block same-key encryption-routine implementation, which does not directly match our target workload. To reduce repeated schedule overhead, we make two key observations: (1) Any 4 sets of 2-block key expansion routine can be seen as 2 independent sets of 4-block key expansion routine; (2) The key expansion and encryption routines of SM4 are similar. Based on these observations, we modify the open-source 4-block same-key encryption-routine implementation, and implement an 4-block key-expansion-routine backend, as well as a 4-block different-key encryption-routine backend. To run four sets of $\{\text{PRG}(k_{\alpha+i}, \text{iv}, \alpha + i; 2\lambda)\}_{i \in [0,4)}$, the former backend is invoked once, and the latter back end is invoked twice. This produces exactly 2λ -bit-output blocks per key for four PRG instances in parallel.

For $\hat{\ell}$ -bit outputs, we exploit the open-source 16-block same-key SM4-NI kernel¹¹ whenever profitable. For $\hat{\ell} = 276$ (17 full blocks plus 4 bytes), we first pre-compute the fixed-tweak counter sequence, then evaluate the first 16 counters with one 16-block call, generate the remaining counters with one extra 4-block call, and finally keep the first 20 bytes of the tail buffer. For $\hat{\ell} = 162$ (10 full blocks plus 2 bytes), experiments show that forcing a 16-block call would over-generate and reduce efficiency. We therefore reuse the 4-block different-key encryption-routine backend and obtain the final outputs by truncation.

On Hygon CPUs. Hygon CPUs provide native SM4 instructions for both round-key generation and round evaluation, which we denote by **SM4RK** and **SM4RND**.

For the 2λ -bit PRG in the seed expansion procedure of [BAVC.Commit](#), we batch four independent PRG instances in one Hygon helper. Given four keys and four pairs of counter blocks, the helper first generates four SM4 key schedules with **SM4RK**, and then encrypts the two counter blocks of each lane with **SM4RND**. Thus, one invocation produces the two SM4 output blocks required by $\{\text{PRG}(k_{\alpha+i}, \text{iv}, \alpha + i; 2\lambda)\}_{i \in [0,4)}$, which differs from the Intel path.

For fixed-tweak long-output PRGs, the Hygon backend exploits the fact that all lanes use the same counter sequence under different keys. For $\hat{\ell} = 276$, we precompute the two counters for the 2λ output, the first 16 counters for the first 256 bytes of the 276-byte output, and two additional counters for the 20-byte tail. The fused Hygon helper processes eight keys at a time as two four-key groups. For each group, it builds four key schedules, encrypts the shared counter sequence under the four keys, writes the 2λ outputs, writes the first 256 bytes, and copies only the first 20 bytes of the tail. For $\hat{\ell} = 162$, we precompute 11 counters, encrypt the first ten full blocks and one tail block with the same expanded key, and copy only the first two bytes of the tail block. Since this

¹¹ https://git.fh-aachen.de/tb3838s/linux/-/blob/c291c9cf76a8fb92ef3d66567e507009236ce90/arch/x86/crypto/sm4-aesni-avx2-asm_64.S

output is much shorter, we avoid the 16-block strategy used for the 276-byte case and instead reuse one Hygon key schedule for the 2λ and 162-byte outputs of each key.

Overall, the Hygon path keeps the same PRG semantics and counter layout as the Intel implementation, but replaces the AES-NI-based SM4 emulation with native SM4 instructions and uses Hygon-specific fused helpers where they reduce scheduling overhead.

PRG design in uBlockith. For the Ballet-CTR-style [5] PRG in uBlockith, we use a lane-packed round-key representation to align multiple-key batching with batched counter-mode encryption.

For the 2λ -bit PRG with $\lambda = 256$, we implement a 4-lane batched design for both key scheduling and encryption. Concretely, four independent seed keys are expanded into lane-packed round-key states, which are then consumed directly by the 4-key batched encryption kernel. For $\hat{\ell}$ -bit outputs, we use the same fixed-tweak batched PRG core with variable output length.

PRG design in Vistrutith. Vistrutith uses Vistrutah-CTR-style PRG with security level $\lambda = 512$ and 64-byte counter blocks. Vistrutah uses AES-based rounds as building blocks, so we use AES-NI for acceleration. For the 2λ -bit PRG, we prepare the Vistrutah key once and implement a dedicated 2-block encryption helper for counter and counter+1. For $\hat{\ell}$ -bit outputs, we use the same fixed-tweak batched PRG core with variable output length.

10.3.2 Acceleration Techniques of Field Operations

Basic field operations. All optimized implementations enable x86 carry-less multiplication with the PCLMULQDQ ISA for $\mathbb{F}_{2^n}$ arithmetic, where n is expected to be an integer multiple of 64. For $\mathbb{F}_{2^{160}}$ in SM4th, field elements are represented using three 64-bit limbs, where only the lower 32 bits of the most significant limb are part of the canonical representation. The optimized multiplication path treats the element as a padded 192-bit value, computes the unreduced carry-less product with PCLMULQDQ, and then reduces the resulting 320-bit product modulo this trinomial-like defining polynomial. For $\mathbb{F}_{2^{256}}$ in uBlockith, the optimized code uses a 2-way Karatsuba decomposition [49] to lower the number of 128-bit carry-less subproducts versus the naive split-product formulation. For $\mathbb{F}_{2^{512}}$ in Vistrutith, we represent elements as eight 64-bit limbs and use PCLMULQDQ with recursive 2-way Karatsuba, followed by tailored reduction modulo $m(t) = t^{512} + t^8 + t^5 + t^2 + 1$.

Constraints in SM4th. We give several acceleration techniques in SM4th.

Precomputing the embedding from $\mathbb{F}_{2^8}$ to $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. In the SM4-based variants of SM4th, the native SM4 byte operations are defined over $\mathbb{F}_{2^8}$, whereas the ZKP constraints are evaluated over the larger field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Our implementation embeds an SM4 byte $b = \sum_{i=0}^7 b_i \alpha^i$ into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ using precomputed powers of α . We further precompute the embeddings of all byte constants under their Frobenius conjugates, *i.e.*, the successive SM4 byte-field squarings. As a result, byte reconstruction, byte squaring, and norm-basis expansion become fixed linear combinations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, instead of repeatedly recomputing byte-field constants and their embeddings. In the optimized implementation, these linear combinations and the resulting $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ multiplications are accelerated with PCLMULQDQ and fused carry-less product accumulation.

Reducing repeated embeddings and field reductions. For SM4th-d3, we specialize the SM4 norm path for the linearized maps used by the byte-level affine layers. For such an affine map M , write $M(z) = c + \sum_{i=0}^7 a_i z^{2^i}$ over the SM4 byte field, where $z \in \mathbb{F}_{2^8}$ is the input byte, $c \in \mathbb{F}_{2^8}$ is the affine constant, and the coefficients $a_i \in \mathbb{F}_{2^8}$ are fixed by M . For the r -th Frobenius conjugate, the embedded value is $\langle M(z)^{2^r} \rangle_{\lambda_{\mathbb{F}}} = \langle c^{2^r} \rangle_{\lambda_{\mathbb{F}}} + \sum_{i=0}^7 \langle a_i^{2^r} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_{\mathbb{F}}}$. After embedding into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, each term $\langle a_i^{2^r} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_{\mathbb{F}}}$ is evaluated as a multiplication in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Concretely, the processor first computes an unreduced $2\lambda_{\mathbb{F}}$ -bit carry-less product and then obtain a $\lambda_{\mathbb{F}}$ -bit field element. Thus, a direct evaluation of one byte-level conjugate performs one $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ multiplication and one field reduction for each nonzero coefficient a_i .

We denote the modular reduction from an unreduced $2\lambda_{\mathbb{F}}$ -bit carry-less product to the $\lambda_{\mathbb{F}}$ -bit field representation by $\text{Red}_{\lambda_{\mathbb{F}}}$. A direct implementation reduces each product separately, computing $\langle c^{2^r} \rangle_{\lambda_{\mathbb{F}}} + \bigoplus_i \text{Red}_{\lambda_{\mathbb{F}}}(\langle a_i^{2^r} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_{\mathbb{F}}})$. Our implementation instead precomputes the $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ embeddings of the fixed SM4 byte-field constants and accumulates the unreduced carry-less products before applying one reduction, computing $\langle c^{2^r} \rangle_{\lambda_{\mathbb{F}}} + \text{Red}_{\lambda_{\mathbb{F}}}(\bigoplus_i \langle a_i^{2^r} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_{\mathbb{F}}})$. Since $\text{Red}_{\lambda_{\mathbb{F}}}$ is

$\mathbb{F}_2$ -linear, the two expressions are equivalent, while the latter avoids repeated reductions inside each fixed linear combination. For the word-level maps L , L' , L^{-1} , and $(L')^{-1}$, we apply the same idea across all products contributing to one output-byte conjugate. The norm-anchor check $h \cdot x^{16} \cdot x^2 + x = 0$ also uses the optimized $\mathbb{F}_{2^8}$ multiplication path for its two field multiplications, namely first $\langle h \rangle_{\lambda_F} \cdot \langle x^{16} \rangle_{\lambda_F}$ and then the product with $\langle x^2 \rangle_{\lambda_F}$.

Only computing and storing necessary conjugate tuples. In SM4th-d3, there are two kinds of constraint checks. The non-anchor checks verify the patched inversion relations $x^2y + x = 0$ and $xy^2 + y = 0$ in S-boxes, where x and y are the input and output of the patched inversion function. The norm-anchor checks verify $h \cdot x^{16} \cdot x^2 + x = 0$, where x is the affine S-box input and h is the claimed norm witness corresponding to x^{-17} .

For a byte value $z \in \mathbb{F}_{2^8}$, we call $(\langle z \rangle_{\lambda_F}, \langle z^2 \rangle_{\lambda_F}, \dots, \langle z^{128} \rangle_{\lambda_F})$ its full conjugate tuple after embedding into $\mathbb{F}_{2^{\lambda_F}}$. This tuple is only a local representation used by the constraint computation; it is not an additional component of the extended witness.

At a norm-anchor position, the full inverse byte is not stored as committed witness bits. The witness contains only the norm coordinate for $h = x^{-17}$. After checking $h \cdot x^{16} \cdot x^2 + x = 0$, the implementation reconstructs the inverse byte as $x^{-1} = x^{16} \cdot h$ and then applies the SM4 output affine map to obtain the S-box output byte. This reconstructed byte becomes a virtual key or state byte used by later SM4 rounds. Since such a virtual byte is not backed by committed output bits, its later propagation through byte-level affine maps and the word-level linear layers L and L' cannot recover the needed Frobenius conjugates by reloading witness bits. Therefore, the constraint computation materializes and carries the full conjugate tuple for these reconstructed virtual bytes.

For non-anchor positions, the corresponding byte values are already present in the extended witness. The patched-inversion check itself only needs $\langle x \rangle_{\lambda_F}$, $\langle x^2 \rangle_{\lambda_F}$, $\langle y \rangle_{\lambda_F}$, and $\langle y^2 \rangle_{\lambda_F}$. Thus, the implementation does not keep a full conjugate tuple solely for this check. If such a byte is later consumed by a linearized affine map or by the SM4 word-level linear layers, its tuple can be constructed on demand from the committed bits. This avoids carrying unnecessary conjugates for ordinary S-box positions, while still retaining full tuples for norm-anchor bytes whose reconstructed virtual values must continue through the SM4 computation.

10.4 Performance Test Results

Table 19 and Table 20 report the performance of our signature schemes for the reference implementation and the x86-64 architecture specific implementation, respectively.

The optimized implementations use accelerations of including PRGs and field operations. As required, the hash-function implementations use the auxiliary functions of SM3 and SM3-based XOFs provided by ICCS without any accelerations. Most implementations were run on a machine with an Intel CPU at 3.85 GHz. For SM4th-lo using SM4 as PRGs, Table 20 also provides the performance for the x86-64 architecture specific implementation on a Hygon CPU at 3.00 GHz, which supports the native SM4 instruction set. All implementations were run using a single core¹². The running times, averaged 1000 runs, are given in milliseconds and million CPU cycles.

All signatures have close signing and verifying times. The term “dn” means the constraint systems in ZKPs are degree- n . The “s” variants have smaller signature sizes but slower running times, while the “f” variants have faster running times but larger signature sizes, achieving different trade-offs by using different VOLE parameters. The EM variants use the Even-Mansour constructions as OWFs with shorter witness and fewer constraints, hence leading to smaller signature sizes and faster running times. The secret-witness keyspace-reduction constraint contributes only one additional OWF constraint in the EM variants and does not change any transmission-size term. For better performance, we recommend to use “EM-s” variants if smaller signature sizes are preferred, and to use “EM-f” variants if the running times are more important.

For SM4th variants, the sizes of the public key and the secret key are both 32 bytes. The key generation times are 3-5 μ s. On Intel CPUs, the signing and verifying times are either 3-5 ms for “f” variants, or 23-35 ms for “s” variants for the optimized implementations, regardless of the loose

¹² We plan to implement multi-core parallelization in the future if needed.

or tight variants. The CPU cycles of SM4th-lo on Hygon CPUs are around 25%-40% smaller than those on Intel CPUs, which benefits from the SM4-specific instruction set instead of accelerating SM4 with AES-NI. As for the reference implementation, the running times are 2-10 $\times$ slower than those on Intel CPUs. The smallest signature size for loose variants is 3 818 bytes for “EM-d2-128s-lo”, and the largest one is 6 724 bytes for “d3-128f-lo”. Tight variants feature 1.8-2 $\times$ larger signature sizes than loose ones. The peak memories for optimized implementations are less than 23 MB for the “s” variants, and less than 5 MB for the “f” variants.

For uBlockith variants, the sizes of the public key and the secret key are both 64 bytes, and the key generation times are 7-9 μ s. The signing and verifying times are either around 10 ms for the “f” variant, or 80-90 ms for “s” variants for the optimized implementation. As for the reference implementation, the running times are 2-4 $\times$ slower. The smallest signature size is 19 056 bytes for the “EM-d3-256s” variant, and the largest one is 31 556 bytes for the “d3-256f” one. The peak memories are around 26 MB for the “s” variants, and around 5 MB for the “f” variants.

For Vistrutith variants, the sizes of the public key and the secret key are both 128 bytes, and the key generation times are 5 μ s. The signing and verifying times are around 50 ms for the “f” variant, or larger than 300 ms for “s” variants for the optimized implementation. The running times of the optimized implementation are 5-7 $\times$ faster than the reference one, benefiting from the AES-NI instruction-set accelerations. The “s” variant has a larger signature size of 83 004 bytes, while the “f” variant requires 106 788 bytes. The peak memories are around 97 MB for the “s” variant, and around 13 MB for the “f” variant.

Table 19: Benchmark results for the reference implementation

Scheme	Runtimes in ms, Mcyc.						Sizes in B			Peak memory (KB)
	KeyGen		Sign		Verify		sk	pk	Sig	
SM4th-d3-128s-lo	0.004	0.011	87.652	245.426	72.949	204.257	32	32	5 056	16 480
SM4th-EM-d2-128s-lo	0.004	0.011	47.970	134.316	45.503	127.408	32	32	3 818	11 108
SM4th-d3-128f-lo	0.003	0.008	32.688	91.526	21.587	60.444	32	32	6 724	3 512
SM4th-EM-d2-128f-lo	0.003	0.008	7.384	20.675	7.054	19.751	32	32	4 932	2 896
SM4th-EM-d2-128s-ti	0.004	0.011	60.585	169.638	58.149	162.817	32	32	7 556	15 844
SM4th-d3-128s-ti	0.004	0.011	109.378	306.258	87.655	245.434	32	32	9 176	22 716
SM4th-d3-128f-ti	0.004	0.011	51.077	143.016	31.561	88.371	32	32	11 864	4 744
SM4th-EM-d2-128f-ti	0.004	0.011	9.010	25.228	8.602	24.086	32	32	9 450	3 564
uBlockith-d3-256s	0.010	0.036	239.959	850.291	241.414	860.909	64	64	24 144	25 564
uBlockith-EM-d3-256s	0.009	0.034	202.116	716.432	198.611	709.134	64	64	19 056	24 412
uBlockith-d3-256f	0.008	0.027	45.085	159.644	47.546	169.479	64	64	31 556	5 160
uBlockith-EM-d3-256f	0.007	0.025	36.000	128.247	37.913	135.849	64	64	25 028	4 896
Vistrutith-d3-512s	0.007	0.024	1780.279	6329.695	1749.544	6236.407	128	128	83 004	97 540
Vistrutith-d3-512f	0.007	0.024	340.649	1210.835	308.497	1100.714	128	128	106 788	18 012

Timing decomposition. Figure 5, Figure 6, Figure 7, Figure 8, and Figure 9 show the timing distributions of the optimized x86-64 accelerated implementations for the SM4th-EM-d2-lo, SM4th-EM-d2-ti, uBlockith-EM-d3, and Vistrutith-d3 instances. The total running time is divided into five categories: PRG, Hash, VOLE Com, ZKP, and Other. Here, the PRG category includes all pseudorandom generation costs, including seed generation, leaf commitment generation, and PRG calls inside the VOLE conversion procedure. The Hash category includes all hash-related operations. The VOLE Com category records the remaining VOLE commitment overhead after removing the PRG- and hash-related parts, while ZKP measures the OWF proving and verifying times. The Other category collects the residual cost not covered by the previous four classes.

Table 20: Benchmark results for the architecture specific implementation on x86-64 CPUs

Scheme	CPU	Runtimes in ms, Mcyc.						Sizes in B			Peak memory (KB)
		KeyGen		Sign		Verify		sk	pk	Sig	
SM4th-d3-128s-lo	Intel	0.004	0.011	27.931	78.209	23.767	66.565	32	32	5 056	16 496
	Hygon	0.004	0.011	17.394	51.972	12.957	38.859				16 704
SM4th-EM-d2-128s-lo	Intel	0.004	0.011	26.231	73.450	23.887	66.891	32	32	3 818	11 112
	Hygon	0.004	0.011	15.250	45.688	12.377	37.119				11 324
SM4th-d3-128f-lo	Intel	0.004	0.011	3.859	10.805	3.526	9.876	32	32	6 724	3 484
	Hygon	0.004	0.011	2.758	8.263	2.303	6.905				3 624
SM4th-EM-d2-128f-lo	Intel	0.003	0.010	3.557	9.958	3.376	9.453	32	32	4 932	2 924
	Hygon	0.004	0.011	2.113	6.337	1.869	5.605				3 228
SM4th-d3-128s-ti	Intel	0.005	0.016	34.979	124.918	30.878	110.259	32	32	9 176	22 752
SM4th-EM-d2-128s-ti	Intel	0.004	0.015	31.600	112.916	28.936	103.383	32	32	7 556	15 916
SM4th-d3-128f-ti	Intel	0.004	0.015	5.047	18.047	4.428	15.835	32	32	11 864	4 880
SM4th-EM-d2-128f-ti	Intel	0.004	0.015	4.042	14.461	3.787	13.550	32	32	9 450	3 712
uBlockith-d3-256s	Intel	0.009	0.033	89.265	312.614	87.927	312.270			24 144	25 480
uBlockith-EM-d3-256s		0.009	0.032	85.265	299.861	79.367	282.958	64	64	19 056	24 128
uBlockith-d3-256f		0.007	0.025	12.200	42.756	11.610	41.544			31 556	5 292
uBlockith-EM-d3-256f		0.007	0.025	10.954	38.449	10.412	37.258			25 028	5 060
Vistrutith-d3-512s	Intel	0.005	0.017	328.160	1150.404	322.205	1144.536	128	128	83 004	97 656
Vistrutith-d3-512f		0.005	0.016	51.107	179.236	46.720	166.249			106 788	12 912

From the distributions, one can see that the SM4th-EM-d2-lo instances are overwhelmingly dominated by the PRG cost, which accounts for roughly 70% of the total running times on Intel CPUs and around 30-40% on Hygon CPUs. The Hash part accounts for around 25% for loose variants on Intel CPUs, but explicitly increases on Hygon CPUs due to the more efficient SM4 ISA. The ZKP-specific cost is only less than 3%. In contrast, for the SM4th-ti, uBlockith-EM-d3, and Vistrutith-d3 instances, the hash component becomes substantially dominant, the relative shares of VOLE commitment become smaller, and the ZKP component becomes more visible. The “fast” variants exhibit a noticeably larger ZKP share than the corresponding “small” variants. Overall, these figures highlight that, under the optimized implementation, PRG and hash computations are the main performance bottlenecks, whereas the remaining overhead is comparatively small. Note that as required by ICCS, we use the C reference implementation of SM3 without any ISA. This also implies that our implementations would achieve better performance after modifying the underlying hash functions into accelerated versions.

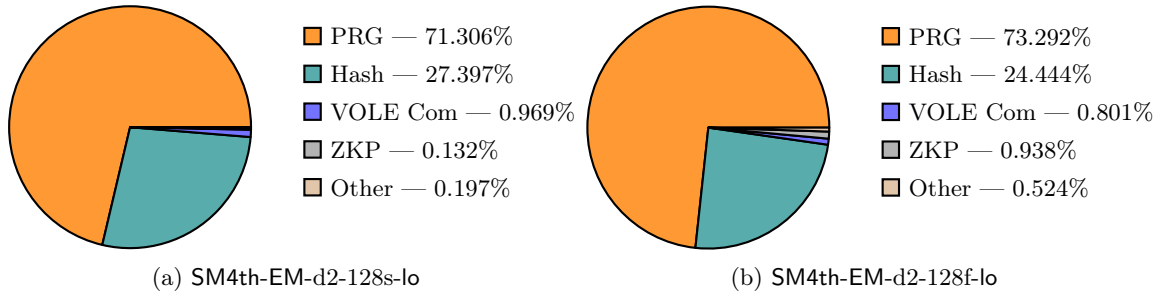


Fig. 5: The timing distributions for SM4th-EM-d2-128s-lo and SM4th-EM-d2-128f-lo under the x86-64 accelerated optimized implementation on Intel CPUs

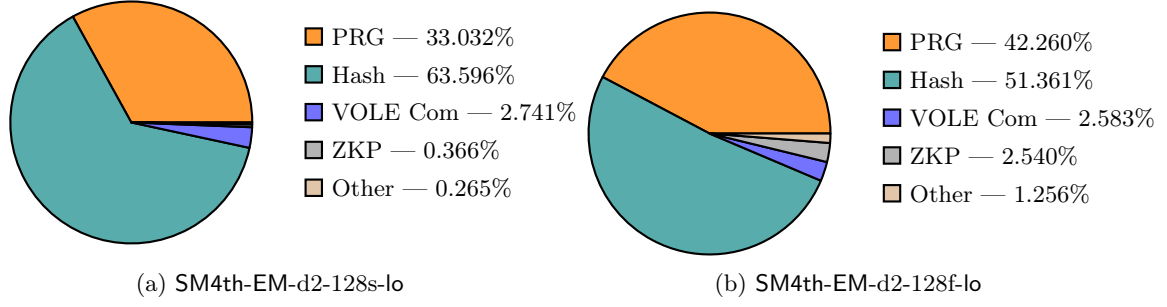


Fig. 6: The timing distributions for SM4th-EM-d2-128s-lo and SM4th-EM-d2-128f-lo under the x86-64 accelerated optimized implementation on Hygon CPUs

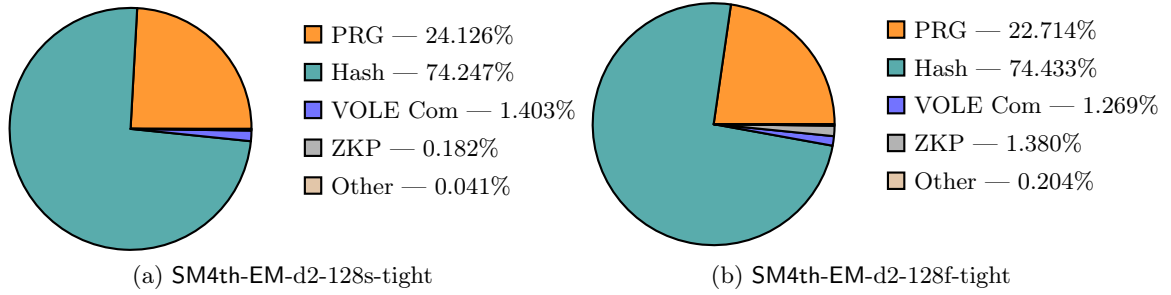


Fig. 7: The timing distributions for SM4th-EM-d2-128s-tight and SM4th-EM-d2-128f-tight under the x86-64 accelerated optimized implementation

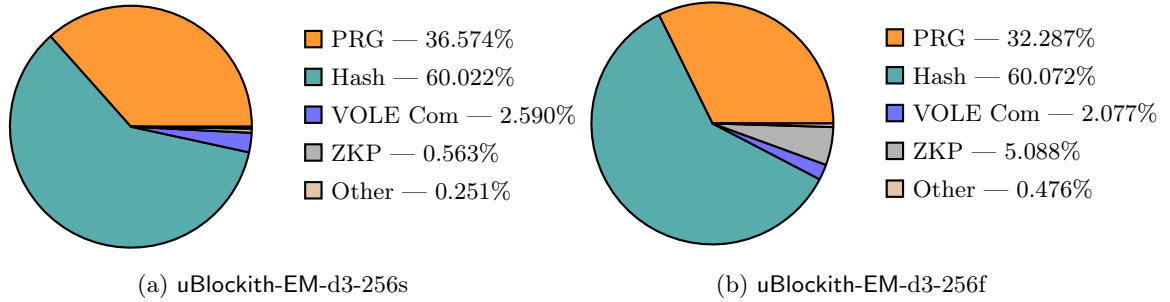


Fig. 8: The timing distributions for uBlockith-EM-d3-256s and uBlockith-EM-d3-256f under the x86-64 accelerated optimized implementation

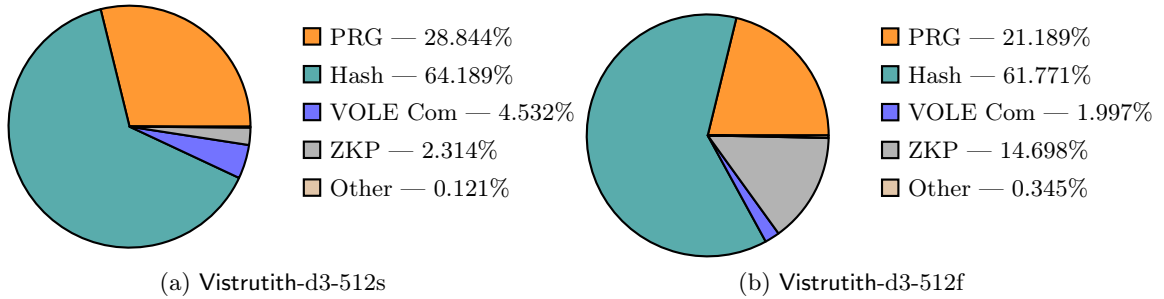


Fig. 9: The timing distributions for Vistrutith-d3-512s and Vistrutith-d3-512f under the x86-64 accelerated optimized implementation

11 Feature Statements

This section summarizes notable features of our signature schemes, including the features required by NGGC, and several additional advantages and limitations.

11.1 Features Required by ICCS

Innovativeness. One of our main contribution is to show that despite USA cryptographic primitives, Chinese block ciphers can also serve as efficient building blocks for VOLEitH-based post-quantum signature schemes. This includes two parts, ZKPs and PRGs. For the former, we propose three proving methods of S-boxes for SM4 and uBlock, achieving optimized signature sizes in VOLEitH-based constructions. For non-EM variants of SM4-based OWFs, we modify the norm technique in FAEST to make it compatible with SM4, which can not be directly instantiated due to the iterative structure of SM4. We also apply this technique to both the key expansion and encryption routines instead of the sole encryption routine as in FAEST. For the EM variants, we directly commit the output bytes of each S-box. For the uBlock with boolean-circuit-based S-Boxes, we use a “meet-in-the-middle” approach to avoid trivially committing all outputs of S-boxes. Our techniques may be helpful if using other block ciphers as OWFs for VOLE-based ZKPs in the future, as many block ciphers use S-boxes belonging to these two categories.

For PRGs, we propose a multiple-key-schedule ISA based on SM4-NI for the Chinese national standard SM4, which may be helpful for other cryptographic applications using SM4. Observing that many PRGs used in the signatures share a common plaintext, we also propose a fixed-plaintext batched PRG core, which may be generally useful for other VOLEitH-based protocols.

From the theoretical perspective, we give a concrete security analysis of VOLEitH-based signature schemes, which are not given explicitly in previous works [20–22].

Simplicity. Our signature schemes have a structurally simple and modular design. All concrete instantiations follow the same VOLEitH and QuickSilver-style three-challenge protocol: the signer first generates VOLE commitments, then commits to the extended witness derived from the block-cipher-based OWF, and finally proves the correctness of the low-degree constraints. Thus, the protocol logic is independent of the particular block ciphers used in the OWF and PRG.

The main scheme-specific part is the representation of the underlying block cipher inside the zero-knowledge proof. The remaining components, including BAVC-based VOLE commitments, transcript hashing, Fiat-Shamir transformation, field arithmetic, and the compressed VOLE/ZK checks, are shared across SM4th, uBlockith, and Vistrutith. This separation makes the construction easier to analyze and implement: PRGs can be chosen for software or hardware efficiency, OWFs can be chosen for ZKP-friendly constraint systems, and hash functions can be instantiated according to the required random-oracle and domain-separation properties.

Moreover, the size and performance behavior of the schemes is governed by a small set of explicit parameters. Public-key size, secret-key size, and signature size are derived directly from the main parameter tuple, while the trade-off between signing time and signature size is mainly controlled by VOLE parameters such as τ , w_{grind} , and T_{open} . This gives the schemes a transparent tuning methodology and makes future variants easier to audit.

Flexibility. Our signatures achieve flexible trade-offs in running times and transmission costs by using different VOLE parameters. Although this document provides two variants (“s” and “f”) for each scheme, more parameter choices are naturally feasible if one wants more aggressive trade-offs.

Compatibility. Our signature schemes use basically existing cryptographic primitives including Chinese-designed block ciphers SM4, uBlock, and Ballet, and Chinese national standard hash functions SM3 [2]. This implies a good compatibility when instantiated with existing Chinese national standard cryptographic optimizations and acceleration hardware.

Extensibility. Our signature schemes use an extensible design in VOLE commitments and ZKPs. The PRGs, OWFs, and hash functions can be easily changed with other alternatives that may improve performance or security. Note that the PRG and OWF may use different block ciphers as built in uBlockith. This means one can separately choose or design software-friendly PRGs to improve speeds and ZKP-friendly PRGs to optimize signature sizes.

Performance advantages on variant platforms. The proposed signatures are well-suited for platform-specific optimization because their dominant operations are symmetric-key computations, hash evaluations, finite-field arithmetic, and VOLE-related linear operations. These operations can naturally benefit from modern vector and cryptographic instruction sets. For example, **SM4th** can exploit SM4-specific instructions on platforms with native SM4 support such as Hygon CPUs, while still allowing efficient emulation or batching on general Intel or AMD x86-64 processors equipped with AES-NI. **uBlockith** benefits from vectorized implementations of Ballet-based PRGs, and **Vistrutith** can take advantage of AES-NI due to the AES-based structures inside Vistrutah.

The modular design further helps performance portability. The common VOLE commitment, BAVC opening, Fiat-Shamir transcript, and ZK checking logic are shared by all variants, while the platform-dependent optimization is mostly confined to the PRG backends. Therefore, new hardware accelerations or optimized cryptographic libraries can be integrated without changing the overall signature scheme. This makes the schemes attractive for various deployment environments, ranging from portable reference implementations to architecture-specific optimized ones.

11.2 Additional Advantages

Minimal assumptions. Our signature schemes use only symmetric primitives, with security relying on the one-wayness, pseudo-randomness, and ideal-cipher properties of PRGs as well as the random-oracle and pseudo-randomness properties of hash functions. They do not assume any number-theoretical and structured assumptions. For the 128-bit loose instantiations, our scheme solely relies on the Chinese national standard primitives SM4 and SM3.

Small sizes for keys and “keys + signatures”. The public-key sizes of our signatures at classical 128, 256, and 512-bit security, are 32, 64, and 128 bytes respectively. These are the same as SLH-DSA, and are $10\times$ smaller than ML-DSA and FL-DSA. As a result, the combined sizes of the public key and the signature are comparably small, *e.g.*, 3 818 bytes for **SM4th-EM-d2-128s-lo**, which is very close to the 3 732 bytes of ML-DSA, and smaller than the 3 938 bytes of AES-based FAEST-EM-128s.

Compared with the 7 888 bytes of SLH-DSA-short at 128-bit security, the signature sizes of **SM4th-d3-128s-lo** and **SM4th-EM-d2-128s-lo** are 50% and 71% smaller, and the sizes of **uBlockith-EM-d3-256s** and **uBlockith-EM-d3-256f** are 38% and 49% smaller in the same security levels, respectively.

Good, general-purpose performance. Our signatures achieve a good trade-off across public key and signature sizes, signing time and verifying time. This makes them stand out for general-purpose usage. Compared with SPHINCS⁺-Haraka (also using PRGs and hashes), apart from the smaller signatures, our signatures also have $2\text{--}5\times$ faster signing times, but with $100\times$ slower verifying times according to its reported figures [30]. Note that our implementations use the C reference hash functions required by ICCS, which would be further improved if using acceleration.

11.3 Limitations

Verifying times. Due to the linear verifier complexity of QuickSilver, our signature schemes feature very close times for signing and verifying. This is in contrast to ML-DSA, FL-DSA and SLH-DSA, where the verifying efficiency can be several times faster than the signing efficiency.

Signature sizes. The signature sizes of our schemes are generally larger than lattice-based schemes. They are also somewhat larger than the combination of VOLEitH and other OWFs based on structured assumptions such as syndrome decoding and the multivariate quadratic problem.

Reference

- [1] Goldwasser S, Micali S, Rackoff C. The knowledge complexity of interactive proof-systems (extended abstract)[C]//ACM Symposium on Theory of Computing (STOC). New York: ACM, 1985: 291–304.
- [2] Information security technology—SM3 cryptographic hash algorithm: GB/T 32905-2016[S]. Standardization Administration of China, 2016.
- [3] Information security technology—SM4 block cipher algorithm: GB/T 32907-2016[S]. Standardization Administration of China, 2016.
- [4] Wu W, Zhang L, Zheng Y, et al. The design of uBlock block cipher (in Chinese)[EB/OL]. (2019-12-10)[2026-04-01]. https://sfjs.cacnet.org.cn/site/term/list_76_1.html.
- [5] Cui T, Wang M, Fan Y, et al. The design of Ballet block cipher (in Chinese)[EB/OL]. (2019-12-10)[2026-04-01]. https://sfjs.cacnet.org.cn/site/term/list_76_1.html.
- [6] Avanzi R, Chakraborti A, Chakraborty B, et al. The large block cipher Vistrutah[J]. IACR Trans. Symmetric Cryptol., 2025, 2025(3): 1–150.
- [7] Fiat A, Shamir A. How to prove yourself: Practical solutions to identification and signature problems[C]//Annual International Cryptology Conference (CRYPTO). Berlin: Springer, 1986: 186–194.
- [8] Baum C, Dittmer S, Scholl P, et al. Sok: vector ole-based zero-knowledge protocols[J]. Design, Codes and Cryptography, 2023, 91(11): 3527–3561.
- [9] Baum C, Braun L, de Saint Guilhem C D, et al. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head[C]//Annual Cryptology Conference (CRYPTO). Berlin: Springer, 2023: 581–615.
- [10] Yang K, Sarkar P, Weng C, et al. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field[C]//ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, 2021: 2986–3001.
- [11] Chase M, Derler D, Goldfeder S, et al. Post-quantum zero-knowledge and signatures from symmetric-key primitives[C]//ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, 2017: 1825–1842.
- [12] Ishai Y, Kushilevitz E, Ostrovsky R, et al. Zero-knowledge from secure multiparty computation[C]//ACM Symposium on Theory of Computing (STOC). New York: ACM, 2007: 21–30.
- [13] Albrecht M R, Rechberger C, Schneider T, et al. Ciphers for MPC and FHE[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT). Berlin: Springer, 2015: 430–454.
- [14] Feneuil T, Rivain M. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments[J]. Journal of Cryptology, 2025, 38(3): 28.
- [15] Baum C, de Saint Guilhem C D, Kales D, et al. Banquet: Short and fast signatures from AES [C]//Public-Key Cryptography (PKC). Berlin: Springer, 2021: 266–297.
- [16] Adj G, Aragon N, Barbero S, et al. Mirath signature scheme: Algorithm specifications and supporting documentation, version 2.0[EB/OL]. (2025-07-25)[2026-04-01]. https://pqc-mirath.org/assets/downloads/mirath_v2.0.1.pdf.
- [17] Benadjila R, Feneuil T, Rivain M. MQ on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem[C]//IEEE European Symposium on Security and Privacy (EuroS&P). Los Alamitos: IEEE, 2024: 468–485.
- [18] Bettaieb S, Bidoux L, Dyseryn V, et al. PERK: compact signature scheme based on a new variant of the permuted kernel problem[J]. Design, Codes and Cryptography, 2024, 92(8): 2131–2157.
- [19] Bidoux L, Chi-Domínguez J, Feneuil T, et al. RYDE: A digital signature scheme based on Rank-Syndrome-Decoding problem with MPCitH paradigm[J]. CoRR, 2023, abs/2307.08726.
- [20] Melchor C A, Bettaieb S, Bidoux L, et al. The syndrome decoding in the head (SD-in-the-head) signature scheme: Algorithm specifications and supporting documentation, version 2.0 [EB/OL]. (2025-02-05)[2026-04-01]. <https://sdith.org/docs/sdith-v2.0.pdf>.

- [21] Baum C, Beullens W, Braun L, et al. FAEST v2: Algorithm specifications version 2.0[EB/OL]. (2025)[2026-04-01]. <https://faest.info/faest-spec-v2.0.pdf>.
- [22] Cui H, Liu H, Yan D, et al. ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-head[C]//Public-Key Cryptography (PKC). Berlin: Springer, 2024.
- [23] Bai S, Ducas L, Kiltz E, et al. CRYSTALS-Dilithium algorithm specifications and supporting documentation (version 3.1)[EB/OL]. (2021-02-08)[2026-04-01]. <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>.
- [24] Fouque P A, Lyubashevsky V, Hoffstein J, et al. Falcon: Fast-Fourier lattice-based compact signatures over NTRU[EB/OL]. (2021-01-10)[2026-04-01]. <https://falcon-sign.info/falcon.pdf>.
- [25] Bos J W, Bronchain O, Ducas L, et al. HAWK: Version 1.1[EB/OL]. (2025-02-05)[2026-04-01]. <https://hawk-sign.info/hawk-spec.pdf>.
- [26] Baldi M, Barenghi A, Battagliola M, et al. CROSS: Codes and restricted objects signature scheme, algorithm specifications and supporting documentation, version 2.2[EB/OL]. (2025-07-31)[2026-04-01]. https://www.cross-crypto.com/CROSS_Specification_v2.2.pdf.
- [27] Baldi M, Barenghi A, Beckwith L, et al. LESS: Linear equivalence signature scheme[EB/OL]. 2025-02-07[2026-04-25]. <https://www.less-project.com/LESS-2025-02-07.pdf>.
- [28] Beullens W, Campos F, Celi S, et al. MAYO: Round 2 version[EB/OL]. (2025)[2026-04-25]. <https://pqmayo.org/assets/specs/mayo-round2.pdf>.
- [29] Aardal M A, Adj G, Aranha D F, et al. SQIsign: Algorithm specifications and supporting documentation, version 2.0.1[EB/OL]. (2025-07-07)[2026-04-25]. <https://sqisign.org/spec/sqisign-20250707.pdf>.
- [30] Aumasson J P, Bernstein D J, Beullens W, et al. SPHINCS+: Submission to the NIST post-quantum project, v.3.1[EB/OL]. (2022-06-10)[2026-04-01]. <https://sphincs.org/data/sphincs+-r3.1-specification.pdf>.
- [31] Goldreich O, Goldwasser S, Micali S. How to construct random functions (extended abstract) [C]//Annual Symposium on Foundations of Computer Science (FOCS). Los Alamitos: IEEE, 1984: 464–479.
- [32] Baum C, Beullens W, Mukherjee S, et al. One tree to rule them all: Optimizing GGM trees and owfs for post-quantum signatures[C]//International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT). Berlin: Springer, 2024: 463–493.
- [33] Roy L. Softspokenot: Quieter OT extension from small-field silent VOLE in the minicrypt model[C]//Annual International Cryptology Conference (CRYPTO). Berlin: Springer, 2022: 657–687.
- [34] National Institute of Standards and Technology. Submission requirements and evaluation criteria for the Post-Quantum Cryptography standardization process[EB/OL]. 2016. <https://csrc.nist.gov/csrc/media/projects/post-quantum-cryptography/documents/call-for-proposals-final-dec-2016.pdf>.
- [35] Sarah D., Peter C. On the practical cost of Grover for AES key recovery[C]//The 5th NIST PQC Standardization Conference. 2024.
- [36] Even S, Mansour Y. A construction of a cipher from a single pseudorandom permutation[J]. Journal of Cryptology, 1997, 10(3): 151–162.
- [37] Chiesa A, Ojha D, Spooner N. Fractal: Post-quantum and transparent recursive proofs from holography[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT). Berlin: Springer, 2020: 769–793.
- [38] Dobraunig C, Kales D, Rechberger C, et al. Shorter signatures based on Tailor-Made minimalist symmetric-key crypto[C]//ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, 2022: 843–857.
- [39] Zou J, Guo N, Li J. Efficient quantum circuit implementations of SM3 and RIPEMD-160 hash functions (in Chinese)[J]. Journal on Communications, 2025, 46(12): 222–233.
- [40] Grassl M, Langenberg B, Roetteler M, et al. Applying grover’s algorithm to AES: quantum resource estimates[C]//Post-Quantum Cryptography - International Workshop (PQCrypto). Berlin: Springer, 2016: 29–43.
- [41] Liu M, Chen J. Improved linear attacks on the Chinese block cipher standard[J]. Journal of Computer Science and Technology, 2014, 29(6): 1123–1133.

- [42] Wang C, Cui J, Li M, et al. Improved integral cryptanalysis on block cipher uBlock (in Chinese)[J]. Journal of Electronics & Information Technology, 2024, 46(5): 2149–2158.
- [43] Liu F, Ji W, Hu L, et al. Analysis of the SMS4 block cipher[C]//Information Security and Privacy - Australasian Conference (ACISP). Berlin: Springer, 2007: 472–492.
- [44] Giacon F, Heuer F, Poettering B. KEM combiners[C]//Public-Key Cryptography (PKC). Berlin: Springer, 2018: 190–218.
- [45] Don J, Fehr S, Huang Y. Adaptive versus static multi-oracle algorithms, and quantum security of a split-key PRF[C]//Theory of Cryptography - International Conference (TCC). Berlin: Springer, 2022: 33–51.
- [46] Chiesa A, Yogev E. Building cryptographic proofs from hash functions[EB/OL]. (2024)[2026-04-01]. <https://snargsbook.org/>.
- [47] Lin D, Xiang Z, Zhang R, et al. Quantum implementation of SM4 (in Chinese)[J]. Journal of Cryptologic Research, 2021, 8(6): 999–1018.
- [48] Brassard G, Høyer P, Tapp A. Quantum cryptanalysis of hash and claw-free functions[C]//LATIN 1998: Theoretical Informatics. 1998: 163–169.
- [49] Karatsuba A, Ofman Y. Multiplication of many-digital numbers by automatic computers[J]. Proceedings of the USSR Academy of Sciences, 1962, 145: 293–294.